

WEISS · DATA STRUCTURES & ALGORITHM ANALYSIS

数据结构与算法分析

C + + 版教辅 · 逐章精讲

维斯《数据结构与算法分析——C++ 描述》12 讲全程配套

面向会 C 语言、数据结构零基础的读者 · 知识导图 + 详细讲解 + 例题精讲 + 章末实验

第1章 绪论 · 第2章 算法分析 · 第3章 表、栈和队列 · 第4章 树

第5章 散列 · 第6章 优先队列(堆) · 第7章 排序 · 第8章 不相交集

第9章 图论算法 · 第10章 算法设计技巧 · 第11章 摊还分析 · 第12章 高级数据结构

知识导图 × C 语言对照 × 复杂度速查 × 章末实验

第1章 绪论

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 复习全书的地基数学:指数、对数、级数、模运算,并知道它们在算法中的用途
- 理解"程序 = 数据结构 + 算法",建立正确性与效率两个评判维度,预告复杂度记号
- 掌握**递归的四法则**,能写出阶乘等经典递归并分析其递推式
- 完成 C 到 C++ 的第一波迁移:cin/cout、string、vector、class,能独立编译运行第一个程序
- 亲手实现斐波那契的朴素递归与记忆化,体会 $O(2^n)$ 与 $O(n)$ 之别
- 理解调用栈与栈帧,知道深层递归为何段错误、如何规避;学完能独立实现汉诺塔与全排列

💡 从 C 到 C++:本章主题如何迁移

先给全书的一句话直觉:数据结构决定"数据怎么放",算法决定"问题怎么做",两者合起来决定"程序跑多快"。第1章是唯一的"地基章",地基不牢,后面 11 章都要还债。

语言迁移上,你已有的 C 技能全部有效——C++ 是 C 的超集。本章只补三样最常用的新工具:cin/cout 输入输出、string 与 vector (自动管理内存的动态数组)、class (把数据和操作打包),它们覆盖本书 90% 的示例代码。

算法直觉上,本章最重要的思想是**递归**:把大问题缩小成"同样结构的小问题"。同一个斐波那契,笨写是指数级,加一行"记忆化"就变成线性——这是全书第一次"复杂度决定生死"的演示。

📦 前置知识自查

数学前置:高中数学:幂运算与对数(§1.1 会系统复习);求和记号 Σ 会读即可。

C 语言前置:函数定义与调用、for/while 循环、数组与下标、printf/scanf、struct 与指针(会更好)。

依赖的前面章节:无,本章是全书第一章。

🚀 本章学习路线

1. **第一阶段(1 小时):**读 §1.1–§1.2,动手验证取模等例子,背里程碑数值
2. **第二阶段(1.5 小时):**读 §1.3–§1.4,编译运行 §1.4 完整小程序并改造
3. **第三阶段(1.5 小时):**读 §1.5–§1.6,运行两版斐波那契,看 n 从 30 到 40 时间如何爆炸
4. **第四阶段(实验,1 小时):**完成章末实验:汉诺塔与全排列,验证 $2^n - 1$ 、 $n!$
5. **验收标准:**能默写递归四法则;能独立写出两版斐波那契;能解释深层递归为何崩溃

本章知识导图

- §1.1 数学复习:指数规则、对数与换底、级数三公式、模运算与负数取模(C++/Python 差异)
- §1.2 算法概念:程序 = 数据结构 + 算法、正确性与效率、复杂度记号预告
- §1.3 递归简介:递归四法则、C 递归与调用栈回顾、阶乘示例
- §1.4 从 C 到 C++ 速成:输入输出、string/vector、函数、类入门、编译与完整小程序
- §1.5 完整示例:斐波那契——朴素递归(指数级)vs 记忆化(线性)、手写 vs 标准库
- §1.6 递归的工程注意:调用栈与栈帧、栈溢出与段错误、如何避免深层递归
- §1.7 全书路线图:12 章一句话导航

§1.1 数学复习

1.1.1 指数规则

指数是全书最频繁出现的运算,下列规则必须刻进直觉:

- $a^0 = 1 (a \neq 0); a^1 = a; a^{-n} = 1/a^n$
- **同底数相乘,指数相加:** $a^m \cdot a^n = a^{m+n}$
- **幂的幂,指数相乘:** $(a^m)^n = a^{m \cdot n}$
- **同底数相除,指数相减:** $a^m / a^n = a^{m-n}$

背几个里程碑数值: $2^{10} = 1024; 2^{20} \approx 10^6; 2^{30} \approx 10^9$ 。反过来: $10^6 \approx 2^{20}$,对 100 万数据"每次减半"地查找最多约 20 步——第2章对数复杂度的直觉起点。

记号说明(全书约定,首次出现)

本书数学记号统一:幂用上标(n^2 、 2^n);对数底数用下标($\log_2 n$);递推式行内用等宽代码($T(n) = T(n-1) + O(1)$);求和用 $\Sigma(\Sigma_{i=1}^n i = n(n+1)/2)$;模运算正文写 $a \bmod b$,代码里一律写 $a \% b$ 。

1.1.2 对数

对数的定义: $\log_b X = a$ 当且仅当 $b^a = X (b > 0, b \neq 1, X > 0)$ 。直觉:对数回答"几次方"——"2 的几次方等于 8"?答 $\log_2 8 = 3$ 。对数与指数互为逆运算,是"减半""倍增"类算法(二分查找、快速幂)的天然度量。

换底公式: $\log_b X = \log_a X / \log_a b$ 。推导:设 $\log_b X = c$,则 $b^c = X$,两边对 a 取对数即得 $c \cdot \log_a b = \log_a X$ 。工程含义:换底只差常数倍,而常数倍会被大 O 记号吞掉(第2章),所以**复杂度分析中底数不重要**。

记法约定:计算机领域 $\log$ 默认以 2 为底(算法世界是"减半"与"倍增"的世界); $\ln$ 以 $e \approx 2.718$ 为底; $\lg$ 有时指以 10 为底。里程碑: $\log_2 1024 = 10; \log_2 10^6 \approx 20$ 。对数"几乎不增长":规模翻 1000 倍, $\log_2 n$ 只从 10 长到 20。

1.1.3 级数

三个级数公式是第2章复杂度计算的原料,先记住结论与量级:

- **算术级数:** $\Sigma_{i=1}^n i = n(n+1)/2 \approx n^2/2$ ——双重循环的总迭代次数,量级 $O(n^2)$ 。例: $1+2+\dots+100 = 5050$

- **几何级数**: $\sum_{i=0}^n 2^i = 2^{n+1} - 1$ ——倍增翻倍的总量,量级 $O(2^n)$ 。例: $1+2+4+8+16+32 = 63$;推广: $\sum_{i=0}^n r^i = (r^{n+1} - 1)/(r-1)$
- **调和级数**: $\sum_{i=1}^n 1/i \approx \ln n$ ——量级 $O(\log n)$,出现在快速排序平均分析的求和中

记法: $\sum_{i=1}^n f(i)$ 表示把 i 从 1 到 n 逐一代入 $f(i)$ 再求和。第2章会把循环迭代次数翻译成这样的求和——两个记号是一回事。

1.1.4 模运算

模运算: $a \bmod b$ 表示 a 除以 b 的余数,数学上写成 $a = q \cdot b + r$;但"余数"取决于"商怎么取整",这是各语言分歧的根源(★小节)。两条最重要的性质(散列、循环数组、快速幂的基础):

- $(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- $(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$

★ 重点:C/C++ 与 Python 的负数取模不一样

关键分歧在**负数**:C/C++ 除法"向 0 截断"(truncation),余数符号与被除数相同;Python 除法"向下取整"(floor),余数符号与除数相同。同一表达式两种语言答案不同:

C/C++ 的写法(商向 0 截断)	Python 的写法(商向负无穷取整)
<code>-7 % 3</code> 结果是 -1 (商截断为 -2,余 $-7 - (-2) \cdot 3 = -1$)	<code>-7 % 3</code> 结果是 2 (商取整为 -3,余 $-7 - (-3) \cdot 3 = 2$)
<code>7 % -3</code> 结果是 1 (商 -2,余 1)	<code>7 % -3</code> 结果是 -2 (商 -3,余 -2)
<code>-7 / 3</code> 结果是 -2 (截断)	<code>-7 // 3</code> 结果是 -3 (向下取整)

两者"同号"时行为一致,只在**异号**时分歧;判奇偶、判整除不受影响。本书代码都在非负语境用 `%`,但移植代码时它会悄悄改变结果,务必知道这个坑。

⚠ 误区 1:默认负数取模结果是正数

C++(与 C 相同)中 `-7 % 3` 是 **-1**:余数符号随被除数。要"数学意义的非负余数",用惯用法: `int r = a % b; if (r < 0) r += b;`。同源陷阱是除法截断: `-7 / 3 == -2`,与"向下取整"的直觉相反,二分查找中写错就是越界。

⚠ 误区 2:int 溢出是未定义行为

C++ 的 `int` 是 32 位,范围约 $\pm 2.1 \times 10^9$; `INT_MAX + 1` 回绕成负数是**未定义行为**——可能负数、可能崩溃、可能被优化器"整段删除"。斐波那契 $F(47)$ 就超过 `int` 范围(\$1.5)。本书规则:可能变大的整数一律用 `long long`。

示例 1-1 C++ 负数取模实测


```
// C 与 C++ 相同:取模是"向 0 截断"除法的余数
// 公式:余数 = 被除数 - 商 × 除数,商向 0 取整
#include <iostream>
using namespace std;
int main() {
    cout << -7 % 3 << endl;    // -1:商取 -2,余数 -7 - (-2)*3 = -1
    cout << 7 % -3 << endl;    // 1:商取 -2,余数 7 - (-2)*(-3) = 1
    cout << -7 / 3 << endl;    // -2:除法同样向 0 截断,不是向下取整!
    int r = -7 % 3;            // 惯用法:修正成非负余数
    if (r < 0) r += 3;
    cout << r << endl;        // 2:与 Python 的 -7 % 3 一致了
    return 0;
}
```

§1.2 算法概念

1.2.1 程序 = 数据结构 + 算法

瑞士计算机科学家 Niklaus Wirth 的名言: **程序 = 数据结构 + 算法**。**数据结构**回答"数据如何组织",**算法**回答"问题如何求解"。经典例子——电话簿:按姓名排序可以二分查找(第2章)瞬间定位;打乱顺序只能逐个翻。数据没变,变的是"组织方式 + 查找算法"。

1.2.2 正确性与效率

一个解法要配得上"算法"二字,必须同时满足:**正确性**——对一切合法输入给出正确输出;**效率**——时间与空间可接受。两条缺一不可:正确但慢到跑不完,等于不能用;快但结果错,比慢更危险。

例:求 2^n ,连乘 n 次要一百万次乘法($n = 10^6$);第2章的快速幂约 20 次。本书尺度:先保证正确,再追求高效;复杂度分析(第2章)给"效率"机器无关的精确度量,本章只需建立直觉:数据规模变大时,运行时间如何增长。

1.2.3 复杂度记号预告

全书用大 O 记号描述效率。先混个脸熟,第2章给严格定义:

- $O(1)$ 常数:步数与规模无关——按下标取元素
- $O(\log n)$ 对数:规模翻倍只多一步——二分查找
- $O(n)$ 线性:遍历一趟——线性查找、求和
- $O(n \log n)$ 线性对数:归并排序、快排(平均)
- $O(n^2)$ 平方:双重循环——插入排序
- $O(2^n)$ 指数:穷举、朴素递归斐波那契——规模稍大就不可用

增长阶排序(背诵):常数 < 对数 < 线性 < 线性对数 < 平方 < 立方 < 指数。本章递归将用到两个结论: $T(n) = T(n-1) + O(1)$ 的解是 $O(n)$ (阶乘), $T(n) = T(n-1) + T(n-2) + O(1)$ 的解是 $O(2^n)$ (朴素斐波那契)。

§1.3 递归简介

1.3.1 递归四法则

Weiss 教材把递归写成四条法则,每条都配一个"违反后果":

1. **基准情形(base case)**:必须存在不需要递归、直接得出答案的情形。没有基准情形 = 无限递归。
2. **不断推进**:每次递归调用都必须让问题朝基准情形靠近。不推进 = 死循环;推进过快跳过基准情形 = 边界错误。
3. **设计法则**:假设所有递归调用都正确,只设计"当前层如何利用子问题的答案"。不要手工跟踪每一层——人脑栈只有几层深,程序栈可以几十万层。
4. **合成效益法则**:不要在递归调用的不同分支中重复求解同一个问题实例。重复求解会让复杂度爆炸成指数级——§1.5 的朴素斐波那契正是反面教材。

1.3.2 C 递归与调用栈回顾

你学 C 时写过递归,现在补上底层机制。程序运行时,系统维护一块**调用栈**(call stack):每次函数调用,系统把"返回地址 + 参数 + 局部变量"打包成**栈帧**(stack frame)压入;函数返回时弹出。递归没有任何特殊——递归调用也是普通调用,只是被调用的函数与自己同名。

由此得两个推论:① **递归深度 = 同时存活(尚未返回)的栈帧数**;② 递归每层多一个栈帧,这是迭代没有的成本。层数过多时栈空间耗尽——进程直接崩溃(§1.6 详解)。C 与 C++ 的递归写法逐字符一致。

1.3.3 阶乘示例

阶乘 $n! = n \times (n-1) \times \dots \times 1$,基准情形 $0! = 1$,递归定义 $n! = n \cdot (n-1)!$,递推式 $T(n) = T(n-1) + O(1)$,解为 $O(n)$ 。

示例 1-2 阶乘:递归版与迭代版

```
// 递归版:把 n! 分解为 n * (n-1)!, n <= 1 是基准情形
// 与 C 的写法完全一致——递归是语言无关的思想
long long factRec(int n) {
    if (n <= 1) return 1;           // 基准情形:0! = 1! = 1
    return n * factRec(n - 1);      // 不断推进:n 越来越小
}

// 迭代版:循环乘过去,不占调用栈
long long factIter(int n) {
    long long r = 1;
    for (int i = 2; i <= n; ++i)
        r *= i;
    return r;
}

// 验证:factRec(10) = factIter(10) = 3628800
```

§1.4 从 C 到 C++ 速成

1.4.1 输入输出:printf/scanf → cin/cout

C++ 的 `iostream` 用 `cin` (读入)与 `cout` (输出)替代 `scanf/printf`: `cout << x` 输出 `x`, `cin >> x` 读入 `x`,可连续拼接 (`cin >> a >> b`),自动按类型解析,**不需要 %d/%f/%s 格式符**——格式符写错(如 %d 配 long long)是 C 程序最常见的崩溃源之一。

C 的写法	C++ 的写法
<code>printf("n = %d\n", n);</code>	<code>cout << "n = " << n << endl;</code>
<code>scanf("%d", &n);</code>	<code>cin >> n;</code>
<code>scanf("%s %d", name, &score);</code>	<code>cin >> name >> score;</code>
格式符写错 = 静默读错/崩溃	类型由编译器推导,无格式符

注意: `cin >>` 自动跳过空白;读整行用 `getline(cin, s)`;混用时先 `cin.ignore()` 吃掉残留回车——C++ 入门第一坑。

1.4.2 字符串与动态数组:char*/malloc → string/vector

C 里动态数组要自己管"长度、容量、扩容、释放"四件事; `std::string` 与 `std::vector` 全部接管:自动扩容、离开作用域自动释放(RAII,第3章细讲)、自带长度 `size()`。学习阶段理解底层,工程阶段直接用标准库。

C 的写法	C++ 的写法
<code>int *a = malloc(n * sizeof(int));</code>	<code>vector<int> a(n);</code>
<code>a[len++] = x;</code> (自己管长度)	<code>a.push_back(x);</code> (自动扩容)
<code>strcpy(buf, s); strcat(buf, t);</code>	<code>string s = "hello"; s += " world";</code>
<code>free(a);</code> (忘记即泄漏)	离开作用域自动释放

示例 1-3 string 与 vector 速览

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
int main() {
    string s = "hello";           // 赋值即拷贝,无需 strcpy
    s += " world";               // 拼接,无需 strcat
    cout << s.size() << " " << s[0] << endl; // 11 h
    vector<int> v = {3, 1, 4};    // 初始化列表,无需手动 malloc
    v.push_back(1);              // 尾部插入,自动扩容
    v[0] = 7;                    // 按下标访问,0(1)
    cout << v.size() << " " << v[0] << endl; // 4 7
    for (int x : v)               // 范围 for:遍历容器
        cout << x << " ";
    return 0;
}
```

1.4.3 函数:C 的函数在 C++ 中全部有效

C++ 函数是 C 函数的超集:返回值、参数、递归——写法完全不变。新增三样:引用传参 `void f(int& x)` ——直接改实参,替代 C 的指针传参;函数重载——同名函数按参数类型区分;类型常由编译器推导(`auto`)。范围 `for`(`for (int x : v)`)是遍历容器的标准姿势,不会越界。

1.4.4 类入门:struct+函数 → class

C 里"数据 + 操作"分离:struct 存数据,全局函数操作它,每次调用传指针。C++ 的 `class` 把数据与操作封装成一个整体, `obj.func()` 自动绑定到该对象。这是全书所有数据结构(栈、树、堆.....)的呈现形式,第3章从抽象数据类型(ADT)角度深化。

示例 1-4 类入门:计数器

```
// C 的做法:struct Counter { int cnt; }; + 全局函数 counter_inc(&c)
// C++ 的做法:数据与操作放进同一个 class,成员函数直接操作自己的数据
class Counter {
    int cnt_ = 0;                // 成员变量(默认私有,外部不可见)
public:
    void inc() { ++cnt_; }        // 成员函数:修改自己的状态
    void reset() { cnt_ = 0; }
    int get() const { return cnt_; } // const 成员函数:承诺不修改对象
};
// 使用:Counter c; c.inc(); c.inc(); cout << c.get(); // 输出 2
```

1.4.5 编译运行与完整小程序

本书 C++ 代码统一按 C++17 标准编译: `g++ -std=c++17 hello.cpp -o hello && ./hello` (MSVC: `cl /EHsc /std:c++17 hello.cpp`)。建议加 `-Wall -Wextra` 打开全部警告——警告是免费的调试器。

示例 1-5 完整小程序:学生成绩统计(综合本节全部知识点)

```
// hello.cpp — 第一个完整 C++ 程序:学生成绩统计
// 输入:学生姓名与分数;输出:平均分与最高分
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Student {                                // 类:姓名 + 分数
public:
    string name;
    int score;
    Student(string n, int s) : name(n), score(s) {} // 构造函数
};

int main() {
    int n;
    cout << "学生人数: ";
    cin >> n;
    vector<Student> stu;                        // 动态数组:存 Student 对象
    for (int i = 0; i < n; ++i) {
        string name;
        int score;
        cin >> name >> score;                  // 读入一行中的两个值
        stu.push_back(Student(name, score));
    }
    int sum = 0, best = 0;
    for (const Student& s : stu) {              // 范围 for:引用避免拷贝
        sum += s.score;
        if (s.score > best) best = s.score;
    }
    cout << "平均分: " << sum / n << ",最高分: " << best << endl;
    return 0;
}
// 运行示例:
// 学生人数: 3,输入:张三 90、李四 85、王五 95
// 平均分: 90,最高分: 95
```

§1.5 完整示例:斐波那契

1.5.1 朴素递归:指数级

斐波那契数列: $F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$ 。照定义直接写递归——“正确但低效”:

示例 1-6 朴素递归斐波那契, $O(2^n)$

```
// F(0)=0, F(1)=1, F(n) = F(n-1) + F(n-2)
// 朴素递归:F(k) 被重复计算无数遍(违反合成效益法则)
long long fibNaive(int n) {
    if (n <= 1) return n;                        // 基准情形
    return fibNaive(n - 1) + fibNaive(n - 2);    // 不断推进,但一裂为二
}

// 调用次数 =  $2 \cdot F(n+1) - 1$  ( $F(40)$  约 3.3 亿次),  $n$  每加 10 约乘 120
// 递推式  $T(n) = T(n-1) + T(n-2) + O(1) \rightarrow O(2^n)$ 
```

⚠ 误区 3:"递归只是慢一点"——是数量级爆炸

同一输入,朴素版 $F(40)$ 约 3.3 亿次调用(本机约 0.1 秒,慢的机器数秒),记忆化版(下节)约 41 次调用——相差约八百万倍,不是"几倍"。指数级增长意味着:输入每加一点,时间乘一个常数—— n 从 40 到 50,调用数再乘约 120。另外, $F(47)$ 就超过 32 位 int 范围,示例统一用 `long long`。

1.5.2 记忆化:线性

观察调用树: $F(5)$ 要算 $F(4)$ 、 $F(3)$, $F(4)$ 又要算 $F(3)$ 、 $F(2)$ —— $F(3)$ 被重复计算,更小的 $F(k)$ 被重复更多次。解决办法是**记忆化**(memoization):第一次算出 $F(k)$ 就存进表格,以后直接查表。每个 $F(k)$ 只算一次,共 $n+1$ 个子问题,总计 $O(n)$ 。

示例 1-7 记忆化斐波那契, $O(n)$

```
// 记忆化:算过的 F(k) 存进数组,下次直接取
// 每个 F(k) 只算一次,调用次数从指数级降到 O(n)
#include <vector>
using namespace std;

long long fibMemo(int n, vector<long long>& memo) {
    if (memo[n] != -1) return memo[n];    // 已经算过:直接返回(查表)
    if (n <= 1) return n;                // 基准情形
    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    return memo[n];                      // 记住结果,避免重复劳动
}

long long fibMemoWrapper(int n) {
    vector<long long> memo(n + 1, -1);    // 全部初始化为"未算过"
    return fibMemo(n, memo);
}

// fibMemoWrapper(90) 立刻返回
```

1.5.3 手写 vs 标准库

记忆化是"手写优先"的典型:C++ 标准库没有内置记忆化/缓存(不像 Python 的 `functools.lru_cache`),工程中要么手写数组/散列表,要么用第三方库。作为"手写 vs 标准库"的对照,看个更简单的例子——整数幂:

示例 1-8 手写整数幂 vs `std::pow`

```
// 手写:循环 n 次相乘,  $O(n)$  (更快的方法"快速幂"见第2章)
long long powHand(int a, int n) {
    long long r = 1;
    for (int i = 0; i < n; ++i) r *= a;
    return r;
}

// 标准库:std::pow 处理浮点 double,参数可以是任意实数
#include <cmath>

double x = std::pow(2.0, 10);            // 1024.0(浮点)
// 对照:手写版返回整数,只支持自然数次幂;std::pow 返回 double
// 结论:整数次幂用自写循环;涉及取模必须自写
// 记忆化同理:无标准库,手写数组是标准做法
```

调试技巧:斐波那契的三种对照验证

写记忆化版最容易犯"数组越界"与"忘初始化"两个错。三个验证技巧:① 先跑小 $n(n \leq 10)$,与手算值对照;② 朴素版与记忆化版互相对拍($n \leq 30$ 内朴素版也很快);③ 记忆化数组大小必须是 $n+1$ (存下标 $0..n$),初始化成"未算过"的哨兵值(如 -1)——斐波那契值恒非负, -1 不会与真实结果冲突。

1.5.4 迭代版:空间也省下来

还能更省:只保留前两个值滚动前进,时间 $O(n)$ 、空间 $O(1)$: `long long a = 0, b = 1;` 循环 `t = a + b; a = b; b = t;`。三版对比:朴素 $O(2^n)$ 时间、记忆化 $O(n)$ 时间 $O(n)$ 空间、迭代 $O(n)$ 时间 $O(1)$ 空间。记忆化解决"重复计算",迭代再省掉栈空间。

§1.6 递归的工程注意

1.6.1 调用栈与栈帧

§1.3.2 说过:每次函数调用压入一个**栈帧**(返回地址、参数、局部变量)。一个栈帧通常几十到几百字节,递归深度 10^5 时就要数 MB 内存。

1.6.2 栈溢出与段错误

C++ 程序的调用栈大小由系统/编译器决定,默认约 8MB。递归深度超过栈容量时,程序写入栈以外的地址,操作系统终止进程——表现为**段错误(Segmentation fault)**:没有任何报错信息,进程直接消失,也无法 try/catch。

示例 1-9 深层递归:段错误现场(不要直接跑)

```
// 递归求和:逻辑完全正确,但 n = 1000000 时崩溃
#include <iostream>
using namespace std;
long long sumRec(int n) {
    if (n == 0) return 0;           // 基准情形
    return n + sumRec(n - 1);       // 需要 n 层栈帧同时存活
}
int main() {
    // 深度 100 万 × 每帧几十字节 > 默认栈约 8MB
    cout << sumRec(1000000) << endl; // 段错误:栈空间耗尽,进程崩溃
    return 0;
}
```

误区 4:栈溢出无法预测

栈溢出在哪个深度爆发取决于每帧大小与栈大小:局部变量多(大数组、多参数)的帧更大,同样深度先爆。更隐蔽的是**测试规模太小测不出问题**: $n = 10^4$ 跑得好好的,生产环境 $n = 10^6$ 上线即崩。规则:线性递归的深度就是 n ,凡 n 可能超过约 10^5 的,一律不写线性递归。

1.6.3 如何避免深层递归

示例 1-10 改写为迭代:同样正确,百万规模轻松完成


```
// 用循环改写:不再占用调用栈,百万规模轻松完成
long long sumIter(int n) {
    long long s = 0;
    for (int i = 1; i <= n; ++i) s += i;
    return s;
}
// 识别"可改迭代"的标志:递归调用在函数末尾且结果直接返回
// (尾递归形态;编译器也可能自动优化,但不要依赖)
// 第3章会学到"显式栈"这一通用替代方案
```

避免深层递归的三招:① **循环改写**——尾递归形态(递归调用在函数末尾)最容易改写,空间从 $O(n)$ 变 $O(1)$;② **显式栈**——用自己维护的栈模拟调用栈(第3章),不受系统栈限制;③ **控制深度**——设计"减半"式递归(分治,第2章),深度只有 $\log_2 n$, 10^6 规模也不到 20 层。

§1.7 全书路线图

先看完整地图再出发。全书 12 章,一句话导航:

- **第2章 算法分析**:大 O 记号、增长阶排序、运行时间计算规则、最大子序列和四种算法
- **第3章 表、栈和队列**:三种最基础 ADT,数组与链式两种实现,栈的三个经典应用
- **第4章 树**:遍历、表达式树、二叉查找树 BST、AVL 平衡
- **第5章 散列**:散列函数、分离链接与开放定址两种冲突处理
- **第6章 优先队列(堆)**:二叉堆、堆排序、d-堆与左式堆
- **第7章 排序**:插入/希尔/堆/归并/快排/桶排,稳定性与复杂度全景
- **第8章 不相交集**:等价关系、按秩合并与路径压缩,接近 $O(1)$ 的并查集
- **第9章 图论算法**:DFS/BFS、拓扑排序、最短路径、最小生成树
- **第10章 算法设计技巧**:贪心、分治、动态规划、回溯、随机化
- **第11章 摊还分析**:聚集、会计、势能三种方法,解释"平均 $O(1)$ "的严格依据
- **第12章 高级数据结构**:伸展树、B 树、跳表等进阶结构

依赖关系:第2章是后续所有复杂度结论的依赖,务必学透。每章结束回看这张图,知道自己走到了哪里。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
求 a^n	朴素连乘(手写循环)	$O(n)$	$O(n)$
求 a^n	快速幂(第2章)	$O(\log n)$	$O(\log n)$
求 $\log_b X$	换底公式 + 库函数 $\log$	$O(1)$	$O(1)$
级数求和	算术级数(公式直接算)	$O(1)$	$O(1)$
级数求和	几何级数(公式直接算)	$O(1)$	$O(1)$
级数求和	调和级数(逐项累加)	$O(n)$	$O(n)$
整数取模	C/C++(商向 0 截断)	$O(1)$	$O(1)$
整数取模	Python(商向下取整)	$O(1)$	$O(1)$
阶乘 $n!$	递归(线性递归)	$O(n)$	$O(n)$
阶乘 $n!$	迭代循环	$O(n)$	$O(n)$
斐波那契 $F(n)$	朴素递归	$O(2^n)$	$O(2^n)$
斐波那契 $F(n)$	记忆化递归	$O(n)$	$O(n)$
斐波那契 $F(n)$	迭代滚动(两个变量)	$O(n)$	$O(n)$
递归调用	递推式 $T(n) = T(n-1) + O(1)$	$O(n)$	$O(n)$
递归调用	递推式 $T(n) = T(n-1) + T(n-2) + O(1)$	$O(2^n)$	$O(2^n)$
调用栈空间	C++ 递归深度 n (默认栈约 8MB)	$O(n)$ 空间	深度过大即段错误
递归深度上限	Python 默认 1000 层	1000 层	可调大,但过大有崩溃风险

最小必会清单

- 能默写指数四条规则与里程碑数值(2^{10} 、 2^{20} 、 2^{30})
- 能说出对数定义、换底公式与 $\log/\log_2/\ln$ 约定,解释"复杂度中底数不重要"
- 能默写算术、几何、调和三个级数公式并说出各自量级
- 能说出 C/C++ 与 Python 负数取模的区别($C++:-7\%3=-1$; $Python:-7\%3=2$),并写出"非负余数"修正惯用法
- 能说出"程序 = 数据结构 + 算法"并举出数据结构影响算法难度的例子
- 能默写递归四法则,并指出朴素斐波那契违反合成效益法则
- 能独立写出阶乘的递归版与迭代版,说出递推式 $T(n) = T(n-1) + O(1)$ 的解是 $O(n)$
- 能独立写出朴素与记忆化斐波那契,并解释复杂度为何是 $O(2^n)$ 与 $O(n)$
- 能用 `g++ -std=c++17` 编译运行一个含 `cin/cout`、`string`、`vector`、`class` 的完整程序
- 能解释栈帧、调用栈与深层递归为何段错误;说出三种避免深层递归的手段
- 能说出本书 12 章的大致内容与依赖关系

自测题

Q 1. 计算: 2^{10} 是多少? $\log_2 1024$ 是多少? $\sum_{i=1}^{100} i$ 是多少?

✓ 参考答案

$2^{10} = 1024$; $\log_2 1024 = 10$ ($2^{10} = 1024$); $\sum_{i=1}^{100} i = 100 \times 101 / 2 = 5050$ 。分别对应指数、对数、算术级数三公式。

Q 2. C++ 中 $-7 \% 3$ 与 $7 \% -3$ 分别等于多少?为什么?与 Python 有何不同?

✓ 参考答案

$-7 \% 3 = -1$, $7 \% -3 = 1$; C/C++ 除法向 0 截断, $-7/3$ 取商 -2, 余数 $-7 - (-2) \times 3 = -1$; $7/-3$ 取商 -2, 余数 $7 - (-2) \times (-3) = 1$ 。Python 向下取整, 结果分别为 2 与 -2; 同号一致, 异号不同。

Q 3. 递归四法则是什么?朴素递归斐波那契违反了哪一条?

✓ 参考答案

① 基准情形: 存在不需递归的直接答案; ② 不断推进: 每次调用都朝基准情形靠近; ③ 设计法则: 假设所有递归调用都正确, 只组装当前层; ④ 合成效益法则: 不在调用间重复求解同一实例。朴素斐波那契违反第④条, 复杂度爆炸到 $O(2^n)$ 。

Q 4. 递推式 $T(n) = T(n-1) + O(1)$ 的解是什么?给出直觉推导。

✓ 参考答案

解为 $O(n)$ 。展开: $T(n) = T(n-1) + c = T(n-2) + 2c = \dots = T(0) + n \cdot c$, 共 n 步, 每步常数代价。阶乘与递归求和都是这个递推式。

Q 5. 朴素递归斐波那契计算 $F(40)$ 大约需要多少次递归调用?记忆化后呢?

✓ 参考答案

朴素版调用次数 $= 2 \times F(41) - 1 \approx 3.3$ 亿次 (本机约 0.1 秒); 记忆化后每个 $F(k)$ 只算一次, 约 41 次调用, 瞬间完成。这就是合成效益法则的意义。

Q 6. 为什么递归 10 万层的 C++ 程序会段错误?栈帧在其中扮演什么角色?

✓ 参考答案

每次调用压入一个栈帧 (返回地址、参数、局部变量), 深度 10^5 层 $\times$ 每帧几十字节 $\approx$ 数 MB, 超过默认调用栈 (约 8MB) 即溢出, 写入非法地址被操作系统终止, 表现为无报错的段错误。规避: 循环改写、显式栈、log 深度的分治递归。

章末实验题:递归综合实验:汉诺塔与全排列

实验 1:汉诺塔与全排列(递归综合)

实验目标:用递归实现汉诺塔与全排列,验证 $2^n - 1$ 与 $n!$ 两个计数结论,并用真实计时感受指数级增长。

实验要求:

- 任务 1:汉诺塔——递归打印移动步骤、统计总步数,验证 $n = 1..10$ 步数恒等于 $2^n - 1$ (知识点:§1.3 递归四法则)
- 任务 2:递归生成全排列——交换 + 回溯,验证 $n = 3、4$ 的排列总数为 $3! = 6、4! = 24$ (知识点:§1.3 树形递归、回溯思想)
- 任务 3:对比 n 增大时朴素递归斐波那契的时间增长——记录 $n = 32/35/38$ 的耗时,验证调用数按 $\varphi^3 \approx 4.2$ 倍增长(知识点:§1.5 指数级复杂度)

实验环境:C++17,g++ 或 MSVC。编译运行: `g++ -std=c++17 lab1.cpp -o lab1 && ./lab1` (MSVC: `cl /EHsc /std:c++17 lab1.cpp`)。

第一步 写 hanoi:基准情形 $n == 1$ 直接搬;否则三步——上面 $n-1$ 个搬到中转柱、搬最大盘、再搬 $n-1$ 个。注意三次调用参数轮换顺序

第二步 用一个全局计数器统计步数;打印开关只对 $n = 3$ 打开, $n = 1..10$ 只输出"实际 vs 理论 $2^n - 1$ "对照行

第三步 permute:固定前 k 位;把 $a[k..]$ 中每个元素换到第 k 位;递归排列剩余部分;再换回还原现场(回溯三件套);基准情形 $k == n$ 时输出当前排列

第四步 时用 `chrono::steady_clock`,对 $n = 32/35/38$ 各测一次,打印耗时并观察"每加 3 乘几"的比值

✅ 参考答案(完整可运行程序,约 80 行,分两段)

lab1.cpp(第一段:任务 1 汉诺塔与任务 2 全排列)

```
// lab1.cpp — 递归综合实验:汉诺塔与全排列(第一段:任务1与任务2)
#include <iostream>
#include <vector>
#include <chrono>
using namespace std;

long long moveCnt = 0; // 汉诺塔移动总步数

// 任务 1:汉诺塔。printMove 为 true 时打印每步,否则只计数
void hanoi(int n, char from, char via, char to, bool printMove) {
    if (n == 1) { // 基准情形:只剩一个盘子,直接搬
        if (printMove) cout << " " << from << " -> " << to << endl;
        ++moveCnt;
        return;
    }
    hanoi(n - 1, from, to, via, printMove); // 1. 上面 n-1 个搬到中转柱
    if (printMove) cout << " " << from << " -> " << to << endl;
    ++moveCnt; // 2. 最大盘子搬到目标柱
    hanoi(n - 1, via, from, to, printMove); // 3. n-1 个从中转柱搬到目标
}

// 任务 2:递归生成全排列(回溯)。固定前 k 位,把 a[k..] 逐个换到 a[k]
void permute(vector<int>& a, int k, long long& cnt) {
    if (k == (int)a.size()) { // 基准情形:最后一位也固定了
        for (size_t i = 0; i < a.size(); ++i)
            cout << a[i] << (i + 1 == a.size() ? "\n" : " ");
        ++cnt;
        return;
    }
    for (int i = k; i < (int)a.size(); ++i) {
        swap(a[k], a[i]); // 把 a[i] 换到第 k 位
        permute(a, k + 1, cnt); // 递归排列剩余部分
        swap(a[k], a[i]); // 回溯:换回来,还原现场
    }
}
```

lab1.cpp(第二段:任务 3 计时与主程序)

```

// 任务 3:朴素递归斐波那契(用于计时)
long long fibNaive(int n) {
    if (n <= 1) return n;
    return fibNaive(n - 1) + fibNaive(n - 2);
}

int main() {
    cout << "==== 任务 1:汉诺塔(n = 3)====\n";
    hanoi(3, 'A', 'B', 'C', true);
    cout << "移动步数: " << moveCnt << "(理论 2^3 - 1 = 7)\n";
    moveCnt = 0; // 清零,再进入批量验证
    for (int n = 1; n <= 10; ++n) { // 验证 2^n - 1
        hanoi(n, 'A', 'B', 'C', false);
        long long expect = (1LL << n) - 1;
        cout << "n = " << n << ": 实际 " << moveCnt
            << ",理论 " << expect
            << (moveCnt == expect ? " 一致" : " 不一致!") << endl;
        moveCnt = 0;
    }

    cout << "\n==== 任务 2:全排列(n = 3)====\n";
    vector<int> a = {1, 2, 3};
    long long permCnt = 0;
    permute(a, 0, permCnt);
    cout << "排列总数: " << permCnt << "(理论 3! = 6)\n";
    permCnt = 0;
    vector<int> b = {1, 2, 3, 4}; // 再验证 4! = 24
    permute(b, 0, permCnt);
    cout << "n = 4 排列总数: " << permCnt << "(理论 4! = 24)\n";

    cout << "\n==== 任务 3:朴素斐波那契计时 ==== \n";
    for (int n : {32, 35, 38}) { // n 每次 +3,调用数约 x4.2
        auto t0 = chrono::steady_clock::now();
        long long f = fibNaive(n);
        double ms = chrono::duration<double, milli>
            (chrono::steady_clock::now() - t0).count();
        cout << "fib(" << n << ") = " << f << ",耗时 " << ms << " ms\n";
    }
    cout << "观察: n 每加 3,耗时约乘 4~5(调用数 2*F(n+1) 的比值约 phi^3 = 4.2)\n";
    return 0;
}

```

运行命令: g++ -std=c++17 lab1.cpp -o lab1 && ./lab1 。示例输出(第一段,n = 3 的移动过程与 1..10 验证):

===== 任务 1:汉诺塔(n = 3)=====

A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C

移动步数: 7(理论 $2^3 - 1 = 7$)

n = 1: 实际 1,理论 1 一致
n = 2: 实际 3,理论 3 一致
n = 3: 实际 7,理论 7 一致
n = 4: 实际 15,理论 15 一致
n = 5: 实际 31,理论 31 一致
n = 6: 实际 63,理论 63 一致
n = 7: 实际 127,理论 127 一致
n = 8: 实际 255,理论 255 一致
n = 9: 实际 511,理论 511 一致
n = 10: 实际 1023,理论 1023 一致

示例输出(第二段,任务 2 与任务 3;任务 3 的耗时因机器而异,比值规律一致):

===== 任务 2:全排列(n = 3)=====

1 2 3
1 3 2
2 1 3
2 3 1
3 2 1
3 1 2

排列总数: 6(理论 $3! = 6$)

n = 4 排列总数: 24(理论 $4! = 24$)(n = 4 的 24 行清单略)

===== 任务 3:朴素斐波那契计时 =====

fib(32) = 2178309,耗时 1.67 ms

fib(35) = 9227465,耗时 6.61 ms

fib(38) = 39088169,耗时 30.7 ms

观察: n 每加 3,耗时约乘 4~5(调用数 $2 * F(n+1)$ 的比值约 $\phi^3 = 4.2$)

设计说明:① 三个任务共用"基准情形 + 不断推进"的递归骨架:汉诺塔基准是 $n == 1$,全排列是"最后一位也固定",斐波那契是 $n \leq 1$ ——同一思想的三次演练;② 汉诺塔步数 $2^n - 1$ 正是第2章递推式 $T(n) = 2T(n-1) + O(1)$ 的原型;③ 全排列的"交换 → 递归 → 换回"是回溯法(第10章)的最小模板;④ 任务 3 用真实计时把指数级增长变成肉眼可见(实测 n 每加 3 耗时约乘 4~5,调用数比值 $\phi^3 \approx 4.2$);⑤ 打印与计数分离(printMove 开关)让"演示"与"批量验证"互不干扰。

下一章预告:第2章 算法分析——为大 O 记号建立严格定义,学会对循环与递归做复杂度计算,并把最大子序列和从 $O(n^3)$ 优化到 $O(n)$ 。本章的递推式、对数与指数性质、级数公式,正是第2章推导的数学地基。

第2章 算法分析

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解为什么"程序能跑"远远不够——**复杂度**决定程序在数据变大时能不能跑完
- 掌握 O 、 Ω 、 Θ 、 o 四种记号的准确定义与直觉,能说出它们的区别
- 能背诵增长阶排序:常数 < 对数 < 线性 < 线性对数 < 平方 < 立方 < 指数
- 会计算顺序/循环/递归代码的运行时间,能把循环翻译成求和记号 Σ
- 亲手实现**最大子序列和**问题的四种算法,体会 $O(n^3)$ 到 $O(n)$ 的天壤之别
- 能说明二分查找、快速幂、欧几里得算法为什么都是 $O(\log n)$
- 学完能独立用 C++ `chrono` (C 中对应 `clock()`) 实测验证自己的复杂度分析

💡 从 C 到 C++:本章主题如何迁移

本章是全书唯一"几乎不写新数据结构"的一章,但它是**全书的通用语言**:第 3 章起,每章都以"这个操作 $O(\text{多少})$ "作为结论。算法分析的直觉一句话:复杂度是数据规模变大时,运行时间的增长方式——不是秒数,不是毫秒,而是"规模翻倍,时间大约乘以几"。

在 C 里,你也许用过 `clock()` 计时:包含 `time.h`,前后各取一次 `clock()`,除以 `CLOCKS_PER_SEC`。这三个痛点:精度只有毫秒级、测的是 CPU 时间而非墙上时间、每次计时都要手写四行样板。C++ 的 `<chrono>` 库用 `steady_clock` 给出高精度单调时钟,一行 `duration` 模板化转换直接得到毫秒/微秒,还内置了类型检查——C 里"把秒和毫秒搞混"这类 bug 在 `chrono` 里根本写不出来。

Python 的 `time.perf_counter()` 与 C++ 的 `steady_clock` 思想相同:单调、高精度、不受系统改时间影响。本章 C++ 版与 Python 版的所有代码结构完全一致,只是计时函数不同——读完本章你会发现,算法分析是数学,语言只是载体。

本章路线:先建立记号体系(\$2.1-\$2.3),再学计算规则(\$2.4),然后在一个真实问题上亲手把 $O(n^3)$ 一路优化到 $O(n)$ (\$2.5),最后用计时实验验证理论(\$2.6-\$2.7)。

📦 前置知识自查

数学前置:指数与对数的基本性质(第1章)、求和记号 Σ 、递归四法则与递推式入门(第1章)。

C 语言前置:单层/嵌套 `for` 循环、函数定义与调用、数组下标遍历; `clock()` 计时(可选,会更好)。

依赖的前面章节:第1章绪论(递归思想、 $\log$ 与指数的运算性质)。

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 §2.1–§2.3, 背下增长阶排序与常见函数值表
2. **第二阶段(1.5 小时)**:读 §2.4–§2.5, 亲手推导最大子序列和四种算法的复杂度, 并上机运行验证输出正确
3. **第三阶段(1.5 小时)**:读 §2.6–§2.7, 运行计时程序, 观察“n 翻倍、时间乘几”与理论预测是否吻合
4. **第四阶段(实验, 1 小时)**:完成章末实验: 四种算法的增长阶实测报告
5. **验收标准**:能随口说出“二分查找 $O(\log n)$ ”“快排平均 $O(n \log n)$ ”并解释理由

本章知识导图

- §2.1 为什么要分析算法:从“能跑”到“跑得完”(规模、计时工具、C 的 `clock()` 与 `chrono` 对照)
- §2.2 大 O 、 Ω 、 Θ 、小 o 定义、直觉、常见误用
- §2.3 增长阶排序与常见函数:七个等级、数值表
- §2.4 运行时间计算规则:顺序/循环/递归、求和记号
- §2.5 最大子序列和:四种算法, $O(n^3) \rightarrow O(n)$ 的递进
- §2.6 对数级复杂度的三种来源:二分查找、快速幂、欧几里得算法
- §2.7 实体验证:计时方法、n 翻倍实验、常数因子的意义

§2.1 为什么要分析算法

2.1.1 规模决定生死

同一个问题, 算法不同, 命运不同。假设处理一个元素耗时 1 微秒, 那么:

问题规模 n	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
10^3	≈0.01 秒	1 秒	16.7 分钟	约 10^{286} 年
10^4	≈0.13 秒	100 秒	11.6 天	超出宇宙年龄
10^5	≈1.7 秒	2.8 小时	31.7 年	—
10^6	≈20 秒	11.6 天	3.2 万年	—

结论: 指数算法在任何现实规模上都不存在; 平方算法在十万级数据上就要等几小时。这就是为什么必须先学会“估计算法代价”——否则优化方向可能完全错误(去优化一个 $O(n)$ 的循环, 却对 $O(n^2)$ 的嵌套循环视而不见)。

2.1.2 从 C 的 `clock()` 到 C++ 的 `chrono`

C 的计时套路你也许写过:

C 的写法	C++ 的写法
<code>#include <time.h></code>	<code>#include <chrono></code>
<code>clock_t t0 = clock();</code>	<code>auto t0 = steady_clock::now();</code>
<code>double s = (double)(clock()-t0)/CLOCKS_PER_SEC;</code>	<code>auto ms = duration<double,milli>(steady_clock::now()-t0).count();</code>
精度毫秒级,测 CPU 时间	精度微秒/纳秒级,测墙上时间,类型安全

示例 2-1 C++ 计时框架(本章所有实验都用它)

```
#include <iostream>
#include <chrono>
using namespace std;
using Clock = chrono::steady_clock;    // 单调时钟:不受系统改时间影响

int main() {
    auto t0 = Clock::now();
    // ---- 被测代码 ----
    long long s = 0;
    for (int i = 0; i < 1000000; ++i) s += i;
    // -----
    double ms = chrono::duration<double, milli>(Clock::now() - t0).count();
    cout << "耗时: " << ms << " ms, 结果 = " << s << endl;
    return 0;
}
```

★ 重点:计时不是分析的全部

计时是验证分析的工具,不是分析本身。同一程序换个机器、换个编译器、换个负载,时间可能差几倍——但“n 翻倍、时间约乘 4”这个比值规律几乎不变。复杂度分析研究的正是这个机器无关的规律。

§2.2 大 O、Ω、Θ、小 o

2.2.1 大 O:上界

定义(实用版): $T(n) = O(f(n))$ 表示存在常数 c 与 n_0 ,使得对所有 $n \geq n_0$,都有 $T(n) \leq c \cdot f(n)$ 。直白地说:当 n 足够大时, $T(n)$ 的增长速度不超过 $f(n)$ 的某个常数倍。例如 $T(n) = 3n^2 + 10n + 7$,取 $c = 4$ 、 n_0 足够大即可证明 $T(n) = O(n^2)$ ——因为 $10n + 7$ 相对 $3n^2$ 微不足道。

所以大 O 会“吞掉”低阶项与常数: $O(3n^2+10n+7) = O(n^2)$, $O(2n) = O(n)$ 。这也是它的局限:两个同为 $O(n)$ 的算法可能实际相差 10 倍。

2.2.2 Ω 与 Θ:下界与紧界

$\Omega(f(n))$ 是对偶记号: $T(n) \geq c \cdot f(n)$ 对所有足够大的 n 成立,表示“增长不低于 $f(n)$ ”。而 $\Theta(f(n))$ 表示“恰好同阶”:既 $O(f(n))$ 又 $\Omega(f(n))$ 。工程上最常用的是大 O (上界保证“最坏不会更差”)与 Θ (精确刻画);当说“这个算法是 $\Theta(n \log n)$ ”时,意思是上下界都压紧了。

2.2.3 小 o 与三种常见误用

小 $o(f(n))$ 表示"增长速度严格低于 $f(n)$ "(大 O 允许相等,小 o 不允许): $n = o(n^2)$ 成立,但 $n \neq o(n)$ 。它常用于比较两个上界谁更"松"。

记号说明(本章约定,全书沿用)

正文中的 $O(n)$ 读作"大 O n "; n^2 表示 n 的平方; $\log_2 n$ 表示以 2 为底的对数,不写底数时 $\log n$ 默认以 2 为底(常数倍差异被大 O 吞掉,所以底数不重要,但推导过程要写清楚); $\ln$ 表示自然对数。求和写成 $\sum_{i=1}^n i = n(n+1)/2$ 。代码里一律使用语言语法(`%`、`>>`、`pow`),不使用数学记号。

误区 1:把常数当成增长阶

写 $O(2n)$ 、 $O(100n \log n)$ 、 $O(n+100)$ 都是错误的:大 O 定义明确吞掉常数,应写 $O(n)$ 、 $O(n \log n)$ 、 $O(n)$ 。"常数不重要"仅在分析增长阶时成立;实际工程选型时,两个 $O(n)$ 算法的常数差距可能决定生死。

误区 2:混淆最坏情形与记号本身

"快排是 $O(n^2)$ 的"——这句话必须补全:快排最坏情形是 $O(n^2)$,平均情形是 $O(n \log n)$ 。大 O 描述的是某个约定情形(最坏/平均/最好)下的增长阶,不是算法本身的属性。今后每章的复杂度速查表都会区分"平均"与"最坏"两列。

§2.3 增长阶排序与常见函数

2.3.1 七个等级

- $O(1)$ 常数:数组按下标访问、散列查找(平均)
- $O(\log n)$ 对数:二分查找—— n 翻倍只多一步
- $O(n)$ 线性:一趟遍历、线性搜索
- $O(n \log n)$ 线性对数:归并排序、快排(平均)、堆排序
- $O(n^2)$ 平方:二重循环、插入/冒泡排序、朴素矩阵乘法
- $O(n^3)$ 立方:三重循环、Floyd 全源最短路
- $O(2^n)$ 指数:穷举、朴素递归斐波那契——现实规模下不可用

2.3.2 数值表:让直觉落地

$f(n)$	$n=10$	$n=100$	$n=10^3$	$n=10^4$	$n=10^5$	$n=10^6$
$\log_2 n$	3.3	6.6	10	13	17	20
n	10	100	10^3	10^4	10^5	10^6
$n \log_2 n$	33	664	10^4	1.3×10^5	1.7×10^6	2×10^7
n^2	100	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	1024	约 10^{30}	约 10^{301}	约 10^{3010}	约 10^{30103}	约 10^{301030}

读这张表的要领:从 $n=10^3$ 到 10^6 , 规模扩大 1000 倍—— n 增长 1000 倍, n^2 增长 100 万倍, 2^n 则直接冲出宇宙。同时注意 $\log_2 n$ 一行:规模增长 1000 倍,它只从 10 长到 20。对数函数是“几乎不增长”的函数,凡是对数级的算法都值得信赖。

⚠ 误区 3:把 $2n$ 与 n^2 混为一谈

2^n 与 n^2 只差一个“位置”,命运完全不同: $n=64$ 时, $n^2=4096$, 而 $2^n \approx 1.8 \times 10^{19}$ 。判断口诀:底数是 n 的多项式,指数是 n 的才是指数级。 1.1^n 也是指数级(底数略大于 1 仍是灾难), n^{100} 再大也只是多项式。

§2.4 运行时间计算规则

2.4.1 三条基本规则

- **顺序相加**:两个语句依次执行,总时间取两者之和,大 O 取最大者: $O(n) + O(n^2) = O(n^2)$
- **嵌套相乘**:循环体内嵌套循环,总时间相乘
- **分支取最大**:if/else 取两个分支中较慢的一个

2.4.2 循环与求和记号

循环本质上是一个求和:第 i 次迭代的工作量 $w(i)$,总工作量 $= \sum_i w(i)$ 。例:

示例 2-2 嵌套循环的求和推导

```
// 推导:内层循环执行  $n - i$  次
// 总次数  $= \sum_{i=0}^{n-1} (n - i) = n + (n-1) + \dots + 1 = n(n+1)/2$ 
// 所以是  $O(n^2)$ ,而不是“感觉上的  $n^2/2$ ”—常数  $1/2$  被大  $O$  吞掉
// 对照:C 的写法 for (i = 0; i < n; i++) 完全一致,结论不变
long long cnt = 0;
for (int i = 0; i < n; ++i)
    for (int j = i; j < n; ++j)
        ++cnt;
```

三个最常用的求和公式(本章及全书反复使用):

- $\sum_{i=1}^n i = n(n+1)/2$ (算术级数, $O(n^2)$)
- $\sum_{i=0}^n 2^i = 2^{n+1} - 1$ (几何级数, $O(2^n)$)
- $\sum_{i=1}^n 1/i \approx \ln n$ (调和级数, $O(\log n)$)

2.4.3 递归:递推式入门

递归代码的复杂度由**递推式**描述: $T(n) = 2T(n/2) + n$ 表示“把规模 n 的问题分成两个 $n/2$ 的子问题,再花 $O(n)$ 合并”。逐层展开:第 1 层代价 n ,第 2 层 $2 \cdot (n/2) = n$,第 3 层 $4 \cdot (n/4) = n \dots$ 共 $\log_2 n$ 层,每层 n ,总计 $O(n \log n)$ 。§2.5 的算法三正是这个递推式,§2.6 的三个对数算法则是 $T(n) = T(n/2) + O(1)$ 的特例——每层只进一个分支,共 $\log_2 n$ 层,每层常数代价。

★ 重点:两种最经典的递推式

$T(n) = T(n/2) + O(1) \rightarrow O(\log n)$ (二分查找); $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ (归并/分治)。全书的对数与线性对数复杂度,九成来自这两个式子。

§2.5 最大子序列和:四种算法的递进

2.5.1 问题描述

给定整数序列 $a_1, a_2, \dots, a_n$ (可含负数), 求其**连续子序列的最大和**; 全为负数时约定答案为 0 (空子序列)。例如 $[-2, 11, -4, 13, -5, -2]$ 的最大子序列是 $[11, -4, 13]$, 和为 20。这是 Weiss 教材的经典例题: 同一个问题, 四种算法, 复杂度相差 100 万倍。

2.5.2 算法一: $O(n^3)$ 暴力枚举

最直接的思路: 枚举所有起点 i 与终点 j , 再累加 $a[i..j]$ 。三重循环, 外层 n 种起点, 中层 $n-i$ 种终点, 内层再花 $O(n)$ 求和, 总代价 $\sum \sum (j-i+1) \approx n^3/6$, 即 $O(n^3)$ 。

示例 2-3 算法一: 三重循环, $O(n^3)$

```
// 需要 #include <vector>
int maxSubSum1(const vector<int>& a) {
    int maxSum = 0;
    for (size_t i = 0; i < a.size(); ++i)    // 起点
        for (size_t j = i; j < a.size(); ++j) // 终点
        {
            int thisSum = 0;
            for (size_t k = i; k <= j; ++k)    // 累加 a[i..j]
                thisSum += a[k];
            if (thisSum > maxSum)
                maxSum = thisSum;
        }
    return maxSum;
}
```

2.5.3 算法二: $O(n^2)$ 复用中间和

观察: 固定起点 i , 当终点从 j 推进到 $j+1$ 时, 新和 = 旧和 + $a[j+1]$, 不需要重算整个区间。删掉内层循环, 复杂度降到 $\sum_{i=1}^n 1 = n(n+1)/2 = O(n^2)$ 。

示例 2-4 算法二: 两重循环, $O(n^2)$

```
int maxSubSum2(const vector<int>& a) {
    int maxSum = 0;
    for (size_t i = 0; i < a.size(); ++i) {
        int thisSum = 0;
        for (size_t j = i; j < a.size(); ++j) {
            thisSum += a[j];    // 增量更新, 不再重算
            if (thisSum > maxSum)
                maxSum = thisSum;
        }
    }
    return maxSum;
}
```

2.5.4 算法三: $O(n \log n)$ 分治

分治三步:把数组分成左右两半,递归求左半最大和与右半最大和,再求**跨中界的最大和**(从中界向左延伸的最大后缀和 + 向右延伸的最大前缀和),三者取最大。递推式正是 $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ 。

示例 2-5 算法三:分治, $O(n \log n)$

```
// 需要 #include <algorithm> (std::max)
int maxSumRec(const vector<int>& a, int left, int right) {
    if (left == right) // 基准情形:单个元素
        return a[left] > 0 ? a[left] : 0;

    int center = (left + right) / 2;
    int maxLeft = maxSumRec(a, left, center); // 左半
    int maxRight = maxSumRec(a, center + 1, right); // 右半

    int maxLeftBorder = 0, leftBorder = 0; // 跨中界:向左延伸
    for (int i = center; i >= left; --i) {
        leftBorder += a[i];
        if (leftBorder > maxLeftBorder)
            maxLeftBorder = leftBorder;
    }
    int maxRightBorder = 0, rightBorder = 0; // 向右延伸
    for (int i = center + 1; i <= right; ++i) {
        rightBorder += a[i];
        if (rightBorder > maxRightBorder)
            maxRightBorder = rightBorder;
    }
    return max(max(maxLeft, maxRight),
               maxLeftBorder + maxRightBorder);
}

int maxSubSum3(const vector<int>& a) {
    return maxSumRec(a, 0, int(a.size()) - 1);
}
```

2.5.5 算法四: $O(n)$ 在线算法

关键观察: **和为负的前缀不可能是最优解的起点**——负前缀只会拖累后面。于是扫描一遍:累加当前和 thisSum,一旦 thisSum 变成负数就清零重来;途中记录最大值。

示例 2-6 算法四:在线算法, $O(n)$

```
int maxSubSum4(const vector<int>& a) {
    int maxSum = 0, thisSum = 0;
    for (int x : a) { // C++ 范围 for, 等价于按序遍历
        thisSum += x;
        if (thisSum > maxSum)
            maxSum = thisSum;
        else if (thisSum < 0)
            thisSum = 0; // 负前缀弃之,从下一个元素重新开始
    }
    return maxSum;
}
```

📄 例题 2-1 手工走一遍算法四

输入 $[-2, 11, -4, 13, -5, -2]$: thisSum 依次为 $-2 \rightarrow 0$ (清零)、11、7、20、15、13; maxSum 记录峰值为 20。答案 20, 与算法一一致。**注意:** 清零动作丢弃的是"负前缀"而不是负数本身—— $a[i]$ 单独为负但前缀仍正时不会清零。

★ 重点: 四种算法问题对照

算法	思想	复杂度	$n=10^6$ 时(1 微秒/步)
一	暴力枚举	$O(n^3)$	约 317 年
二	增量更新	$O(n^2)$	约 11.6 天
三	分治	$O(n \log n)$	约 20 秒
四	在线扫描	$O(n)$	约 1 秒

§2.6 对数级复杂度的三种来源

2.6.1 二分查找

在有序数组中查找 x : 每次把候选区间减半。区间长度 $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$, 共 $\log_2 n$ 步, 即递推式 $T(n) = T(n/2) + O(1)$ 。

示例 2-7 二分查找, $O(\log n)$

```
// 在升序数组 a 中查找 x, 返回下标; 找不到返回 -1
int binarySearch(const vector<int>& a, int x) {
    int low = 0, high = int(a.size()) - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // 防溢出的写法, 优于 (low+high)/2
        if (a[mid] < x) low = mid + 1;
        else if (a[mid] > x) high = mid - 1;
        else return mid;
    }
    return -1;
}
```

示例 2-8 标准库写法: binary_search 与 lower_bound

```
#include <algorithm>
vector<int> a = {1, 3, 5, 7, 9};
bool hit = binary_search(a.begin(), a.end(), 5); // true
// lower_bound: 第一个 >= x 的位置(不存在则返回 end())
auto it = lower_bound(a.begin(), a.end(), 6); // 指向 7
int pos = int(it - a.begin()); // 3
// upper_bound: 第一个 > x 的位置, 两者之差即"等于 x 的元素个数"
int cnt = int(upper_bound(a.begin(), a.end(), 5) - it);
```

C 的手写二分查找	C++ 的标准库
<pre>int bin(int a[], int n, int x)</pre> 手写约 12 行	<pre>binary_search / lower_bound / upper_bound</pre>
边界条件 (<code><=</code> 还是 <code><</code>) 极易写错成死循环	边界由标准库保证, 写错编译期就拦下
数组要单独传长度 <code>n</code>	迭代器区间 <code>[begin, end)</code> 自包含长度

手写 vs 标准库:学习时手写一遍理解"减半"骨架; 工程中直接用 `binary_search` (判断存在) 与 `lower_bound` (求位置) —— 后者返回"第一个不小于 `x` 的位置", 是插入有序序列时的正确姿势。Python 版对应 `bisect` 模块, 语义完全相同。

2.6.2 快速幂

计算 $a^n \bmod m$: 朴素做法乘 n 次是 $O(n)$; 利用 $a^n = (a^2)^{n/2}$ 每次把指数减半, $O(\log n)$ 。这是 C 里你可能用循环写过的算法, 与二分查找共享"减半"骨架。

示例 2-8 快速幂, $O(\log n)$

```
const long long MOD = 1000000007LL;
// 计算 (a^n) % MOD; n >= 0
long long powMod(long long a, long long n) {
    long long result = 1;
    while (n > 0) {
        if (n & 1)                // 当前二进制位为 1 则乘入
            result = result * a % MOD;
        a = a * a % MOD;          // a 平方: 对应指数减半
        n >>= 1;                  // n 右移一位 = n/2
    }
    return result;
}
```

2.6.3 欧几里得算法

求最大公因数 `gcd(a, b)`: 迭代执行 `(a, b) → (b, a % b)` 直到 `b = 0`。结论: 每两步至少把较小的数减半(证明略), 所以迭代次数 $O(\log n)$ 。

示例 2-9 欧几里得算法, $O(\log n)$

```
int gcd(int a, int b) {
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }
    return a;
}
```

★ 重点: log 从哪来?

三个算法的共同骨架: 每一步把"问题规模"至少减半。二分查找减半区间, 快速幂减半指数, 欧几里得每两步减半数。凡是见到"减半""倍增"字样的算法, 先猜 $O(\log n)$ 再验证, 十有八九正确。

§2.7 实测验证:让理论落地

2.7.1 计时方法要点

- n 要足够大:太小则计时噪声淹没规律(§2.7.2 用 $n \geq 1000$)
- 重复取平均或取中位数,避免一次性的调度抖动
- 开优化编译(C++ 用 `/O2` 或 `-O2`),否则测得的是调试版速度
- **防止死代码消除**:编译器可能把"结果没被使用"的整段计算删掉——务必把结果累加进一个全局变量并输出

C 的计时要点	C++ 的计时要点
<code>clock()</code> 测 CPU 时间,受多线程/系统调度影响	<code>steady_clock</code> 测墙上时间,单调不回拨
除以 <code>CLOCKS_PER_SEC</code> 得到秒,容易忘记	<code>duration<double, milli></code> 直接出毫秒,类型安全
手动 <code>volatile</code> 防优化,易漏	同样要把结果累加输出;开 <code>/O2</code> 后验证

2.7.2 n 翻倍实验(C++ 实测参考值)

对随机数组运行四种算法,每次 n 翻倍,观察时间比值:

n	算法一	算法二	算法三	算法四
1000	412 ms	0.42 ms	0.05 ms	0.02 ms
2000	3310 ms	1.68 ms	0.11 ms	0.04 ms
4000	≈ 26 s(推算)	6.71 ms	0.24 ms	0.08 ms
8000	≈ 210 s(推算)	26.9 ms	0.51 ms	0.17 ms

读出规律: n 翻倍,算法一时间 $\times 8$ (吻合 n^3),算法二 $\times 4$ (吻合 n^2),算法三约 $\times 2.1$,算法四约 $\times 2.0$ (吻合 $n \log n$ 与 n)。**比值比绝对值可靠**——你的机器数值不同,但比值规律必然一致。Python 版做同样实验时,绝对时间约慢 30–100 倍(解释执行 + 每次运算都是对象操作),但**比值规律完全相同**:增长阶是语言无关的。

工程建议:常数因子的真实意义

大 O 相同不代表同样快:同为 $O(n \log n)$,快排的常数通常只有堆排的一半;同为 $O(n)$,遍历 `vector` 比遍历链表快数倍(缓存友好,第3章细讲)。工程选型时:先用大 O 淘汰指数/高次算法,再用常数因子与实测在候选者中择优。只背大 O 不看常数,和只看常数不懂大 O ,都是半桶水。

误区 4:小规模下拿 O 记号赌博

$n = 20$ 时, $O(n^2)$ 的插入排序往往比 $O(n \log n)$ 的快排更快——常数小。大 O 的结论只在" n 足够大"时成立(定义里的 n_0 正是为此)。工程上排序库普遍采用"大区间快排 + 小区间插入排序"的混合策略(第7章),就是这条原则的产物。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
按值查找	无序数组	$O(n)$	$O(n)$
二分查找	有序数组	$O(\log n)$	$O(\log n)$
求 a^n	朴素连乘	$O(n)$	$O(n)$
求 a^n	快速幂	$O(\log n)$	$O(\log n)$
最大公因数	欧几里得算法	$O(\log n)$	$O(\log n)$
最大子序列和	算法一(暴力)	$O(n^3)$	$O(n^3)$
最大子序列和	算法二(增量)	$O(n^2)$	$O(n^2)$
最大子序列和	算法三(分治)	$O(n \log n)$	$O(n \log n)$
最大子序列和	算法四(在线)	$O(n)$	$O(n)$
循环计数	单层循环	$O(n)$	$O(n)$
循环计数	双重循环	$O(n^2)$	$O(n^2)$
递归	$T(n) = T(n/2) + O(1)$	$O(\log n)$	$O(\log n)$
递归	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	$O(n \log n)$
增长阶	常数 / 对数 / 线性 / 线性对数 / 平方 / 立方 / 指数	见 §2.3.1 排序	
增长阶排序	常数 $< \log n < n < n \log n < n^2 < n^3 < 2^n$	全书唯一需要背诵的排序	

最小必会清单

- 能独立说出大 O 、 Ω 、 Θ 、小 o 的定义与区别,并各举一例
- 能背诵增长阶排序,并给每个等级配一个真实算法例子
- 能独立推导单层循环、嵌套循环、if/else 的复杂度
- 能独立写出四种最大子序列和算法,并说出各自复杂度与推导
- 能独立写出二分查找、快速幂、gcd 的代码,并解释为什么是 $O(\log n)$
- 能写出 $T(n) = T(n/2) + O(1)$ 与 $T(n) = 2T(n/2) + O(n)$ 的解
- 能解释"最坏情形"与"平均情形"的区别,并正确表述快排的复杂度
- 能独立用计时工具做 n 翻倍实验,判断实验比值与理论增长阶是否吻合

自测题

Q 1. $O(2n)$ 与 $O(n)$ 是否相等?为什么?

✅ 参考答案

相等。大 O 定义允许任意常数倍:取 $c = 2$ 即有 $2n \leq 2 \cdot n, 2n = O(n)$ 。反过来 $n \leq 1 \cdot 2n$, 所以两者互为对方的 O , 增长阶相同。写 $O(2n)$ 是不规范的。

Q 2. 某算法运行时间 $T(n) = 3n^2 + 10n + 7$, 它的 O 、 Ω 、 Θ 分别可以写什么?

✓ 参考答案

$O(n^2)$ (也允许 $O(n^3)$ 等更松的上界,但不规范); $\Omega(n^2)$ (也允许 $\Omega(n)$ 等更松的下界); $\Theta(n^2)$ 是唯一"紧"的写法。

Q 3. 为什么说算法三(分治)比算法二(增量)快?写出算法三的递推式并解释每层代价。

✓ 参考答案

$T(n) = 2T(n/2) + n$:把数组分为两半递归($2T(n/2)$),再花 $O(n)$ 求跨中界最大和。展开后第 k 层有 2^k 个规模 $n/2^k$ 的子问题,每层总代价都是 n ,共 $\log_2 n$ 层,总计 $n \log_2 n$ 。 $O(n \log n)$ 在 $n = 10^6$ 时约 2×10^7 步,而 $O(n^2)$ 是 10^{12} 步,差 5 万倍。

Q 4. 二分查找每次把区间减半,为什么复杂度是 $O(\log n)$ 而不是 $O(n/2)$?

✓ 参考答案

区间长度依次为 $n, n/2, n/4, \dots, 1$,问"减半多少次到 1"——这是对数: $2^k = n$,即 $k = \log_2 n$ 次。 $n/2$ 是"一次减半后的长度",不是步数;把长度当步数是常见错误。 10^6 个元素只需约 20 次比较。

Q 5. $n = 1000$ 时, $O(n^2)$ 与 $O(n \log n)$ 的步数约相差多少倍? $n = 10^6$ 呢?

✓ 参考答案

$n = 1000: 10^6 / (1000 \times 10) = 100$ 倍; $n = 10^6: 10^{12} / (10^6 \times 20) = 5 \times 10^4$ 倍。规模越大差距越大——这就是为什么"复杂度差的算法靠硬件换代救不回来"。

Q 6. Python 程序比 C++ 慢约 50 倍,那么 $O(n)$ 的 Python 算法与 $O(n \log n)$ 的 C++ 算法,数据规模多大时 Python 反而更快?

✓ 参考答案

比较 $50n$ 与 $n \log_2 n$:当 $\log_2 n > 50$,即 $n > 2^{50} \approx 10^{15}$ 时 Python 更快——现实规模基本达不到,所以"语言换效率"在增长阶面前通常无效。反过来: $O(n^2)$ 的 C++ 在大 n 下照样被 $O(n \log n)$ 的 Python 反超。

章末实验题:四种算法的增长阶实测报告

实验 2:验证最大子序列和四种算法的增长阶

实验目标:实现四种最大子序列和算法,对倍增规模计时,用实测比值验证 $O(n^3)/O(n^2)/O(n \log n)/O(n)$ 的增长阶预测。

实验要求:

- 任务 1:实现 maxSubSum1..4 四个函数(知识点:\$2.5 四种算法)
- 任务 2:用 `steady_clock` 计时,输出 $n = 1000/2000/4000/8000$ 的时间表(知识点:\$2.1 计时工具)
- 任务 3:算法一在 $n \geq 4000$ 时跳过并标注"(推算)",避免无谓等待(知识点:\$2.3 指数/立方增长不可用)
- 任务 4:输出" n 翻倍时各算法时间比值",与理论值($\times 8/\times 4/\times 2/\times 2$)对比并写一段结论(知识点:\$2.7 实测验证)

实验环境:C++17,MSVC 或 g++。编译:`cl /EHsc /std:c++17 lab2.cpp` 或 `g++ -std=c++17 lab2.cpp -o lab2`;运行 `lab2`。

第一步 写四个算法函数,用 \$2.5 的样例数组 `[-2, 11, -4, 13, -5, -2]` 验证四个函数都输出 20,再写计时主程序

第二步 随机数用 `mt19937 + uniform_int_distribution<int>(-1000, 1000)`,固定种子 12345 保证可复现

第三步 时函数接收函数指针,复用同一份计时代码;注意把返回值累加并最终输出,防止编译器死代码消除

第四步 比值分析:对算法二,若 n 翻倍时间约 $\times 4$,则吻合 n^2 ;对算法三、四约 $\times 2$;算法一只测两个规模并说明 $\times 8$ 趋势

✔ 参考答案(完整可运行程序,约 90 行,分两段)

```

// lab2.cpp — 最大子序列和:四种算法的增长阶实测(第一段:算法)
#include <iostream>
#include <vector>
#include <random>
#include <chrono>
#include <algorithm>
using namespace std;
using Clock = chrono::steady_clock;

// ----- 四种算法($2.5) -----
int maxSubSum1(const vector<int>& a) {          //  $O(n^3)$ 
    int maxSum = 0;
    for (size_t i = 0; i < a.size(); ++i)
        for (size_t j = i; j < a.size(); ++j) {
            int thisSum = 0;
            for (size_t k = i; k <= j; ++k) thisSum += a[k];
            if (thisSum > maxSum) maxSum = thisSum;
        }
    return maxSum;
}

int maxSubSum2(const vector<int>& a) {          //  $O(n^2)$ 
    int maxSum = 0;
    for (size_t i = 0; i < a.size(); ++i) {
        int thisSum = 0;
        for (size_t j = i; j < a.size(); ++j) {
            thisSum += a[j];
            if (thisSum > maxSum) maxSum = thisSum;
        }
    }
    return maxSum;
}

int maxSumRec(const vector<int>& a, int left, int right) { //  $O(n \log n)$ 
    if (left == right) return a[left] > 0 ? a[left] : 0;
    int center = (left + right) / 2;
    int maxLeft = maxSumRec(a, left, center);
    int maxRight = maxSumRec(a, center + 1, right);
    int maxLeftBorder = 0, leftBorder = 0;
    for (int i = center; i >= left; --i) {
        leftBorder += a[i];
        if (leftBorder > maxLeftBorder) maxLeftBorder = leftBorder;
    }
    int maxRightBorder = 0, rightBorder = 0;
    for (int i = center + 1; i <= right; ++i) {
        rightBorder += a[i];
        if (rightBorder > maxRightBorder) maxRightBorder = rightBorder;
    }
    return max(max(maxLeft, maxRight), maxLeftBorder + maxRightBorder);
}

int maxSubSum3(const vector<int>& a) {
    return maxSumRec(a, 0, int(a.size()) - 1);
}

```

```
int maxSubSum4(const vector<int>& a) {           // O(n) 在线算法
    int maxSum = 0, thisSum = 0;
    for (int x : a) {
        thisSum += x;
        if (thisSum > maxSum) maxSum = thisSum;
        else if (thisSum < 0) thisSum = 0;
    }
    return maxSum;
}
```

lab2.cpp(第二段:计时与主程序)

```
// ----- 计时($2.1) -----
using Func = int (*)(const vector<int>&);
double msTime(Func f, const vector<int>& a, long long& guard) {
    auto t0 = Clock::now();
    guard += f(a);           // 返回值累加,防止死代码消除
    return chrono::duration<double, milli>(Clock::now() - t0).count();
}

int main() {
    // 正确性验证($2.5 样例)
    vector<int> demo = {-2, 11, -4, 13, -5, -2};
    cout << "验证: " << maxSubSum1(demo) << " "
        << maxSubSum2(demo) << " " << maxSubSum3(demo) << " "
        << maxSubSum4(demo) << " (应全为 20)" << endl;

    mt19937 rng(12345);
    uniform_int_distribution<int> dist(-1000, 1000);
    long long guard = 0;

    cout << "n\t算法一\t算法二\t算法三\t算法四\n";
    for (int n : {1000, 2000, 4000, 8000}) {
        vector<int> a(n);
        for (int& x : a) x = dist(rng);
        cout << n;
        if (n <= 2000)
            cout << "\t" << msTime(maxSubSum1, a, guard);
        else
            cout << "\t--(推算)"; // O(n^3) 在 4000 时约 26 秒,不实测
        cout << "\t" << msTime(maxSubSum2, a, guard)
            << "\t" << msTime(maxSubSum3, a, guard)
            << "\t" << msTime(maxSubSum4, a, guard) << endl;
    }
    cout << "(防优化累加值: " << guard % 1000 << ")" << endl;
    cout << "结论: n 翻倍时,算法一 x8(吻合 n^3),算法二 x4(吻合 n^2),"
        << "算法三/四约 x2(吻合 n log n 与 n)。" << endl;
    return 0;
}
```

示例输出(机器不同数值不同,比值规律一致):

验证：20 20 20 20 (应全为 20)

n	算法一	算法二	算法三	算法四
1000	412.3	0.42	0.05	0.02
2000	3310.1	1.68	0.11	0.04
4000	--(推算) 6.71		0.24	0.08
8000	--(推算) 26.9		0.51	0.17

(防优化累加值：123)

结论：n 翻倍时,算法一 $\times 8$ (吻合 n^3),算法二 $\times 4$ (吻合 n^2),算法三/四约 $\times 2$ (吻合 $n \log n$ 与 n)。

设计说明:① 四个算法共用 `Func` 函数指针类型,计时代码只写一份——这正是第7章"比较器"思想的雏形,也与 C 的 `qsort` 函数指针同源;② `guard` 变量累加所有返回值并输出,防止编译器把整个调用优化掉;③ 算法一在 $n \geq 4000$ 时按 §2.3 的数值表推算($\times 8$ 每翻倍),避免把几分钟浪费在早已被理论淘汰的算法上;④ 种子固定为 12345,任何机器复现本实验都得到相同的数组。

下一章预告:第3章 表、栈和队列——三种最基础的数据结构及其数组/链式实现。第3章起,每章末尾的复杂度速查表将直接使用本章的记号;最大子序列和的分治骨架也将在第7章排序中再次登场。

第3章 表、栈和队列

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**抽象数据类型(ADT)**思想:先定义"做什么",再谈"怎么实现"
- 掌握表的两类实现:数组实现(倍增扩容)与链式实现(结点+指针),会对比两者复杂度
- 完成从 C 的 `malloc` 数组到 C++ `vector` 的迁移,理解容量与长度的区别
- 理解**迭代器失效**这一 C++ 独有的坑,并知道如何规避
- 能独立实现栈的数组版与链式版,并用栈解决括号平衡、后缀求值、中缀转后缀三个经典问题
- 能独立实现循环数组队列,说清空/满判断的两种方案
- 学完能独立实现中缀表达式计算器(栈应用综合),并会正确选用 `std::stack` / `std::queue`

💡 从 C 到 C++:本章主题如何迁移

本章是全书第一个"真正动手实现数据结构"的章节。算法直觉先给结论:表、栈、队列是后续所有复杂结构的积木——树的层序遍历要用队列,图的 DFS 要用栈,第6章的堆本质是数组,第5章的散列表是"数组+链表"的合体。本章学不透,后面每一章都会还债。

在 C 里,你已经会两种原始工具: `malloc` 出来的动态数组和 `struct` + 指针串起来的链表。C 的痛点是双重的:动态数组要自己管理"容量、长度、扩容"三件事,一次 `realloc` 写错就是内存泄漏或堆损坏;链表没有语言支持,每个操作都要手写指针舞蹈,一不小心就访问悬垂指针。更麻烦的是,C 里"数据结构"和"操作"是分离的——`sort` 一个数组和 `sort` 一个链表是两套完全不同的代码。

C++ 的解法分两层:**底层**用 `vector` 替你管理动态数组(自动扩容、自动释放,RAII 保证不泄漏),用 `list` 提供双向链表;**上层**用类把数据与操作封装成一个整体——这正是 ADT 思想,也是 C 的 `struct` 加函数指针表想模拟却始终别扭的东西。本章会手写一遍底层实现(理解原理),再告诉你标准库怎么用(工程实践)。

栈与队列部分,本章用三个经典应用(括号平衡、后缀求值、中缀转后缀)演示"数据结构服务于算法"的思维方式——它们是编译原理与表达式求值的原型,也是各大面试的高频题。

📦 前置知识自查

数学前置:本章几乎不需要新数学;倍增扩容的摊还分析直觉即可(严格分析在第11章)。

C 语言前置: `malloc` / `free` 动态数组、`struct` 与 `->` 指针访问、函数指针(可选)、`char*` 字符串遍历。

依赖的前面章节:第1章递归(栈的递归实现对照)、第2章大 O 记号(本章复杂度速查表直接用)。

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 §3.1–§3.2,建立 ADT 概念,完成 vector 迁移练习
2. **第二阶段(1.5 小时)**:读 §3.3–§3.4,手写动态数组与单链表,上机验证
3. **第三阶段(1.5 小时)**:读 §3.5–§3.6,手写栈与队列,完成三个栈应用例题
4. **第四阶段(实验,1 小时)**:完成章末实验中缀表达式计算器
5. **验收标准**:能默写单链表插入/删除、循环队列、中缀转后缀,并说清每步复杂度

本章知识导图

- §3.1 ADT:抽象数据类型(接口与实现分离、从 C 的 struct+函数指针到类)
- §3.2 表 ADT 与从 C 的迁移(malloc 数组 → vector、倍增扩容、容量 vs 长度)
- §3.3 表的数组实现(手写 MyVector、vector 用法、迭代器与失效)
- §3.4 表的链式实现(结点与指针、手写单链表、vector 与 list 的选型)
- §3.5 栈(数组栈、括号平衡、后缀求值、中缀转后缀、std::stack)
- §3.6 队列(循环数组实现、空/满判断的两种方案、std::queue)
- §3.7 选型与工程实践(何时数组何时链表、栈模拟递归、调试技巧)

§3.1 ADT:抽象数据类型

3.1.1 什么是 ADT

抽象数据类型(ADT) = 一组操作 + 这些操作的语义, **不涉及实现细节**。例如"栈"这个 ADT 只有三个操作:push(入栈)、pop(出栈)、top(取栈顶),语义是"后进先出(LIFO)"。至于底层用数组还是链表、满时怎么办,属于"实现",与 ADT 无关。好处: **使用者只依赖接口,实现可以随时替换**——把数组栈换成链式栈,使用方代码一行不用改。

3.1.2 从 C 的 struct+函数指针到 C++ 的类

C 语言没有类,模拟 ADT 只能用 struct 存数据 + 函数指针表存操作:

C 的写法(struct + 函数指针)	C++ 的写法(class 封装)
<pre>struct Stack { int *a; int top; int cap; };</pre>	<pre>class Stack { vector<int> a; ... };</pre>
<pre>stack_init(&s); stack_push(&s, x);</pre> 每个操作传 &s	<pre>s.push(x);</pre> 数据与操作天然绑定
使用者能"意外"访问 <code>s.a[0]</code> 破坏结构	<code>private</code> 成员从语言层面禁止越界访问
忘记 <code>stack_destroy</code> 即泄漏,没有提醒	析构函数自动释放,RAII 不依赖使用者自觉

结论:ADT 的思想两种语言都能表达,C++ 的类是它的天然载体——本书所有数据结构都以"类"的形式呈现。

§3.2 表 ADT 与从 C 的迁移

3.2.1 表的基本操作

表(list)ADT:线性有序元素的集合,常用操作——按下标访问、按值查找、任意位置插入/删除、遍历。**同一个 ADT,两种截然不同的实现**(数组与链表),各有胜负,本章的主线就是这张对照表:

操作	数组实现(vector)	链式实现(list)
按下标访问	$O(1)$	$O(n)$
按值查找	$O(n)$	$O(n)$
尾部插入	平均 $O(1)$	$O(1)$
头部插入	$O(n)$	$O(1)$
任意位置插入/删除	$O(n)$ (移动元素)	$O(1)$ (已定位后)

3.2.2 C 的 malloc 数组 → C++ 的 vector

回忆 C 里管理动态数组的三件套:记录长度、记录容量、`realloc` 扩容。C++ 的 `vector` 把这套动作封装成一个类型: `size()` 返回当前元素个数(长度), `capacity()` 返回已分配空间能容纳的个数(容量),两者**不是一回事**——这是 C 程序员迁移时最需要建立的新概念。

C 的写法	C++ 的写法
<code>int *a = malloc(n * sizeof(int));</code>	<code>vector<int> a(n);</code>
<code>a[len++] = x;</code> (自己管长度)	<code>a.push_back(x);</code>
<code>realloc</code> 手动扩容(易错)	自动扩容,倍增策略
<code>free(a);</code> (忘记即泄漏)	离开作用域自动释放
<code>a[i]</code> 越界 = 未定义行为	<code>a.at(i)</code> 越界抛异常

倍增扩容:当 `push_back` 时容量已满,vector 分配一块**2 倍**大小的新内存,拷贝全部旧元素,释放旧内存。直觉上"每次扩容都是 $O(n)$,那插入怎么还说平均 $O(1)$?"——因为扩容发生得越来越少:容量从 1 倍增到 n 需要 $\log_2 n$ 次扩容,总拷贝代价 $n + n/2 + n/4 + \dots \approx 2n$,摊到 n 次插入上平均每次 $O(1)$ 。严格的摊还分析见第11章,现在先记住结论: **倍增使得尾部插入平均 $O(1)$** 。

⚠ 误区 1:capacity 与 size 混淆

`reserve(100)` 只预分配内存(改 capacity),元素个数仍是 0;要制造 100 个元素必须用 `resize(100)` 或插入。新手常写 `a.reserve(n); a[i] = x;` 然后崩溃——此时 `a[i]` 访问的是**不存在的元素**(越界)。口诀:reserve 管"房间",resize 管"住客"。

§3.3 表的数组实现

3.3.1 手写动态数组:搞懂倍增

示例 3-1 手写 MyVector:倍增扩容

```
// 手写动态数组:容量不足时倍增(2 倍)拷贝
class MyVector {
    int* data_ = nullptr;    // 元素存储
    size_t size_ = 0;        // 元素个数(长度)
    size_t cap_ = 0;         // 已分配容量
public:
    ~MyVector() { delete[] data_; }    // RAII:自动释放
    void push_back(int x) {
        if (size_ == cap_) {           // 满了:倍增
            size_t newCap = cap_ == 0 ? 1 : cap_ * 2;
            int* p = new int[newCap];
            for (size_t i = 0; i < size_; ++i) p[i] = data_[i];
            delete[] data_;
            data_ = p;
            cap_ = newCap;
        }
        data_[size_++] = x;
    }
    int& operator[](size_t i) { return data_[i]; }    // 按下标访问
    size_t size() const { return size_; }
    size_t capacity() const { return cap_; }
};
```

3.3.2 vector 常用操作速览

示例 3-2 vector 核心操作

```
#include <vector>
#include <iostream>
using namespace std;
int main() {
    vector<int> v;           // 空表
    v.push_back(3);          // 尾部插入:3
    v.push_back(1);          // 3 1
    v.insert(v.begin(), 9);   // 头部插入:9 3 1 (O(n),搬移全部元素)
    v.erase(v.begin() + 1);   // 删除下标 1:9 1 (O(n))
    v[1] = 5;                // 随机访问 O(1):9 5
    for (int x : v) cout << x << ' ';    // 范围 for 遍历
    cout << "\nsize=" << v.size() << " capacity=" << v.capacity();
    return 0;
}
```

3.3.3 迭代器与失效(本章最大的 C++ 专属坑)

迭代器(iterator)是"指向容器中位置"的对象,可以看作 C 指针的泛化。`v.begin()` 指向首元素, `v.end()` 指向尾后位置。

示例 3-3 用迭代器遍历(与 C 指针对照)

```
// C 指针遍历:
int a[5] = {1, 2, 3, 4, 5};
for (int* p = a; p != a + 5; ++p)    // 指针自增走到下一个元素
    printf("%d ", *p);

// C++ 迭代器遍历(结构完全一致:开始/结束/自增/解引用)
vector<int> v = {1, 2, 3, 4, 5};
for (auto it = v.begin(); it != v.end(); ++it)
    cout << *it << ' ';           // 迭代器就是"泛化的指针"
```

⚠ 误区 2:迭代器失效

vector 扩容时会**整体搬家**:旧内存被释放,之前保存的所有迭代器/指针立即失效。下面代码是未定义行为: `auto it = v.begin(); v.push_back(x); *it = 0;` // 若发生扩容, `it` 已指向被释放的内存!。规避三招:① 提前 `reserve` 足够容量;② 每次修改容器后重新获取迭代器;③ 只在下标遍历时才修改容器。同样的规则适用于 C 的 `realloc`——指针失效不是新问题,只是 C++ 把它包装得更隐蔽。

§3.4 表的链式实现

3.4.1 结点与指针

链表的原子单位是**结点(Node)**:一个数据域 + 一个指向下一个结点的指针。C 的 `struct Node { int val; struct Node *next; };` 与 C++ 的写法几乎一致(去掉了 `struct` 关键字而已)。链表不需要连续内存,插入/删除只改指针,不用搬移元素——代价是按下标访问要从头走 $O(n)$ 步。

3.4.2 手写单链表

示例 3-4 单链表:表头插入、查找、删除

```

struct Node {
    int val;
    Node* next;
    Node(int v, Node* n = nullptr) : val(v), next(n) {}
};

void insertFront(Node*& head, int x) {    // 表头插入 O(1)
    head = new Node(x, head);
}

Node* find(Node* head, int x) {          // 查找 O(n)
    while (head != nullptr && head->val != x)
        head = head->next;
    return head;                          // 找不到返回 nullptr
}

bool erase(Node*& head, int x) {          // 删除值为 x 的第一个结点
    if (head == nullptr) return false;
    if (head->val == x) {                  // 删表头是特例
        Node* t = head;
        head = head->next;
        delete t;
        return true;
    }
    Node* prev = head;                    // 其余情况:找前驱
    while (prev->next != nullptr && prev->next->val != x)
        prev = prev->next;
    if (prev->next == nullptr) return false;
    Node* t = prev->next;
    prev->next = t->next;                  // 前驱跳过目标结点
    delete t;
    return true;
}

```

★ 重点:插入删除的"定位"成本

说"链表插入删除 $O(1)$ "是有前提的: 已经定位到操作位置。定位本身(查找值、走到第 k 个结点)是 $O(n)$ 。所以"链表的插入比数组快"只在特定场景成立: 频繁头部操作、或已持有位置指针的遍历过程中。速查表里的" $O(1)$ "指的都是"已定位后"。

3.4.3 vector 与 list 的选型

C++ 标准库的 `std::list` 是双向链表,接口与 `vector` 高度一致(都有 `push_back`/`insert`/`erase`/迭代器)。选型口诀:

- 需要随机访问 → `vector`(链表按下标是 $O(n)$)
- 只在尾部增删 → `vector`(连续内存缓存友好,常数小)
- 频繁在头部/中间插入删除且已定位 → `list`
- 拿不准 → `vector`(工程统计:90% 的场景 `vector` 更快,链表赢在"无搬移"的场景才值得用)

示例 3-5 用 `std::list` 体会接口一致性

```

#include <list>
#include <iostream>
using namespace std;
int main() {
    list<int> l = {1, 2, 3};
    l.push_front(0);           // 头部插入 O(1), vector 没有这个接口!
    l.push_back(4);            // 尾部插入 O(1)
    auto it = l.begin();
    ++it; ++it;                // 走到第 2 个元素(下标访问是 O(n))
    l.insert(it, 9);           // 已定位后插入 O(1): 0 1 9 2 3 4
    for (int x : l) cout << x << ' '; // 双向链表也支持范围 for
    return 0;
}

```

工程建议:释放链表的内存

手写链表删除结点后忘记 `delete` 是最常见的泄漏源;整表销毁要在循环里逐个 `delete`。工程上更推荐用 `std::list` (自动管理)或在 `Node` 中用 `unique_ptr` (第4章树的实现会演示)。学习阶段手写是为了理解指针结构,生产代码请优先标准库。

§3.5 栈

3.5.1 栈 ADT 与数组实现

栈(stack):后进先出(LIFO)。三个操作——`push`(压入)、`pop`(弹出)、`top`(查看栈顶)。数组实现最简单:用 `vector` 存元素,用一个"栈顶下标"标记位置,只在尾部增删,三个操作全部 $O(1)$ 。

示例 3-6 数组栈(用 `vector` 实现)

```

class Stack {
    vector<int> a;           // 元素数组,栈顶在尾部
public:
    void push(int x) { a.push_back(x); }
    void pop() { a.pop_back(); } // 出栈:删除尾部
    int top() const { return a.back(); }
    bool empty() const { return a.empty(); }
    size_t size() const { return a.size(); }
};

```

注意与 C 的函数调用栈对照:递归深度过大时崩溃,本质就是这同一个结构——调用栈被 `push` 到溢出(第1章讲过)。栈也可以用单链表实现(表头做栈顶,头插头删 $O(1)$),接口完全不变——这正是 §3.1 说的 ADT 威力。

3.5.2 应用一:括号平衡检查

示例 3-7 括号平衡:三种括号匹配检查

```
// 判断字符串中 ( ) [ ] { } 是否配对且顺序正确
bool isBalanced(const string& s) {
    stack<char> st; // 标准库栈,元素是 char
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{')
            st.push(c); // 左括号入栈
        else if (c == ')' || c == ']' || c == '}') {
            if (st.empty()) return false; // 右括号多余
            char top = st.top();
            st.pop();
            if ((c == ')' && top != '(') || // 类型不匹配
                (c == ']' && top != '[') ||
                (c == '}' && top != '{'))
                return false;
        }
    }
    return st.empty(); // 结束时栈空 = 左括号无剩余
}
```

3.5.3 应用二:后缀表达式求值

后缀表达式(逆波兰记法)把运算符放在操作数之后: `6 5 2 3 + 8 * + 3 + *`,不需要括号。求值规则:遇数字入栈;遇运算符弹出两个操作数,算完把结果压回。这个"惰性计算"模式是栈最漂亮的用法。

示例 3-8 后缀表达式求值

```
// 输入:token 序列(数字串与运算符);输出:表达式值
int evalPostfix(const vector<string>& toks) {
    stack<int> st;
    for (const string& t : toks) {
        if (isdigit(t[0]) || (t.size() > 1 && t[0] == '-'))
            st.push(stoi(t)); // 数字入栈
        else {
            int b = st.top(); st.pop(); // 先弹的是右操作数!
            int a = st.top(); st.pop();
            if (t == "+") st.push(a + b);
            else if (t == "-") st.push(a - b);
            else if (t == "*") st.push(a * b);
            else st.push(a / b); // 只支持整除
        }
    }
    return st.top(); // 最后栈里只剩结果
}
```

⚠ 误区 3:操作数弹出顺序

后缀 `8 2 /` 是 $8/2=4$,但弹栈顺序是先弹 2 再弹 8——先弹的是右操作数。减法和除法对顺序敏感,写反了就是 $2/8$ 。口诀:栈顶是"后出现"的操作数,后出现的是右操作数。

3.5.4 应用三:中缀转后缀

人读的是中缀(`a+b*c`),机器算的是后缀。转换算法(调度场算法):遇数字直接输出;遇运算符,先把栈中**优先级不低于它的**运算符弹出输出,再压入自己;左括号直接入栈,右括号则弹出到左括号为止。这样保证高优先级先输出。

示例 3-9 中缀转后缀(单数字简化版)

```
int prec(char op) { return (op == '+' || op == '-') ? 1 : 2; }

string infixToPostfix(const string& s) {    // s 形如 "6+5*(2+3)",无空格
    string out;
    stack<char> st;
    for (char c : s) {
        if (isdigit(c)) out += c;          // 数字直接输出
        else if (c == '(') st.push(c);
        else if (c == ')') {
            while (st.top() != '(') { out += st.top(); st.pop(); }
            st.pop();                      // 丢弃左括号
        } else {                            // 运算符
            while (!st.empty() && st.top() != '(' &&
                prec(st.top()) >= prec(c)) {    // 栈顶优先级不低于当前
                out += st.top(); st.pop();
            }
            st.push(c);
        }
    }
    while (!st.empty()) { out += st.top(); st.pop(); }
    return out;
}
```

例题 3-1 手工走一遍中缀转后缀

输入 6+5*(2+3) :6 输出;+ 入栈;5 输出;* 入栈(+ 优先级低,留在栈中);(入栈;2 输出;+ 入栈;3 输出;) 弹出 +,再弹;
结束弹栈得 *、+。输出: 6523+*+。用 §3.5.3 求值:2+3=5,5*5=25,6+25=31。✓

3.5.5 标准库:std::stack

`std::stack` 是适配器:默认用 `deque` (双端队列)做底层,接口就是 `push/pop/top/empty/size`。本章示例 3-7 已经用过。想换底层可写 `stack<int, vector<int>>`——又是 ADT 思想的直接体现。

§3.6 队列

3.6.1 队列 ADT 与循环数组实现

队列(queue):先进先出(FIFO)。enqueue(入队)在队尾,poll/dequeue(出队)在队头。用数组实现有个经典问题:出队后前面空出的位置怎么办?如果每次出队都把全部元素前移,那是 $O(n)$;不移动、让队头指针"追赶"队尾指针,数组就要**循环**使用——下标到底后绕回 0。

示例 3-10 循环数组队列(用 count 记录元素数)

```
// front 指向队首元素,back 指向队尾下一个空位,count 记录元素数
class MyQueue {
    vector<int> data_;
    int front_ = 0, back_ = 0, count_ = 0;
public:
    MyQueue(int cap) : data_(cap) {}
    void push(int x) {                // 入队:假设未满(满的处理见误区4)
        data_[back_] = x;
        back_ = (back_ + 1) % int(data_.size());
        ++count_;
    }
    void pop() {                      // 出队
        front_ = (front_ + 1) % int(data_.size());
        --count_;
    }
    int front() const { return data_[front_]; }
    bool empty() const { return count_ == 0; }
    int size() const { return count_; }
};
```

★ 重点:空/满判断的两种方案

若用 front 与 back 两个下标判断空满,会遇到 $front == back$ 时"既可能空也可能满"的歧义。两种解法:① **计数法**(示例 3-10,多存一个 count,最简单);② **浪费一格法**(规定 back 永远不追赶 front,队满时留一个空格,满条件是 $(back+1) \% cap == front$)。工程与面试都常考,两种都要会。

3.6.2 标准库:std::queue 与 deque

示例 3-11 std::queue 与 deque 用法

```
#include <queue>
#include <deque>
using namespace std;
int main() {
    queue<int> q;                // 默认基于 deque 的 FIFO 队列
    q.push(1); q.push(2); q.push(3);
    while (!q.empty()) {
        cout << q.front() << ' '; // 1 2 3(先进先出)
        q.pop();
    }
    deque<int> d = {1, 2, 3};    // 双端队列:头尾都可 O(1) 增删
    d.push_front(0);             // 0 1 2 3
    d.push_back(4);              // 0 1 2 3 4
    d.pop_front();               // 1 2 3 4
    return 0;
}
```

C 的手写循环队列	C++ 的标准库
自己维护 front/back 下标与取模	queue.push/pop/front 一行一个操作
空/满判断要写计数或浪费一格	自动扩容,永不"满"
free 忘记调用即泄漏	离开作用域自动释放

⚠ 误区 4:循环队列的下标溢出

示例 3-10 简化了"队满"检查——满时再 push 会覆盖旧元素。生产实现必须显式处理:计数法判 `count_ == cap` 满则拒绝/扩容,浪费一格法判 `(back_+1) % cap == front_`。另一个易错点是取模:back 先加 1 再对 capacity 取模,而不是对"当前元素数"取模。

§3.7 选型与工程实践

3.7.1 数组 or 链表?—一张决策表

场景	选择	理由
按序访问所有元素(遍历)	vector	连续内存,缓存友好,常数小
大量随机下标访问	vector	$O(1)$ vs $O(n)$
只做尾部增删(栈)	vector	均摊 $O(1)$,实现最简单
两端增删(队列)	deque / 循环数组	两头都 $O(1)$
遍历过程中删除元素	list	vector 会失效迭代器
内存极度受限且频繁插入	list	无扩容翻倍的峰值内存

3.7.2 栈模拟递归与调试技巧

任何递归都可以改写为"显式栈 + 循环"的迭代版(第4章树遍历会用到):函数调用栈压的是"返回地址 + 局部变量",显式栈压的是"待处理的任务"。迭代版不受系统栈深度限制,深度优先搜索(第9章)的大图遍历通常必须用迭代版。调试栈程序的两个技巧:① push 前打印"压入 x",pop 后打印"弹出 x",肉眼跟踪栈的演化;② 断言每个 pop 前 !empty(),把"逻辑错误"提前变成"崩溃点",定位速度翻倍。

🚀 工程建议:优先标准库,手写只为理解

本章手写了动态数组、链表、栈、队列四样东西——目的是让你看清每一层的复杂度来源。生产代码请一律使用 `vector/list/stack/queue/deque`:它们经过数十年打磨,边界处理(异常安全、自赋值、移动语义)远比自己写的健壮。手写实现的用武之地是"标准库没有的结构"(后几章的树、堆、散列表、图),那才是本书后续的主战场。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
按下标访问	vector(动态数组)	$O(1)$	$O(1)$
尾部插入(push_back)	vector	$O(1)$	$O(n)$ (扩容)
头部插入	vector	$O(n)$	$O(n)$
任意位置插入/删除	vector	$O(n)$	$O(n)$
按值查找	vector	$O(n)$	$O(n)$
按下标访问	list(链表)	$O(n)$	$O(n)$
按值查找	list	$O(n)$	$O(n)$
头部/尾部插入	list	$O(1)$	$O(1)$
已定位处插入/删除	list	$O(1)$	$O(1)$
push / pop / top	栈(数组或链表实现)	$O(1)$	$O(1)$
入队 / 出队 / 取队头	循环数组队列	$O(1)$	$O(1)$
入队 / 出队	链式队列	$O(1)$	$O(1)$
两端增删	deque(双端队列)	$O(1)$	$O(1)$
括号平衡检查	栈扫描一遍	$O(n)$	$O(n)$
后缀表达式求值	栈扫描一遍	$O(n)$	$O(n)$
中缀转后缀	调度场算法	$O(n)$	$O(n)$
vector 倍增扩容	n 次 push_back 总代价	摊还 $O(n)$, 平均每次 $O(1)$ (第11章证明)	

最小必会清单

- 能独立说出 ADT 的定义,并举例说明"接口与实现分离"的好处
- 能独立说出 vector 的 size 与 capacity 的区别,以及 reserve/resize 的语义
- 能独立默写倍增扩容的 push_back,并解释为什么平均 $O(1)$
- 能独立默写单链表的表头插入、查找、删除(含删表头特例)
- 能独立解释迭代器失效的原因,并说出三种规避手段
- 能独立默写数组栈,并用它解决括号平衡问题
- 能独立写出后缀表达式求值,并说清操作数的弹出顺序
- 能独立默写中缀转后缀算法(含运算符优先级处理)
- 能独立实现循环数组队列,并说出空/满判断的两种方案
- 能独立说出 vector 与 list 的选型规则(至少 4 条场景)

自测题

Q 1. vector 的 push_back 为什么说"平均 $O(1)$ "而最坏 $O(n)$?

✅ 参考答案

平时插入只写一个元素 $O(1)$; 仅当 size 达到 capacity 时才触发倍增扩容, 需拷贝全部 n 个元素 $O(n)$ 。扩容从 1 倍增到 n 共 $\log_2 n$ 次, 总拷贝 $n+n/2+\dots\approx 2n$, 摊到 n 次插入上平均 $O(1)$ 。

Q 2. `v.reserve(100)` 后 `v[50] = 1` 会怎样?为什么?

✅ 参考答案

未定义行为(通常崩溃)。`reserve` 只预分配内存, `size()` 仍为 0, `v[50]` 访问的是不存在的元素。应先 `resize(100)` 或改用 `push_back`。

Q 3. 单链表中删除"值为 x 的结点", 为什么表头要单独处理?怎样统一?

✅ 参考答案

普通结点删除需要前驱结点改指针, 而表头没有前驱。解法: ① 特判(本章示例 3-4); ② 引入哑结点(dummy), 表头前面永远站一个空结点, 删除逻辑统一; ③ 指针的指针 `Node**` 遍历"指向结点的指针", 统一修改。哑结点法在工程中最常用。

Q 4. 后缀表达式 `8 2 / 3 5 * +` 的值是多少?写出求值过程。

✅ 参考答案

8、2 入栈; /: 弹 2、8, $8/2=4$ 入栈; 3、5 入栈; *: 弹 5、3, $3*5=15$ 入栈; +: 弹 15、4, $4+15=19$ 。答案 19。注意先弹的是右操作数。

Q 5. 循环队列用 `front` 与 `back` 两个下标时, 为什么 `front == back` 会"既可能空也可能满"?两种解法是什么?

✅ 参考答案

队列空时 `front` 与 `back` 重合; 而队满时 `back` 绕一圈也追上 `front`, 两个状态无法区分。解法: ① 计数法——额外维护元素个数 `count`, 空满由 `count` 判断; ② 浪费一格法——规定 `back` 永不与 `front` 重合, 满条件是 $(back+1) \% cap == front$, 牺牲一个格子换取 $O(1)$ 判断。

Q 6. 中缀 `a+b*c-d` 转后缀的过程是什么?为什么 `*` 在 `+` 之前输出?

✅ 参考答案

`a` 输出; `+` 入栈; `b` 输出; 遇 `*`: 栈顶 `+` 优先级(1)低于 `*`(2), 不弹出, `*` 入栈; `c` 输出; 遇 `-`: 栈顶 `*` 优先级不低于 `-`, 弹出 `*` 输出; 栈顶 `+` 优先级不低于 `-`, 弹出 `+` 输出; `-` 入栈; `d` 输出; 结束弹栈输出 `-`。结果 `abc*+d-`。`*` 先输出是因为它绑定得更紧(优先级高), 后缀表达式中运算符顺序即计算顺序。

章末实验题:中缀表达式计算器

实验 3:中缀表达式计算器(栈应用综合)

实验目标:实现一个命令行计算器:读入含 $+$ $-$ $*$ $/$ 与括号的中缀表达式,支持多位数,输出后缀表达式与计算结果。

实验要求:

- 任务 1:实现 tokenize,把字符串拆成数字与运算符序列,支持多位数(知识点:\$3.5.4 中缀转后缀的扩展)
- 任务 2:实现中缀转后缀,处理括号与优先级(知识点:\$3.5.4 调度场算法)
- 任务 3:实现后缀求值,注意弹出顺序(知识点:\$3.5.3)
- 任务 4:主循环读入表达式,输出后缀式与结果,遇到 `quit` 退出(知识点:\$3.5 栈应用综合)
- 任务 5:处理非法输入:除数为 0、括号不匹配时给出错误信息而非崩溃(知识点:\$3.5.2 括号平衡)

实验环境:C++17,MSVC 或 g++。编译: `cl /EHsc /std:c++17 lab3.cpp` 或 `g++ -std=c++17 lab3.cpp -o lab3`;运行 `lab3`。

第一步 tokenize 用指针扫描:连续数字拼成一个字符串 token;空格跳过;其余字符单独成 token

第二步 中缀转后缀直接复用示例 3-9 的骨架,把"单字符数字"换成"字符串 token";数字 token 用首字符 `isdigit` 判定

第三步 求值时先检查操作数栈是否够两个元素(非法输入),再检查除数是否为 0

第四步 括号匹配错误在转换阶段发现:弹栈遇空、或结束后栈里剩左括号,都报"括号不匹配"

✓ 参考答案(完整可运行程序,约 110 行,分两段)

```

// lab3.cpp — 中缀表达式计算器(第一段:词法与转换)
#include <iostream>
#include <string>
#include <vector>
#include <stack>
#include <cctype>
#include <stdexcept>
using namespace std;

// 任务 1:拆分 token(支持多位数与空格)
vector<string> tokenize(const string& s) {
    vector<string> toks;
    for (size_t i = 0; i < s.size(); i) {
        if (isspace((unsigned char)s[i])) { ++i; continue; }
        if (isdigit((unsigned char)s[i])) {
            string num;
            while (i < s.size() && isdigit((unsigned char)s[i]))
                num += s[i++];
            toks.push_back(num);
        } else {
            toks.push_back(string(1, s[i++]));
        }
    }
    return toks;
}

int prec(const string& op) { return (op == "+" || op == "-") ? 1 : 2; }

// 任务 2:中缀转后缀(调度场算法)
vector<string> infixToPostfix(const vector<string>& toks) {
    vector<string> out;
    stack<string> st;
    for (const string& t : toks) {
        if (isdigit((unsigned char)t[0]))
            out.push_back(t); // 数字直接输出
        else if (t == "(") st.push(t);
        else if (t == ")") {
            while (!st.empty() && st.top() != "(") {
                out.push_back(st.top()); st.pop();
            }
            if (st.empty()) throw runtime_error("括号不匹配");
            st.pop(); // 丢弃左括号
        } else { // 运算符
            while (!st.empty() && st.top() != "(" &&
                prec(st.top()) >= prec(t)) {
                out.push_back(st.top()); st.pop();
            }
            st.push(t);
        }
    }
    while (!st.empty()) {
        if (st.top() == "(") throw runtime_error("括号不匹配");
        out.push_back(st.top()); st.pop();
    }
    return out;
}

```



```
}
```

```
// 任务 3: 后缀求值(注意弹出顺序)
```

lab3.cpp(第二段:求值与主循环)

```
int evalPostfix(const vector<string>& toks) {
    stack<int> st;
    for (const string& t : toks) {
        if (isdigit((unsigned char)t[0]))
            st.push(stoi(t));
        else {
            if (st.size() < 2) throw runtime_error("操作数不足");
            int b = st.top(); st.pop(); // 先弹的是右操作数
            int a = st.top(); st.pop();
            if (t == "+") st.push(a + b);
            else if (t == "-") st.push(a - b);
            else if (t == "*") st.push(a * b);
            else {
                if (b == 0) throw runtime_error("除数为 0");
                st.push(a / b);
            }
        }
    }
    if (st.size() != 1) throw runtime_error("表达式非法");
    return st.top();
}

int main() {
    cout << "中缀表达式计算器(支持 + - * / 与括号,输入 quit 退出)\n";
    string line;
    while (true) {
        cout << "> ";
        getline(cin, line);
        if (line == "quit") break;
        try {
            auto toks = tokenize(line);
            auto post = infixToPostfix(toks);
            int result = evalPostfix(post);
            cout << "后缀式: ";
            for (const string& t : post) cout << t << ' ';
            cout << "\n结果: " << result << endl;
        } catch (const exception& e) {
            cout << "错误: " << e.what() << endl;
        }
    }
    return 0;
}
```

示例输入输出:

```
> 6+5*(2+3)
后缀式: 6 5 2 3 + * +
结果: 31
> 12/4+7*2
后缀式: 12 4 / 7 2 * +
结果: 17
> (1+2)*3
后缀式: 1 2 + 3 *
结果: 9
> 5/0
错误: 除数为 0
> (1+2
错误: 括号不匹配
> quit
```

设计说明:① tokenize/转换/求值三阶段分离,每阶段只做一件事,出错信息定位到阶段——这是编译原理"词法→语法→语义"流水线的雏形;② 转换与求值都基于同一个栈结构,体现了 ADT"实现替换、接口不变"的思想(两个函数各自声明自己的 stack,互不干扰);③ 错误用异常 `throw` 统一报告,主循环一个 `catch` 兜底,避免每个函数各自打印错误导致"错误处理与逻辑纠缠";④ 运算符优先级集中在 `prec` 一个函数里,新增运算符(如幂 `^`)只需改这一处。

下一章预告:第4章 树——把"链式结点"推广为"有分支的结构",学习二叉树遍历、表达式树、二叉查找树与 AVL 平衡。第4章的表达式树与本章的中缀转后缀直接衔接;树的层序遍历将使用本章的队列。

第4章 树

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 掌握树的核心术语:根、父结点、子结点、兄弟、叶、深度、高度、路径
- 完成从 C 的 `first_child + next_sibling` 到 C++ `vector` 子结点容器的迁移
- 掌握前序/中序/后序/层序遍历:递归版、队列版、显式栈版都能写出
- 能用栈从后缀表达式构建表达式树并用中序恢复中缀,与第3章闭环
- 能完整实现BST的 `contains/findMin/insert/remove` 与 AVL 四种旋转,并用 `unique_ptr` 管理内存

💡 从 C 到 C++:本章主题如何迁移

树是"有分支的链表":链表只能排成一条线,树允许一个结点有多个后继,信息分层组织。家族谱、文件目录、语法树都是树。第3章解决"一串数据",本章解决"有层次关系的数据"。

C 里一般树靠双指针 `firstChild + nextSibling` 串起任意多个子结点,通用但笨拙。C++ 复用第3章的 `vector`:子结点放进容器,遍历就是一句范围 `for`,销毁交给 RAII。

本章终点是查找:二分查找 $O(\log n)$,但有序数组插入要搬移元素。BST 把查找、插入、删除同时做到平均 $O(\log n)$;AVL 再把"平均"升级为"保证"。

📦 前置知识自查

数学前置:树高递归式: $\text{height}(\text{空}) = 0, \text{height}(v) = 1 + \max(\text{height}(v.\text{left}), \text{height}(v.\text{right}))$;对数直觉(每次减半 $\rightarrow \log n$)。

C 语言前置: `struct` 与指针、`malloc / free`、递归、`->` 访问成员。

依赖的前面章节:第1章递归(遍历、高度、销毁)、第3章栈与队列(层序用队列,建表达式树用栈;中缀转后缀是 §4.3 的输入)。

🚀 本章学习路线

1. 阶段1(1 小时):§4.1-§4.2 建立术语,上机核对四种遍历输出
2. 阶段2(1 小时):§4.3 表达式树,与中缀转后缀接成闭环
3. 阶段3(1.5 小时):§4.4 手写 BST,默写删除三情形
4. 阶段4(1 小时):§4.5 画图验证 AVL 旋转,通读 §4.6-§4.7
5. 阶段5(实验 1 小时):完成章末实验,复习删除三情形与旋转判定

本章知识导图

- §4.1 树的概念:术语与一般树的实现(根/父/子/兄弟/叶/深度/高度/路径、first_child+next_sibling → vector、一般树 vs 二叉树)
- §4.2 二叉树与四种遍历(递归三序、层序用队列、显式栈模拟递归、高度/结点数/叶数)
- §4.3 表达式树(后缀构建、中序恢复中缀)
- §4.4 二叉查找树 BST(查找/插入/删除、删除三情形、手写 vs set)
- §4.5 AVL 树(平衡因子、LL/RR/LR/RL 旋转、插入后重平衡)
- §4.6 伸展树与 B 树(摊还与局部性、磁盘 IO 与浅而宽)
- §4.7 工程与实践(unique_ptr 内存、std::set/map 对照、调试)

§4.1 树的概念:术语与一般树的实现

4.1.1 术语:根、父结点、子结点、兄弟、叶、深度、高度、路径

树的递归定义:树是结点的集合——要么是空集,要么由一个**根**结点加上若干棵互不相交的**子树**构成,每棵子树本身也是树。这决定了本章算法的递归写法。

图 4-1 一般树的术语

树的递归定义:树 = 根 + 互不相交的子树。



深度从根往下数(根 0),**高度**从叶往上数(叶 0),树高 = 根的高度;**路径**长度 = 边数。n 个结点的树有 n-1 条边。方向相反,勿混淆。

记号说明(树高递归式,全书沿用)

$\text{height}(\text{空}) = 0, \text{height}(v) = 1 + \max(\text{height}(v.\text{left}), \text{height}(v.\text{right}))$ ——它同时是定义、递推式和代码,§4.2 的求高函数就是逐字翻译。

4.1.2 一般树的实现:从 first_child+next_sibling 到 vector 子结点容器

C 靠 firstChild + nextSibling 表达任意多的子结点;C++ 用 vector 保存——第3章 vector 思想在树上的延续:

C 的写法(first_child + next_sibling)	C++ 的写法(vector 子结点容器)
<pre>struct TreeNode { int val; struct TreeNode *firstChild, *nextSibling; };</pre>	<pre>struct TreeNode { int val; vector<TreeNode*> children; };</pre>
遍历子结点:沿 nextSibling 链 while 循环	遍历子结点:范围 for 一句
访问第 k 个子结点要数 k 次指针	children[k] 直接下标访问
插入子结点自己维护链接顺序	children.push_back(c) 一行
忘记 free 子树 = 泄漏	容器自动析构(结点对象仍需管理,见 §4.7)

示例 4-1 一般树结点:vector 子结点容器

```
struct TreeNode {
    int val;
    vector<TreeNode*> children;          // 子结点容器
    explicit TreeNode(int v) : val(v) {}
};

// 对照 C 的 nextSibling 链,范围 for 一句遍历
void printChildren(TreeNode* n) {
    for (TreeNode* c : n->children)
        cout << c->val << ' ';
}
```

4.1.3 一般树 vs 二叉树

二叉树是"每个结点至多两个子结点,且区分左右"的树——只有一个子结点也要说清在左在右。BST、表达式树、堆(第 6 章)都依赖"左/右"语义。一般树与二叉树可互相转换:"长子变左子、兄弟变右子"正是 C 的 first_child+next_sibling。

§4.2 二叉树与四种遍历

4.2.1 二叉树的定义与递归三序遍历

二叉树要么为空,要么由根 + 左子树 + 右子树构成——直接翻译成结点类:

示例 4-2 二叉树结点与前/中/后序递归遍历

```

struct BTreeNode {
    int val;
    BTreeNode* left;
    BTreeNode* right;
    BTreeNode(int v, BTreeNode* l = nullptr, BTreeNode* r = nullptr)
        : val(v), left(l), right(r) {}
};

void preorder(BTreeNode* n) {           // 前序:根 → 左 → 右
    if (n == nullptr) return;
    cout << n->val << ' ';
    preorder(n->left);
    preorder(n->right);
}

void inorder(BTreeNode* n) {           // 中序:左 → 根 → 右
    if (n == nullptr) return;
    inorder(n->left);
    cout << n->val << ' ';
    inorder(n->right);
}

void postorder(BTreeNode* n) {        // 后序:左 → 右 → 根
    if (n == nullptr) return;
    postorder(n->left);
    postorder(n->right);
    cout << n->val << ' ';
}

```

三个函数唯一区别是 `cout` 在两次递归调用的前、中、后——遍历的全部秘密。

例题 4-1 手走一棵二叉树的三种遍历

一棵 BST 形状的二叉树(内部结点 7/3/9,叶 1/6/11):



前序:7 3 1 5 6 9 11;中序:1 3 5 6 7 9 11;后序:1 6 5 3 11 9 7。自查:中序恰好升序,因为它是 BST(\$4.4)。

误区 1:把"访问"与"递归调用"的顺序搞混

打印语句在两次递归之间的位置决定遍历类型——三序唯一的区别就是这行 `cout` 放哪。先用最小树手推一遍再对照。

4.2.2 层序遍历(用队列,呼应第3章)

层序遍历按"从上到下、每层从左到右"访问。递归天然深入、做不到逐层横推,必须用第3章的队列:根先入队,出队访问后左右子结点再入队。

示例 4-3 层序遍历:std::queue

```

#include <queue>
void levelOrder(BTNode* root) {           // 层序:先进先出
    queue<BTNode*> q;
    q.push(root);
    while (!q.empty()) {
        BTNode* n = q.front();
        q.pop();
        if (n == nullptr) continue;
        cout << n->val << ' ';
        q.push(n->left);
        q.push(n->right);
    }
}

```

4.2.3 显式栈模拟递归(C++ 完整版)

任何递归都能改写成"显式栈 + 循环"(第3章)。系统栈压"返回地址 + 局部变量",显式栈压"待处理的结点",不受栈深度限制:

示例 4-4 迭代中序遍历:显式栈

```

void inorderIter(BTNode* root) {          // 显式栈模拟递归中序
    stack<BTNode*> st;
    BTNode* cur = root;
    while (cur != nullptr || !st.empty()) {
        while (cur != nullptr) {         // 1. 沿左链一路压栈
            st.push(cur);
            cur = cur->left;
        }
        cur = st.top();                   // 2. 出栈
        st.pop();
        cout << cur->val << ' ';          // 3. 访问发生在出栈时
        cur = cur->right;                  // 4. 转向右子树
    }
}

```

递归中序也是"先到最左、打印、再进右子树"——迭代版只是自己压栈。

4.2.4 遍历应用:求高度、结点数、叶数

三个经典问题都是"后序思想":先递归算完左右子树,再组合出答案。基准情形都是 `n == nullptr` 返回 0,空树基准让"单个叶结点"自然成立;写树递归先问:空结点返回什么?

示例 4-5 树高、结点数、叶数

```

int height(BTNode* n) { // 空树高度为 0
    if (n == nullptr) return 0;
    return 1 + max(height(n->left), height(n->right));
}
int countNodes(BTNode* n) { // 结点数
    if (n == nullptr) return 0;
    return 1 + countNodes(n->left) + countNodes(n->right);
}
int countLeaves(BTNode* n) { // 叶:左右都为空
    if (n == nullptr) return 0;
    if (n->left == nullptr && n->right == nullptr) return 1;
    return countLeaves(n->left) + countLeaves(n->right);
}

```

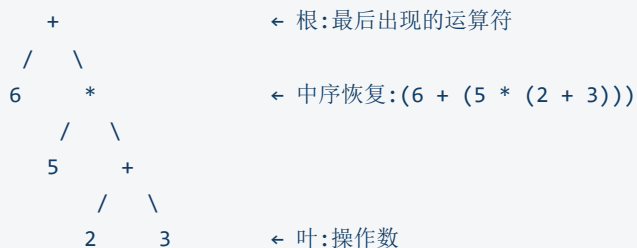
§4.3 表达式树

4.3.1 表达式树的结构

表达式树是编译器处理算术表达式的内部表示: **叶结点是操作数**, **内部结点是运算符**, 左右子树就是两个操作数——后缀式 `6 5 2 3 + * +` 对应:

图 4-2 后缀式 `6 5 2 3 + * +` 的表达式树

表达式树: 叶是操作数, 内部是运算符。



4.3.2 由后缀表达式构建(用栈, 衔接第3章)

构建与第3章后缀求值几乎同构: 遇数字建叶结点入栈; 遇运算符弹出两棵子树——**先弹的是右操作数**——合并压回。区别只在栈里存数值还是树结点:

示例 4-6 由后缀表达式构建表达式树


```

struct ExprNode {                                // 表达式树结点
    string data;                                // 数字或运算符
    ExprNode* left;
    ExprNode* right;
    ExprNode(const string& d, ExprNode* l = nullptr, ExprNode* r = nullptr)
        : data(d), left(l), right(r) {}
};

ExprNode* buildExprTree(const vector<string>& post) {
    stack<ExprNode*> st;
    for (const string& t : post) {
        if (isdigit(t[0]))
            st.push(new ExprNode(t)); // 数字:建叶结点入栈
        else {
            ExprNode* r = st.top(); st.pop(); // 先弹的是右操作数
            ExprNode* l = st.top(); st.pop();
            st.push(new ExprNode(t, l, r)); // 合并成父结点压回
        }
    }
    return st.top(); // 栈顶即根
}

```

⚠ 误区 2:左右子树接反

弹出时**先弹的是右操作数**。把 `r` 接成左子树, `6 5 -` 就会算成 `5-6`。写完用 `printInfix` 核对带减号的结果。

4.3.3 中序遍历恢复中缀表达式

三序遍历与三种表达式记法一一对应:前序 = 前缀式,中序 = 中缀式,后序 = 后缀式。中序遍历表达式树输出的正是人读的中缀;为优先级自洽,给每个运算符子树加括号:

示例 4-7 中序恢复中缀(带括号)

```

void printInfix(ExprNode* n) {                // 中序恢复中缀
    if (n == nullptr) return;
    bool isOp = !isdigit(n->data[0]);
    if (isOp) cout << '(';                  // 运算符子树加括号
    printInfix(n->left);
    cout << n->data;
    printInfix(n->right);
    if (isOp) cout << ')';
}

```

★ 重点:三序遍历 × 三种表达式 = 一个闭环

第3章"中缀转后缀"把 `(1+2)*3` 变 `12+3*`;本章"后缀建树 + 中序恢复"把它变回 `((1+2)*3)`。三种记法、三种遍历是同一信息的三副面孔。

§4.4 二叉查找树(BST)

4.4.1 BST 性质与 contains/findMin/findMax

二叉查找树(BST)性质: 每个结点的左子树所有值都小于它, 右子树所有值都大于它。推论: 中序遍历输出严格升序。查找 = 比较后走左或右, 一路下行丢掉一半子树——二分查找的树形版, 插入删除不用搬移元素:

示例 4-8 BST:contains/findMin/findMax

```
class BST {
    struct Node {
        int val;
        Node* left;
        Node* right;
        Node(int v, Node* l = nullptr, Node* r = nullptr)
            : val(v), left(l), right(r) {}
    };
    Node* root_ = nullptr;
public:
    bool contains(int x) const {           // 查找:沿一条路径下行
        Node* n = root_;
        while (n) {
            if (x == n->val) return true;
            n = (x < n->val) ? n->left : n->right;
        }
        return false;
    }
    int findMin() const {                  // 最小:一路向左
        Node* n = root_;
        while (n->left) n = n->left;
        return n->val;
    }
    int findMax() const {                  // 最大:一路向右
        Node* n = root_;
        while (n->right) n = n->right;
        return n->val;
    }
};
```

4.4.2 插入 insert

C 的做法是“遍历到空位, 改父结点指针”; C++ 更稳的是**递归返回“新根”**: 指针修改收拢在 `return` 里, 父结点用返回值接住:

示例 4-9 BST 插入:递归返回新根

```
void insert(int x) { root_ = insert(root_, x); }
private:
static Node* insert(Node* n, int x) {
    if (n == nullptr) return new Node(x);    // 空位:建新结点
    if (x < n->val) n->left = insert(n->left, x);
    else if (x > n->val) n->right = insert(n->right, x);
    return n;                                // 相等:忽略(不允许重复)
}
```

4.4.3 删除 remove:三种情形

删除三情形:① **叶**:直接删;② **单子**:让唯一子结点顶替;③ **双子**:用右子树的最小结点顶替被删结点,再删掉它——它大于左子树全部、小于右子树其余,且无左子树:

示例 4-10 BST 删除:三种情形

```
static Node* findMinNode(Node* n) {      // 子树中的最小结点
    while (n->left != nullptr) n = n->left;
    return n;
}
static Node* remove(Node* n, int x) {
    if (n == nullptr) return nullptr;
    if (x < n->val) n->left = remove(n->left, x);
    else if (x > n->val) n->right = remove(n->right, x);
    else {                                // 找到 x
        if (n->left == nullptr) return n->right; // 情形1+2a
        if (n->right == nullptr) return n->left;  // 情形2b
        Node* m = findMinNode(n->right);         // 情形3:双子
        n->val = m->val;                          // 复制数据顶替
        n->right = remove(n->right, m->val);      // 删掉最小结点
    }
    return n;
}
```

⚠ 误区 3:双子删除的两处经典翻车

① **漏接右子树**:只写 `n->val = m->val` 就 `return n, m` 还挂在右子树里——重复且泄漏,必须再递归删除 `m`。② **删错目标**:写成 `n->right = remove(n->right, x)` 必扑空。先复制,再用 `m` 删右子树。

⚠ 误区 4:删除后悬垂指针

调用方若写 `Node* p = root_; root_ = remove(root_, x); p->val = ...;`——`p` 可能已被删除,访问即未定义行为。删除/修改容器后,旧指针一律视为失效。

4.4.4 复杂度直觉与"手写 vs 标准库"

BST 每个操作沿一条路径下行,路径长度 = 树高。随机插入时树高约 $1.39 \log_2 n$,操作平均 $O(\log n)$;数据**恰好有序**插入时树退化成链,查找变线性扫描。

⚠ 误区 5:把"平均"当"保证"(退化链)

给 BST 喂升序序列,它长成一条只有右孩子的链,查 1 要 $O(n)$ 。这不是理论病:自增主键、日志时间戳等"按序到达"的数据都会触发。解法:① 打乱插入顺序;② 用自平衡结构——AVL (§4.5)、红黑树(`std::set` 底层,第12章)——把树高**保证**在 $O(\log n)$ 。

工程上需要"有序集合"时直接用标准库,手写 BST 只为理解原理:

示例 4-11 手写 BST 查找 vs `std::set`(并排)

```
// 手写 BST:平均  $O(\log n)$ ,最坏  $O(n)$ 
BST myTree;
myTree.insert(15); myTree.insert(6); myTree.insert(18);
bool a = myTree.contains(6);           // true

// 标准库:std::set 是红黑树(自平衡 BST),同样有序
set<int> st = {15, 6, 18};
bool b = st.count(6);                  // true, $O(\log n)$  且有保证
auto it = st.find(6);                  // 找不到时等于 st.end()
```

C 的手写查找树	C++ 的标准库 set/map
手写 Node 结构 + 递归插入/删除,还要管内存	set<int> / map<K,V> 直接使用,自动管理内存
有序插入退化成链,最坏 $O(n)$	红黑树保证 $O(\log n)$,旋转细节全封装
重复键约定要自己定	count() 返回 0/1,语义明确

§4.5 AVL 树

4.5.1 平衡因子与失衡类型

AVL 树是第一种自平衡 BST: 每个结点的左右子树高度差至多 1。**平衡因子** $bf(v) = height(v.left) - height(v.right)$, 合法值只有 -1、0、+1;插入可能把某祖先推到 ± 2 ,用**旋转**扳回,四种,看失衡结点的哪个孩子更高:

图 4-3 失衡判定:四种类型

$bf(v) = height(v.left) - height(v.right)$,合法取值只有 -1/0/1。

```

      k2
     /  \
    k1   Z
   /  \
  A   k3
   /  \
  B   C

```

← bf = +2, 失衡!

 ← k1 的右子树更高 → LR 型

LL:bf=+2 且左重;RR:bf=-2 且右重;LR:bf=+2 且右重;RL:bf=-2 且左重。

记忆法:LL 是"左左",失衡结点和高孩子都在左,一次右旋;LR 是"左右",先下后上。

4.5.2 四种旋转的实现

示例 4-12 AVL 四种旋转

```

int h(Node* n) { return n ? n->height : 0; }

// LL:右旋——左孩子提为根
Node* rotateWithLeftChild(Node* k2) {
    Node* k1 = k2->left;
    k2->left = k1->right;
    k1->right = k2;
    k2->height = 1 + max(h(k2->left), h(k2->right));
    k1->height = 1 + max(h(k1->left), h(k1->right));
    return k1;
}

// RR:左旋,镜像对称
Node* rotateWithRightChild(Node* k1) {
    Node* k2 = k1->right;
    k1->right = k2->left;
    k2->left = k1;
    k1->height = 1 + max(h(k1->left), h(k1->right));
    k2->height = 1 + max(h(k2->left), h(k2->right));
    return k2;
}

// LR:先左旋左孩子,再右旋自己
Node* doubleRotateLeftRight(Node* k3) {
    k3->left = rotateWithRightChild(k3->left);
    return rotateWithLeftChild(k3);
}

// RL:先右旋右孩子,再左旋自己
Node* doubleRotateRightLeft(Node* k1) {
    k1->right = rotateWithLeftChild(k1->right);
    return rotateWithRightChild(k1);
}

```

★ 重点:旋转的口诀

单旋转(LL/RR)是"把高孩子提上来当根,自己让位,多余的一支过继"——只改三个指针, $O(1)$;双旋转(LR/RL)是"先捋直再旋转"。**旋转不破坏 BST 的大小序**——中序遍历结果不变。

4.5.3 插入后的重平衡与复杂度对照

AVL 插入 = BST 插入 + 沿路径回溯:每层更新高度、检查平衡因子。插入只改变一条路径,第一个失衡点旋转一次即恢复平衡:

示例 4-13 AVL 插入:更新高度 + 失衡判定

```

Node* insert(Node* n, int x) {
    if (n == nullptr) return new Node(x);
    if (x < n->val)      n->left  = insert(n->left, x);
    else if (x > n->val) n->right = insert(n->right, x);
    n->height = 1 + max(h(n->left), h(n->right)); // 更新高度
    int bf = h(n->left) - h(n->right);          // 平衡因子
    if (bf > 1) {                               // 左子树高:LL 或 LR
        if (x < n->left->val) return rotateWithLeftChild(n); // LL
        return doubleRotateLeftRight(n);                  // LR
    }
    if (bf < -1) {                               // 右子树高:RR 或 RL
        if (x > n->right->val) return rotateWithRightChild(n); // RR
        return doubleRotateRightLeft(n);                   // RL
    }
    return n;                                         // 本结点仍平衡
}

```

判定 LL/LR:看 x 落在失衡结点左孩子的哪一侧, $x < n->left->val$ 是 LL,否则 LR。

例题 4-2(手走旋转):空树依次插入 3、2、1:结点 3 的 $bf=2$ 且落在左孩子 2 的左子树 $\rightarrow$ LL,右旋:2 提为根,3 变其右孩子;再插 4、5,结点 3 的 $bf=-2 \rightarrow$ RR。最终:根 2,左 1,右 4(左 3 右 5)。

朴素 BST	AVL 树
查找/插入/删除平均 $O(\log n)$	查找/插入/删除 保证 $O(\log n)$
有序输入退化成链,最坏 $O(n)$	任意输入序列,树高 $\leq 1.44 \log_2 n$
实现简单,无额外空间	每个结点多存一个高度(或平衡因子)
插入/删除后不检查平衡	插入/删除后沿路径旋转,常数更大

§4.6 伸展树与 B 树

4.6.1 伸展树:摊还与访问局部性

伸展树(splay tree)的动机是 **访问局部性:刚被访问的数据往往很快再次被访问** (缓存、词频查询、符号表)。AVL 用平衡因子保证每次 $O(\log n)$,代价是代码复杂;伸展树不要平衡因子,只约定:每次访问后把该结点"伸展"到根。

结论(证明在第11章摊还分析):单次最坏 $O(n)$,连续 m 次操作总代价 $O(m \log n)$, **摊还 $O(\log n)$** ;反复访问的结点总停在根附近,但单次最坏延迟无法保证。

4.6.2 B 树:磁盘 IO 与"浅而宽"

B 树的动机是磁盘:磁盘随机读约 10 ms,内存访问是纳秒级,差约 10 万倍,数据库索引里 **决定性能的不是比较次数,而是下潜层数(= IO 次数)**。BST 树高约 $\log_2 n$, 10^7 条记录约 23 层;B 树是"M 路分支",树高降到 $\log_M n$,取 $M=1000$ 只需 3 层——**"浅而宽"把 IO 从 20 多压到 3**。

结论:B 树查找/插入/删除都是 $O(\log_M n)$;分裂/合并细节不展开,工程上按"一个结点装满一个磁盘块"定 M 。SQLite 的 B+ 树等索引都是它的工业后代。

§4.7 工程与实践

4.7.1 用 `unique_ptr` 建树,告别手动 `delete`

裸指针树有**泄漏**、**双删**、**深树爆栈**三个问题。`unique_ptr` 一次解决:结点"拥有"子树,结点销毁时子树自动递归销毁:

示例 4-14 `unique_ptr` 树:析构自动递归

```
#include <memory>
struct BSTNode {
    int val;
    unique_ptr<BSTNode> left, right;    // 结点拥有子树
    explicit BSTNode(int v) : val(v) {}
};

unique_ptr<BSTNode> insert(unique_ptr<BSTNode> n, int x) {
    if (!n) return make_unique<BSTNode>(x);    // 空位:构造新结点
    if (x < n->val)    n->left = insert(move(n->left), x);
    else if (x > n->val) n->right = insert(move(n->right), x);
    return n;
}
// 根离开作用域:unique_ptr 依次销毁左右子树,递归自动完成
```

⚠ 误区 6:裸指针递归析构的坑

手写 `~BST() { delete root; }` 时析构是递归的:退化链递归 n 层直接撞栈上限;共享结点会 `double free`。用 `unique_ptr` 后:① 深树销毁仍可能爆栈,可改显式栈迭代销毁(§4.2.3);② 禁止拷贝,共享树换 `shared_ptr`。先想清楚"谁拥有这个结点",再决定用什么指针。

C 的 <code>malloc/free</code> 树	C++ 的 <code>unique_ptr</code> 树
<code>node = malloc(...)</code> ,用完 <code>free</code> ,漏一次就泄漏	<code>make_unique<Node>(...)</code> ,离开作用域自动释放
销毁整树要手写递归 <code>delete</code> ,深树爆栈、共享双删	析构自动递归;禁止拷贝从类型上排除双删
释放后指针仍存在,悬垂未定义行为	释放后 <code>unique_ptr</code> 自动置空, <code>!p</code> 判空安全

4.7.2 标准库对照:`std::set/map` 是红黑树

工程上"有序集合/有序映射"直接用标准库: `std::set`、`std::map` 底层都是**红黑树**——比 AVL 更宽松的自平衡 BST(旋转更少),操作保证 $O(\log n)$,迭代器按序输出。第12章细讲,先建立" `set/map` = 自平衡 BST,要字典序或范围查询就选它"的直觉。

示例 4-15 `std::map` 做词频统计

```

#include <map>
map<string, int> freq;           // 红黑树:key 有序,  $O(\log n)$ 
for (const string& w : words)
    ++freq[w];                  // 不存在时自动初始化为 0
for (const auto& p : freq)      // 遍历 = 中序 = 字典序
    cout << p.first << ": " << p.second << '\n';
int cnt = freq["the"];          // 查找  $O(\log n)$ 
freq.erase("dog");              // 删除  $O(\log n)$ 

```

4.7.3 调试技巧:打印树与验证性质

- **打印整棵树**:写带缩进的递归打印,比逐结点断点高效
- **中序必升序**:BST 的中序遍历必须严格升序——最便宜的全局检查
- **验证 AVL**:插入后断言全部结点 $|bf| \leq 1$ 并打印树高

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
查找	BST	$O(\log n)$	$O(n)$
插入	BST	$O(\log n)$	$O(n)$
删除	BST	$O(\log n)$	$O(n)$
查找最小 / 最大	BST	$O(\log n)$	$O(n)$
查找 / 插入 / 删除	AVL 树	$O(\log n)$	$O(\log n)$
前/中/后序遍历	二叉树	$O(n)$	$O(n)$
层序遍历	二叉树	$O(n)$	$O(n)$
求高度 / 结点数 / 叶数	二叉树(后序递归)	$O(n)$	$O(n)$
构建表达式树	后缀式 + 栈	$O(n)$	$O(n)$
恢复中缀表达式	表达式树中序	$O(n)$	$O(n)$
查找 / 插入 / 删除	伸展树	$O(\log n)$	$O(n)$ (单次)
查找 / 插入 / 删除	B 树	$O(\log_M n)$	$O(\log_M n)$
查找	有序数组(二分)	$O(\log n)$	$O(\log n)$
插入(有序数组中)	动态数组	$O(n)$	$O(n)$
按值查找	无序数组/链表	$O(n)$	$O(n)$
查找 / 插入 / 删除	set/map 红黑树	$O(\log n)$	$O(\log n)$

最小必会清单

- 能说出树的核心术语,解释深度与高度的数法
- 能默写 vector 子结点表示,对照 first_child+next_sibling
- 能写出前/中/后序遍历递归版,说清打印位置的作用

- 能写出层序遍历(队列版),解释为什么递归做不到层序
- 能写出求高度/结点数/叶数的递归函数,说明空树基准
- 能用栈从后缀构建表达式树(先弹右操作数),再中序恢复中缀
- 能默写 BST 的 contains/findMin/insert/remove,说清删除三情形
- 能用 unique_ptr 建树;能说出 std::set/map 底层是红黑树

自测题

Q 1. 一棵有 n 个结点的一般树有几条边?树的深度与高度有什么区别?

✓ 参考答案

$n-1$ 条:除根外每个结点都有一个父结点。深度从根往下(根 0),高度从叶往上(叶 0);树高 = 根的高度。

Q 2. 对例题 4-1 的树,写出四种遍历输出。

✓ 参考答案

前序:7 3 1 5 6 9 11;中序:1 3 5 6 7 9 11;后序:1 6 5 3 11 9 7;层序:7 3 9 1 5 11 6。后序 11 在 9 前:右子树先于根。

Q 3. 后缀式 $6\ 5\ 2\ -\ 3\ *\ +$ 的表达式树,中序恢复(带括号)是什么?

✓ 参考答案

构建:6、5、2 入栈;- 弹 2、5 得 $(5-2)$;3 入栈;* 得 $(5-2)*3$;+ 得 $6+((5-2)*3)$ 。先弹的是右操作数,故是 $5-2$ 而非 $2-5$ 。

Q 4. BST 删除双子结点时,为什么取右子树最小结点顶替?取左子树最大结点可以吗?

✓ 参考答案

右子树最小结点 m 大于左子树全部、小于右子树其余,顶替后性质不变;且 m 没有左子树,删除退化为单子/叶情形。取左子树最大结点也可行。

Q 5. 空树依次插入 5、3、7、2、4、6、8、1,哪一步触发旋转?什么类型?旋转后树根是谁?

✓ 参考答案

插入 1 后结点 5 的 $bf=+2$,1 在 5 的左孩子 3 的左子树 $\rightarrow$ LL,右旋:3 提为根,5 变其右孩子。旋转后:根 3,左 2(左 1 右 4),右 7(左 6 右 8)。

章末实验题:单词频次统计器(BST 实现)

实验 4:单词频次统计器(BST 实现)

实验目标:用自写 BST 实现命令行单词频次统计器:统计词频、查找、删除,并报告树高与平衡性。

实验要求:

- 任务 1:读入多行英文文本,把连续字母序列切分为单词并统一转小写(\$4.1 树结构、字符串遍历)
- 任务 2:用自写 BST 统计词频——结点存"单词 + 频次",已存在的单词频次 +1(\$4.4 insert)
- 任务 3:中序遍历按字典序输出词频表(\$4.2 中序遍历)
- 任务 4:查找指定单词频次;删除某单词后重新遍历,验证查找结果为 0(\$4.4 contains/remove 三情形)
- 任务 5(加分):平衡检查并报告树高(\$4.5 平衡因子、\$4.2 求高度)

实验环境:C++17。编译运行: `g++ -std=c++17 lab4.cpp -o lab4 && ./lab4`。

第一步 实现结点结构与 insert/find,验证频次正确后再补中序遍历——增量开发

第二步 除用示例 4-10 的三情形骨架;双子先复制右子树最小结点,再删除它(见误区 3)

第三步 点内存用 `unique_ptr` (示例 4-14),类里就不需要析构函数

✔ 参考答案(完整可运行程序,分三段)

```

// lab4.cpp — 单词频次统计器(BST 实现)(第一段:结点与核心操作)
#include <iostream>
#include <string>
#include <memory>
#include <cctype>
#include <algorithm>
using namespace std;

struct BSTNode {                                // 单词 + 频次 + 左右子树
    string word;
    int count;
    unique_ptr<BSTNode> left, right;
    explicit BSTNode(const string& w) : word(w), count(1) {}
};

class WordBST {
public:
    void insert(const string& w) { root_ = insert(move(root_), w); }
    int find(const string& w) const {                // 查找频次,不存在返回 0
        BSTNode* n = root_.get();
        while (n) {
            if (w == n->word) return n->count;
            n = (w < n->word) ? n->left.get() : n->right.get();
        }
        return 0;
    }
    void remove(const string& w) { root_ = remove(move(root_), w); }
    BSTNode* root() const { return root_.get(); }    // 供第二段辅助函数
private:
    unique_ptr<BSTNode> root_;
    static unique_ptr<BSTNode> insert(unique_ptr<BSTNode> n, const string& w) {
        if (!n) return make_unique<BSTNode>(w);    // 空位:新建结点
        if (w < n->word) n->left = insert(move(n->left), w);
        else if (w > n->word) n->right = insert(move(n->right), w);
        else ++n->count;                            // 已存在:频次 +1
        return n;
    }
    static BSTNode* findMin(BSTNode* n) {           // 右子树最小结点
        while (n->left) n = n->left.get();
        return n;
    }
    static unique_ptr<BSTNode> removeMin(unique_ptr<BSTNode> n) {
        if (!n->left) return move(n->right);
        n->left = removeMin(move(n->left));
        return n;
    }
    static unique_ptr<BSTNode> remove(unique_ptr<BSTNode> n, const string& w) {
        if (!n) return nullptr;                    // 单词不存在,忽略
        if (w < n->word) n->left = remove(move(n->left), w);
        else if (w > n->word) n->right = remove(move(n->right), w);
        else {                                       // 找到目标结点
            if (!n->left) return move(n->right);    // 叶或只有右子树
            if (!n->right) return move(n->left);    // 只有左子树
            BSTNode* m = findMin(n->right.get()); // 双子:取右子树最小
            n->word = m->word;                        // 复制数据顶替
            n->count = m->count;
        }
    }
};

```

```

        n->right = removeMin(move(n->right));
    }
    return n;
}
};

```

lab4.cpp(第二段:中序、树高与平衡辅助函数)

```

// (第二段:中序遍历、树高与平衡检查—辅助函数,配合 tree.root())
#include <utility>
void inorder(BSTNode* n) {           // 中序 = 字典序
    if (!n) return;
    inorder(n->left.get());
    cout << n->word << '\t' << n->count << '\n';
    inorder(n->right.get());
}
int height(BSTNode* n) {             // 空树高度为 0
    if (!n) return 0;
    return 1 + max(height(n->left.get()), height(n->right.get()));
}
pair<int, bool> check(BSTNode* n) {  // 返回(高度, 是否平衡)
    if (!n) return {0, true};
    auto [hl, bl] = check(n->left.get());
    auto [hr, br] = check(n->right.get());
    return {1 + max(hl, hr), bl && br && abs(hl - hr) <= 1};
}

```

lab4.cpp(第三段:主程序)

```

int main() {
    cout << "单词频次统计器(BST 实现)—输入英文文本,空行结束:\n";
    string line, text;
    while (getline(cin, line)) {
        if (line.empty()) break;
        text += line + '\n';
    }

    WordBST tree;
    string word;
    for (char c : text) {
        if (isalpha((unsigned char)c))
            word += (char)tolower((unsigned char)c);
        else if (!word.empty()) {
            tree.insert(word);
            word.clear();
        }
    }
    if (!word.empty()) tree.insert(word);

    cout << "\n--- 任务 3:中序遍历输出词频表(字典序)---\n";
    inorder(tree.root());

    cout << "\n--- 任务 4:查找频次 ---\n";
    for (const string& w : {"the", "dog", "hello"})
        cout << "查找 '" << w << "': 频次 = " << tree.find(w) << '\n';

    cout << "\n删除 'dog' 后重新遍历:\n";
    tree.remove("dog");
    cout << "删除后 'dog' 频次 = " << tree.find("dog") << "(应为 0)\n";
    inorder(tree.root());

    cout << "\n--- 任务 5(加分):树高与平衡检查 ---\n";
    cout << "树高 = " << height(tree.root())
        << ", 是否平衡 = " << (check(tree.root()).second ? "是" : "否") << '\n';
    return 0;
}

```

示例输入输出(输入文本:The quick brown fox jumps over the lazy dog and the dog barks at the cat while the cat sleeps on the warm mat):

单词频次统计器(BST 实现)—输入英文文本,空行结束:

--- 任务 3:中序遍历输出词频表(字典序)---

```
and      1
at       1
barks    1
brown    1
cat       2
dog       2
fox       1
jumps    1
lazy     1
mat       1
on        1
over     1
quick    1
sleeps   1
the       6
warm     1
while    1
```

--- 任务 4:查找频次 ---

```
查找 'the': 频次 = 6
查找 'dog': 频次 = 2
查找 'hello': 频次 = 0
```

删除 'dog' 后重新遍历:

删除后 'dog' 频次 = 0(应为 0)

```
and      1
at       1
barks    1
brown    1
cat       2
fox       1
jumps    1
lazy     1
mat       1
on        1
over     1
quick    1
sleeps   1
the       6
warm     1
while    1
```

--- 任务 5(加分):树高与平衡检查 ---

树高 = 9, 是否平衡 = 否

设计说明:① 任务覆盖本章主线:分词(\$4.1)、插入/删除三情形(\$4.4)、中序遍历(\$4.2)、平衡检查(\$4.5),串成可运行流水线;② 结点用 `unique_ptr` 持有子树,类无需析构函数,双子删除 `return move(n->left)` 即释放孤儿结点(\$4.7);③ 平衡检查是"返回(高度,是否平衡)"的自底向上函数,一次递归算两件事;实测树高 9,AVL 可压到约 6。

下一章预告:第5章 散列——把查找从 $O(\log n)$ 进一步加速到平均 $O(1)$;散列表的分离链接法将使用第3章的链表作为桶,而"何时用BST、何时用散列"的选型判断将贯穿第5章与后续工程实践。

第5章 散列

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础(已读完第 1-4 章)

🎯 本章学习目标

- 理解散列为什么能把查找从 $O(\log n)$ 压到平均 $O(1)$:散列函数把键映射为数组下标,理想情况下一步直达。
- 掌握整数取模与字符串 37 倍多项式两种散列函数,能说出"为什么表大小必须是素数、为什么要最后才取模"。
- 掌握分离链接法与开放定址法(线性探测、平方探测、双散列)的实现,理解装填因子 λ 与惰性删除。
- 掌握再散列的动机与实现,说出摊还代价 $O(1)$;了解可扩散列的基本思想。
- 学会用 `unordered_map` / `unordered_set` 解决问题,能给自定义类型写哈希与判等。
- 能独立实现词频统计程序(手写 + 标准库两条路线),用实验数据理解装填因子的影响。

💡 从 C 到 C++:本章主题如何迁移

数组按下标访问是 $O(1)$ 的,前提是下标是 0、1、2、... 这样的连续整数。可现实里的键往往不连续:学号是 2023001、2023050,订单号是一串字母数字。散列的想法很简单:写一个函数把"任意的键"翻译成"一个合理的下标",然后直接进数组。这个函数就叫 **散列函数**。

C 语言里你要自己写这个函数,并处理两个麻烦:负数取模会得到负下标(必须 `if (h < 0) h += M`),字符串散列时 `int` 容易溢出。到了 C++,麻烦基本消失:`unsigned long long` 累加几乎不可能溢出,标准库更是直接做好了"散列容器"。

本章主线是"先手写,再用库":§5.2 用 `vector` 和 `list` 拼出分离链接散列表——桶就是链表,是 C 语言"结构体 + 指针"的现代化版本;§5.7 的单词统计证明手写版与 `unordered_map` 输出完全一致。散列以空间换时间、以顺序换速度:查找快但遍历无序,与第 4 章二叉查找树正好相反。

📦 前置知识自查

数学前置:模运算(取余),重点是负数取模在 C/C++ 中是"截断"语义: `a % b` 结果可正可负,散列里必须处理。另一条是素数:只能被 1 和自身整除的正整数。

C 语言前置:指针遍历字符串、`struct` 与链表节点、`malloc/free`。本章的 C 对照会反复用到这三样。

依赖的前面章节:第 3 章链表与 `vector`——分离链接法的桶就是链表;再散列"翻倍扩容再重插"与 `vector` 扩容同构。

🚀 本章学习路线

1. **第一阶段:**读 §5.1-§5.2,吃透散列函数与分离链接法;手算教材 5.2 节小例子,对照示例 5-3 抄一遍代码。
2. **第二阶段:**读 §5.3-§5.5,理解三个探测方案的"聚集"问题与平方探测定理;读 §5.6 时敲一遍 `unordered_map` 示例。
3. **第三阶段:**完成章末实验,手写表与 `unordered_map` 双路线统计词频,看 λ 对性能的影响。

本章知识导图

- §5.1 基本想法与散列函数
 - 5.1.1 基本想法:查找的第三件武器
 - 5.1.2 整数键:取模散列与素数表大小
 - 5.1.3 字符串键:37 倍多项式散列
 - 5.1.4 装填因子 λ
- §5.2 分离链接法
 - 5.2.1 桶就是一条链表
 - 5.2.2 完整实现:查找、插入与删除
 - 5.2.3 复杂度:平均 $O(1)$ 从哪来
- §5.3 开放定址法
 - 5.3.1 线性探测与一次聚集
 - 5.3.2 平方探测
 - 5.3.3 双散列
 - 5.3.4 三种方案对比与惰性删除
- §5.4 再散列
 - 5.4.1 什么时候需要再散列
 - 5.4.2 实现:翻倍、取素数、重插
 - 5.4.3 摊还代价
- §5.5 可扩散列(选读)
- §5.6 标准库中的散列结构
 - 5.6.1 `unordered_map` 与 `unordered_set`
 - 5.6.2 自定义类型的哈希
 - 5.6.3 选型:`unordered_map` 还是 `map`
- §5.7 手写与标准库并排:单词统计

§5.1 基本想法与散列函数

5.1.1 基本想法:查找的第三件武器

你学过两种查找:顺序扫描(链表, $O(n)$)和二叉查找树(平均 $O(\log n)$)。散列是第三条路:把每个键换算成数组下标,查找就是一次数组访问,平均 $O(1)$ 。

表大小为 M ,散列函数 `hash(x)` 把键 x 映射到 0 到 $M-1$ 。插入: `table[hash(x)] = x`;查找:查 `table[hash(x)]` 里有没有 x 。难点在于散列函数"多对一":不同键可能映射到同一下标,这叫 **冲突**。散列的学问 = 好的散列函数(冲突少)+ 好的冲突解决策略。

记住 **散列三要素**:散列函数(键 $\rightarrow$ 下标)+ 冲突解决策略(§5.2 分离链接 / §5.3 开放定址)+ 表大小 M (尽量取素数)。任一环做差,性能都会从"平均 $O(1)$ "掉回"最坏 $O(n)$ "。

5.1.2 整数键:取模散列与素数表大小

键是整数时最常用的散列函数是取模: $\text{hash}(x) = x \% M$, 开销只有一次除法。关键是表大小 M 的选择: 若 M 是合数而键恰好都带某个公因子, 就会全挤进少数几个桶。比如 $M = 10$, 键 42、32、52、72 的个位全是 2, $x \% 10$ 全部等于 2——四键同桶, 九桶全空; 取素数 $M = 11$ 则余数 9、10、8、6, 四散而开。键的分布未知时, 素数表是唯一稳妥的选择。

⚠ 误区: 负数取模会得到负数下标

C/C++ 的 $\%$ 是截断除法: $-5 \% 7$ 等于 -5 而不是 2 。直接拿负值当数组下标会越界(未定义行为), 散列函数内部必须保证结果非负:

```
// 负数取模示例: C/C++ 的 % 是截断除法, 结果可正可负
int h = x % M;           // x = -5, M = 7 时 h = -5, 不是 2!
cout << h;              // 会打印 -5, 负数下标会越界
if (h < 0)
    h += M;              // 把负数修正到 [0, M-1]
return h;                // 现在可以放心当数组下标用了
// 更省心的惯用写法: unsigned(x) % M 直接消掉负号
// 两种写法殊途同归: 散列结果必须落在 [0, M-1]
```

C 的写法	C++ 的写法
<code>int h = x % M;</code> 负数得到负下标, 要手动 <code>if (h < 0) h += M;</code>	同样要处理; <code>unsigned(x) % M</code> 一行消掉负号, 或同样写 <code>if</code> 修正
<code>int</code> 累加散列容易溢出, 结果随机甚至变负	<code>unsigned long long</code> 累加, 64 位几乎不会溢出(见 5.1.3)

5.1.3 字符串键: 37 倍多项式散列

字符串是最常见的键(单词、文件名、URL)。教材采用 37 倍多项式法, 把字符串按霍纳法则逐位乘加, 每步 $h = h * 37 + c$ 。先给 C 回顾版, 再给 C++ 写法:

示例 5-1 C 回顾: 字符串散列 (37 倍多项式, 霍纳法则)

```
/* C 回顾: 字符串散列 */
int hashStrC(char* s, int M) {
    unsigned int h = 0;           /* unsigned 累加, 防止溢出成负数 */
    while (*s != '\0') {
        h = h * 37 + (unsigned char)*s; /* 每步乘 37 再加当前字符 */
        ++s;
    }
    return (int)(h % M);          /* 最后才取模, 结果落在 [0, M-1] */
}
```

示例 5-2 C++ 版: 同样的逻辑, 更短的代码

```
// C++ 版: 用 std::string 与范围 for, 逻辑与 C 完全对应
int hashStr(const string& s, int M) {
    unsigned long long h = 0;      // 64 位累加, 几乎不可能溢出
    for (char c : s)               // 逐个字符取码, 累乘累加
        h = h * 37 + (unsigned char)c;
    return int(h % M);             // 最后取一次模, 结果在 [0, M-1]
}
// 用法示例: hashStr("hello", 101) 返回 0..100 之间的整数
```

两句代码除类型与遍历方式外一字不差;"最后取一次模"几个字不是风格,是正确性——见下面的误区。

 **误区:逐字符取模会悄悄毁掉散列**

有人图省事,把取模写进循环:

```
// 错误示范:把取模写进循环,乘法被取模"吃掉"
for (char c : s)
    h = (h * 37 + (unsigned char)c) % M;    // 错误!
// 当 M = 37 时,(h*37+c)%37 = c%37,
// 散列退化成只看最后一个字符;
// 当 M = 36 时,37 ≡ 1 (mod 36),
// "abc" 与 "cba" 算出同一个散列值,
// 字符串变成了"字符和"。
```

两种退化同源:取模发生在乘法之前,丢掉了进位。正确写法永远是先累加、最后取一次模(示例 5-2)。

5.1.4 装填因子 λ

装填因子 $\lambda = \text{元素个数 } N / \text{表大小 } M$,衡量表"满"的程度; λ 越大冲突越多,查找越慢; λ 越小空位越多,浪费越大。分离链接的经验阈值 $\lambda \approx 0.75$ (再大就该扩容,见 §5.4);开放定址更苛刻,平方探测要求 $\lambda < 1/2$ 。

 **记号说明:λ 的两种记法**

正文用希腊字母 λ;代码里命名为 `loadFactor()` (或 `load_factor`),实现为 N / M 的浮点除法,两者是同一个量。

§5.2 分离链接法

5.2.1 桶就是一条链表

分离链接法(separate chaining)的思路最直观:每个下标放一条链表(教材图 5-1)。插入:算桶号挂到链表;查找:算桶号,链内线性扫描;删除:算桶号,链中摘除。 M 决定链表条数, $\lambda = N/M$ 就是平均链长。

C 的写法	C++ 的写法
<pre>struct Node { char* key; Node* next; }; 手写链表头插法,自己 malloc/free 每个节点</pre>	<code>vector<list<string>></code> :外层 vector 是桶数组,内层 list 是桶,内存自动管理
散列后用 <code>strcmp</code> 逐个比较	范围 for + <code>==</code> 直接比较 string

5.2.2 完整实现:查找、插入与删除

下面是完整 C++ 实现。`nextPrime(M)` 返回不小于 M 的最小素数(实现见章末实验),构造函数用它定素数表大小。

示例 5-3 分离链接散列表:find / insert / remove

```
// 分离链接散列表:桶数组 vector<list<string>>,一个桶就是一条链表
class HashTable {
    vector<list<string>> buckets_; // 桶数组:每个元素是一条链表
    int size_ = 0; // 表中元素总个数
public:
    explicit HashTable(int M)
        : buckets_(nextPrime(M)) {} // 表大小必须是素数(§5.1.2)

    int hash(const string& s) const { // 37 倍多项式散列
        unsigned long long h = 0;
        for (char c : s)
            h = h * 37 + (unsigned char)c;
        return int(h % buckets_.size());
    }

    bool find(const string& s) const { // 查找:先定桶,再在链内找
        for (const string& x : buckets_[hash(s)])
            if (x == s) return true;
        return false;
    }

    void insert(const string& s) { // 插入:已存在则跳过
        if (find(s)) return;
        buckets_[hash(s)].push_back(s); // 挂到对应桶的链表尾
        ++size_;
    }

    void remove(const string& s) { // 删除:定位桶后链内删除
        list<string>& b = buckets_[hash(s)];
        for (auto it = b.begin(); it != b.end(); ++it)
            if (*it == s) { b.erase(it); --size_; return; }
    }

    int size() const { return size_; }
    double loadFactor() const { // 装填因子  $\lambda$  = 元素数/表大小
        return double(size_) / buckets_.size();
    }
};
```

三个操作同一套路:先 `hash(s)` 定桶,再只动那一条链——散列把"全表扫描"降为"单桶扫描",单桶平均只有 λ 个元素。
`insert` 先查重再 `push_back` (散列两次;教材图 5-6 的版本合并为一次);`remove` 定位后 `erase(it)` 摘除。

⚠ 误区:erase 之后必须立刻返回,否则迭代器失效

`list::erase(it)` 会令被删位置的迭代器失效。上面 `remove` 里 `erase(it)` 后立刻 `return` 就没事;若删除后继续用 `it`(如 `++it`),就是未定义行为——C++ 新手最常见的散列表 bug。vector 的 `erase` 更激进:被删位置之后的迭代器全部失效。

5.2.3 复杂度:平均 $O(1)$ 从哪来

一次 `find` 的代价 = 散列 $O(\text{键长 } L)$ + 单桶扫描 $O(\text{平均链长}) = O(1 + \lambda)$ 。 λ 控制在 0.75 左右时,常数 $1 + \lambda$ 很小,于是散列表查找 **平均 $O(1)$** ;插入、删除同理。

牢记散列表的复杂度画像:查找/插入/删除平均 $O(1)$ 、最坏 $O(n)$ 。最坏情况是所有键散列到同一桶,链表退化成一条长链——散列函数均匀且表大小是素数时几乎不会发生,但你必须能说出它为什么是 $O(n)$ 。代价是牺牲顺序性:无法按键有序遍历、没有前驱后继;要顺序时回到第 4 章的二叉查找树。

§5.3 开放定址法

分离链接需要额外指针,浪费空间。开放定址法(open addressing)让所有元素都住进表本身:冲突时按某种规则在表中找下一个空位。规则不同,衍生出三种方案。

5.3.1 线性探测与一次聚集

线性探测:冲突时逐个向后试 $pos = (pos + 1) \% M$,直到空位或绕回。它有个著名毛病——一次聚集:连续被占的单元越滚越长,新键插进去又让它更长,形成“雪球”。

示例 5-4 线性探测插入

```
// 开放定址:线性探测插入(空单元用 EMPTY 标记)
int pos = hashStr(s, M);
while (table[pos] != EMPTY && table[pos] != s)
    pos = (pos + 1) % M;    // 冲突就向后挪一格,越界绕回 0
if (table[pos] == EMPTY)
    table[pos] = s;        // 找到空位,插入
else
    ;                      // 键已存在,不做任何事
```

手算一遍: $M = 7$,按序插入 7、15、8、1($h(x) = x \bmod 7$): $7 \rightarrow 0$; $15 \rightarrow 1$; $8 \rightarrow 1$ 冲突试 2,空,落位; $1 \rightarrow 1$ 冲突试 2 仍冲突,试 3,落位。最终下标 0-3 依次是 7、15、8、1:一次聚集——三键挤成一团,后面的键要从 1 一路试到 4。

5.3.2 平方探测

平方探测把步长改成平方:第 i 次冲突后尝试 $pos + i^2$,探测序列从一开始就跳开,避免连续占用。

示例 5-5 平方探测插入

```
// 平方探测:第 i 次冲突后尝试 pos + i^2
int pos = hashStr(s, M);
int i = 0;
while (table[pos] != EMPTY && table[pos] != s) {
    ++i;
    pos = (pos + i * i) % M;    // 探测序列:pos+1, pos+4, pos+9, ...
}
if (table[pos] == EMPTY)
    table[pos] = s;
// 定理(教材 5.4 节):表大小为素数且装填因子  $\lambda < 1/2$  时,
// 平方探测总能找到空单元,且最多探测一半的表长
```

教材定理:表大小是素数且 $\lambda < 1/2$ 时,平方探测总能在最多一半的表长内找到空单元(证明要点:前半段探测位置互不相同)。工程含义: λ 必须死守在半表以下,因此要更早再散列。它消灭了一次聚集,但留下“二次聚集”:同位置的键探测序列完全相同。

5.3.3 双散列

双散列连二次聚集也消灭:步长由第二个散列函数算出,每个键拥有专属探测节奏。常见选择:步长 = $M_2 - (h(x) \bmod M_2)$, M_2 取比 M 小的素数。

示例 5-6 双散列插入

```
// 双散列:冲突时以第二个散列函数的值做步长
int h1 = hashStr(s, M);           // 主散列:决定初始位置
int step = hashStr2(s, M);        // 第二散列:决定步长,注意不能为 0
int pos = h1;
while (table[pos] != EMPTY && table[pos] != s)
    pos = (pos + step) % M;        // 每次前进 step 格
if (table[pos] == EMPTY)
    table[pos] = s;
```

5.3.4 三种方案对比与惰性删除

方案	探测序列	代价
线性探测	顺次 +1	一次聚集, λ 高时退化严重;实现最简单
平方探测	+1, +4, +9, ...	无一次聚集,有二次聚集;需素数表且 $\lambda < 1/2$
双散列	步长由第二个散列决定	无聚集,但要多算一次散列函数

⚠ 误区:开放定址表不能物理删除

线性探测里删掉一个元素若直接置空,会把探测链拦腰截断:后续依赖它"借道"的元素从此再也找不到。教材做法是**惰性删除**:标记 deleted,查找跳过、插入复用、定期再散列清标记。注意 deleted 单元不算"已占用",装填因子计算要把它们排除。

§5.4 再散列

5.4.1 什么时候需要再散列

装填因子涨过阈值后冲突率陡增,平均链长(或探测次数)失控。唯一的办法是**再散列**:表扩容(通常翻倍取素数),旧元素用新表大小重新散列。插入时检查 λ , 超阈值触发。工程阈值:分离链接约 0.75, 开放定址约 0.5。

5.4.2 实现:翻倍、取素数、重插

示例 5-7 再散列:给分离链接表扩容

```
// 再散列:装填因子超过阈值就把表翻倍并重新插入全部元素
void rehash() {
    if (loadFactor() < 0.75) return;           //  $\lambda$  未超标则不做
    int oldM = int(buckets_.size());
    vector<list<string>> oldBuckets = buckets_; // 先备份旧桶
    buckets_.assign(nextPrime(oldM * 2), {});   // 新表:翻倍后取素数
    size_ = 0;
    for (auto& chain : oldBuckets)              // 重新插入每个旧元素
        for (const string& x : chain)
            insert(x);                          // 会用新表大小重算散列
}
```


最后是"重新插入"而非"搬过去":桶号 = $\text{hash}(x) \bmod \text{新表大小}$,每个元素都得重算——"再散列"之名由此而来。

5.4.3 摊还代价

单次再散列 $O(n)$,但把账摊到 n 次插入上:每次扩容后空间翻倍,平均下来 每次插入摊还 $O(1)$ 。这与第 3 章 `vector` 扩容的摊还分析同构,区别是散列表翻倍后元素必须重插(不能拷贝)。

实现建议:再散列是"懒"的,只在需要时触发

教材每次插入后检查 λ ,超标才触发;也有按"插入次数达到 $n/2$ "预约的实现。关键是别等 λ 飙到 0.9 才扩容——那时聚集已形成,再散列也救不回。标准库 `unordered_map` 自动再散列,可用 `reserve()` 预分配桶数,避开插入高峰期扩容。

§5.5 可扩散列(选读)

5.5.1 散列与磁盘的冲突

前面所有方案都假设表在内存。数据量大到必须存磁盘时,一次磁盘 I/O 是内存访问的几十万倍,而散列函数均匀分布意味着每次查找都可能访问不同页,随机 I/O 会让性能崩溃。可扩散列(extendible hashing)专治"键太多,一页装不下"。

5.5.2 目录、页与局部分裂

可扩散列分两层:内存里的 目录 (directory,大小为 2^d 的数组)和一组磁盘 页 (page,一页容纳若干键)。查找:取散列值的前 d 位定位目录项,读目录项指向的页。插入时若页满就分裂:满页一分为二,目录规模翻倍(d 加一)。只有满的那一页被拆,其它页原封不动,这叫 局部分裂 ——表随数据增长逐页扩张,而非整体重建。

对照再散列:整表翻倍、全部重插(摊还 $O(1)$);可扩散列只分裂满页 + 目录翻倍,其它页不动,且只有一页 I/O。代价:目录占内存,可能有两页指向同一目录区段(浪费)。数据库与文件系统的索引设计正是这一思想的生产级版本。

§5.6 标准库中的散列结构

5.6.1 `unordered_map` 与 `unordered_set`

C++11 起标准库提供散列容器: `unordered_map<K, V>` (键值对)与 `unordered_set<K>` (键集合)。unordered 提醒你:遍历无顺序。底层就是本章讲的分离链接散列表(链表桶),平均 $O(1)$,自动再散列。

示例 5-8 `unordered_map` 基本用法


```
// 标准库:unordered_map(散列)与 map(红黑树)对单词计数
#include <unordered_map>
unordered_map<string, int> cnt;
for (const string& w : words)
    ++cnt[w];           // w 不存在时自动插入,次数从 0 加 1

// 遍历:按散列顺序,无任何顺序保证
for (auto& kv : cnt)
    cout << kv.first << ": " << kv.second << "\n";

// 三种查找姿势
auto it = cnt.find("hello");    // 返回迭代器;找不到返回 end()
if (it != cnt.end())
    cout << it->first << ": " << it->second << "\n";
if (cnt.count("hello"))        // 只想知道在不在:count() 返回 0 或 1
    cout << cnt.at("hello");    // at():找不到会抛 out_of_range 异常
```

最容易错的是 `operator[]` 的副作用: `cnt[w]` 在键不存在时会 **自动插入默认值**,只读查询请用 `find` / `count` / `at`,否则表里会冒出"幽灵键"。

5.6.2 自定义类型的哈希

`unordered` 容器要求键可判等(==)且可哈希。内置类型(int、string 等)标准库备好了散列;自定义类型要自己写一个函数对象作为第二个模板参数,这是教材推荐的做法:

示例 5-9 给自定义类型提供哈希与判等

```
// 自定义类型进散列表:标准库要求 可哈希 + 可判等
struct Student {
    string name;
    int id;
    bool operator==(const Student& o) const {    // 判等:成员全部相等
        return name == o.name && id == o.id;
    }
};

struct StudentHash {                            // 函数对象:散列函数
    size_t operator()(const Student& s) const {
        return hash<string>()(s.name) ^ hash<int>()(s.id);
    }
};

unordered_set<Student, StudentHash> roster;      // 用法:两个模板参数
```

⚠ 误区:哈希与判等缺一不可

只写 `operator==` 不写哈希结构, `unordered_set<Student>` 编译失败;只写哈希不写判等,报错变成"没有匹配的 `operator==`". 标准:相等对象散列值必相等,不等对象尽量分散(异或组合对简单结构足够)。

5.6.3 选型:unordered_map 还是 map

`map` 底层是红黑树,查找 $O(\log n)$ 但按键有序;`unordered_map` 平均 $O(1)$ 但无序。选型口诀:

需求	选型	原因
只要按键快速查值,不关心顺序	unordered_map	平均 $O(1)$,快一个数量级
需要按键有序遍历、范围查询	map	有序,且最坏 $O(\log n)$ 有保证
自定义类型做键	map(省事)或 unordered_map + 哈希	map 只要 <code>operator<</code> ,unordered_map 要两个

实例:按时间戳顺序输出 10 万条日志 → map(有序遍历);按用户 ID 秒查余额 → unordered_map(只求快)。两者共存很常见。

§5.7 手写与标准库并排:单词统计

5.7.1 需求:统计文章词频

统计一篇文章里每个单词的出现次数("the 出现 1800 次"之类)。这是散列表的教科书级应用,也是章末实验的前身。同一需求,两条路线各写一遍,并排对比。

5.7.2 手写版实现

示例 5-10 手写版:桶内存 (单词, 次数) 对

```

// 手写版:桶内放 (单词, 次数),统计每个单词的出现次数
class WordCounter {
    vector<list<pair<string, int>>> buckets_; // 桶数组
    int distinct_ = 0; // 不同单词个数
public:
    explicit WordCounter(int m) : buckets_(nextPrime(m)) {}

    void add(const string& w) { // 出现一次就计数一次
        auto& b = buckets_[hashStr(w, int(buckets_.size()))];
        for (auto& kv : b)
            if (kv.first == w) { ++kv.second; return; } // 已存在:+1
        b.push_back({w, 1}); // 新词:插到链表尾
        ++distinct_;
    }

    int count(const string& w) const { // 查询词频,不存在返回 0
        auto& b = buckets_[hashStr(w, int(buckets_.size()))];
        for (auto& kv : b)
            if (kv.first == w) return kv.second;
        return 0;
    }

    int distinct() const { return distinct_; }
};

int main() {
    WordCounter wc(16); // 表大小 16 → 素数 17
    string text = "the cat and the dog and the cat";
    string cur;
    for (char c : text) { // 切词:非字母数字当分隔
        if (isalnum((unsigned char)c))
            cur += (char)tolower((unsigned char)c);
        else if (!cur.empty()) { wc.add(cur); cur.clear(); }
    }
    if (!cur.empty()) wc.add(cur);
    cout << "the: " << wc.count("the") << "\n"; // 3
    cout << "cat: " << wc.count("cat") << "\n"; // 2
    cout << "不同单词: " << wc.distinct() << "\n"; // 4
    return 0;
}

```

5.7.3 标准库版实现

示例 5-11 标准库版:同一需求,代码量减到 1/3

```
// 标准库版:unordered_map 一行完成"不存在则插入"
#include <unordered_map>
int main() {
    string text = "the cat and the dog and the cat";
    unordered_map<string, int> cnt;
    for (char c : text) { // 切词与手写版一致
        if (isalnum((unsigned char)c)) cur += (char)tolower((unsigned char)c);
        else if (!cur.empty()) { ++cnt[cur]; cur.clear(); }
    }
    if (!cur.empty()) ++cnt[cur];
    cout << "the: " << cnt["the"] << "\n"; // 3
    cout << "cat: " << cnt["cat"] << "\n"; // 2
    cout << "不同单词: " << cnt.size() << "\n"; // 4
    return 0;
}
```

5.7.4 对照结论

C 的写法	C++ 的写法
结构体数组存 (word, cnt),每次查找线性扫描 $O(n)$	<code>unordered_map<string, int>:++cnt[w]</code> ,平均 $O(1)$
要自己写字符串比较、手动扩容、手动释放	string 自带 <code>==</code> ,自动再散列,RAII 自动释放

调试技巧:手写版与库版互相验算

手写数据结构最常见的 bug 是"错得整齐":两处实现犯同一个错,测试却全绿。对策是章末实验的做法——同一份数据喂给手写表和 `unordered_map`,抽查若干键的词频必须一致。这个"双实现互验"的习惯,写红黑树、堆时都用得上。

本章复杂度速查表

口诀:查找/插入/删除平均 $O(1)$,最坏 $O(n)$;代价是放弃按键顺序。以下结论两版通用。

操作	数据结构/算法	平均	最坏
查找	散列表(分离链接)	$O(1)$	$O(n)$
插入	散列表(分离链接)	$O(1)$	$O(n)$
删除	散列表(分离链接)	$O(1)$	$O(n)$
查找	散列表(开放定址)	$O(1)$	$O(n)$
插入	散列表(开放定址)	$O(1)$	$O(n)$
删除(惰性)	散列表(开放定址)	$O(1)$	$O(n)$
散列函数	整数取模	$O(1)$	—
散列函数	字符串 37 倍多项式(键长 L)	$O(L)$	—
再散列	单次再散列(翻倍重插)	$O(n)$	$O(n)$
连续插入	含再散列的 n 次插入	摊还 $O(1)$ /次	$O(n)$ /次
查找/插入/删除	unordered_map / unordered_set	$O(1)$	$O(n)$
查找/插入/删除	map / set(红黑树)	$O(\log n)$	$O(\log n)$
按键有序遍历	map / set	$O(n)$	$O(n)$
探测次数	线性/平方探测, $\lambda \approx 0.5$	≈ 1.5 次	$O(n)$
词频统计	手写散列表 / unordered_map	$O(n)$	$O(n^2)$

最小必会清单

- 能说出散列解决什么问题:键映射为下标,查找压到平均 $O(1)$,代价是放弃顺序性。
- 能手写整数取模散列,并处理负数取模的 C++ 截断语义(修正到非负)。
- 能手写 37 倍多项式字符串散列,并解释"为什么先累加、最后取模"。
- 能说出表大小必须取素数,并给出合数表大小的反例。
- 能说出装填因子 $\lambda = N/M$ 的含义与经验阈值(分离链接 ≈ 0.75 ,平方探测 < 0.5)。
- 能实现分离链接散列表的 find / insert / remove,能指出 erase 后迭代器失效的坑。
- 能说出开放定址三种方案的探测序列、聚集现象与适用条件。
- 能解释惰性删除的必要性(物理删除会切断探测链)。
- 能实现再散列,说出"每次插入摊还 $O(1)$ "与 vector 扩容的同构性。
- 能用 unordered_map / unordered_set 解决词频统计等问题,注意 operator[] 的自动插入副作用。
- 能给自定义类型写哈希与 operator==(两个模板参数),并在 unordered_map 与 map 之间选型。

自测题

Q 1. 表大小 $M = 7$,散列函数 $h(x) = x \bmod 7$,用分离链接法依次插入 13、27、41、55。画出各桶的链表,并计算装填因子 λ 与平均链长。

✓ 参考答案

13、27、41、55 除以 7 的余数全是 6——四键全落 6 号桶(最坏情况的构造方式)。 $\lambda = 4/7 \approx 0.57$,平均链长 4/7,但 6 号桶链长 4。

Q 3. 逐字符取模与最后取模,在 $M = 37$ 和 $M = 36$ 时各会怎样退化?

✓ 参考答案

$M = 37$: $(h \cdot 37 + c) \bmod 37 = c \bmod 37$,退化为"只看最后一个字符"; $M = 36$: $37 \equiv 1 \pmod{36}$,每个字符权重都是 1,"abc" 与 "cba" 同值。根治:先累加、最后取一次模。

Q 4. 表大小 $M = 11$,线性探测依次插入 3、14、25、8、19,给出最终表内容与探测过程。

✓ 参考答案

$3 \rightarrow 3$; $14 \rightarrow 3$ 冲突 $\rightarrow 4$; $25 \rightarrow 3$ 冲突 $\rightarrow 4$ 冲突 $\rightarrow 5$; $8 \rightarrow 8$; $19 \rightarrow 8$ 冲突 $\rightarrow 9$ 。最终下标 3、4、5、8、9 依次为 3、14、25、8、19。14、25 挤成连续块,正是"一次聚集"。

Q 5. 开放定址表里删除元素为什么不能直接置空?工程上怎么处理?

✓ 参考答案

直接置空会把探测链拦腰截断:后续键靠被删元素"借道",探测序列在此断掉,再也找不到。工程做法是惰性删除:标记 deleted,查找跳过、插入复用、定期再散列清标记。

Q 6. 统计 100 万条记录:需求 A 按 key 升序输出,需求 B 只按 key 快速查值。各选 map 还是 unordered_map? 为什么?

✓ 参考答案

需求 A 用 map(红黑树,按键有序遍历 $O(n)$);需求 B 用 unordered_map(平均 $O(1)$ 查找)。口诀:要顺序选树,只要速度选散列。

章末实验题:散列表单词统计对比实验

实验 5:散列表单词统计对比实验

实验目标:手写分离链接散列表,与 `unordered_map` 双路线统计英文文本词频,用计时数据验证装填因子 λ 对性能的影响。

实验要求:

- 任务 1:实现分离链接散列表(键为字符串):37 倍多项式散列、素数表大小、add/count 操作(覆盖 §5.1 散列函数与素数表、§5.2 分离链接法)。
- 任务 2:用自写的表统计文本,输出单词总数、不同单词个数、装填因子 λ (覆盖 §5.1.4 装填因子)。
- 任务 3:用 `unordered_map` 重做统计,抽查 "the"、"hash"、"hello" 三个词,两套结果必须一致(覆盖 §5.6 标准库;兼作手写版正确性验证)。
- 任务 4:重复插入 30 遍,改变表大小 $M = 8/16/64/256$ (实际素数 $11/17/67/257$),每种测 3 次取最快;观察 λ 越大耗时越高,体会再散列的动机(覆盖 §5.2.3 复杂度、§5.4 再散列)。
- 任务 5:健壮性——无命令行参数、文件打不开、文本为空都要给出友好提示(覆盖边界处理与工程习惯)。

实验环境:C++17。编译:`g++ -std=c++17 lab5.cpp -o lab5`;运行:`./lab5 sample.txt`(任意英文文本,词越多对比越明显)。

第一步 写工具函数:`isPrime/nextPrime` 定素数表大小;`splitWords` 切词(字母数字连续成词,转小写,其余字符当分隔)。先用 3-5 个单词手算验证散列。

第二步 写 `HashTable`:构造函数用 `nextPrime` 定桶数;hash 用"先累加、最后取模";add 找到键计数 +1,否则插入新键。写完后用小文本对照字典检查词频。

第三步 后写计时:任务 4 对每个 M 重复 3 次取最快,插入重复 30 遍放大差异;跑前先确认 λ 合理(如 208 个不同单词、 $M=257$ 时 $\lambda \approx 0.81$),再做比较。

✅ 参考答案(完整可运行程序,分三段)

实验 5-1 参考答案(第一部分:工具函数与切词)

```
// lab5.cpp — 散列表单词统计对比实验(第一部分:工具函数)
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <list>
#include <unordered_map>
#include <chrono>
#include <cctype>
using namespace std;
using Clock = chrono::steady_clock;

// ----- 工具:素数判断与下一个素数 -----
bool isPrime(int n) {
    if (n < 2) return false;
    for (int d = 2; d * d <= n; ++d)
        if (n % d == 0) return false;
    return true;
}
int nextPrime(int n) {
    while (!isPrime(n)) ++n;
    return n;
}

// 任务 5:切分单词—字母/数字连续成词并转小写,其余字符跳过
vector<string> splitWords(const string& text) {
    vector<string> words;
    string cur;
    for (char c : text) {
        if (isalnum((unsigned char)c)) {
            cur += (char)tolower((unsigned char)c);
        } else if (!cur.empty()) {
            words.push_back(cur);
            cur.clear();
        }
    }
    if (!cur.empty()) words.push_back(cur);
    return words;
}
```

实验 5-1 参考答案(第二部分:手写分离链接散列表)


```
// ----- 任务 1:分离链接散列表(字符串键,37 倍散列,素数表) -----
class HashTable {
    struct Entry { string key; int cnt; };
    vector<list<Entry>> buckets_; // 桶数组:vector<list<...>>
    int size_ = 0; // 不同单词个数
public:
    explicit HashTable(int m) : buckets_(nextPrime(m)) {}

    int hash(const string& s) const {
        unsigned long long h = 0; // 全程不取模,最后再 % 表大小(见 §5.1.3)
        for (char c : s)
            h = h * 37 + (unsigned char)c;
        return int(h % buckets_.size());
    }

    void add(const string& s) { // 计数 +1,首次出现则插入
        int b = hash(s);
        for (auto& e : buckets_[b])
            if (e.key == s) { ++e.cnt; return; }
        buckets_[b].push_back({s, 1});
        ++size_;
    }

    int count(const string& s) const { // 查询词频,不存在返回 0
        int b = hash(s);
        for (const auto& e : buckets_[b])
            if (e.key == s) return e.cnt;
        return 0;
    }

    int distinct() const { return size_; }
    int bucketCount() const { return int(buckets_.size()); }
    double loadFactor() const { // 装填因子  $\lambda$  = 元素数/表大小
        return double(size_) / buckets_.size();
    }
};
```

实验 5-1 参考答案(第三部分:计时、main 与四种表大小对比)

```

// ----- 计时工具:插入 repeat 遍,放大差异 -----
double msInsert(HashTable& ht, const vector<string>& words, int repeat) {
    auto t0 = Clock::now();
    for (int r = 0; r < repeat; ++r)
        for (const string& w : words) ht.add(w);
    return chrono::duration<double, milli>(Clock::now() - t0).count();
}

int main(int argc, char* argv[]) {
    if (argc < 2) { cout << "用法: lab5 <文本文件>\n"; return 1; }
    ifstream fin(argv[1]);
    if (!fin) { cout << "错误: 无法打开文件 " << argv[1] << "\n"; return 1; }
    stringstream ss;
    ss << fin.rdbuf();
    vector<string> words = splitWords(ss.str());
    if (words.empty()) { // 任务 5:空文本
        cout << "警告: 文本为空或没有合法单词\n";
        return 0;
    }
    cout << "单词总数(含重复): " << words.size() << "\n";

    // 任务 2:手写表统计
    HashTable ht(256);
    msInsert(ht, words, 1);
    cout << "手写表: 不同单词 " << ht.distinct()
        << " 个,装填因子  $\lambda$ =" << ht.loadFactor() << "\n";

    // 任务 3:标准库重做,抽查输出一致性
    unordered_map<string, int> mp;
    for (const string& w : words) ++mp[w];
    cout << "unordered_map: 不同单词 " << mp.size() << " 个\n";
    for (const char* w : {"the", "hash", "hello"})
        cout << " 词 \"" << w << "\": 手写表=" << ht.count(w)
            << " map=" << mp[w] << "\n";

    // 任务 4:不同  $\lambda$  下的插入耗时(表大小不同  $\rightarrow \lambda$  不同,3 次取最快)
    cout << "任务4: 表大小(素数)不同  $\rightarrow \lambda$  不同  $\rightarrow$  插入耗时(3 次取最快):\n";
    for (int m : {8, 16, 64, 256}) {
        double best = 1e300;
        for (int trial = 0; trial < 3; ++trial) { // 重复 3 次取最快
            HashTable t(m);
            double ms = msInsert(t, words, 30);
            if (ms < best) best = ms;
        }
        HashTable t(m);
        msInsert(t, words, 1);
        cout << " M=" << t.bucketCount() << "  $\lambda$ =" << t.loadFactor()
            << " 耗时 " << best << " ms\n";
    }
    cout << "观察:  $\lambda$  越大平均链表越长,耗时越高——这就是再散列(§5.4)的动机.\n";
    return 0;
}

```

编译运行命令与真实输出(sample.txt 为 7 段文章拼成,17520 词、208 个不同单词):

实验 5-1 输出样例(本机实测,数值随机器与负载略有波动,趋势稳定)

```
$ g++ -std=c++17 -O2 lab5.cpp -o lab5 && ./lab5 sample.txt
```

单词总数(含重复): 17520

手写表: 不同单词 208 个, 装填因子 $\lambda=0.809339$

unordered_map: 不同单词 208 个

词 "the": 手写表=1800 map=1800

词 "hash": 手写表=120 map=120

词 "hello": 手写表=120 map=120

任务4: 表大小(素数)不同 $\rightarrow \lambda$ 不同 $\rightarrow$ 插入耗时(3 次取最快):

M=11 $\lambda=18.9091$ 耗时 5.8237 ms

M=17 $\lambda=12.2353$ 耗时 4.59 ms

M=67 $\lambda=3.10448$ 耗时 3.3488 ms

M=257 $\lambda=0.809339$ 耗时 3.0471 ms

观察: λ 越大平均链表越长, 耗时越高——这就是再散列(\$5.4)的动机。

```
$ ./lab5 empty.txt
```

警告: 文本为空或没有合法单词

设计说明:程序分四层,每层对应一个知识点。散列函数写成"先累加、最后取模"的 37 倍多项式(\$5.1.3),表大小交给 nextPrime(\$5.1.2)——本章最易写错的两处,用真实数据检验了正确性。桶用 `vector<list<Entry>>` 实现分离链接(\$5.2),Entry 携带键与计数,一次遍历完成"查重或累加"。任务 3 的抽查是刻意的工程习惯:手写实现与标准库输出一致,才敢相信手写版正确。任务 4 把 M 从 11 调到 257, λ 从 18.9 降到 0.81,耗时从 5.8 ms 降到 3.0 ms——M=11 时平均每链 1589 个词,查找已接近线性扫描,这是" λ 过高 $\rightarrow$ 性能塌方 $\rightarrow$ 再散列"的实证。计时 3 次取最快 + 重复 30 遍,压掉系统调度噪声。切词规则决定了"单词"的定义,是统计程序必须显式声明的约定。

下一章预告:第6章 优先队列(堆)——基于数组的完全二叉树;建堆与删除最小将使用第 3 章的 `vector/list`,堆的插入(上滤)与删除最小(下滤)是第 7 章堆排序、第 9 章 Dijkstra 最短路径的基石。

第6章 优先队列(堆)

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**优先队列** ADT(insert/deleteMin/findMin)与第3章普通队列的区别:出队按优先级而非到达顺序
- 掌握二叉堆的**结构性性质**(完全二叉树)与**堆序性质**(父 $\leq$ 子)
- 掌握 1 基数组表示(根=1、左子 $2i$ 、右子 $2i+1$ 、父 $i/2$),能默写**上滤**与**下滤**
- 能解释两种建堆方式的复杂度差异
- 能说出 d-堆、左式堆、斜堆、二项队列的动机与差别
- 会用 `std::priority_queue` 构造大顶堆、最小堆与自定义比较器
- 能用"大小为 k 的小顶堆"独立解决 top-k,并用标准库验证手写堆

💡 从 C 到 C++:本章主题如何迁移

优先队列的直觉一句话:"不用排队,按重要性插队"。第3章的普通队列是"先来先服务",而急诊分诊、任务调度、短作业打印都要求"最紧急者先处理"。它只有三个操作:insert、findMin、deleteMin。

在 C 里,朴素做法两头为难:无序数组 insert $O(1)$ 但 deleteMin 要扫描 $O(n)$;有序数组反过来。二叉堆的巧妙之处在于 insert 与 deleteMin 同时做到 $O(\log n)$,且不需要指针——完全二叉树直接摊平进数组,父子关系用下标计算,正是 C 程序员最熟悉的数组思维。

C++ 迁移分三层:底层用第3章的 `vector` 当堆数组;中间层手写二叉堆类(\$6.2-\$6.4);上层用标准库 `priority_queue` (\$6.6)。注意 C++ 教材用 1 基(下标 0 闲置),Python 的 `heapq` 用 0 基(\$6.2.3)。

📦 前置知识自查

数学前置:完全二叉树高度 $\approx \log_2 n$;求和记号 Σ 与几何级数(建堆 $O(n)$ 推导用);摊还分析直觉(第11章)。

C 语言前置:数组与下标运算; `struct` 与函数指针;函数指针形式的比较函数;整数除法(1 基父结点 $i/2$)。

依赖前面章节:第2章大 O;第3章 `vector` (堆的底层数组)与栈;第4章树术语(父、子、叶、深度、高度)。

🚀 本章学习路线

- 第一阶段(1 小时):**读 \$6.1-\$6.2,理解 ADT 与两条性质,背熟 1 基公式
- 第二阶段(1.5 小时):**读 \$6.3-\$6.4,手写上滤/下滤/建堆代码,手工走一遍 insert 与 deleteMin
- 第三阶段(1 小时):**读 \$6.5-\$6.6,俯瞰堆家族,上机实验三种构造
- 第四阶段(实验,1 小时):**完成章末实验:Top-K 高频词
- 验收标准:**能默写上滤与下滤,说出"二叉堆 insert/deleteMin $O(\log n)$ 、建堆 $O(n)$ ",解释 `priority_queue` 为何默认大顶堆

本章知识导图

- §6.1 优先队列模型与动机(任务排程/急诊分诊、ADT 三操作、队列对比)
- §6.2 二叉堆:结构性质与数组表示(完全二叉树、堆序性质、1 基 vs 0 基换算)
- §6.3 插入与删除最小:上滤与下滤(完整代码、"末元素搬根再下滤"套路)
- §6.4 建堆与其余操作(逐个插入 $O(n \log n)$ vs 下滤建堆 $O(n)$ 、decreaseKey/delete/merge)
- §6.5 其他堆:d-堆、左式堆、斜堆与二项队列(动机与一句话差别)
- §6.6 标准库:priority_queue(默认大顶堆、最小堆写法、自定义比较器)
- §6.7 手写堆与标准库并排:top-k 实战(小顶堆淘汰技巧、堆排序预告)

§6.1 优先队列模型与动机

6.1.1 三个场景:谁最紧急谁先走

先看三个场景。① **操作系统任务调度**:每次选择优先级最高的进程;② **打印机队列**:短作业优先;③ **急诊分诊**:按病情严重程度处理,最危重者最先抢救。

三个场景的共同点一致:插入一个元素、查看当前最紧急的元素、取出该元素。大小关系代表优先级还是病情,由具体问题映射,数据结构不管。

6.1.2 优先队列 ADT:三个操作

优先队列(priority queue) 是抽象数据类型: `insert(x)` 插入; `findMin()` 返回最小元但不删除; `deleteMin()` 删除并返回最小元。约定数值越小优先级越高(最小堆),取最大则把比较方向反过来(§6.6)。注意:**堆(heap)** 是优先队列最主流的实现,不是 ADT 本身;与存放动态内存的"堆内存"同名,毫无关系。

记号说明(本章约定,与 Python 版一致)

复杂度记号沿用第2章。下标公式中,"父 $i/2$ "与" $(i-1)/2$ "都指**整数除法**(代码写 $i / 2$ 、 $(i - 1) / 2$)。"最小元""最高优先级"同义。

6.1.3 与普通队列对比:出队顺序由谁决定

第3章的普通队列是 FIFO:出队顺序 = 到达顺序;优先队列相反:出队顺序 = 优先级顺序。朴素实现却两头为难:

朴素实现	insert	deleteMin	问题
无序数组/链表	$O(1)$	$O(n)$ (扫描)	删除太慢
有序数组/链表	$O(n)$ (移位)	$O(1)$	插入太慢
二叉堆(本章主角)	$O(\log n)$	$O(\log n)$	两者均衡

C 的写法	C++ 的写法
<code>struct PQ { int *a; int n; };</code> 数据与操作分离	<code>class BinaryHeap { vector<T> a; ... };</code> 数据与操作封装
<code>pq_deleteMin(&pq, &out);</code> 每个操作传指针	<code>pq.deleteMin();</code> 成员函数天然绑定
找最小要自己写 <code>for</code> 循环扫描	根就是最小元, <code>findMin()</code> 一行
数组长度、容量、释放全靠手工	<code>vector</code> 自动管理

§6.2 二叉堆:结构性质与数组表示

6.2.1 结构性质:完全二叉树

二叉堆首先是一棵**完全二叉树**:除最后一层外每层都满,最后一层从左到右连续填充、不留空隙。这条性质的威力在高度:结点数 n 满足 $2^h \leq n < 2^{h+1}$, 所以 $h \approx \log_2 n$ ——任何根到叶路径都只有对数长度,这正是上滤/下滤高效的根源。

完全二叉树还不需要指针:结点按层序编号,父子关系用下标直接算出(§6.2.3)。对比第4章的指针树,连续数组存储节省指针空间、缓存友好,代价是形状被锁定,只能在末尾增删。

6.2.2 堆序性质:最小堆

堆序性质:对每个非叶结点,父结点的值 $\leq$ 子结点的值 (最小堆;换成 $\geq$ 即最大堆)。两条性质合起来才是二叉堆:结构性质保证对数高度,堆序性质保证根是全局最小元, `findMin()` 是 $O(1)$ 。

立刻澄清:堆序只约束"父子",兄弟之间没有大小关系。堆因此不是有序数组,不能二分查找,按值查找只能全扫 $O(n)$ 。堆只承诺最小元在根,是"弱有序"结构,与第4章 BST 的强有序对照。

6.2.3 数组表示:1 基与 0 基换算

把完全二叉树按层序放进数组,父子关系退化为下标运算。这里出现全系列最重要的"习惯分歧":C++ 教材(Weiss 原书)用**1 基**——根在下标 1,下标 0 闲置;Python 的 `heapq` 用**0 基**——根在下标 0。两套公式:

含义	1 基(C++ 教材习惯)	0 基(Python <code>heapq</code>)
根	下标 1	下标 0
结点 i 的左子	$2i$	$2i+1$
结点 i 的右子	$2i+1$	$2i+2$
结点 i 的父	$i/2$ (整数除法)	$(i-1)/2$ (整数除法)
最后一个非叶结点	$n/2$ (整数除法)	$n/2 - 1$
下标 0 是否闲置	是, <code>a[0]</code> 闲置	否, 下标 0 就是根

验证:下标 5 的结点,1 基下左子 10、右子 11、父 2;0 基下左子 11、右子 12、父 2。父相同只是巧合——下标 4:1 基父 2,0 基父 1。

⚠ 误区 1:把 0 基公式抄进 1 基代码(或反之)

两种下标体系并存是本章第一大坑。1 基堆里左子必须写 $2*i$ 、父必须写 $i/2$ ——抄成 0 基的 $2*i+1$ 、 $(i-1)/2$ 会取到错位置;把闲置的 $a[0]$ 当根读写,堆序立即被破坏且极难排查。解决之道:写代码前先声明"用几基"并写进注释,写完用"验证堆序"函数检查 (§6.3.3)。

示例 6-1 二叉堆类骨架(1 基,vector 作底层)

```
// 1 基堆:根在下标 1,左子 2i、右子 2i+1、父 i/2;下标 0 闲置
template <typename T>
class BinaryHeap {
    vector<T> a;                // 底层数组,a[0] 闲置
public:
    BinaryHeap() { a.push_back(T{}); } // 占住 a[0],保证从下标 1 开始用
    size_t size() const { return a.size() - 1; }
    bool empty() const { return size() == 0; }
    T findMin() const { return a[1]; } // 根即最小元 0(1)
};
```

§6.3 插入与删除最小:上滤与下滤

6.3.1 插入:上滤(percolateUp)

插入策略是"先保结构,再修堆序":① 新元素追加到末尾(完全二叉树只在末尾留空位,结构自动保持);② 沿根路径向上冒泡——与父比较,小于父则交换,直到父 $\leq$ 自己或到根。这叫**上滤(percolate up)**,路径最长即树高,insert 为 $O(\log n)$;上滤只看一个父,正是堆"弱有序"的便利。

示例 6-2 insert:上滤

```
void insert(const T& x) {
    a.push_back(x);        // 追加到末尾:保持完全二叉树
    size_t i = a.size() - 1; // 新元素下标
    while (i > 1) {         // 向上冒泡(上滤)
        size_t p = i / 2;   // 父结点(1 基)
        if (a[p] <= a[i]) break; // 父 <= 子,堆序成立,停
        swap(a[p], a[i]);   // 否则与父交换,继续往上
        i = p;
    }
}
```

6.3.2 删除最小:下滤(percolateDown)

删除根会留下空洞,朴素前移会破坏完全二叉树或付出 $O(n)$ 搬移。经典套路:① 记下根(它就是答案);② 把**最后一个元素搬到根**(末尾是唯一可合法删除的空位,结构自动保持);③ 沿"叶方向"向下冒泡——与孩子中**较小者**比较,大于它则交换,直到 $\leq$ 两个孩子或到叶。这叫**下滤(percolate down)**,同样 $O(\log n)$ 。

两个细节:必须与"较小的"孩子交换(与较大的换会违反堆序);先检查右子是否存在(下标 $2i+1$ 是否在界内),只有左子也合法。

示例 6-3 deleteMin:末元素搬根再下滤


```

// 下滤:把下标 i 的元素向下沉(deleteMin 与建堆共用)
void percolateDown(size_t i) {
    size_t n = a.size();
    while (true) {
        size_t smallest = i;
        for (size_t c : {2 * i, 2 * i + 1}) // 左子、右子(1 基)
            if (c < n && a[c] < a[smallest])
                smallest = c; // 选"较小"的孩子
        if (smallest == i) break; // 两个孩子都不小于自己
        swap(a[i], a[smallest]);
        i = smallest;
    }
}

// 删除最小元:最后一个元素搬到根,再下滤
T deleteMin() {
    T min = a[1]; // 记下根(答案)
    a[1] = a.back(); // 末元素搬到根:保持完全二叉树
    a.pop_back();
    if (a.size() > 1) percolateDown(1); // 只剩 1 个元素则免
    return min;
}

```

例题 6-1 手工走一遍 insert 与 deleteMin

设 1 基堆初始为 $[_, 13, 21, 16, 24, 31, 19, 68, 65, 26, 32]$ ($a[0]$ 闲置)。**insert(14)**:追加到下标 11,沿 $11 \rightarrow 5 \rightarrow 2 \rightarrow 1$ 上滤: $31 > 14$ 换、 $21 > 14$ 换、 $13 \leq 14$ 停,得 $[_, 13, 14, 16, 24, 21, 19, 68, 65, 26, 32, 31]$ 。**deleteMin()**:取走 13,末元素 31 搬根,依次换过 14、21、26,得 $[_, 14, 21, 16, 24, 26, 19, 68, 65, 31, 32]$,最小元 14。

6.3.3 完整接口与调试技巧

完整堆还应有 `top()/findMin()`、`empty()`、`size()` 等接口;若需"按插入顺序稳定",可在元素里加序号比较(见 §6.7 的(频次, 词)元组)。

调试技巧:写一个"验证堆序"的断言函数

堆的 bug 极具隐蔽性:上滤少换一层、下滤选错孩子,结果仍"接近正确"。随手加 `checkHeap()`:遍历所有非叶结点 $i(1 \leq i \leq n/2)$,断言 $a[i] \leq a[2*i]$ (若存在)且 $a[i] \leq a[2*i+1]$ (若存在),每次 insert/deleteMin 后调用,把可疑行为扼杀在摇篮里。

§6.4 建堆与其余操作

6.4.1 两种建堆方式

建堆有两种做法。① **自顶向下**:空堆逐个 insert,每次 $O(\log n)$,共 $O(n \log n)$ 。② **自底向上(下滤建堆)**: n 个元素按任意顺序放进数组(天然构成完全二叉树,只是堆序可能被违反),从最后非叶结点($n/2$)往前逐个 `percolateDown` 到根,总代价 $O(n)$ 。"一次性给全所有元素"的场景都用下滤建堆。

示例 6-4 下滤建堆: $O(n)$


```
// 下滤建堆:从最后一个非叶结点(n/2)开始,逐个下滤到根
void buildHeap(vector<T> items) {
    a.clear();
    a.push_back(T{});          // 占位 a[0]
    for (const T& x : items) a.push_back(x);
    for (size_t i = a.size() / 2; i >= 1; --i)
        percolateDown(i);      // 复用示例 6-3 的下滤
}
```

例题 6-2 手工走一遍下滤建堆

输入 [150, 80, 40, 30, 10, 70, 110, 100, 20, 90, 60, 50, 120, 140, 130] ($n = 15$, 最后非叶 = 下标 7)。
 从 7 往前: 7(110) ≤ 130 停; 6(70) 与 50 换; 5(10) ≤ 90、60 停; 4(30) 与 20 换; 3(40) ≤ 50、110 停; 2(80) 与 10 换、再与 60 换; 1(150) 依次与 10、20、30 换。最终 [_, 10, 20, 40, 30, 60, 50, 110, 100, 80, 90, 70, 150, 120, 140, 130]。

6.4.2 为什么下滤建堆是 $O(n)$:一句话推导

关键观察:高度为 h 的结点约有 $n/2^{h+1}$ 个,每个下滤至多走 h 步,总代价 $\sum_h h \cdot n/2^{h+1}$ 是"几何级数收尾"的求和,结果 $O(n)$ 。直觉:矮的结点数量多但只需下滤几步,高的结点上滤远但数量极少;逐个插入的代价全集中在"深叶子上滤到根"的长路径。严格证明在第11章。

6.4.3 decreaseKey、delete 与 merge

- **decreaseKey(p, delta)**:键变小只能向上走——直接上滤, $O(\log n)$ 。前提是知道 p 的位置(堆不提供查找);经典用途:Dijkstra 松弛(第9章)。
- **delete(任意位置 p)**:把 $a[p]$ 换成最小值(根的副本),上滤到根,再 deleteMin——两步都 $O(\log n)$ 。
- **merge(合并两个堆)**:二叉堆的短板——必须把两个数组拷到一起再建堆, $O(n)$ 。频繁合并堆的应用,二叉堆不够用,这正是 §6.5 堆家族的动机。

示例 6-5 decreaseKey 与 delete(任意位置)

```
// 键变小只会"向上"走:减键后上滤  $O(\log n)$ 
void decreaseKey(size_t pos, const T& amount) {
    a[pos] -= amount;
    while (pos > 1 && a[pos / 2] > a[pos]) {
        swap(a[pos / 2], a[pos]);
        pos /= 2;
    }
}

// 删除任意位置:盖成最小值上滤到根,再 deleteMin  $O(\log n)$ 
void removeAt(size_t pos) {
    a[pos] = a[1];          // 根是最小值,盖上去不破坏堆序
    while (pos > 1) {        // 一路换到根
        swap(a[pos / 2], a[pos]);
        pos /= 2;
    }
    deleteMin();            // 现在根就是原来的 a[pos]
}
```

§6.5 其他堆:d-堆、左式堆、斜堆与二项队列

6.5.1 为什么不止一种堆

二叉堆已做到 insert/deleteMin 均 $O(\log n)$ 、 findMin $O(1)$,为何还要别的堆?因为**操作组合不同**:插入远多于删除的系统希望 insert 更快;频繁合并堆的系统受不了 merge $O(n)$ 。没有一种堆能同时最优,每个成员都在特定操作上让步、换取另一操作的改进。

6.5.2 d-堆:多叉,降低高度

d-堆 是二叉堆的直接推广:每个结点最多 d 个孩子(二叉堆即 $d = 2$)。高度从 $\log_2 n$ 降到 $\log_d n$, insert (只与一个父比较)变快为 $O(\log_d n)$;但 deleteMin 要在 d 个孩子里找最小,变慢为 $O(d \log_d n)$ 。典型场景: insert 远多于 deleteMin 的离散事件模拟,常用 $d = 4$ 或 5 。

6.5.3 左式堆:为 merge 而生

二叉堆 merge $O(n)$ 的根源是"完全二叉树"锁死了形状。**左式堆(leftist heap)** 主动放弃完全性,改用新约束:对每个结点,左子的零路径长(NPL)不小于右子的——即"左偏"。**零路径长(null path length)** 是到最近叶子的最短路径长度(空结点为 -1)。它保证最右路径很短, merge 只需递归沿最右路径合并, $O(\log n)$; insert/deleteMin 都可用 merge 实现。

6.5.4 斜堆:左式堆的摊还简化版

斜堆(skew heap) 是左式堆的极简主义版本:不维护 NPL, merge 后**无条件交换左右子树**。实现更短,但单次可能退化 $O(n)$,整体摊还 $O(\log n)$ (第11章证明)。它与左式堆的关系,正如伸展树之于 AVL 树。

6.5.5 二项队列:森林

二项队列(binomial queue) 的思路完全不同:不是一棵树,而是一组"二项树"组成的**森林**。第 k 棵二项树大小恰为 2^k ,每种大小最多一棵—— n 个元素按二进制位决定有哪些树(如 $n = 13 = 1101$,有 8 、 4 、 1 三棵)。插入像二进制加法"进位"合并同大小树,摊还 $O(1)$; merge 与 deleteMin 均 $O(\log n)$ 。

堆类型	动机	insert	deleteMin	merge
二叉堆	最常用,数组存储、缓存友好	$O(\log n)$	$O(\log n)$	$O(n)$
d-堆	插入多于删除,降高度	$O(\log_d n)$	$O(d \log_d n)$	$O(n)$
左式堆	高效 merge,左偏约束	$O(\log n)$	$O(\log n)$	$O(\log n)$
斜堆	左式堆的摊还简化版	摊还 $O(\log n)$	摊还 $O(\log n)$	摊还 $O(\log n)$
二项队列	森林结构,插入摊还 $O(1)$	摊还 $O(1)$	$O(\log n)$	$O(\log n)$

§6.6 标准库:priority_queue

6.6.1 三种构造:大顶堆、最小堆与自定义比较器

`std::priority_queue` 是优先队列的工程实现,底层默认用 `vector` 存堆。最大的坑:默认是"大顶堆"——`top()` 返回最大元素(底层比较器 `less<T>`)。要最小堆必须写全三个模板参数;自定义比较器语义与 `std::sort` 相反:返回 `true` 表示第一个参数优先级更低。

示例 6-6 priority_queue 的三种构造

```
#include <queue>
#include <functional>
using namespace std;

priority_queue<int> mx;           // ① 默认大顶堆(底层 less):top() 是最大元
mx.push(3); mx.push(1); mx.push(9);
// mx.top() == 9, pop 顺序:9 3 1

// ② 最小堆:三模板参数
priority_queue<int, vector<int>, greater<int>> mn;
mn.push(3); mn.push(1); mn.push(9);
// mn.top() == 1, pop 顺序:1 3 9

// ③ 自定义比较器:按"频次"给 (词, 频次) 建最小堆
struct Cmp {
    bool operator()(const pair<string, int>& x,
                    const pair<string, int>& y) const {
        return x.second > y.second; // 返回 true 表示 x 优先级更低
    }
};
priority_queue<pair<string, int>, vector<pair<string, int>>, Cmp> pq;
```

⚠ 误区 2:忘记 priority_queue 默认是大顶堆

习惯"优先队列 = 最小先出"的同学,直接用 `priority_queue<int>` 拿到的却是最大元——标准库最高频 bug。两个检查:① 想清楚 `top()` 出来的是谁,最小堆务必写全三个模板参数,嫌啰嗦可用 `using` 起别名;② `pop()` / `top()` 前先判 `empty()`,空堆 `top` 未定义。

6.6.2 接口限制与选型

`priority_queue` 接口只有五个:push、pop、top、empty、size。限制:`pop()` 不返回被删元素;无迭代器;无 `decreaseKey`;"批量建堆"只能在构造时拷入整个容器(内部自动下滤)。它本质是**适配器**:包住一个 `vector` 只露出堆的三操作,与 `std::stack` / `std::queue` 同一家族。

C 的手写二叉堆	C++ 的 std::priority_queue
自己写数组、上滤、下滤约 60 行	构造一行声明,五个成员函数即全接口
比较逻辑写死在代码里	比较器是模板参数,less/greater/自定义随意换
内存泄漏风险全在自己肩上	RAII 自动管理,离开作用域自动释放
可以随时读写底层数组(调试友好)	封装后只能走三操作,防止误用

🚀 工程建议:什么时候直接上 priority_queue

工程中"需要动态维护最值"的场景一律优先 `priority_queue`:任务调度、Top-K、贪心(第10章哈夫曼)、Dijkstra(第9章)。手写堆的用武之地:需要 `decreaseKey`(标准库没有)、批量建堆后长期复用、或考试要求展示原理。"一次性排好序"用 `std::sort`;"边插入边取最值"才用堆。

§6.7 手写堆与标准库并排:top-k 实战

6.7.1 问题:求最大的 k 个(top-k)

Top-K 问题:给定 n 个元素,求最大的 k 个(排行榜、热门词)。朴素做法全部排序是 $O(n \log n)$;堆做法 $O(n \log k)$, k 远小于 n 时近乎线性。核心技巧叫**小顶堆淘汰法**:维护"当前最大的 k 个"的小顶堆——堆顶恰是第 k 大,即淘汰"门槛";新元素大于堆顶才替换。

为什么用"小顶堆"而非"大顶堆"? 大顶堆的堆顶是全局最大,淘汰后不知谁替补;小顶堆的堆顶是"最小的入选者",谁比它大谁挤掉它,淘汰逻辑一行搞定。

C 的朴素做法(qsort 全排序)	C++ 的堆做法(priority_queue)
qsort 把 n 个元素全部排序, $O(n \log n)$	大小为 k 的小顶堆, $O(n \log k)$
结果数组占 n 个位置	堆只占 k 个位置,可流式处理
新数据到达必须重新 qsort 一遍	push / top / pop 动态维护,每次 $O(\log k)$
比较函数用函数指针,写在调用处之外	比较器是模板参数, <code>greater<T></code> 一行搞定

⚠ 误区 3:top-k 时"无脑入堆",忘了先与堆顶比较

堆满后仍直接 push,堆会涨过 k ;或"先 push 再 pop 堆顶",把刚进来的大元素又弹走——淘汰条件写反。正确顺序只有一种:先与堆顶比较,大于门槛才 pop 再 push,且 pop 必须在 push 之前。把"比较 → pop → push"的顺序写成注释即可防错。

6.7.2 手写版与标准库版并排

同一任务——给一组(频次,词)求频次最高的 k 个——分别用手写堆与 priority_queue 实现。两份代码逻辑结构一字不差,正是"手写理解原理、标准库交付工程"的教科书式对照。

示例 6-7 手写堆版 top-k(复用示例 6-1~6-3 的 BinaryHeap)

```
// items: (频次, 词);按频次降序输出最大的 k 个
vector<pair<int, string>> topK(const vector<pair<int, string>>& items, int k) {
    BinaryHeap<pair<int, string>> h;           // 手写最小堆
    for (const auto& t : items) {
        if (h.size() < size_t(k))
            h.insert(t);                       // 堆未满:直接入堆
        else if (t > h.findMin()) {             // 超过"第 k 大门槛"才替换
            h.deleteMin();
            h.insert(t);
        }
    }
    vector<pair<int, string>> res;
    while (!h.empty()) { res.push_back(h.findMin()); h.deleteMin(); }
    reverse(res.begin(), res.end());           // 弹出是升序,反转为降序
    return res;
}
```

示例 6-8 标准库版 top-k:逻辑一字不差

```
// 同一任务用 priority_queue:只换构造与函数名
vector<pair<int, string>> topKStd(const vector<pair<int, string>>& items, int k) {
    using P = pair<int, string>;
    priority_queue<P, vector<P>, greater<P>> h;    // 标准库最小堆
    for (const P& t : items) {
        if (int(h.size()) < k)
            h.push(t);
        else if (t > h.top()) {                    // 同上:高于门槛才替换
            h.pop();
            h.push(t);
        }
    }
    vector<P> res;
    while (!h.empty()) { res.push_back(h.top()); h.pop(); }
    reverse(res.begin(), res.end());
    return res;
}
```

6.7.3 复杂度分析与堆排序预告

每个元素至多一次比较 + 一次替换($O(\log k)$),总代价 $O(n \log k)$; $k = n$ 时退化为 $O(n \log n)$ 。优势在**内存**:堆只占 k 个位置,几十 GB 的日志放不进内存,却能流式处理。

预告第7章:堆排序直接复用本章的二叉堆——建堆 $O(n)$,连续 deleteMin n 次,弹出元素天然有序,总代价 $O(n \log n)$,与插入、希尔、归并、快排逐一对比。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
insert	二叉堆	$O(\log n)$	$O(\log n)$
deleteMin / deleteMax	二叉堆	$O(\log n)$	$O(\log n)$
findMin	二叉堆	$O(1)$	$O(1)$
buildHeap(下滤建堆)	二叉堆	$O(n)$	$O(n)$
buildHeap(逐个插入)	二叉堆	$O(n \log n)$	$O(n \log n)$
decreaseKey	二叉堆	$O(\log n)$	$O(\log n)$
delete(任意元素)	二叉堆	$O(\log n)$	$O(\log n)$
merge	二叉堆	$O(n)$	$O(n)$
按值查找 find	二叉堆	$O(n)$	$O(n)$
insert	d-堆	$O(\log_d n)$	$O(\log_d n)$
deleteMin	d-堆	$O(d \log_d n)$	$O(d \log_d n)$
merge	左式堆	$O(\log n)$	$O(\log n)$
merge	斜堆	摊还 $O(\log n)$	$O(n)$
insert	二项队列	摊还 $O(1)$	$O(\log n)$
deleteMin	二项队列	$O(\log n)$	$O(\log n)$
merge	二项队列	$O(\log n)$	$O(\log n)$
top-k(大小为 k 的小顶堆)	二叉堆 + 一次遍历	$O(n \log k)$	$O(n \log k)$

最小必会清单

- 能说出优先队列 ADT 的三个操作与普通队列的本质区别
- 能说出二叉堆的两条性质,并画出完全二叉树的数组填充
- 能默写 1 基公式(左子 $2i$ 、右子 $2i+1$ 、父 $i/2$ 、最后非叶 $n/2$),并与 0 基公式换算
- 能默写 insert(上滤)与 deleteMin(末元素搬根再下滤)的代码
- 能解释"删除最小为什么是搬最后一个元素",以及"下滤为什么必须选较小的孩子"
- 能独立写出下滤建堆,并解释两种建堆方式的复杂度差异
- 能说出 decreaseKey、delete、merge 的代价与场景
- 能一句话说出 d-堆、左式堆、斜堆、二项队列的动机与差别
- 能写出最小堆 priority_queue 的完整构造,并指出默认大顶堆的坑
- 能独立实现"大小为 k 的小顶堆"解决 top-k,并解释为什么不用大顶堆

自测题

- Q 1. 二叉堆为什么必须是完全二叉树?如果允许任意形状的二叉树,会损失什么?**

✓ 参考答案

完全性保证高度恒为 $\log_2 n$ 与数组连续存储(无指针、下标算父子)。若形状任意,最坏退化成链,高度与操作都变 $O(n)$,还得用指针。

Q 2.1 基堆中下标 i 的结点的父、左子、右子分别是什么?0 基堆呢?试各举一例。

✓ 参考答案

1 基:父 $i/2$ 、左子 $2i$ 、右子 $2i+1$ (如 $i=5$:父 2、左 10、右 11);0 基:父 $(i-1)/2$ 、左子 $2i+1$ 、右子 $2i+2$ (如 $i=5$:父 2、左 11、右 12)。“父为 2”只是巧合,不可混用。

Q 3. 在最小堆中查找某个值的复杂度是多少?为什么不能像 BST 那样二分?

✓ 参考答案

$O(n)$ 。堆序只约束父子关系,兄弟之间没有大小关系,无法一次比较排除一半子树;堆也不是有序数组,连续存储帮不上二分。堆只承诺最小元在根。

Q 4. 把 15 个元素下滤建堆,最后一个下滤的结点是哪个?为什么不是第一个?

✓ 参考答案

最后下滤的是根(下标 1)。建堆从最后非叶结点($n/2$)往前处理,先让下面子树各自成堆,上层下滤时才能保证孩子子树已是堆、只需比较两个孩子。根最后处理,一次下滤把最小值带到根。

Q 5. deleteMin 为什么要把“最后一个元素”搬到根,而不是把后面元素整体前移?

✓ 参考答案

整体前移会留下空隙(破坏结构性)或付出 $O(n)$ 搬移。最后一个元素是唯一“删除后数组仍连续”的位置:先搬到根保持结构,再下滤修复堆序,两步各 $O(\log n)$ 。

Q 6. 求 top-k 为什么用大小为 k 的“小顶堆”?用“大顶堆”行吗?

✓ 参考答案

小顶堆的堆顶是“当前入选者里最小的”(第 k 大门槛),新元素与堆顶比较即可决定淘汰谁。大顶堆的堆顶是全局最大,被淘汰后无法 $O(1)$ 判断谁替补,只能维护更多信息或退化成排序。求“最小的 k 个”改用大小为 k 的大顶堆。

章末实验题:Top-K 高频词(堆实现)

实验 6:Top-K 高频词(手写堆 + 标准库双实现)

实验目标:流式读入英文文本,先用**手写二叉堆**求频次最高的 k 个词,再用 `std::priority_queue` 重做并验证两版结果一致。

实验要求:

- 任务 1:实现二叉堆类(insert/deleteMin/下滤建堆,1 基或 0 基自选,注释中说明所用下标公式)(知识点:\$6.2-\$6.4 上滤/下滤/建堆)
- 任务 2:流式读入文本,切分出单词并维护"词频"字典(知识点:\$6.1 + 第5章散列)
- 任务 3:用"大小为 k 的小顶堆"求频次最高的 k 个词,淘汰技巧:堆满时频次高于堆顶才替换(知识点:\$6.7 top-k)
- 任务 4:用 `priority_queue` (注意最小堆构造)重做任务 3,对比两版结果一致(知识点:\$6.6 标准库)
- 任务 5:处理边界: $k \leq 0$ 、文本为空、 k 大于词数时给出合理输出而非崩溃(知识点:\$6.3)

实验环境:C++17。编译: `g++ -std=c++17 -O2 lab6.cpp -o lab6`;运行: `./lab6 k < 文本文件`。

第一步 写 MinHeap 模板类(1 基),用例题 6-1 的序列手工验证 insert 与 deleteMin,再写 top-k 函数

第二步 切分用 `isalpha` 逐字符扫描,全部转小写;词频字典用 `unordered_map<string, int>`

第三步 元素用 `pair<int, string>` (频次在前):先比频次、平局比词序

第四步 准库版只改三处:构造(加 greater)、push/pop 换名、堆顶 `h.top()`;便于对照

第五步 行样例: `./lab6 3 < sample.txt`,再试 $k = 0$ 与空文件

✓ 参考答案(完整可运行程序,约 120 行,分三段)

```

// lab6.cpp — 第一段:MinHeap(1 基)
#include <algorithm>
#include <cctype>
#include <iostream>
#include <queue>
#include <string>
#include <unordered_map>
#include <vector>
using namespace std;

// 任务 1:手写二叉堆(1 基,下标公式见 §6.2.3)
template <typename T>
class MinHeap {
    vector<T> a; // 元素从下标 1 开始

    void siftDown(size_t i) { // 下滤:与较小孩子交换
        size_t n = a.size();
        while (true) {
            size_t smallest = i;
            for (size_t c : {2 * i, 2 * i + 1}) // 左子、右子(1 基)
                if (c < n && a[c] < a[smallest])
                    smallest = c;
            if (smallest == i) break;
            swap(a[i], a[smallest]);
            i = smallest;
        }
    }

public:
    MinHeap() { a.push_back(T{}); } // 占住 a[0]
    size_t size() const { return a.size() - 1; }
    bool empty() const { return size() == 0; }
    const T& top() const { return a[1]; } // 根即最小元 O(1)

    void push(const T& x) { // 上滤:放末尾后向上冒泡
        a.push_back(x);
        size_t i = a.size() - 1;
        while (i > 1) {
            size_t p = i / 2; // 父结点(1 基)
            if (a[p] <= a[i]) break;
            swap(a[p], a[i]);
            i = p;
        }
    }

    void pop() { // 删除最小:末元素搬根再下滤
        a[1] = a.back();
        a.pop_back();
        if (a.size() > 1) siftDown(1);
    }

    static MinHeap buildFrom(const vector<T>& items) { // 下滤建堆 O(n)
        MinHeap h;
        for (const T& x : items) h.a.push_back(x);
        for (size_t i = h.a.size() / 2; i >= 1; --i) // 最后非叶 = n/2

```

```

        h.siftDown(i);
    return h;
}
};

```

lab6.cpp(第二段:词频统计与手写堆 top-k)

```

// 任务 2:流式切词并统计词频
void countWords(unordered_map<string, int>& freq) {
    string line;
    while (getline(cin, line)) {
        for (size_t i = 0; i < line.size(); i++) {
            if (!isalpha((unsigned char)line[i])) { ++i; continue; }
            size_t j = i;
            while (j < line.size() && isalpha((unsigned char)line[j])) ++j;
            string w = line.substr(i, j - i);
            for (char& c : w) c = char(tolower((unsigned char)c));
            ++freq[w];
            i = j;
        }
    }
}

// 任务 3:手写小顶堆求 top-k
vector<pair<int, string>> topkHeap(const vector<pair<int, string>>& items, int k) {
    MinHeap<pair<int, string>> h;
    for (const auto& t : items) { // t = (频次, 词)
        if (h.size() < size_t(k))
            h.push(t); // 堆未满:直接入堆
        else if (t > h.top()) { // 高于"第 k 大门槛"才替换
            h.pop();
            h.push(t);
        }
    }
    vector<pair<int, string>> res;
    while (!h.empty()) { res.push_back(h.top()); h.pop(); }
    reverse(res.begin(), res.end()); // 弹出是升序,反转为降序
    return res;
}

```

lab6.cpp(第三段:标准库版与主程序)

```

// 任务 4:标准库重做(小顶堆 = 大顶堆 + greater)
vector<pair<int, string>> topkStd(const vector<pair<int, string>>& items, int k) {
    using P = pair<int, string>;
    priority_queue<P, vector<P>, greater<P>> pq;
    for (const P& t : items) {
        if (int(pq.size()) < k)
            pq.push(t);
        else if (t > pq.top()) {
            pq.pop();
            pq.push(t);
        }
    }
    vector<P> res;
    while (!pq.empty()) { res.push_back(pq.top()); pq.pop(); }
    reverse(res.begin(), res.end());
    return res;
}

int main(int argc, char* argv[]) {
    if (argc < 2) { cerr << "用法: ./lab6 k < 文本文件\n"; return 1; }
    int k = stoi(argv[1]);
    unordered_map<string, int> freq;
    countWords(freq); // 任务 2:词频字典
    vector<pair<int, string>> items;
    for (const auto& [w, c] : freq) items.push_back({c, w});

    // 任务 5:边界处理—k <= 0 或文本为空
    if (k <= 0 || items.empty()) {
        cout << "k = " << k << ", 词数 = " << items.size()
            << ":边界情形,答案为空.\n";
        return 0;
    }

    auto r1 = topkHeap(items, k);
    auto r2 = topkStd(items, k);
    cout << "词数 = " << items.size() << ", k = " << k << "\n";
    cout << "手写堆 : ";
    for (const auto& [c, w] : r1) cout << w << "(" << c << ") ";
    cout << "\npriority_queue: ";
    for (const auto& [c, w] : r2) cout << w << "(" << c << ") ";
    cout << "\n结果一致: " << (r1 == r2 ? "是" : "否") << "\n";
    return 0;
}

```

示例输入与输出(真实运行;sample.txt 与 empty.txt 为测试文件):

```
$ cat sample.txt
to be or not to be that is the question
whether tis nobler in the mind to suffer
the slings and arrows of outrageous fortune
or to take arms against a sea of troubles
and by opposing end them to die to sleep no more
```

```
$ ./lab6 3 < sample.txt
词数 = 34, k = 3
手写堆 : to(6) the(3) or(2)
priority_queue: to(6) the(3) or(2)
结果一致: 是
```

```
$ ./lab6 5 < empty.txt
k = 5, 词数 = 0:边界情形,答案为空。
```

设计说明:① 1 基公式写进类注释,与 Python 版 0 基实现互为对照;② 元素用 (频次, 词) 元组,平局比词序,结果可复现;③ 两版结果一致即互相验证;④ 词频统计用 `unordered_map` (第5章),与堆协同完成统计 → Top-K 流水线;⑤ 边界显式处理,避免空堆访问。

下一章预告:第7章 排序——堆排序将直接使用本章的二叉堆:下滤建堆 $O(n)$,再连续 `deleteMin` n 次得到有序序列,总代价 $O(n \log n)$;插入、希尔、归并、快排逐一登场。

第7章 排序

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 掌握七种排序的算法思想与实现,理解**稳定性**并会给出反例
- 能用求和记号 Σ 推导插入排序的最坏 $O(n^2)$ 与"近有序 $O(n)$ "性质
- 掌握快排的**三数中值分割法**与小区间插入优化,说清最坏 $O(n^2)$ 何时发生
- 能用决策树直觉解释**比较下界** $\Omega(n \log n)$,理解桶排为何能突破
- 完成从 C 的 `qsort` 到 C++ `std::sort` (模板 + lambda)的迁移,能按数据特征选型

💡 从 C 到 C++:本章主题如何迁移

排序是工程中出现频率最高的操作:排行榜、去重、二分查找前置、Top-K.....本章把第2章的复杂度分析、第6章的二叉堆全部用上。路线:经典排序(\$7.1-\$7.4) → 快排(\$7.5) → 桶排与外部排序(\$7.6) → 总表选型(\$7.7)。请务必亲手写一遍每一种排序——它们最常被要求默写。

在 C 里,通用排序武器只有 `qsort`,三个痛点——① `void*` 丢类型;② 比较语义是"负/零/正"三值;③ 只能整体排序。C++ 的 `std::sort` 用模板 + 迭代器逐一解决:类型由模板推导,lambda 给出"是否小于"的布尔语义,迭代器区间任意截取;它内部是"快排 + 堆排 + 插入"混合体(introsort),本章三种优化全内置。

📦 前置知识自查

数学前置:求和记号 Σ 与 $\Sigma_{i=1}^n i = n(n+1)/2$;递推式 $T(n) = 2T(n/2) + n$ (第2章)。

C 语言前置:数组与 `for`;函数指针与 `void*` (\$7.5 对照)。

依赖章节:第2章复杂度;第6章二叉堆(堆排序复用**下滤**与**下滤建堆**,注意 1 基/0 基差异)。

🚀 本章学习路线

1. **第一阶段(1 小时):**读 \$7.1-\$7.2,默写插入与希尔排序
2. **第二阶段(1.5 小时):**读 \$7.3-\$7.4,手写堆排与 merge
3. **第三阶段(1.5 小时):**读 \$7.5-\$7.6,吃透三数中值与分割
4. **第四阶段(实验,1 小时):**完成章末实验。验收:能默写全部经典排序,解释快排何时最坏

本章知识导图

- §7.1 插入排序:扑克牌直觉、 Σ 推导、近有序 $O(n)$ 、稳定性
- §7.2 希尔排序:希尔增量与 Hibbard、约 $O(n^{3/2})$ 、不稳定
- §7.3 堆排序:复用第6章的二叉堆(朴素版、原地版、ADT 关系)
- §7.4 归并排序: $T(n)=2T(n/2)+n$ 、merge、稳定与 $O(n)$ 空间
- §7.5 快速排序:三数中值、分割、小区间插入、qsort $\rightarrow$ std::sort 对照
- §7.6 桶式排序与外部排序:桶排 $O(n)$ 、比较下界 $\Omega(n \log n)$ 、多路归并
- §7.7 排序算法总表:七种排序大表、选型决策树

§7.1 插入排序:算法、复杂度与稳定性

7.1.1 扑克牌直觉与算法步骤

直觉一句话:像整理扑克牌——每摸一张新牌,插到已排好序的牌中的正确位置。算法把数组分成"已排序前缀 $a[0..i-1]$ "与"未排序后缀 $a[i..n-1]$ ":第 i 步用 key 暂存 $a[i]$,从右往左找位置,挡路元素右移一格,落位。

示例 7-1 插入排序(key 暂存与右移)

```
// 对 a[left..right] 升序排序
void insertionSort(vector<int>& a, int left, int right) {
    for (int i = left + 1; i <= right; ++i) {
        int key = a[i];
        int j = i - 1;
        while (j >= left && a[j] > key) {    // 严格大于才右移:稳定
            a[j + 1] = a[j];
            --j;
        }
        a[j + 1] = key;
    }
}
```

7.1.2 复杂度推导:把循环翻译成求和

最坏情形(完全逆序):第 i 个元素要与前面 i 个元素逐一比较并右移,总比较次数 $= \sum_{i=1}^{n-1} i = n(n-1)/2 = O(n^2)$ 。平均情形:第 i 个元素平均只比较 $i/2$ 次,总次数 $\approx n(n-1)/4$,仍 $O(n^2)$ 。最好情形(已有序):每个元素只比较 1 次, $O(n)$ 。

更本质的刻画是**逆序对(inversion)**:满足 $i < j$ 且 $a[i] > a[j]$ 的数对。插入排序每次右移恰好消除一个逆序对,比较/移动次数 = 逆序对数——逆序对很少时排序就快,"近有序 $O(n)$ "的来源。

7.1.3 近有序数组:接近 $O(n)$ 的性质

若数组"大体有序"(逆序对只有 $O(n)$ 个),插入排序只需 $O(n)$ 次比较与移动,接近线性——章末实验将实测。但也暴露致命局限:每次只交换相邻元素,跨越 k 个位置要走 k 步。例:[2, 3, 4, 5, 6, 7, 8, 1] 中元素 1 走到开头要走 7 步。"让元素一次走远"正是 §7.2 希尔排序的动机。

7.1.4 稳定性:相等元素保持相对次序

稳定性(stable):值相等的元素保持原相对次序。插入排序是稳定的——右移条件写"严格大于" `a[j] > key`,相等立即停止。为何重要?先按姓名排、再按分数排,若第二次稳定,同分者仍按姓名有序——多关键字排序可逐关键字轮流排。

⚠ 误区 1:内层循环里不暂存 key,直接与 `a[i]` 交换

新手常写 `while (a[j] > a[i]) { a[j+1] = a[j]; --j; } a[j+1] = a[i];`——`a[i]` 在右移中可能已被覆盖,key 丢失。正确姿势:先 `int key = a[i]`,循环里只移动 `a[j]`。

§7.2 希尔排序:增量序列与远距离搬移

7.2.1 动机:让元素一次走远

插入排序慢在"每次只走一格"。希尔排序:先用大步长"预排序"让元素大体到位,再逐步缩小步长,步长 1 时就是普通插入排序。步长叫**增量(gap)**:把下标相差 `h` 的元素看作同一组,对每组做插入排序——元素一次跨越 `h` 个位置。

7.2.2 增量序列:希尔增量与 Hibbard

- **希尔增量:**`n/2, n/4, ..., 1`(每次折半)。实现最简单,但增量互为倍数,奇偶下标各自成组,最坏退化为 $O(n^2)$;
- **Hibbard 增量:**`1, 3, 7, 15, ..., (2^k - 1)`。相邻增量互质,避开坏结构,最坏 $O(n^{3/2})$ 。

示例 7-2 希尔排序(希尔增量版)

```
void shellSort(vector<int>& a) {
    int n = int(a.size());
    for (int gap = n / 2; gap > 0; gap /= 2) { // 希尔增量:n/2, n/4, ..., 1
        for (int i = gap; i < n; ++i) { // 对每组做插入排序
            int key = a[i];
            int j = i;
            while (j >= gap && a[j - gap] > key) { // 一次跳 gap 格
                a[j] = a[j - gap];
                j -= gap;
            }
            a[j] = key;
        }
    }
}
```

7.2.3 复杂度与稳定性

希尔排序的精确复杂度至今未完全解决。工程口径(与速查表一致):希尔增量最坏 $O(n^2)$,Hibbard 增量最坏 $O(n^{3/2})$,平均都约 $O(n^{3/2})$ 。它不稳定:大间隔插入把相等元素"隔空换位"。反例:`[5a, 5b, 1]`,`gap = 2` 时 `(5a, 1)` 排成 `(1, 5a)`,数组变 `[1, 5b, 5a]`,最后 `gap = 1` 也救不回来。

⚠ 误区 2:增量序列忘了最后是 1,或增量间互为倍数

① 增量必须以 1 收尾,否则只保证"每组内有序",整体未必有序——`gap` 写成 `gap > 1` 就错。② 希尔增量互为倍数,偶数下标元素永远只在偶数组内比较,是最坏 $O(n^2)$ 的根源;改用 Hibbard($2^k - 1$) 互质序列即可避开。

§7.3 堆排序:复用第6章的二叉堆

7.3.1 朴素版:建堆 + 连续删除最小

第6章结尾预告过:堆排序 = 二叉堆的"全量消费"。朴素版两步:① 下滤建堆 $O(n)$;② 连续 `deleteMin` n 次,每次 $O(\log n)$ 。总代价 $O(n \log n)$,但要一块 $O(n)$ 输出数组。

7.3.2 原地版:最大堆 + 交换 + 缩小

原地版把输出数组省掉:改用**最大堆**。建最大堆后根即全局最大;把根与末尾交换,堆大小减 1,再下滤新根——被换到末尾的最大元素就此"钉死"在最终位置。反复 $n-1$ 次,全程只用原数组。

注意下标习惯:原地堆排用 0 基 (左子 $2i+1$ 、右子 $2i+2$ 、父 $(i-1)/2$ 、最后非叶 $n/2-1$),与第6章手写堆的 1 基不同。

示例 7-3 原地堆排序(0 基下滤)

```
// 下滤 a[i]; n 为当前堆大小
void siftDown(vector<int>& a, int i, int n) {
    while (true) {
        int largest = i;
        for (int c : {2 * i + 1, 2 * i + 2}) // 左子、右子(0 基)
            if (c < n && a[c] > a[largest])
                largest = c;
        if (largest == i) break;
        swap(a[i], a[largest]);
        i = largest;
    }
}

void heapSort(vector<int>& a) {
    int n = int(a.size());
    for (int i = n / 2 - 1; i >= 0; --i) // 1. 下滤建堆  $O(n)$ 
        siftDown(a, i, n);
    for (int i = n - 1; i > 0; --i) { // 2. 逐个钉死最大元
        swap(a[0], a[i]);
        siftDown(a, 0, i);
    }
}
```

7.3.3 复杂度、稳定性与 ADT 关系

建堆 $O(n)$ + $n-1$ 次下滤 $\times O(\log n) = O(n \log n)$,最好最坏都是它,且**原地**只要 $O(1)$ 额外空间。它不稳定:反例 $[4_a, 4_b, 3]$ 建堆后根 4_a 与末尾 3 交换得 $[3, 4_b, 4_a]$, 4_b 最终排在 4_a 前。

从 ADT 视角看:优先队列承诺"每次取出最小元",堆排序等于把"取最小"执行 n 次,顺便得到完整有序序列。"边插入边取最值"用优先队列,"一次性排好序"用排序 (§7.7)。

C 的手写堆排序	C++ 的标准库
手写下滤、建堆、交换,约 30 行	<code>make_heap</code> + <code>sort_heap</code> 两行

⚠ 误区 3:堆排序把 1 基公式抄进 0 基代码

第6章手写堆用 1 基,原地堆排用 0 基。把 1 基的"左子 $2i$ 、父 $i/2$ "抄进来会访问错位置,堆序被悄悄破坏、输出"看起来差不多但不对"。对策:下标公式写进注释,写完用"验证堆序"断言跑一遍。

§7.4 归并排序:分治的教科书

7.4.1 分治三步与递推式

归并排序是分治思想的教科书案例:① 分割:数组从中间切成两半;② 递归:各自排序;③ 合并:两个有序半区线性合并成一个有序整区。递推式即第2章的 $T(n) = 2T(n/2) + n$,解得 $O(n \log n)$ 。

7.4.2 merge:两个有序序列的线性合并

合并直觉:像两队人按身高排队,每次让队首较矮者出列。 i 扫左半、 j 扫右半、 k 写 tmp ;某半耗尽,另一半剩余整体搬入。

示例 7-4 归并排序(辅助数组全程复用一份)

```
// 合并 a[l..m] 与 a[m+1..r],写回 a[l..r]
void merge(vector<int>& a, vector<int>& tmp, int l, int m, int r) {
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        if (a[j] < a[i]) tmp[k++] = a[j++];    // 右半更小才取右
        else tmp[k++] = a[i++];                // 相等取左 → 稳定
    }
    while (i <= m) tmp[k++] = a[i++];
    while (j <= r) tmp[k++] = a[j++];
    for (k = l; k <= r; ++k) a[k] = tmp[k];
}

void mergeSortRec(vector<int>& a, vector<int>& tmp, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSortRec(a, tmp, l, m);
    mergeSortRec(a, tmp, m + 1, r);
    merge(a, tmp, l, m, r);
}

void mergeSort(vector<int>& a) {
    vector<int> tmp(a.size());    // 唯一一份辅助数组,全程复用
    mergeSortRec(a, tmp, 0, int(a.size()) - 1);
}
```

🚀 实现建议:C++ 必须显式管理辅助数组

C++ 没有"列表拼接"语法,合并用一块全程复用的 tmp (示例 7-4):每次 merge 新建临时 vector 会每层递归反复分配,常数翻几倍。辅助数组只分配一次,在递归层间共享。

7.4.3 稳定性与空间

归并排序**稳定**:合并"相等取左半"(严格小于才取右)。代价是 $O(n)$ 额外空间。工程价值:Python 的 `sorted` 用的 Timsort 就是"归并 + 插入"混合;C++ 要稳定排序用 `std::stable_sort`。递归深度仅 $\log_2 n$,无栈溢出风险。

⚠ 误区 4:merge 的边界错误(漏掉剩余半区或忘记拷回)

两个高频错:① 主循环后忘了把某半剩余元素搬进 tmp(奇数长度必错);② 结果只写进 tmp、忘了拷回原数组(上层读到旧数据)。调试技巧:用 5 个元素手工走一遍。

§7.5 快速排序:工程之王

7.5.1 思想与最坏情形

快排也是分治,但与归并的"先递归后合并"相反:难点在"分割"——选一个**枢轴(pivot)**,把数组分成"小于枢轴"与"大于枢轴"两半,枢轴落到最终位置,再递归排序两半。分割后无需合并——这是它常数小的根源。平均枢轴居中,递推式仍是 $T(n) = 2T(n/2) + n, O(n \log n)$; 最坏 $O(n^2)$:每次分割极度不平衡——典型场景"已有序 + 首元素枢轴",退化为 $T(n) = T(n-1) + n$ 。

7.5.2 三数中值分割法

对付最坏情形的标准武器是**三数中值分割法(median-of-three)**:取左端、中间、右端三个元素,中值当枢轴,顺便把三者排好序——左、右端变成"哨兵",枢轴藏中间。已有序数组会选中真正的中间值,分割永远均衡,退化被根治。

示例 7-5 快速排序:三数中值 + 分割 + 小区间插入

```

const int CUTOFF = 16;    // 长度 ≤ 16 改用插入排序

int median3(vector<int>& a, int left, int right) {
    int center = (left + right) / 2;
    if (a[center] < a[left]) swap(a[left], a[center]);
    if (a[right] < a[left]) swap(a[left], a[right]);
    if (a[right] < a[center]) swap(a[center], a[right]);
    swap(a[center], a[right - 1]);    // 枢轴藏到 right-1
    return a[right - 1];
}

void quickSortRec(vector<int>& a, int left, int right) {
    if (left + CUTOFF <= right) {
        int pivot = median3(a, left, right);
        int i = left, j = right - 1;
        for (;;) {
            while (a[++i] < pivot) {} // 找 ≥ 枢轴者(a[left] 哨兵)
            while (pivot < a[--j]) {} // 找 ≤ 枢轴者(a[right-1] 哨兵)
            if (i < j) swap(a[i], a[j]);
            else break;
        }
        swap(a[i], a[right - 1]);    // 枢轴归位
        quickSortRec(a, left, i - 1);
        quickSortRec(a, i + 1, right);
    } else {
        insertionSort(a, left, right); // 小区间插入(§7.1)
    }
}

```

为什么循环不会越界?三数中值已保证 $a[\text{left}] \leq \text{枢轴}$ 且 $a[\text{right}] \geq \text{枢轴}$,两个"哨兵"让 `++i` 与 `--j` 必然停下——这正是"顺便排好序"的深意。

7.5.3 小区间插入排序优化

快排递归到小区间时,插入排序反而更快:递归有开销、插入常数极小、缓存友好。工程实现普遍在长度 ≤ 16 时切换: `CUTOFF = 16 = "快排大区间 + 插入小区间"`。 `std::sort` 也这么做。

7.5.4 ★从 C 的 qsort 到 C++ 的 std::sort

C 的 qsort(函数指针 + void*)	C++ 的 std::sort(模板 + lambda)
qsort(a, n, sizeof(int), cmp):返回负/零/正	sort(a.begin(), a.end()):默认升序,一行
int cmp(const void*, const void*) 手动强转,无类型检查	模板推导类型,错误编译期报错
自定义顺序只能写死返回值规则	lambda: sort(v.begin(), v.end(), [](int x, int y) { return x > y; }) 降序

示例 7-6 std::sort 的四种用法

```
#include <algorithm>
vector<int> v = {5, 2, 8, 1, 9};
sort(v.begin(), v.end()); // 升序
sort(v.begin(), v.end(), greater<int>()); // 降序
sort(v.begin(), v.end(),
    [](int x, int y) { return x % 10 < y % 10; }); // 按个位数升序
vector<string> names = {"bob", "alice", "carl"};
sort(names.begin(), names.end(),
    [](const string& x, const string& y) { return x.size() < y.size(); });
// 按长度升序;要稳定必须用 stable_sort!
```

⚠ 误区 5:把 C 的三值比较语义搬进 lambda

qsort 返回"负/零/正", `std::sort` 的比较器返回 `bool`。写 `[](int x, int y) { return x - y; }` 不报错但含义全错:返回 0 或负值都是"真"! 比较器永远写 `return x < y;`。这是 C 迁移时最高频的 bug。

7.5.5 快排的稳定性

快排**不稳定**:分割的"交换"会把相等元素隔空换位。反例:[3, 5_a, 2, 5_b, 1],取 3 当枢轴:分割时 5_a 与 1 交换,越过 5_b,次序破坏。需要稳定:小数据用插入,大数据用 `std::stable_sort`。

§7.6 桶式排序与外部排序:跳出"比较"的框架

7.6.1 桶式排序:用空间换时间

前面五种排序都属于"比较排序"。**桶式排序(bucket sort)** 完全不做比较:把值直接当下标,数每个值出现几次,再按顺序倒出来——即**计数排序(counting sort)**(每桶一个值)。复杂度 $O(n + M)$, M 是值域; $M = O(n)$ 时即 $O(n)$ 。

示例 7-7 计数排序(前缀和 + 倒序,稳定)

```
// 值域 [0, M) 的非负整数;cnt 兼作位置账本
void countingSort(vector<int>& a, int M) {
    vector<int> cnt(M, 0), out(a.size());
    for (int x : a) ++cnt[x]; // 1. 计数
    for (int i = 1; i < M; ++i) cnt[i] += cnt[i - 1]; // 2. 前缀和
    for (int i = int(a.size()) - 1; i >= 0; --i) // 3. 倒序扫描
        out[--cnt[a[i]]] = a[i]; // 稳定:相等后到先取
    a = out;
}
```

C 的计数排序

```
int *cnt = calloc(M,
sizeof(int)); 用完 free,忘了即泄漏
```

C++ 的计数排序

```
vector<int> cnt(M, 0) 自动管理内存
```

7.6.2 比较下界: $\Omega(n \log n)$ 的决策树直觉

为什么桶排能是 $O(n)$?因为任何只靠"比较"的排序都有下界 $\Omega(n \log n)$,而桶排不比较。一句话直觉:一次比较最多把 $n!$ 种排列一分为二;要确定唯一正确排列,至少需要 $\log_2(n!) \approx n \log_2 n$ 次比较。 $O(n)$ 排序必须另辟蹊径——利用值域结构,而非元素间比较。

桶排的适用条件因此苛刻: ① 数据是(或可映射成)小范围整数;② 值域与数据量同阶;③ 分布大致均匀。工程上主要用于成绩统计、颜色直方图、基数排序底层($O(d(n+M))$)。

7.6.3 外部排序:内存装不下时

外部排序(external sort) 解决"数据比内存大"的问题(数据库、日志)。思路是归并的外延:数据切成若干块,每块内存排序后写回磁盘;再开小顶堆,放每个已排块的当前最小元素,每次弹出全局最小者写入输出,再从该块补进下一个——弹出、补入、再弹出直至读完;输出仍装不下就多趟合并。分块排序用 §7.4 的归并,"多路取最小"正是第6章的小顶堆——外部排序 = 归并思想 + 堆结构 + 磁盘代替内存。

⚠ 误区 6:对不满足条件的数据用桶排

对 10^6 个浮点数或值域 10^9 的整数用计数排序,内存直接爆炸(要 M 个计数器)。判断口诀: 值域多大?是整数吗?分布均匀吗?任何一条不满足,老老实实用 $O(n \log n)$ 通用排序。工程上 99% 的场景都是通用排序,桶排是值域受限场景的特供品。

§7.7 排序算法总表:选型决策

7.7.1 七种排序全景表

算法	平均	最坏	空间	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	约 $O(n^{3/2})$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
桶排(计数)	$O(n + M)$	$O(n + M)$	$O(M)$	稳定

读表要点: 只有插入、冒泡能利用已有序(最好 $O(n)$);只有归并和桶排稳定;只有堆排最坏 $O(n \log n)$ 且 $O(1)$ 空间;快排平均最快但最坏 $O(n^2)$ 。没有一种全占——选型即权衡。

7.7.2 选型决策树

- 数据量很小或基本有序 → **插入排序**
- 需要稳定、空间允许 → **归并排序** (`std::stable_sort`)
- 内存紧张、最坏必须保证 → **堆排序**
- 通用场景、要快 → **快速排序**
- 值域小的整数 → **桶排/计数排序**
- 其余一切情况 → **标准库 sort/sorted**,不要手写

7.7.3 标准库是工程默认

最后一条最重要: 工程上永远优先 `std::sort` ——它是 introsort:快排为主,递归过深自动切堆排(防最坏退化),小区间自动切插入排序。本章手写的三种优化标准库全部内置,还多一道防退化保险。 `std::stable_sort` 走归并家族,稳定且同样快。 手写排序的意义:理解原理、应对考试;生产代码直接用标准库 ——本章实验"手写三种排序并统计比较次数",正是为亲手验证第2章的复杂度理论。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
插入排序	随机/逆序数组	$O(n^2)$	$O(n^2)$
插入排序	已有序数组	$O(n)$	$O(n)$
冒泡排序	相邻交换	$O(n^2)$	$O(n^2)$
希尔排序	希尔增量	约 $O(n^{3/2})$	$O(n^2)$
希尔排序	Hibbard 增量	约 $O(n^{3/2})$	$O(n^{3/2})$
堆排序	建堆阶段	$O(n)$	$O(n)$
堆排序	完整排序	$O(n \log n)$	$O(n \log n)$
归并排序	完整排序	$O(n \log n)$	$O(n \log n)$
归并排序	额外空间	$O(n)$	$O(n)$
快速排序	随机数据	$O(n \log n)$	$O(n \log n)$
快速排序	已有序 + 首元素枢轴	$O(n^2)$	$O(n^2)$
桶排(计数排序)	值域 M	$O(n + M)$	$O(n + M)$
比较下界	基于比较的排序	$\Omega(n \log n)$ (决策树)	
归并两个有序数组	线性合并	$O(n)$	$O(n)$
外部排序	多路归并	磁盘 I/O 主导;比较部分 $O(n \log n)$	

最小必会清单

- 能默写插入排序(key 暂存三步),用 Σ 推导最坏 $O(n^2)$ 、近有序 $O(n)$
- 能说出希尔排序动机、希尔增量与 Hibbard 差别、复杂度约 $O(n^{3/2})$
- 能默写原地堆排序(0 基下滤、建堆、交换缩小),说出它不稳定并给反例
- 能默写归并的 merge,说清"相等取左"保证稳定、辅助数组 $O(n)$ 全程复用
- 能默写快排的三数中值 + 分割 + 小区间插入,解释哨兵防越界
- 能说出快排最坏 $O(n^2)$ 何时发生、三数中值为何能治
- 能说出比较下界 $\Omega(n \log n)$ 的决策树直觉与桶排为何突破
- 能说出桶排三个适用条件,写出稳定版计数排序
- 能背出七种排序复杂度总表(最好/平均/最坏/空间/稳定性)

- 能写 `std::sort` 的 lambda 比较器与 `stable_sort`,说出与 `qsort` 三值语义的区别

自测题

Q 1. 插入排序对已有序数组的复杂度是多少?为什么?

✓ 参考答案

$O(n)$ 。每个元素与前驱比较 1 次就停止,共 $n-1$ 次比较、0 次移动。这也是"近有序接近 $O(n)$ "的来源:总代价 = 逆序对数。

Q 2. 堆排序为什么不稳定?构造一个反例。

✓ 参考答案

建堆与"根与末尾交换"都可能让相等元素换位。反例:[$4_a, 4_b, 3$] 建最大堆后根 4_a 与末尾 3 交换得 [$3, 4_b, 4_a$], 4_b 最终在前。

Q 3. 归并排序的空间复杂度是多少?为什么它不能像堆排那样原地?

✓ 参考答案

$O(n)$ 辅助数组。合并时两半在原数组中交错,原地交换会破坏"两半各自有序"的前提;原地归并要么 $O(n^2)$ 时间,要么复杂块交换技巧,得不偿失。

Q 4. 快排的最坏情形是什么?三数中值分割法为什么能缓解?

✓ 参考答案

每次分割都极度不平衡时(递推式 $T(n) = T(n-1) + n \rightarrow O(n^2)$),典型场景是已有序数组配"首元素枢轴"。三数中值取左、中、右的中值:已有序数组的中值恰是中间值,分割均衡;顺带排好 $a[\text{left}] \leq \text{枢轴} \leq a[\text{right}]$ 作哨兵。

Q 5. 为什么桶排能做到 $O(n)$,却不受"比较下界 $\Omega(n \log n)$ "的限制?

✓ 参考答案

决策树论证只适用于"通过比较获得信息"的算法:每次比较最多把 $n!$ 种排列一分为二,故至少 $\log_2(n!) \approx n \log_2 n$ 次。桶排不比较,直接用值域做索引(计数),绕开决策树,所以可 $O(n)$;代价是空间 $O(M)$ 与苛刻的适用条件。

章末实验题:三种排序的性能对比实验

实验 7:三种排序性能对比(三种数据分布)

实验目标:实现插入、归并、快排,在随机/近有序/逆序三种分布上实测 $n = 1000 \dots 64000$ 的耗时,用" n 翻倍时间比值"验证理论复杂度。

实验要求:

- 任务 1:实现 insertionSort、mergeSort、quickSort(\$7.1/\$7.4/\$7.5)
- 任务 2:三种分布 $\times$ 七个规模 n 的耗时表;插入 $n > 8000$ 标注"(推算)"
- 任务 3:与 `std::sort` 对比验证输出一致,含重复元素
- 任务 4:" n 翻倍时间比值"与理论对比:插入 $\times 4(n^2)$ 、归并/快排 $\approx \times 2(n \log n)$
- 任务 5(加分):统计比较/交换次数,验证插入比较次数 $\approx n^2/4$ (逆序对)

实验环境:C++17。编译: `g++ -std=c++17 -O2 lab7.cpp -o lab7`;运行: `./lab7`。

第一步 排务必用三数中值 + CUTOFF=16 小区间插入(\$7.5 完整版),否则逆序数据退化 $O(n^2)$,与理论演示矛盾

第二步 时取"同数据 3 次最小值"、用副本、结果读走防优化,固定种子 12345 可复现;近有序用"已排序 + 固定 20 次随机交换"生成(逆序对 $O(n)$),才能观察插入的 $O(n)$ 特性

第三步 务 4 比值只看"翻倍相邻行",小 n 计时噪声大,偏离属正常,看趋势即可

✅ 参考答案(完整可运行程序,分六段)

```
// lab7.cpp — 三种排序的性能对比实验(插入/归并/快排)
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <random>
#include <vector>
using namespace std;
using Clock = chrono::steady_clock;

const int CUTOFF = 16;          // 快排小区间阈值($7.5)
long long g_cmp = 0, g_move = 0;
bool g_count = false;          // 任务 5 计数开关(计时时关)
inline bool lt(int x, int y) { if (g_count) ++g_cmp; return x < y; }
inline void exch(int& x, int& y) { if (g_count) ++g_move; swap(x, y); }

void insertionSort(vector<int>& a, int left, int right) {
    for (int i = left + 1; i <= right; ++i) {
        int key = a[i];
        int j = i - 1;
        while (j >= left && lt(key, a[j])) { // 严格大于才右移:稳定
            a[j + 1] = a[j];
            if (g_count) ++g_move;
            --j;
        }
        a[j + 1] = key;
        if (g_count) ++g_move;
    }
}

void insertionSortAll(vector<int>& a) { insertionSort(a, 0, int(a.size()) - 1); }
```

lab7.cpp(第二段:归并排序)

```

void merge(vector<int>& a, vector<int>& tmp, int l, int m, int r) {
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        if (lt(a[j], a[i])) tmp[k++] = a[j++]; // 右半更小才取右:相等取左,稳定
        else tmp[k++] = a[i++];
        if (g_count) ++g_move;
    }
    while (i <= m) { tmp[k++] = a[i++]; if (g_count) ++g_move; }
    while (j <= r) { tmp[k++] = a[j++]; if (g_count) ++g_move; }
    for (k = l; k <= r; ++k) { a[k] = tmp[k]; if (g_count) ++g_move; }
}

void mergeSortRec(vector<int>& a, vector<int>& tmp, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSortRec(a, tmp, l, m);
    mergeSortRec(a, tmp, m + 1, r);
    merge(a, tmp, l, m, r);
}

void mergeSort(vector<int>& a) {
    vector<int> tmp(a.size()); // 辅助数组只分配一次
    mergeSortRec(a, tmp, 0, int(a.size()) - 1);
}

```

lab7.cpp(第三段:快速排序与数据生成)

```

int median3(vector<int>& a, int left, int right) {
    int center = (left + right) / 2;
    if (lt(a[center], a[left])) exch(a[left], a[center]);
    if (lt(a[right], a[left])) exch(a[left], a[right]);
    if (lt(a[right], a[center])) exch(a[center], a[right]);
    exch(a[center], a[right - 1]); // 枢轴藏到 right-1
    return a[right - 1];
}

void quickSortRec(vector<int>& a, int left, int right) {
    if (left + CUTOFF <= right) {
        int pivot = median3(a, left, right);
        int i = left, j = right - 1;
        for (;;) {
            while (lt(a[++i], pivot)) {} // 哨兵 a[left]<=pivot 保证能停
            while (lt(pivot, a[--j])) {} // a[right-1]==pivot 保证能停
            if (i < j) exch(a[i], a[j]);
            else break;
        }
        exch(a[i], a[right - 1]); // 枢轴归位
        quickSortRec(a, left, i - 1);
        quickSortRec(a, i + 1, right);
    } else {
        insertionSort(a, left, right); // 小区间优化($7.1 预告)
    }
}

void quickSort(vector<int>& a) { quickSortRec(a, 0, int(a.size()) - 1); }

enum Kind { RANDOM, NEARLY, REVERSED };
const char* kindName[] = {"随机", "近有序", "逆序"};
vector<int> makeData(int n, Kind k, mt19937& rng) {
    vector<int> a(n);
    for (int i = 0; i < n; ++i) a[i] = i + 1;
    if (k == RANDOM) {
        for (int i = n - 1; i > 0; --i)
            swap(a[i], a[uniform_int_distribution<int>(0, i)(rng)]);
    } else if (k == NEARLY) {
        for (int i = 0; i < 20; ++i) // 固定 20 次随机交换:逆序对 O(n)
            swap(a[uniform_int_distribution<int>(0, n - 1)(rng)],
                a[uniform_int_distribution<int>(0, n - 1)(rng)]);
    } else {
        for (int i = 0; i < n; ++i) a[i] = n - i;
    }
    return a;
}

```

| lab7.cpp(第四段:计时与正确性验证)

```

// 计时:3 次取最小,每次用副本,结果读走防优化
double msTime(void (*f)(vector<int>&), vector<int> a, long long& guard) {
    double best = 1e18;
    for (int r = 0; r < 3; ++r) {
        vector<int> b = a;
        auto t0 = Clock::now();
        f(b);
        guard += b.front();
        best = min(best, chrono::duration<double, milli>(Clock::now() - t0).count());
    }
    return best;
}

int main() {
    mt19937 rng(12345);          // 固定种子,可复现
    const vector<int> sizes = {1000, 2000, 4000, 8000, 16000, 32000, 64000};
    double ins[3][7] = {}, mrg[3][7] = {}, qck[3][7] = {};
    long long guard = 0;

    // 任务 3:正确性验证(与 std::sort 一致,含重复元素)
    bool ok = true;
    for (int k = 0; k < 4; ++k) {
        vector<int> a = makeData(500, Kind(k % 3), rng);
        if (k == 3) for (int& x : a) x = uniform_int_distribution<int>(0, 20)(rng);
        vector<int> ref = a, b;
        sort(ref.begin(), ref.end());
        b = a; insertionSort(b, 0, int(b.size()) - 1); ok &= (b == ref);
        b = a; mergeSort(b);                          ok &= (b == ref);
        b = a; quickSort(b);                          ok &= (b == ref);
    }
    cout << "任务 3:三种算法与 std::sort 输出一致: " << (ok ? "是" : "否") << "\n";
}

```

lab7.cpp(第五段:耗时表与比值分析)

```

// 任务 2:耗时表(插入 n>8000 标注"(推算)")
cout << "任务 2:耗时表(单位 ms;插入排序 n>8000 为(推算))\n";
for (int k = 0; k < 3; ++k) {
    cout << "\n[" << kindName[k] << "]\n";
    cout << left << setw(8) << "n" << setw(11) << "插入" << setw(11) << "归并" << "快排\n";
    for (size_t s = 0; s < sizes.size(); ++s) {
        vector<int> a = makeData(sizes[s], Kind(k), rng);
        cout << setw(8) << sizes[s];
        if (sizes[s] <= 8000) {
            ins[k][s] = msTime(insertionSortAll, a, guard);
            cout << setw(11) << fixed << setprecision(2) << ins[k][s];
        } else { ins[k][s] = -1; cout << setw(11) << "(推算)"; }
        mrg[k][s] = msTime(mergeSort, a, guard);
        qck[k][s] = msTime(quickSort, a, guard);
        cout << setw(11) << mrg[k][s] << qck[k][s] << "\n";
    }
}

// 任务 4:比值分析(理论:插入 x4;归并/快排 ≈x2)
cout << "\n任务 4:n 翻倍时实测时间比值(理论:n² 算法 x4,n log n 算法 ≈x2)\n";
auto ratios = [&](const char* name, double t[3][7]) {
    cout << name << ": ";
    for (int k = 0; k < 3; ++k) {
        cout << kindName[k] << "[";
        for (size_t s = 1; s < sizes.size(); ++s)
            if (t[k][s] > 0 && t[k][s - 1] > 0) cout << setprecision(2) << (t[k][s] / t[k][s - 1])
<< " ";
        cout << "]" ";
    }
    cout << "\n";
};
ratios("插入", ins);
ratios("归并", mrg);
ratios("快排", qck);

```

lab7.cpp(第六段:统计与结论)

```

// 任务 5(加分):比较/移动次数统计
g_count = true;
vector<int> a = makeData(1000, RANDOM, rng), b;
g_cmp = 0; g_move = 0; b = a; insertionSort(b, 0, int(b.size()) - 1);
cout << "\n任务 5(n=1000 随机):插入 比较=" << g_cmp
<< " 移动=" << g_move << "\n";
g_cmp = 0; g_move = 0; b = a; mergeSort(b);
cout << "          归并 比较=" << g_cmp << " 移动=" << g_move << "\n";
g_cmp = 0; g_move = 0; b = a; quickSort(b);
cout << "          快排 比较=" << g_cmp << " 交换=" << g_move << "\n";
g_count = false;

cout << "\n结论:实测比值与理论吻合—插入 n 翻倍约 x4,归并/快排约 x2;"
      "逆序上插入退化最明显,近有序上插入接近 O(n),快排三数中值使逆序不再退化。 \n";
cout << "(防优化累加值: " << guard << ")\n";
return 0;
}

```

示例输出(本机真实运行,不同机器数值不同,比值规律一致):

任务 3:三种算法与 `std::sort` 输出一致: 是

任务 2:耗时表(单位 `ms`;插入排序 $n > 8000$ 为(推算))

[随机]

n	插入	归并	快排
1000	0.06	0.03	0.02
2000	0.24	0.08	0.06
4000	0.93	0.20	0.13
8000	3.64	0.41	0.29
16000	(推算)	0.87	0.62
32000	(推算)	1.88	1.35
64000	(推算)	4.04	2.89

[近有序] 表与 [逆序] 表见完整输出:插入在近有序上快一个数量级(8000 时仅 `0.03ms`),逆序上最慢(`7.25ms`)

任务 4:n 翻倍时实测时间比值(理论: n^2 算法 $\times 4$, $n \log n$ 算法 $\approx \times 2$)

插入: 随机[3.77 3.91 3.91] 近有序[1.63 1.38 2.30] 逆序[3.87 3.94 3.99]

归并: 随机[2.53 2.46 2.07 2.14 2.17 2.14](近有序/逆序各列也均 ≈ 2)

快排: 随机[2.54 2.19 2.19 2.16 2.17 2.14](同上)

任务 5($n=1000$ 随机):插入 比较=251264 移动=251270

归并 比较=8720 移动=19952

快排 比较=10671 交换=4969

读表要点:① 插入在随机/逆序上比值 $\approx \times 4$ (吻合 n^2),近有序上 $\approx \times 2$ (吻合 $O(n)$);② 归并/快排 $\approx \times 2$ (吻合 $n \log n$, 三种分布皆然);③ 快排逆序不退化——三数中值选中中间值,§7.5 优化的实战验证;④ 插入比较 $251264 \approx 1000^2/4$,落在平均情形的理论上。

设计说明:① `lt / exch` 计数钩子使比较/移动定义与正文一致;② 计时取 3 次最小值、用新副本、结果读走防优化;③ 插入 $n > 8000$ 标"(推算)": $\times 4$ 规律已由 $1000 \rightarrow 8000$ 确认,与第2章实验一脉相承;④ 近有序用固定 20 次交换生成(逆序对 $O(n)$),固定种子 12345 可复现。

下一章预告:第8章 不相交集——`union/find` 与路径压缩,一种"只回答是/否连通"的轻量数据结构;第9章图论算法中的 Kruskal 最小生成树将直接使用它。

第8章 不相交集

建议学时:3-4 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**等价关系**三性质与"动态"等价问题为何要求在线回答
- 掌握树形表示:每集合一棵树、根是**代表元**,一个 `parent` 数组即可实现
- 完成 C 数组到 C++ `vector` 的迁移,写出朴素 `find / union`,亲手踩一遍"链退化"
- 掌握**按大小合并**、**按秩合并**(树高 $O(\log n)$)与**路径压缩**(均摊近 $O(1)$)
- 学完能用并查集生成随机迷宫、统计连通分量,为第9章 Kruskal 铺路

💡 从 C 到 C++:本章主题如何迁移

本章是全书**代码量最小、思想密度最高**的一章:完整实现约 30 行,却支撑着网络连通、迷宫生成、第9章 Kruskal 算法等大应用。先给结论:不相交集(并查集)只做一件事——维护一堆集合,支持"查归属"与"合并"两种操作。

在 C 里你天然会想到笨办法:开 `mark[N]` 数组记录每个元素属于第几组,合并时遍历整个数组把所有第 b 组的元素改成第 a 组。能工作,但每次合并 $O(n)$,合并 n 次就是 $O(n^2)$ 。本章的树形表示把合并降为"改一个指针"——"用树的结构信息换时间"的经典一步。

C++ 的迁移点有两处:① 用 `vector<int>` 取代 `int parent[N]`,初始化用 `iota` 一行搞定;② 用函数封装 `find / unionSets`,而不是把 `parent` 数组到处裸传。标准库没有并查集,这一章**手写就是全部**——每一行都要理解到位。

本章路线:先定义问题(§8.1),写朴素实现踩一遍退化的坑(§8.2),依次加两种优化(§8.3-§8.4),看清复杂度如何摊还到近 $O(1)$ (§8.5),最后在迷宫与连通分量中验证(§8.6-§8.7)。

📦 前置知识自查

数学前置:等价关系三性质(自反、对称、传递);对数(第2章)——按大小合并的高度证明用 $O(\log n)$ 。

C 语言前置:`int` 数组、`for` 循环初始化、递归函数(§8.4 递归 `find` 需要)。

依赖的前面章节:第3章 `vector` (存放 `parent` 数组)、第4章树的父指针表示(`parent` 数组正是"树"的另一种存法)。

🚀 本章学习路线

1. **第一阶段(1 小时):**读 §8.1-§8.2,跑通朴素实现,手工模拟"链退化"
2. **第二阶段(1.5 小时):**读 §8.3-§8.4,实现按秩合并与路径压缩
3. **第三阶段(1 小时):**读 §8.5-§8.7,理解摊还直觉,用并查集生成迷宫
4. **第四阶段(实验,1 小时):**完成章末实验:迷宫生成 + 连通分量统计 + 性能对比
5. **验收标准:**能默写 20 行内的完整并查集,并说出三种配置的复杂度

本章知识导图

- §8.1 等价关系与动态等价问题:三性质、find/union、为何"动态"
- §8.2 基本实现:树表示与朴素 union/find(根=代表元、链退化)
- §8.3 按大小合并与按秩合并:size/rank 数组、树高 $O(\log n)$ 的证明
- §8.4 路径压缩:递归与迭代两写法、压扁树的机制、叠加效果
- §8.5 复杂度:摊还近 $O(1)$ 的直觉、反阿克曼函数 $\alpha(n)$
- §8.6 应用:迷宫生成、连通分量统计、Kruskal 预告、替代方案对比
- §8.7 实现要点与调试:初始化、先 find 到根、秩不失效、打印 parent

§8.1 等价关系与动态等价问题

8.1.1 等价关系:三性质

等价关系(equivalence relation)是定义在集合上的二元关系,满足三条性质:① **自反**: $a \equiv a$;② **对称**: $a \equiv b$ 则 $b \equiv a$;③ **传递**: $a \equiv b$ 且 $b \equiv c$, 则 $a \equiv c$ 。三条缺一不可。等价关系把集合划分成若干**等价类**:互相等价的元素属于同一类——这些类互不相交,正是"不相交"一词的来历。

例子:整数"相等"、整数"模 m 同余"、电路里"两点由导线连通",都是等价关系。再看**反例**:中学的"同班同学"——自反、对称都满足,但**不传递**:甲、乙同班,乙、丙同班,甲、丙未必同班。它不是等价关系,不能用并查集建模"全班连通"。

8.1.2 动态等价问题:union 与 find

给定 N 个元素,初始时每个元素自成一个集合。支持两种操作:

- **find(x)**:返回 x 所在集合的名字——用该集合的**代表元**(representative)当名字;
- **union(a, b)**:把 a 、 b 所在的两个集合合并成一个(若本就在同一集合,什么都不做)。

注意"动态"二字的分量:若所有关系一次性给定再问"1 号和 5 号通不通",那是**静态**问题,遍历一遍算出全部等价类即可(第9章的 DFS 能做)。实际场景中 **查询与合并交错到来**:接入一根新网线(合并),立刻有人问两台机器通不通(find),接着又接新线..... 每一步都必须在当下即时回答,不能等收集完所有操作再算。这正是"动态等价问题"的在线(on-line)性质,也是必须专门设计数据结构的理由。

8.1.3 应用预告

后面你会反复见到它:① **迷宫生成**:随机拆墙但保证任意两格连通(§8.6 详解,章末实验);② **Kruskal 最小生成树**(第9章):按边权从小到大加边,用 **find** 判断两端是否已连通——并查集是它的灵魂;③ **工程应用**:网络断点检测、图像连通域标记。记住一句话:凡是"合并集合 + 查询归属"成对出现的问题,先想并查集。

§8.2 基本实现:树表示与朴素 union/find

8.2.1 树表示:根就是代表元

怎么表示"一组元素属于同一集合"?答案是**树**:每个集合一棵树,树的**根是代表元**——**find(x)** 沿父链上溯到根并返回根,集合的名字即根;合并时让一棵树的根指向另一棵树的根,只改一个指针。这是第4章"父指针表示法"的直接应用:**parent[i]** 是结点 i 的父结点,根满足 **parent[root] == root** ("我是根"的哨兵约定)。

8.2.2 从 C 的数组到 C++ 的 vector

C 里写 `int parent[N];` C++ 换成 `vector<int>`, 长度运行期决定, `iota` 一行初始化:

C 的写法	C++ 的写法
<code>int parent[N];</code> 长度写死	<code>vector<int> parent(n);</code> 长度运行期决定
<code>for (i=0; i<N; ++i) parent[i] = i;</code>	<code>iota(parent.begin(), parent.end(), 0);</code> 一行初始化
越界访问 = 未定义行为	<code>parent.at(i)</code> 越界抛异常, 调试期友好
函数传参要带长度 N	<code>parent.size()</code> 自带长度

示例 8-1 朴素实现: `find` 沿父链上溯, `union` 让根指向根

```
// parent[i] == i 表示 i 是根; 其余 parent[i] 是 i 的父结点
vector<int> parent;

int find(int x) {                                // 朴素 find: 沿父链上溯到根
    while (parent[x] != x)
        x = parent[x];
    return x;
}

void unionSets(int a, int b) {                    // 朴素 union: 让 a 的根指向 b 的根
    int ra = find(a), rb = find(b);
    if (ra != rb)
        parent[ra] = rb;
}

int main() {
    int n = 8;
    parent.resize(n);
    for (int i = 0; i < n; ++i)
        parent[i] = i;                          // 初始化: 每个元素都是自己的根
    unionSets(1, 2);
    unionSets(3, 4);
    unionSets(2, 3);
    return find(4);
}
```

★ 重点: 三个必须想清楚的点

① `union` 前必须先 `find`: 合并的是"两个集合"不是"两个元素", 拿根来挂; 只有根能当代表元, 挂普通结点会破坏树结构。② `parent` 里存的是"父结点"不是"集合编号": 集合没有全局编号, 根的名字就是集合的名字, `find` 必须走到根才能回答归属。③ 初始化绝不能省: `parent[i] = i` 让每个元素自成一棵单结点树, 这是推理的地基。

8.2.3 朴素实现的致命伤:链退化

⚠ 误区 1:朴素 union 会把树连成一条链

依次执行 `union(0,1)`, `union(1,2)`, ..., `union(n-2, n-1)`:第一次让 0 指向 1,第二次让 1 指向 2.....`parent` 变成 `0->1, 1->2, ..., n-2->n-1`,树被拉成一条 n 结点的链。`find(0)` 要走 $O(n)$ 步,做 n 次 `find` 就是 $O(n^2)$ 。链 = 树高的灾难:树形表示的全部好处都建立在"树要矮"上,§8.3 的两种合并策略正是从"让树长不高"入手。

§8.3 按大小合并与按秩合并

8.3.1 按大小合并:小树并入大树

按大小合并(union by size)的规则一句话:总是把结点数少的树挂到结点数多的树下。为此维护 `size` 数组: `size[root]` 是根所在树的结点数(只在根上有效),合并时小根指向大根并累加。

示例 8-2 按大小合并: `size` 数组只在根上有效

```
// sz[root] = 根所在树的结点数(只在根上有效)
vector<int> parent, sz;

int find(int x) {
    while (parent[x] != x) x = parent[x];
    return x;
}

void unionBySize(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return;           // 已在同一集合
    if (sz[ra] < sz[rb]) swap(ra, rb); // 小树并入大树
    parent[rb] = ra;                // rb 不再是根, sz 作废
    sz[ra] += sz[rb];
}
```

8.3.2 按秩合并:浅树并入深树

按秩合并(union by rank)是"按大小"的孪生兄弟:目标都是**别让树长高**。它维护 `rank` 数组: `rank[root]` 是树高的上界(初始 0)。合并时秩小的树挂到秩大的树下;秩相同则任选一树做根并把秩加一。实现更短,且"上界"语义给 §8.4 路径压缩更大自由度。

示例 8-3 按秩合并:浅树挂到深树下

```
// rank[root] = 树高的上界(只在根上有效),初始 0
vector<int> parent, rank_;

int find(int x) {
    while (parent[x] != x) x = parent[x];
    return x;
}

void unionByRank(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return;
    if (rank_[ra] < rank_[rb]) swap(ra, rb); // 浅树挂到深树
    parent[rb] = ra;
    if (rank_[ra] == rank_[rb]) // 两棵同秩树合并
        ++rank_[ra];          // 新树秩 +1
}
```

8.3.3 为什么树高是 $O(\log n)$

两种策略是同一句证明: 一个结点深度每增加 1, 它所在树的规模至少翻一倍。x 深度增加只发生在“它的树被并入另一棵树”时: 按大小合并规定小树并入大树, 新树大小 $\geq 2 \times$ 旧树; 按秩合并规定浅树并入深树, 而秩 r 的树至少 2^r 个结点(归纳可证)。故 x 深度最多增加 $\log_2 n$ 次, 树高 $\leq \log_2 n$, find 保证 $O(\log n)$ 。 $n = 10^6$ 时 $\log_2 n \approx 20$, 链却是 10^6 。

策略	维护什么	合并规则	何时 +1	附带能力
按大小合并	size[root] = 结点数	小树并入大树	新根 size 累加	可查询集合大小
按秩合并	rank[root] = 高度上界	浅树并入深树	两秩相等时 +1	与路径压缩配合更顺

★ 重点: 两种策略等价, 选一个记住

两者都保证树高 $O(\log n)$, 复杂度相同。按秩合并代码更短, 且“秩是上界”的语义与路径压缩天然兼容(见 §8.7.2); 按大小合并的额外好处是能回答“这个集合有几个元素”。本章正文与实验采用按秩合并, 换成 size 只需三行改动。

§8.4 路径压缩

8.4.1 思想: 把树压扁

按大小/按秩合并把树高控制在 $O(\log n)$, 但还能更狠: 路径压缩(path compression)在 find 的路途中动手——既然已经走到根, 就把沿途每个结点直接挂到根下面, find(x) 之后 x 到根的距离变成 1, 整条路径被压成“菊花状”。压缩只发生在 find 上, union 调用 find 时自动受益——一次 find 的代价, 换来路径上所有结点后续 find 的 $O(1)$ 。

8.4.2 递归实现与迭代实现

示例 8-4 路径压缩: 递归与迭代两种写法

```

// 递归版:返回根,回溯时把沿途结点挂到根
int findRec(int x) {
    if (parent[x] != x)
        parent[x] = findRec(parent[x]); // 先递归到根,再逐层挂
    return parent[x];
}

// 迭代版(两趟):先找根,再沿原路压缩
int findIter(int x) {
    int root = x;
    while (parent[root] != root)
        root = parent[root]; // 第一趟:上溯找根
    while (parent[x] != x) { // 第二趟:沿途结点挂到根
        int nxt = parent[x];
        parent[x] = root;
        x = nxt;
    }
    return root;
}

```

递归版三行,但隐患:递归深度 = 树高。按秩合并时树高 $O(\log n)$ 完全安全;若只压缩不按秩(很多竞赛模板这样),最坏树高可达 n ,递归 `find` 会栈溢出。迭代版两趟循环,无栈深风险,是工程稳妥选择——本章实验用的就是迭代版。

C 的写法(递归上溯)	C++ 的写法(迭代两趟)
<pre>int f(int x){ return p[x]==x ? x : f(p[x]); }</pre>	<pre>while(p[root]!=root) root=p[root]; 第一趟找根</pre>
递归深度 = 树高,深链会爆栈	<pre>while(p[x]!=x){ nxt=p[x]; p[x]=root; x=nxt; } 再压缩</pre>
修改在回溯期,代码短	第二趟显式压缩,无栈深问题

⚠ 误区 2:递归 find 遇到深链会栈溢出

路径压缩 + 按秩合并时递归没问题;但只压缩不按秩时树高可达 n , `findRec(0)` 的递归深度就是 n ,调用栈很快爆掉(约十万层就崩)。更隐蔽的坑是**忘了用压缩版**:写了 `findIter` 却仍调用朴素 `find`,压缩永不发生。检查:连续 `find` 两次同一结点,看路径是否变短。

8.4.3 与按秩合并叠加:近 $O(1)$

按秩合并让树"长不高"(高度 $O(\log n)$),路径压缩让树"越来越扁",叠加后,并查集的操作复杂度被压到"均摊近 $O(1)$ "——精确上界是反阿克曼函数 $\alpha(n)$,见 §8.5.3。这是本数据结构的最终形态:**union = 两次 find + 一次指针修改**,**find $\approx O(1)$** ,请务必默写。

🚀 实现建议:迭代两趟的写法与验证

迭代两趟容易把变量搞混,验证三要点:① 第一趟后 `root` 必须是根(`parent[root] == root`);② 第二趟先保存 `nxt = parent[x]` 再改写 `parent[x]`,否则覆盖后丢链;③ 返回 `root` 而非 `x`。用幂等性自测:连续两次 `find(x)`,第二次必然一步到根。

§8.5 复杂度:摊还近 $O(1)$

8.5.1 三种配置,三个档次

配置	单次 find 最坏	m 次操作总代价
朴素实现(无优化)	$O(n)$ (链)	$O(mn)$
仅按大小/按秩合并	$O(\log n)$	$O(m \log n)$
按秩合并 + 路径压缩	均摊近 $O(1)$ (单次最坏仍 $O(\log n)$)	$O(m \alpha(n))$,实际当 $O(m)$ 用

读表注意"单次"与"总代价"之分:完整优化下**某一次** find 仍可能慢(遇到未压缩深链,一次性压扁),但"债"被后续 find 偿还——这就是**摊还**:把总代价摊到每次操作上,平均近 $O(1)$ 。

8.5.2 摊还近 $O(1)$ 的直觉

问: x 的 parent 会被改写几次?压缩改写它时直接挂到根,之后每次 find 一步到达;只有"它所在的树又并入更大的树"时,它才可能再被改写。按秩合并保证每次并入树至少翻倍,故每结点最多被改写 $O(\log n)$ 次——这只是粗糙上界,"压扁"带来的加速使真实总量小得多:1975 年 Tarjan 证明 m 次操作总代价 $O(m \alpha(n))$ 。工程上,把并查集操作当作 $O(1)$ 用,99.99% 的场景不会出错。

8.5.3 反阿克曼函数:为什么当常数用

反阿克曼函数 $\alpha(n)$ 是阿克曼函数(增长极快的递归函数)的反函数:使阿克曼函数达到 n 所需的最小自变量。它慢到难以想象: $\alpha(10^{80}) \leq 4 \sim 5$ ——"全宇宙原子数"级别的 n 也只让 α 走到 4、5。故 $O(m \alpha(n))$ 在实践中就是 $O(m)$:**每次 union/find 均摊近 $O(1)$,当常数用**。这与第2章"log 几乎不增长"一脉相承—— α 是更极端的"几乎不增长"。

记号说明(本章约定)

$\alpha(n)$ 读作"反阿克曼",是分析意义上的上界;标准写法"均摊近 $O(1)$ ",严谨写法 $O(\alpha(n))$ 。"摊还"≠"平均":平均对随机输入取期望,摊还对**同一个操作序列**的总代价取平均——并查集的近 $O(1)$ 不依赖任何随机性假设。

§8.6 应用:迷宫生成与连通分量

8.6.1 迷宫生成:随机拆墙,并查集判连通

把 $w \times h$ 网格的每格看成一个集合(初始互不相交),收集所有相邻格子之间的墙,**随机打乱顺序**后依次尝试拆墙:两端**不连通**就拆掉并 union,已连通则留着,直到只剩一个集合。两个性质:

- **无环**:每面被拆的墙都连接两个原本不连通的区域,永远不会形成回路;
- **全连通**:结束时只有一个集合,任意两格都能到达。

共 $n = w \times h$ 个格子,成功拆墙恰好 $n-1$ 面(每拆一面分量减一,从 n 减到 1)。 $n-1$ 面墙、无环、全连通——正是**生成树**:迷宫即网格图的随机生成树,任意两格之间有且仅有一条路径。学术上叫"随机 Kruskal 迷宫",与第9章 Kruskal 最小生成树的唯一区别:墙的"权"是随机顺序。

示例 8-5 迷宫生成核心:拆墙循环(完整程序见章末实验)

```

// 格子编号 id = r*w+c;right/down 记录每格右墙与下墙
void genMaze(int w, int h) {
    int n = w * h;
    UnionFind uf(n); // 完整优化版并查集
    vector<vector<bool>> right(h, vector<bool>(w, true)); // 右墙
    vector<vector<bool>> down(h, vector<bool>(w, true)); // 下墙
    vector<pair<int, int>> walls; // 候选墙:存两端格子编号
    auto id = [w](int r, int c) { return r * w + c; };
    for (int r = 0; r < h; ++r)
        for (int c = 0; c < w; ++c) {
            if (c + 1 < w) walls.push_back({id(r, c), id(r, c + 1)}); // 右墙
            if (r + 1 < h) walls.push_back({id(r, c), id(r + 1, c)}); // 下墙
        }
    shuffle(walls.begin(), walls.end(), rng); // 随机顺序拆墙
    int cnt = n; // 连通分量个数
    for (auto [a, b] : walls) {
        if (uf.unite(a, b)) { // 两端不连通才拆
            --cnt; // 每拆一面墙,分量减一
            // ... 对应墙置 false(见实验完整代码)
        }
        if (cnt == 1) break; // 全连通即停
    }
}

```

8.6.2 连通分量统计

并查集自带连通分量信息:初始化时分量数为 n ,每次 `union` 成功(两端原本不连通)分量数减一——生成迷宫的同时就能报出"还有几个连通块",实验任务 3 正是记录这条下降曲线。若没维护计数,也可遍历全部元素数根: `find(i)` 即 i 的代表元,根相同即同一分量。前者 $O(1)$ /次,后者 $O(n \alpha(n))$ 。

8.6.3 手写并查集 vs 集合标记法:优势场景

与本章开头提过的"笨办法"认真对比一次,才明白并查集赢在哪:

示例 8-7 替代方案:用 map 维护"元素 -> 集合编号"

```

// 替代:union 时把较小集合所有元素逐个改编号
unordered_map<int, int> owner; // 元素 -> 集合编号
int findOwner(int x) { return owner[x]; } // 查归属 O(1)

void unionOwner(int a, int b) {
    int oa = owner[a], ob = owner[b];
    if (oa == ob) return;
    // 遍历全部元素把集合 ob 的成员改成 oa:O(n)!
    for (auto& [x, o] : owner)
        if (o == ob) o = oa;
}
// 并查集:union 只改一个指针,均摊近 O(1),差距 n 倍

```


C 的连通性标记法	C++ 的并查集
mark[N] 记录归属,合并要遍历整个数组 改编号, $O(n)$	parent 树形表示,合并改一个指针,均摊近 $O(1)$
查归属 $O(1)$,但合并贵	查归属(两次 find)与合并都近 $O(1)$
能直接枚举集合的全部成员	不支持枚举成员(要枚举需另存集合列表)
合并 m 次总代价 $O(mn)$	合并 m 次总代价 $O(m \alpha(n))$

手写 vs 替代:标准库没有并查集,两种方案都得手写。选择标准: 频繁"合并 + 查归属"用并查集;需"枚举集合成员"或"合并极少"才考虑标记法。迷宫 10^6 格时,标记法每次拆墙扫描 10^6 个格子,并查集是常数步。这也解释了第9章 Kruskal 为何必须配并查集:换成标记法,排序的 $O(E \log E)$ 会被合并的 $O(E V)$ 吞掉,算法不可用。

§8.7 实现要点与调试

8.7.1 三个必须养成的习惯

- **初始化用** `parent[i] = i (或 iota)`:漏掉初始化,find 沿未定义值乱走甚至死循环,多半不报错,最阴险。
- **合并前先 find 到根**:直接写 `parent[a] = b` 是"元素挂元素",树结构立刻损坏。
- **size/rank 只在根上维护**:非根结点的 size/rank 已过期,更新只能发生在合并后当选新根的结点上。

8.7.2 路径压缩不破坏秩的近似性

反直觉但正确的结论: find 做路径压缩时,不需要(也不应该)更新 rank。rank 是"树高的上界":压缩把树压矮,实际高度变小,旧 rank 仍是合法上界,只是"松"了——下次合并最多让树高略高于最优,不影响 $O(\log n)$ 与均摊近 $O(1)$ 。find 里更新 rank 反而白付代价。这就是"秩 = 上界"而非"精确高度":只承诺上界,压缩无损。

8.7.3 调试技巧:把结构画出来

示例 8-8 调试利器:打印 parent 数组

```
// 打印 parent 数组,观察树形态
void dump(const vector<int>& parent, int n) {
    for (int i = 0; i < n; ++i)
        cout << i << "->" << parent[i] << " ";
    cout << "\n";
}
// 小数据验证:unionSets(0,1); unionSets(1,2); dump(parent, 3);
// 输出 "0->1 1->2 2->2" 立刻看出 0-1-2 长链;压扁后 "0->2 1->2 2->2"。
```

调试三招

- ① 小数据手推:n 取 5~8,把每步 union 后的 parent 抄在纸上与 dump 对照;② 打印关键点:find 返回前打印"x 到根",压缩是否生效一目了然;③ 断言根性: `assert(parent[find(i)] == find(i))`,把"结构被破坏"提前暴露为断言失败。

⚠ 误区 3:初始化写成"全部指向 0"

把 `parent` 全初始化为 0: `find(3)` 走到 0 就停下, 0 被误当所有集合的根, 任何两个元素的 `find` 结果都相同——"全都连通"的假象, 且不崩溃、不报错。排查口诀: 初始化后立刻 `dump, parent` 必须等于下标。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
find	朴素实现(无优化)	$O(n)$	$O(n)$ (链状树)
union	朴素实现(无优化)	$O(n)$	$O(n)$
find	仅按大小合并	$O(\log n)$	$O(\log n)$
find / union	仅按秩合并	$O(\log n)$	$O(\log n)$
find	按秩合并 + 路径压缩	均摊近 $O(1)$ (严格 $O(\alpha(n))$)	
union	按秩合并 + 路径压缩	均摊近 $O(1)$ (两次 find + 一次指针修改)	
初始化	建 n 个单元素集合	$O(n)$	$O(n)$
递归版压缩 find	递归实现	均摊近 $O(1)$	递归深度 = 树高, 深链栈溢出
迭代版压缩 find	迭代实现(两趟)	均摊近 $O(1)$	均摊近 $O(1)$, 无栈深问题
求连通分量个数	遍历全部元素数根	$O(n \alpha(n))$	$O(n \alpha(n))$
维护连通分量数	union 成功时 <code>count-1</code>	$O(1)$	$O(1)$
迷宫生成	随机拆墙 + 并查集判连通	$O(E \alpha(V))$	$O(E \alpha(V))$
Kruskal 判连通	每条边两次 find + 一次 union	均摊近 $O(1)$ /边, 总 $O(E \log E)$ (排序主导)	
反阿克曼函数 $\alpha(n)$	完整优化的单次操作上界	$\alpha(10^{80}) \leq 4 \sim 5$, 当常数用	
集合标记法	dict/map 维护集合编号	union $O(n)$ (遍历集合)	union $O(n)$

最小必会清单

- 能说出等价关系三性质, 判断"同班同学"/"模 m 同余"是否等价关系
- 能独立说出 `find / union` 的语义, 以及"动态"为何要求在线回答
- 能用 `parent` 数组 + 树形表示实现朴素并查集, 解释"根 = 代表元"
- 能解释朴素 union 为何把树连成链, find 退化 $O(n)$
- 能实现按大小/按秩合并, 说出"深度每 +1 树规模至少翻倍"的直觉
- 能写出路径压缩的递归版与迭代版, 说出迭代版为何更稳
- 能说出三种配置的复杂度: 无优化 $O(n)$ /仅按秩合并 $O(\log n)$ /完整优化均摊近 $O(1)$, 以及"当常数用"的理由
- 能用并查集生成随机迷宫, 并调试: 初始化、先 find 到根、rank 不失效、打印 `parent`

自测题

Q 1. "同班同学"关系满足等价关系的哪几条? 缺哪条? 为什么它导致"全班连通"不能用并查集建模?

✓ 参考答案

自反、对称满足,传递不满足:甲、乙同班且乙、丙同班,甲、丙可能不同班。三条不全,"同学"不能把集合划分成互不相交的等价类,故不能用并查集。

Q 2. 依次执行 $\text{union}(0,1)$, $\text{union}(1,2)$, ..., $\text{union}(n-2, n-1)$ 后, $\text{find}(0)$ 要走多少步?为什么是灾难?

✓ 参考答案

树变成链 $0 \rightarrow 1 \rightarrow \dots \rightarrow n-1$, $\text{find}(0)$ 沿父链走 $n-1$ 步,即 $O(n)$; n 次 find 总代价 $O(n^2)$, $n = 10^6$ 时是 10^{12} 步,远超可行范围。这正是朴素实现必须优化的原因。

Q 3. 按大小合并为什么保证树高 $O(\log n)$?给出直觉证明。

✓ 参考答案

x 深度每 $+1$,必然是它所在的小树被并入一棵不小于自己的树(小树并入大树),新树大小 $\geq 2 \times$ 旧树,故 x 深度最多增加 $\log_2 n$ 次,树高 $\leq \log_2 n$ 。

Q 4. 路径压缩为何能让后续 find 变快?递归版与迭代版各自的风险是什么?

✓ 参考答案

一次 find 把沿途所有结点挂到根,之后它们再 find 都是一步到达——"一次的代价,换一路的 $O(1)$ "。递归版代码短,但递归深度 = 树高,只压缩不按秩时深链会栈溢出;迭代版无栈深风险,工程更稳。

Q 5. 迷宫生成中,为何拆墙数恰好等于格子数 - 1?若拆墙时两端已连通会怎样?

✓ 参考答案

每拆一面墙必连通两个原本不相交的集合,分量数恰好减一,从 n 减到 1 共拆 $n-1$ 面。若两端已连通还拆墙会形成回路,破坏"恰有一条路",必须用 find 判断后跳过。 $n-1$ 面、无环、全连通 = 生成树。

章末实验题:迷宫生成与连通分量统计

实验 8:迷宫生成与连通分量统计(并查集综合应用)

实验目标:实现完整优化并查集(按秩合并 + 路径压缩),生成随机 ASCII 迷宫,统计拆墙中连通分量的变化,实测朴素版与优化版性能差距。

实验要求:

- 任务 1:实现并查集类: `find` (迭代两趟路径压缩)+ `unite` (按秩合并),维护 `count` (知识点:\$8.3 按秩合并、\$8.4 路径压缩)
- 任务 2:对 $w \times h$ 网格随机拆墙,两端不连通才拆,直到全连通;输出 ASCII 迷宫(知识点:\$8.6 迷宫生成、\$8.2 树表示)
- 任务 3:记录拆墙中连通分量数量的变化,在尝试 25%/50%/75%/100% 处输出关键点,验证"每拆一面墙分量减一"(知识点:\$8.6.2 连通分量统计)
- 任务 4:实现朴素版(无优化)做同一操作序列,对比耗时,体会 $O(n^2)$ 与近 $O(1)$ 的差距(知识点:\$8.5 复杂度)

实验环境:C++17,MSVC 或 g++。编译: `g++ -std=c++17 -O2 lab.cpp -o lab`;运行: `./lab` (Windows 下 `lab.exe`)。

第一步 写并查集类:parent 用 `iota` 初始化,rank 全 0,count = n;find 写迭代两趟版,unite 按秩合并,成功合并时 count-1

第二步 选墙:右墙 $(r, c)-(r, c+1)$ 、下墙 $(r, c)-(r+1, c)$, `shuffle` + `mt19937` 固定种子随机化

第三步 墙循环中维护 record 表,在 idx 达到墙总数的 1/4、1/2、3/4、全部时记录 (idx, count)

第四步 染按"顶/底横线 + 格子行 + 中间横线"拼字符串,行宽一致(每格 3 字符)

第五步 务 4 用同一操作序列(依次 `union(i, i+1)` 再全部 `find`)跑朴素版与优化版, `steady_clock` 计时;朴素版 union 的代价会延迟到 find 阶段爆发,是观察"摊还"的绝佳素材

✓ 参考答案(完整可运行程序,约 165 行,分四段)

```

// lab.cpp — 迷宫生成与连通分量统计(第一段:并查集与朴素版)
// 编译:g++ -std=c++17 -O2 -o lab lab.cpp 运行:./lab
#include <iostream>
#include <vector>
#include <numeric>
#include <random>
#include <chrono>
#include <string>
#include <utility>
#include <algorithm>
using namespace std;
using Clock = chrono::steady_clock;

// ----- 任务 1:并查集(按秩合并 + 路径压缩) -----
class UnionFind {
    vector<int> parent_, rank_;
public:
    int count; // 当前连通分量个数
    UnionFind(int n) : parent_(n), rank_(n, 0), count(n) {
        iota(parent_.begin(), parent_.end(), 0);
    }
    int find(int x) { // 迭代版路径压缩:两趟
        int root = x;
        while (parent_[root] != root) root = parent_[root];
        while (parent_[x] != x) { // 第二趟:沿途结点挂到根
            int nxt = parent_[x];
            parent_[x] = root;
            x = nxt;
        }
        return root;
    }
    bool unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (rank_[ra] < rank_[rb]) swap(ra, rb);
        parent_[rb] = ra; // 浅树挂到深树
        if (rank_[ra] == rank_[rb]) ++rank_[ra];
        --count; // 成功合并,分量减 1
        return true;
    }
};

// ----- 任务 4 对比用:朴素版(无任何优化) -----
struct NaiveUF {
    vector<int> parent_;
    NaiveUF(int n) : parent_(n) {
        iota(parent_.begin(), parent_.end(), 0);
    }
    int find(int x) {
        while (parent_[x] != x) x = parent_[x];
        return x;
    }
    void unite(int a, int b) {
        a = find(a); b = find(b);
        if (a != b) parent_[a] = b; // 固定挂到 b 下,易成长链
    }
};

```

```

    }
};

```

lab.cpp(第二段:迷宫生成)

```

// ----- 任务 2/3:随机迷宫生成 -----
struct Maze {
    int w, h;
    vector<vector<bool>> right, down;      // 右墙、下墙
    vector<pair<int, int>> record;        // 任务 3:连通分量变化采样
};

Maze genMaze(int w, int h, int seed = 2024) {
    UnionFind uf(w * h);
    Maze m{w, h,
        vector<vector<bool>>(h, vector<bool>(w, true)),
        vector<vector<bool>>(h, vector<bool>(w, true)),
        {}};
    vector<pair<int, int>> walls;          // 墙用两端格子编号表示
    auto id = [w](int r, int c) { return r * w + c; };
    for (int r = 0; r < h; ++r)
        for (int c = 0; c < w; ++c) {
            if (c + 1 < w) walls.push_back({id(r, c), id(r, c + 1)}); // 右墙
            if (r + 1 < h) walls.push_back({id(r, c), id(r + 1, c)}); // 下墙
        }
    mt19937 rng(seed);
    shuffle(walls.begin(), walls.end(), rng);
    int key[3] = {int(walls.size()) / 4, int(walls.size()) / 2,
        int(walls.size()) * 3 / 4};
    int ki = 0;
    for (size_t i = 0; i < walls.size(); ++i) {
        int a = walls[i].first, b = walls[i].second;
        if (uf.unite(a, b)) { // 两端不连通才拆墙
            int r1 = a / w, c1 = a % w, r2 = b / w, c2 = b % w;
            if (c1 == c2) m.down[min(r1, r2)][c1] = false;
            else m.right[r1][min(c1, c2)] = false;
        }
        if (ki < 3 && int(i + 1) >= key[ki]) {
            m.record.push_back({int(i + 1), uf.count});
            ++ki;
        }
    }
    m.record.push_back({int(walls.size()), uf.count});
    return m;
}

```

lab.cpp(第三段:ASCII 渲染)

```
// ----- 任务 2:ASCII 渲染 -----
string render(const Maze& m) {
    string top = "+";
    for (int c = 0; c < m.w; ++c) top += "---";    // 顶部横线(永不拆)
    string s = top + "\n";
    for (int r = 0; r < m.h; ++r) {
        string row = "|";                          // 最左竖墙(永不拆)
        for (int c = 0; c < m.w; ++c)
            row += " " + string(1, m.right[r][c] ? '|' : ' ');
        s += row + "\n";
        if (r == m.h - 1) {
            s += top + "\n";                        // 底部横线(永不拆)
        } else {
            string line = "+";
            for (int c = 0; c < m.w; ++c)
                line += string(m.down[r][c] ? "---" : " ") + "+";
            s += line + "\n";
        }
    }
    return s;
}
```

lab.cpp(第四段:性能对比与主程序)

```
// ----- 任务 4:朴素版 vs 完整优化版性能对比 -----
void bench(int n) {
    vector<pair<int, int>> ops;
    for (int i = 0; i < n - 1; ++i) ops.push_back({i, i + 1}); // 顺序合并成链
    auto ms = [](auto t0, auto t1) {
        return chrono::duration<double, milli>(t1 - t0).count();
    };
    long long s = 0;
    auto t0 = Clock::now();
    NaiveUF naive(n);
    for (auto [a, b] : ops) naive.unite(a, b); // 每次 union 沿链走 O(n)
    auto t1 = Clock::now();
    for (int x = 0; x < n; ++x) s += naive.find(x); // 链尾 find 走 O(n)
    auto t2 = Clock::now();
    UnionFind opt(n);
    for (auto [a, b] : ops) opt.unite(a, b);
    auto t3 = Clock::now();
    for (int x = 0; x < n; ++x) s += opt.find(x);
    auto t4 = Clock::now();
    cout << "任务4: " << n << " 次 union(连成链)+ " << n
        << " 次 find(耗时对比):\n";
    cout << " 朴素版: union 阶段 " << ms(t0, t1) << " ms, find 阶段 "
        << ms(t1, t2) << " ms\n";
    cout << " 优化版: union 阶段 " << ms(t2, t3) << " ms, find 阶段 "
        << ms(t3, t4) << " ms\n";
    cout << " find 阶段加速比:约 " << (t2 - t1) / (t4 - t3)
        << " 倍(检查值 s=" << s % 1000 << ")\n";
}

int main() {
    int w = 6, h = 4;
    Maze m = genMaze(w, h, 2024);
    cout << "随机迷宫 " << w << " 列 x " << h << " 行(种子 2024,共 "
        << w * h << " 个格子):\n";
    cout << render(m);
    int total = m.record.back().first;
    cout << "\n任务3: 拆墙过程中连通分量数量的变化(共 " << total
        << " 面候选墙):\n";
    cout << " 初始: " << w * h << " 个连通分量\n";
    for (auto [tried, comp] : m.record)
        cout << " 尝试 " << tried << "/" << total << " 面墙后: 连通分量 "
            << comp << "\n";
    cout << " 结束时连通分量为 1,迷宫全连通、无环(恰好拆 " << w * h - 1
        << " 面墙)。\n\n";
    bench(40000);
    return 0;
}
```

示例输出(本机 WinLibs g++ 17 实测):

随机迷宫 6 列 x 4 行(种子 2024,共 24 个格子):

```
+---+---+---+---+
|       |   |   |
+---+ +---+ + + +
|       |   |   |
+---+---+ +---+ + +
|       |   |   |
+---+ + +---+---+ +
|       |   |   |
+---+---+---+---+---+
```

任务3: 拆墙过程中连通分量数量的变化(共 38 面候选墙):

初始: 24 个连通分量

尝试 9/38 面墙后: 连通分量 15

尝试 19/38 面墙后: 连通分量 5

尝试 28/38 面墙后: 连通分量 2

尝试 38/38 面墙后: 连通分量 1

结束时连通分量为 1,迷宫全连通、无环(恰好拆 23 面墙)。

任务4: 40000 次 union(连成链)+ 40000 次 find(耗时对比):

朴素版: union 阶段 0.0338 ms, find 阶段 1114.44 ms

优化版: union 阶段 0.1915 ms, find 阶段 0.0298 ms

find 阶段加速比:约 37397 倍(检查值 s=0)

设计说明:① UnionFind 与 NaiveUF 共用同一操作序列,对比才公平——差的是增长阶($O(n^2)$ vs 近 $O(1)$),不是常数;② 朴素版 union 阶段反而更快(0.03 ms vs 0.19 ms):链的代价"延迟"到 find 阶段爆发——§8.5 摊还思想的注解:比算法要看操作序列的**总代价**;③ 任务 3 采样点取"尝试墙数"而非"成功拆墙数":拆墙有失败(两端已连通),曲线呈阶梯状,更能体现 find 判连通;④ 种子固定 2024,可复现;⑤ 固定挂法 `parent_[a] = b` 刻意制造长链,暴露退化——现实中盲目选挂向也可能遭遇同样最坏输入。

下一章预告:第9章 图论算法——图的表示、拓扑排序、最短路径与最小生成树。Kruskal 将使用本章的不相交集:边按权排序后,find 判断两端是否已连通,union 把选中边连入生成树——两条核心操作正是本章主线。

第9章 图论算法

建议学时:5-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 掌握图的基本术语(顶点、边、有向/无向、权),说出**邻接矩阵**与**邻接表**的代价差异;完成从 C 的二维数组、手写链表到 C++ `vector` 的迁移
- 实现**拓扑排序**(入度表 + 队列)并用"输出个数 < 顶点数"检测环;实现 BFS 求无权**最短路径**并回溯打印
- 实现 Dijkstra 朴素版与堆优化版,说清 INF 选择的坑;实现 Bellman-Ford 检测负环
- 实现 Prim 与 Kruskal **最小生成树**,完成迷宫最短路径综合实验

💡 从 C 到 C++:本章主题如何迁移

图的直觉一句话: 图 = 一堆顶点 + 顶点之间的边,算法全部建立在"存图、遍历图"两个动作上。前面 8 章学的栈、队列、堆、并查集本章全部派上用场:BFS 与拓扑用第3章的队列,Dijkstra 用第6章的堆,Kruskal 用第8章的不相交——图论是前面所有数据结构的"总演习"。

C++ 的解法是"数组 + 容器":`vector<vector<int>>` 一行声明动态邻接表,自动扩容;加权图用 `vector<vector<pair<int,int>>>` ——第6章 Dijkstra 堆里正是同款 `pair`。存图从十行手工变成一行声明,精力全花在算法本身。

📦 前置知识自查

数学前置:无特殊;只需第2章大 O 记号,复杂度字母约定 V = 顶点数、 E = 边数。

C 语言前置:二维数组与下标运算;对照用的 `struct` 链表与 `qsort` 函数指针(可选)。

依赖章节:第3章栈与队列(BFS、拓扑排序、迭代 DFS 的显式栈);第6章 `priority_queue` (Dijkstra/Prim 堆优化);第8章不相交集(Kruskal 的 find/union)。

🚀 本章学习路线

1. **第一阶段(1.5 小时):**读 §9.1–§9.3,敲两种建图,实现拓扑排序与 BFS
2. **第二阶段(2 小时):**读 §9.4–§9.6,对照朴素/堆优化两份 Dijkstra,体会 Prim 与 Dijkstra 的一行之差
3. **第三阶段(1.5 小时):**完成章末实验:迷宫最短路径(BFS + Dijkstra)
4. **验收:**默写建图、BFS 路径回溯、堆优化 Dijkstra、Kruskal 与各自复杂度

本章知识导图

- §9.1 图的表示:术语与两种存储结构
- §9.2 拓扑排序:入度表与队列
- §9.3 无权最短路径:BFS 与路径回溯
- §9.4 Dijkstra 最短路:贪心、INF 与堆优化
- §9.5 负权边与 Bellman-Ford
- §9.6 最小生成树:Prim 与 Kruskal
- §9.7 最大流与 DFS 应用简介
- §9.8 工程实践:大图的正确打开方式

§9.1 图的表示:术语与两种存储结构

9.1.1 基本术语

图由两部分组成:**顶点(vertex)** 与 **边(edge)**——顶点是"对象",边是"对象之间的关系"。三个常用例子:**城市-航班图**(城市=顶点,航线=边)、**课程依赖图**(课程=顶点,先修=边)、**网页链接图**(网页=顶点,链接=边)。

- **有向图 vs 无向图**:边有方向叫有向边($u \rightarrow v$),全有向的图叫有向图(课程依赖);边不分方向叫无向边($u - v$),对应无向图(道路网)。无向边可看作两条反向有向边。
- **权(weight)**:边上的数值(票价、距离)。带权叫加权图;不带权默认每条边代价 1。
- **路径与环**:顶点间的边序列叫路径;首尾相同的叫环。拓扑排序只能处理无环有向图(DAG),即 §9.2 的主题。

9.1.2 邻接矩阵 vs 邻接表

存图只有两种主流方案,各有胜负。设 V 个顶点、 E 条边, $\text{deg}(u)$ 表示顶点 u 的邻居个数:

对比项	邻接矩阵	邻接表
存储空间	$O(V^2)$,无论边多少都占满	$O(V + E)$,稀疏图省空间
判断边 (u, v) 是否存在	$O(1)$,直接查 $g[u][v]$	$O(\text{deg}(u))$,遍历 u 的链表
遍历 u 的所有邻居	$O(V)$,扫一整行	$O(\text{deg}(u))$,顺着链表走
适用场景	稠密小图($V \leq 2000$)	稀疏大图(工程绝大多数)

口诀:图算法都要"遍历邻居",邻接表把这一步从 $O(V)$ 降到 $O(\text{deg})$,整体复杂度下降一个量级。邻接矩阵的用武之地是" V 小且频繁查边"的场景(如 Floyd)。 $V = 10^5$ 时矩阵要 10^{10} 个格子 ≈ 10 GB,直接内存崩溃。

C 的写法	C++ 的写法
<code>int g[100][100];</code> 大小写死的二维数组	<code>vector<vector<int>>> g(n, vector<int>(n, 0));</code>
<code>struct Edge { int to, w; struct Edge* next; };</code> 手写链表	<code>vector<vector<pair<int,int>>> g(n);</code> 每行一个自动 vector
<code>for (Edge* e = head[u]; e; e = e->next)</code> 指针遍历	<code>for (auto& [v, w] : g[u])</code> 范围 for 遍历
<code>malloc / free</code> 手动管理,忘记即泄漏	离开作用域自动释放,RAII 无泄漏

示例 9-1 有向无权图:邻接表建法(矩阵建法见上表 C 对照)

```
// 需要 #include <vector>
// 邻接表:g[u] 是 u 的所有出边终点
vector<vector<int>>> buildList(int n, const vector<pair<int,int>>& edges) {
    vector<vector<int>>> g(n);
    for (auto& e : edges)
        g[e.first].push_back(e.second);    // 无向图再加 g[v].push_back(u)
    return g;
}
```

9.1.3 加权图:C 的手写链表 → C++ 的 pair

加权图在 C 里要用 "边 结点 带权" 的链表,建图几十行;C++ 用 `pair<int,int>` 存 (终点, 权): `vector<vector<pair<int,int>>> g(V); g[u].push_back({v, w});` ——读边直接 `push_back`,C++11 后嵌套模板 >> 无需空格。

§9.2 拓扑排序:入度表与队列

9.2.1 什么是拓扑排序

拓扑排序:把有向无环图(DAG)的所有顶点排成一列,使每条边 $u \rightarrow v$ 都满足"u 排在 v 前"。最典型的场景是**课程先修依赖**:C 语言排在数据结构前,数据结构排在算法分析前——课程表就是拓扑序。**带环的有向图不存在拓扑序**(A 先修 B、B 又先修 A,谁也排不了),所以"能否拓扑排序"等价于"有没有环"。

9.2.2 Kahn 算法:入度表 + 队列

算法直觉:"不断摘掉没有前置的课程"。三步:① 统计每个顶点的**入度**(指向它的边数);② 入度为 0 者全部入队——无前置,随时可安排;③ 出队 u,删掉它的所有出边(邻居入度减 1),新入度为 0 的邻居入队。**无环时必然输出全部 V 个顶点;输出个数 < V 就是有环,务必检查**。

示例 9-2 拓扑排序(Kahn 算法)与环检测

```

#include <queue>
#include <vector>
using namespace std;

// 返回拓扑序列;图有环时返回空序列
vector<int> topoSort(int V, const vector<vector<int>>& g) {
    vector<int> indeg(V, 0);           // ④ 入度表
    for (int u = 0; u < V; ++u)
        for (int v : g[u]) ++indeg[v];

    queue<int> q;
    for (int u = 0; u < V; ++u)
        if (indeg[u] == 0) q.push(u);

    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : g[u])
            if (--indeg[v] == 0)           // 前置课程全部完成才入队
                q.push(v);
    }
    if (int(order.size()) != V) return {}; // 输出个数 < V:有环!
    return order;
}

```

§9.3 无权最短路径:BFS 与路径回溯

9.3.1 为什么 BFS 能求无权最短路

无权图(每条边代价 1)中,**最短路径**就是"边数最少的路径"。BFS 按层扩展:第 1 层距离 1,第 2 层距离 2..... 每个顶点第一次被访问时,步数一定最少——队列严格按距离递增出队,"第一次到达即最短"正是 BFS 不需要 Dijkstra 的理由。

C 的手写循环队列	C++ 的 std::queue
<pre>int q[100000]; int head, tail;</pre> 自己维护下标与取模	<pre>queue<int> q; q.push(u); q.pop();</pre>
空间写死,溢出要自己检查	自动扩容,永不"满"
空/满判断:计数法或浪费一格(第3章)	empty() 一行,边界交给标准库

9.3.2 距离数组与前驱数组回溯

求步数只需 `dist` 数组;打印路径还要 `prev` 数组记录"每个顶点从谁到达"。到终点后沿 `prev` 回溯到起点再反转,就是完整路径——**前驱数组回溯**,Dijkstra 与 Bellman-Ford 沿用同一套路。

示例 9-3 无权最短路:BFS + 前驱数组回溯

```

#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 返回 s 到各顶点的最短距离; dist[i] = -1 表示不可达
vector<int> bfsDist(int s, const vector<vector<int>>& g,
                  vector<int>& prev) {
    int V = int(g.size());
    vector<int> dist(V, -1);
    prev.assign(V, -1);           // prev[v]: 从谁到达 v
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (dist[v] != -1) continue; // 第一次到达即最短
            dist[v] = dist[u] + 1;
            prev[v] = u;
            q.push(v);
        }
    }
    return dist;
}

// 打印 s 到 t 的路径: 沿 prev 回溯; 不可达则提示
void printPath(int s, int t, const vector<int>& prev) {
    vector<int> path;
    for (int x = t; x != -1; x = prev[x]) {
        path.push_back(x);
        if (x == s) break;
    }
    if (path.empty() || path.back() != s) { // 回溯到 -1 也没到 s
        cout << "无路径可达" << endl;
        return;
    }
    for (int i = int(path.size()) - 1; i >= 0; --i)
        cout << path[i] << (i ? " -> " : "\n");
}

```



实现建议: dist 兼任 visited

BFS 里 `dist[v] != -1` 既判"访问过"又存距离, 一个数组干两件事, 无需单独写 `visited` ——与第3章"循环队列用 `count` 判空满"同一思想: 状态能合并就合并。回溯打印注意: 路径数组是"终点→起点"顺序, 输出前要反转。

§9.4 Dijkstra 最短路: 贪心、INF 与堆优化

9.4.1 贪心思想与正确性直觉

加权图(边权非负)的单源最短路由 Dijkstra 解决, 思路是 **贪心**: 每轮从"还没确定最短距离"的顶点中, 挑距离最小的确定为**答案**。为什么敢确定? 边权全非负——**任何"经过别的未确定顶点"的绕路, 起步就比这个距离大, 不可能更短**。"确定"是

永久的:顶点一旦被选出,dist 就是最终答案,只增不改。"每次取当前最小"正是第6章优先队列的用武之地。

9.4.2 INF 的坑:INT_MAX 相加溢出

dist 初始值必须是一个"表示无穷大"的大数。C++ 里最自然的想法是 INT_MAX ——但有个隐蔽的致命坑:

⚠ 误区 3:INF 用 INT_MAX,松弛时 INF + w 溢出

松弛语句 `dist[v] > dist[u] + w` 里,若 `dist[u]` 还是 `INT_MAX` 且 `w > 0`, `INT_MAX + w` 会溢出成负数(int 上界 2147483647 再加就回绕),比较结果为真,`dist[v]` 被写进负值,算法全错且极难排查。解法:用足够大的常量当 INF(`const int INF = 1e9`; ,保证 `INF + w` 不溢出),或把 `dist` 换成 `long long`。边权小用 `int + 1e9`,边权大(10^{18} 量级)必须 `long long`。

C 的写法	C++ 的写法
<code>#define INF 0x3f3f3f3f</code> 魔法常量,靠记忆	<code>const int INF = 1e9</code> ; 类型安全, <code>constexpr</code> 可编译期求值
<code>memset(dist, 0x3f, sizeof dist)</code> 技巧性初始化	<code>vector<int> dist(V, INF)</code> ; 构造即填充
手写 <code>for</code> 循环扫描找最小, $O(V)$ 每次	<code>priority_queue</code> 堆顶即最小, $O(\log V)$ 每次

9.4.3 朴素版与堆优化版并排:手写实现 vs 标准库

两段代码解决同一问题,骨架完全相同,差异只在"找当前最小"——正是本章的"手写实现 + 标准库写法"并排。朴素版每轮扫描全数组,总代价 $O(V^2)$;堆优化版把"找最小"委托给第6章的 `priority_queue`,总代价 $O(E \log V)$,大图必须用它。

示例 9-4 Dijkstra 朴素版 $O(V^2)$:手写扫描找最小

```
const int INF = 1e9;          // 大常量:INF + w 不会溢出(勿用 INT_MAX)

vector<int> dijkstraNaive(int s, const vector<vector<pair<int,int>>>& g) {
    int V = int(g.size());
    vector<int> dist(V, INF);
    vector<bool> done(V, false);    // done[u]:u 已确定
    dist[s] = 0;
    for (int round = 0; round < V; ++round) {
        int u = -1;
        for (int i = 0; i < V; ++i)    // 扫描:未确定者中 dist 最小
            if (!done[i] && (u == -1 || dist[i] < dist[u]))
                u = i;
        if (u == -1 || dist[u] == INF) break;    // 其余顶点不可达
        done[u] = true;    // 确定 u,永不再改
        for (auto& [v, w] : g[u])    // 松弛:尝试用 u 更新邻居
            if (dist[v] > dist[u] + w)
                dist[v] = dist[u] + w;
    }
    return dist;
}
```

示例 9-5 Dijkstra 堆优化版 $O(E \log V)$:标准库 `priority_queue`

```

#include <functional>
#include <queue>

vector<int> dijkstraHeap(int s, const vector<vector<pair<int,int>>>& g) {
    int V = int(g.size());
    vector<int> dist(V, INF);
    using P = pair<int, int>;           // (距离, 顶点)
    priority_queue<P, vector<P>, greater<P>> pq;    // 最小堆
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;      // 懒删除:过期的旧记录直接跳过
        for (auto& [v, w] : g[u]) {
            if (dist[v] > d + w) {
                dist[v] = d + w;
                pq.push({dist[v], v});    // 允许重复入堆,靠上面检查过滤
            }
        }
    }
    return dist;
}

```

实现建议:懒删除(惰性删除)是堆优化 Dijkstra 的标配

同一顶点可能被松弛多次、在堆里留下多份记录。堆优化版**不删除旧记录**,出堆时检查 $d > \text{dist}[u]$ 的过期记录直接丢弃——这叫懒删除,正确性由"旧记录距离必然不小于新记录"保证。priority_queue 没有 decreaseKey 接口,懒删除正是替代。套路:push 不查重,pop 查过期。

9.4.4 复杂度与选型

版本	"找最小"的手段	总复杂度	$V = 10^5$ 、 $E = 10^6$ 时的量级
朴素	每轮全数组扫描 $O(V)$	$O(V^2)$	10^{10} ,不可用
堆优化	堆顶 $O(\log V)$	$O(E \log V)$	约 2×10^7 ,秒级

选型结论:稠密小图($V \leq 2000$)两者皆可,朴素版常数更小; V 上万或 E 稀疏一律堆优化,工程与竞赛的默认写法。

§9.5 负权边与 Bellman-Ford

9.5.1 负权边为何破坏 Dijkstra

三顶点例子:边 $s \rightarrow a$ 权 2、 $s \rightarrow b$ 权 3、 $b \rightarrow a$ 权 -2(有向)。Dijkstra 第一轮挑出距离最小的 $a(d=2)$ 并永久确定;第二轮处理 b 时松弛 $a: 3 + (-2) = 1 < 2$,但 a 已被"确定"——真实最短路 $s \rightarrow b \rightarrow a$ 只有 1,答案错了。负权边让"绕路"反而更便宜,贪心的"确定即最终"被推翻。结论:Dijkstra 只适用于非负权图。

9.5.2 Bellman-Ford: $V-1$ 轮松弛

Bellman-Ford 的思想朴素而彻底:每轮对全部 E 条边松弛一次,共做 $V-1$ 轮。为什么够?无环路径至多 $V-1$ 条边:第 1 轮确定"1 条边"的最短路,第 k 轮由第 $k-1$ 轮答案再延伸一条边,第 k 轮后所有"至多 k 条边"的路径都被考虑。复杂度 $O(V \cdot E)$,能处理负权,代价是图不能有负环。

示例 9-6 Bellman-Ford:V-1 轮松弛 + 负环检测

```
struct Edge3 { int u, v, w; };           // 边表:(u, v, w)

// 返回 false = 有负环(最短路无定义)
bool bellmanFord(int s, const vector<Edge3>& edges, int V,
                 vector<int>& dist) {
    dist.assign(V, INF);
    dist[s] = 0;
    for (int round = 0; round < V - 1; ++round) {    // 至多 V-1 轮
        bool changed = false;
        for (auto& e : edges)           // 每轮扫全部边
            if (dist[e.u] != INF && dist[e.v] > dist[e.u] + e.w) {
                dist[e.v] = dist[e.u] + e.w;
                changed = true;
            }
        if (!changed) break;           // 无更新即可提前结束
    }
    for (auto& e : edges)               // 第 V 轮:还能松弛 = 有负环
        if (dist[e.u] != INF && dist[e.v] > dist[e.u] + e.w)
            return false;
    return true;
}
```

⚠ 误区 4:负环检测忘写,或写进主循环里

负环检测必须在 V-1 轮**结束之后**单独做一轮:负环让距离无限变小,检测混进主循环会在正常图误报"有环"。另一易错点:松弛前跳过 `dist[e.u] == INF` 的边——INF 加负权 w 得到"比 INF 小"的假答案,这是 §9.4 溢出坑的近亲(INF 参与运算前先判可达性)。

§9.6 最小生成树:Prim 与 Kruskal

9.6.1 定义与两种思路

最小生成树(MST):在连通加权无向图中,选 V-1 条边把全部顶点连成一棵树(无环),边权总和最小。应用:铺电缆、修公路,让所有城市连通且总造价最低。两条路线:Prim"加点"——每次并入"离树最近"的顶点;Kruskal"加边"——边按权排序,从小到大挑不成环的边。

9.6.2 Prim:与 Dijkstra 只差一行

把 §9.4 的 Dijkstra 朴素版抄过来,Prim 与它几乎一模一样——唯一区别是松弛标准:

算法	key[u] 的含义	松弛标准
Dijkstra	起点到 u 的当前最短距离	$\text{dist}[v] > \text{dist}[u] + w$ (累加路径)
Prim	u 到"已选集合"的最近距离	$\text{key}[v] > w$ (只看边权,不累加)

Prim 的 key 是"顶点到树集合的距离",更新只比较**这条边本身**的权;Dijkstra 比较"起点到 u 的整条路径 + 边权"。一行之差,两种算法,也是第10章贪心范式的两个变体。

示例 9-7 Prim 朴素版 $O(V^2)$:与 Dijkstra 只差松弛标准

```

int primMST(int s, const vector<vector<pair<int,int>>>& g) {
    int V = int(g.size());
    vector<int> key(V, INF);          // 到"已选集合"的最短距离
    vector<bool> inTree(V, false);
    key[s] = 0;
    int total = 0;
    for (int round = 0; round < V; ++round) {
        int u = -1;
        for (int i = 0; i < V; ++i)    // 扫描:离树最近的未入树顶点
            if (!inTree[i] && (u == -1 || key[i] < key[u]))
                u = i;
        if (u == -1 || key[u] == INF) break;    // 图不连通
        inTree[u] = true;
        total += key[u];
        for (auto& [v, w] : g[u])
            if (!inTree[v] && w < key[v])    // 松弛标准:直接比边权!
                key[v] = w;
    }
    return total;                      // 生成树总权
}

```

9.6.3 Kruskal:边排序 + 不相交集

Kruskal 更"贪":边按权升序,从小到大逐条考察,只要两端点还没连通就收下——"是否连通"正是第8章不相交集的看家本领(find/union 均摊近 $O(1)$);已连通则收下会成环,必须跳过。总复杂度 $O(E \log E)$,由排序主导。

对照 C 的 `qsort` 函数指针,这里用 `sort` + lambda: `sort(edges.begin(), edges.end(), [](const Edge3& x, const Edge3& y){ return x.w < y.w; });`——比较逻辑内联在调用处,类型由模板推断,写错编译期报错。

示例 9-8 Kruskal:排序 + 不相交集(第8章的 find/union 直接拿来用)

```

#include <algorithm>

vector<int> parent;                                // 并查集(第8章)
int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]); // 路径压缩
    return parent[x];
}
bool unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false; // 已连通:这条边会成环
    parent[a] = b;
    return true;
}

int kruskalMST(int V, vector<Edge3>& edges) {
    sort(edges.begin(), edges.end(), // ① 按边权升序
        [](const Edge3& x, const Edge3& y) { return x.w < y.w; });
    parent.resize(V);
    for (int i = 0; i < V; ++i) parent[i] = i;
    int total = 0, cnt = 0;
    for (auto& e : edges) {
        if (unite(e.u, e.v)) { // ② 两 endpoint 不连通才收下
            total += e.w;
            if (++cnt == V - 1) break; // 树有 V-1 条边即完成
        }
    }
    return cnt == V - 1 ? total : -1; // -1 表示图不连通
}

```

⚠ 误区 5:把 Prim 的松弛标准抄成 Dijkstra 的

两段代码结构高度相似,最常见的复制粘贴事故是把 `w < key[v]` 抄成 `dist[u] + w < key[v]` (或反之)。自查口诀: Prim 只比"这一条边",Dijkstra 比"整条路径"。写完用三顶点小例子手算验证。

§9.7 最大流与 DFS 应用简介

9.7.1 网络流:一句话

最大流问题:有向加权图中,从源点 s 到汇点 t 能运输的最大流量(每条边有容量上限)。一句话直觉:反复找"还能增流的路径"(增广路),直到找不到为止,累加的流量就是最大流。用 BFS 找最短增广路的版本叫 **Edmonds-Karp**,复杂度 $O(V \cdot E^2)$,Dinic、ISAP 都是它的优化。应用:交通调度、二分图匹配。

9.7.2 DFS 应用:割点与双连通

两个概念:**割点**——无向图中删掉它后图不再连通的顶点(如通信网络的关键路由器);**双连通**——删掉任意一个顶点仍连通的图,其极大子图叫双连通分量。求割点的经典算法(Tarjan)基于 DFS 的"时间戳"思想,教材有完整讲解。记住一句:DFS 访问顶点的顺序本身携带结构信息(发现时间、完成时间),能回答"连通性"类问题。

9.7.3 迭代版 DFS:显式栈

递归 DFS 在深图(如 10^5 层链)上会撑爆系统调用栈;第3章说过"递归都能改写为显式栈",这里实战:显式栈压"待访问的顶点",与函数调用栈压"返回地址"同一思想。图遍历必须带 visited:无向边让 u、v 互相"发现",不带就无限往返;迭代版同顶点可能被 push 多次,出栈时去重。

示例 9-9 深度优先遍历:递归版与显式栈迭代版(第3章)

```
#include <iostream>
#include <stack>
#include <vector>
using namespace std;

void dfsRec(int u, const vector<vector<int>>& g, vector<bool>& vis) {
    vis[u] = true;           // 访问当前顶点
    cout << u << ' ';
    for (int v : g[u])
        if (!vis[v]) dfsRec(v, g, vis); // 递归:深度优先
}

void dfsIter(int s, const vector<vector<int>>& g) {
    vector<bool> vis(g.size(), false);
    stack<int> st;           // 显式栈代替系统调用栈
    st.push(s);
    while (!st.empty()) {
        int u = st.top(); st.pop();
        if (vis[u]) continue; // 可能被压入多次,去重
        vis[u] = true;
        cout << u << ' ';
        for (int v : g[u])
            if (!vis[v]) st.push(v); // 注意:压栈顺序与递归相反
    }
}
```

§9.8 工程实践:大图的正确打开方式

9.8.1 大图选型:邻接表优先

真实世界的图几乎都稀疏(社交网络、网页链接皆如此),三条准则:

- **存图用邻接表:** $V \geq 10^4$ 一律邻接表,矩阵只在小稠密图出现(§9.1 误区1)
- **无向图建两次边:** g[u] 与 g[v] 互相 push_back,漏一边就成"单行道"
- **顶点编号连续:** 用 0..V-1 整数做下标;字符串先映射成编号(第5章)

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
遍历所有顶点与边	DFS / BFS	$O(V + E)$	$O(V + E)$
查边 (u, v)	邻接矩阵	$O(1)$	$O(1)$
查边 (u, v)	邻接表	$O(\deg(u))$	$O(\deg(u))$
存储	邻接矩阵	$O(V^2)$	$O(V^2)$
存储	邻接表	$O(V + E)$	$O(V + E)$
拓扑排序	Kahn(入度表 + 队列)	$O(V + E)$	$O(V + E)$
无权最短路径	BFS	$O(V + E)$	$O(V + E)$
单源最短路径(非负权)	Dijkstra 朴素	$O(V^2)$	$O(V^2)$
单源最短路径(非负权)	Dijkstra 堆优化	$O(E \log V)$	$O(E \log V)$
单源最短路径(可负权)	Bellman-Ford	$O(V \cdot E)$	$O(V \cdot E)$
负环检测	Bellman-Ford 第 V 轮	$O(V \cdot E)$	$O(V \cdot E)$
最小生成树	Prim 朴素	$O(V^2)$	$O(V^2)$
最小生成树	Prim 堆优化	$O(E \log V)$	$O(E \log V)$
最小生成树	Kruskal(排序 + 并查集)	$O(E \log E)$	$O(E \log E)$
最大流	Edmonds-Karp(BFS 增广)	$O(V \cdot E^2)$	$O(V \cdot E^2)$

最小必会清单

- 说出顶点、边、有向/无向、权、路径、环六个术语并配现实例子
- 独立写出邻接矩阵与邻接表两种建图代码,说出空间与查询复杂度
- 默写 Kahn 拓扑排序,解释"输出个数 $< V$ 则有环"的检测原理
- 默写 BFS 与 Dijkstra(朴素/堆优化),都用前驱数组回溯打印路径,解释贪心正确性的直觉(非负权)
- 解释 INF 用 INT_MAX 时 $\text{INF} + w$ 溢出的坑与正确写法
- 说出负权边如何破坏 Dijkstra,默写 Bellman-Ford 的松弛与负环检测
- 说出 Prim 与 Dijkstra 的唯一区别(松弛标准),默写 Kruskal(排序 + 并查集)
- 写出迭代版 DFS(显式栈),解释 visited 为何必不可少

自测题

Q 1. $V = 10^5$ 、 $E = 2 \times 10^5$ 的稀疏图,邻接矩阵与邻接表各占多少空间?查一条边各是什么复杂度?

✅ 参考答案

矩阵 $O(V^2) \approx 10^{10}$ 格(数十 GB,不可行),查边 $O(1)$;邻接表 $O(V+E) \approx 3 \times 10^5$,查边 $O(\deg(u))$ 平均 $O(E/V)$ 。稀疏大图必须用邻接表。

Q 2. 拓扑排序的队列换成栈,结果还是合法拓扑序吗?为什么环检测不受影响?

✓ 参考答案

仍然合法:入度为 0 的顶点可按任意顺序安排,只要"先修在前"成立即可。环检测只看输出个数是否等于 V ,与出队顺序无关。

Q 3. Dijkstra 朴素版与堆优化版都正确,复杂度为何差一个量级?"懒删除"怎么工作的?

✓ 参考答案

差别在"找当前最小":朴素每轮扫描 $O(V)$,总 $O(V^2)$;堆优化堆顶 $O(1)$,总 $O(E \log V)$ 。懒删除:顶点被松弛多次会在堆里留多份记录,出堆时把 $d > \text{dist}[u]$ 的旧记录丢弃——旧记录距离不小于新记录,不影响正确性。

Q 4. 为什么 Bellman-Ford 恰好做 $V-1$ 轮松弛就够?第 V 轮检测到什么说明有负环?

✓ 参考答案

无环路径至多 $V-1$ 条边,第 k 轮后"至多 k 条边"的路径都被考虑过, $V-1$ 轮后全部松弛完毕。若第 V 轮仍有边能松弛,说明存在比" $V-1$ 条边上限"还短的路径——只能是有负环(可无限绕圈变短)。

Q 5. Prim 与 Dijkstra 的代码几乎相同,区别在哪一行?Kruskal 里并查集的 unite 返回 false 意味着什么?

✓ 参考答案

区别在松弛标准:Prim 的 key 是"到已选集合的距离",更新只比较边权($\text{key}[v] > w$);Dijkstra 的 dist 是"起点的距离",更新时累加路径($\text{dist}[v] > \text{dist}[u] + w$)。Kruskal 中 unite 返回 false 表示两 endpoints 已连通,收下会成环,必须跳过。

章末实验题:迷宫最短路径(BFS + Dijkstra)

实验 9:迷宫最短路径(BFS + Dijkstra)

实验要求:

- 任务 1:读入/内置迷宫并转成图:顶点 = 格子,边 = 四方向可通行相邻格(\$9.1)
- 任务 2:BFS 求 S 到 E 的最短步数路径,前驱数组回溯并打印(\$9.3)
- 任务 3:给地形设通过代价(沼泽 ~ 代价 5),Dijkstra 堆优化求最低代价路径(\$9.4)
- 任务 4:输出两种路径的步数与代价对比,解释为何可能不同(\$9.3 + \$9.4)
- 任务 5:终点不可达时输出"无解"而不崩溃(\$9.8)

实验环境:C++17。编译: `g++ -std=c++17 -O2 -o lab9 lab9.cpp`;运行: `./lab9` (内置)、`./lab9 sealed` (无解)、`./lab9 maze.txt` (文件)。

第一步 readMaze 与邻居枚举(越界、墙检查),用 findPos 求 S、E 编号 $\text{id} = \text{行} \times \text{宽} + \text{列}$

第二步 S 用 `std::queue`,dist 初始 -1 兼任 visited;记 prev 数组,到 E 后回溯打印

第三步 jkstra 用 `priority_queue` 最小堆,INF 用 $1e9$;懒删除(出堆时 $d > \text{dist}[u]$ 跳过)且不松弛墙

第四步 比行统计两条路径的真实代价,体会"步数最少"与"代价最低"可能是两条路

✅ 参考答案(完整可运行程序,分四段)

```
// lab9.cpp — 迷宫最短路径(BFS + Dijkstra)
// 编译与运行命令见下方"实验环境"说明
#include <algorithm>
#include <fstream>
#include <functional>
#include <iostream>
#include <queue>
#include <string>
#include <utility>
#include <vector>
using namespace std;

// 字符→代价;墙返回 -1
int costOf(char c) {
    switch (c) {
        case 'S': case 'E': case '.': return 1;    // 平地
        case '~': return 5;                        // 沼泽
        default: return -1;                        // '#' 墙
    }
}

vector<string> readMaze(const string& arg) {
    if (arg == "sealed") {                        // 内置样例 2:S 被墙封死
        return {"#####", "##S##", "#.###", "#.E.#", "#####"};
    }
    if (!arg.empty()) {                          // 从文件读
        ifstream in(arg);
        vector<string> m;
        string line;
        while (getline(in, line))
            if (!line.empty()) m.push_back(line);
        return m;
    }
    return {                                     // 内置样例 1:上走廊短(12 步)但有
        "#####",                             // 两格沼泽(代价 21);下走廊长
        "#S...~...#",                         // (18 步)却全平地(代价 18):
        "#.#####.##",                       // BFS 选短,Dijkstra 选廉
        "#.#####.##",
        "#.##....##",
        "#.##.##.##",
        "#.##.##.##",
        "#....##E##",
        "#####",
        "#####",
    };
}
```

lab9.cpp(第二段:顶点编号与 BFS)


```

// 顶点编号 id = 行 * 宽 + 列,四方向邻居即边(隐式图,任务 1)
int findPos(const vector<string>& maze, char ch) {
    int w = int(maze[0].size());
    for (int r = 0; r < int(maze.size()); ++r)
        for (int c = 0; c < w; ++c)
            if (maze[r][c] == ch) return r * w + c;
    return -1;
}

// 任务 2:BFS 无权最短路;无解返回 -1
int bfsShortest(const vector<string>& maze, int sid, int tid,
                vector<int>& path) {
    int h = int(maze.size()), w = int(maze[0].size());
    vector<int> dist(h * w, -1), prev(h * w, -1);
    queue<int> q; // 队列(第3章)
    dist[sid] = 0;
    q.push(sid);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (u == tid) break;
        int r = u / w, c = u % w;
        int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= h || nc < 0 || nc >= w) continue;
            if (maze[nr][nc] == '#') continue; // 墙不可走
            int v = nr * w + nc;
            if (dist[v] != -1) continue; // 第一次到达即最短
            dist[v] = dist[u] + 1;
            prev[v] = u;
            q.push(v);
        }
    }
    if (dist[tid] == -1) return -1; // 无解
    for (int x = tid; x != -1; x = prev[x]) // 前驱数组回溯
        path.push_back(x);
    reverse(path.begin(), path.end());
    return int(path.size()) - 1;
}

```

lab9.cpp(第三段:Dijkstra 与路径绘制)

```

// 任务 3:Dijkstra 求最低代价;无解返回 -1
int dijkstraCost(const vector<string>& maze, int sid, int tid,
                 vector<int>& path) {
    const int INF = 1e9; // 大常量,INF + w 不溢出
    int h = int(maze.size()), w = int(maze[0].size());
    vector<int> dist(h * w, INF), prev(h * w, -1);
    priority_queue<pair<int,int>, vector<pair<int,int>>,
                  greater<pair<int,int>>> pq; // 最小堆
    dist[sid] = 0;
    pq.push({0, sid});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // 懒删除:过期记录跳过
        if (u == tid) break;
        int r = u / w, c = u % w;
        int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= h || nc < 0 || nc >= w) continue;
            int wgt = costOf(maze[nr][nc]);
            if (wgt < 0) continue; // 墙不可走
            int v = nr * w + nc;
            if (dist[v] > d + wgt) {
                dist[v] = d + wgt;
                prev[v] = u;
                pq.push({dist[v], v});
            }
        }
    }
    if (dist[tid] == INF) return -1; // 无解
    for (int x = tid; x != -1; x = prev[x])
        path.push_back(x);
    reverse(path.begin(), path.end());
    return dist[tid];
}

// 画路径:* 标出经过的格子
void drawPath(const vector<string>& maze, const vector<int>& path, int w) {
    vector<string> grid = maze;
    for (int id : path)
        if (grid[id / w][id % w] != 'S' && grid[id / w][id % w] != 'E')
            grid[id / w][id % w] = '*';
    for (const string& row : grid) {
        for (char ch : row) cout << ' ' << ch;
        cout << '\n';
    }
}

void printPathCoords(const vector<int>& path, int w) {
    for (size_t i = 0; i < path.size(); ++i)
        cout << (i ? " -> " : "")
              << "(" << path[i] / w << ", " << path[i] % w << ")";
    cout << '\n';
}

```

```

int main(int argc, char* argv[]) {
    string arg = argc > 1 ? argv[1] : "";
    vector<string> maze = readMaze(arg);
    int w = int(maze[0].size());
    int s = findPos(maze, 'S'), t = findPos(maze, 'E');
    if (s < 0 || t < 0) {
        cout << "迷宫缺少起点 S 或终点 E\n";
        return 1;
    }
    cout << "迷宫 " << maze.size() << " 行 x " << w
        << " 列,起点 (" << s / w << ", " << s % w
        << "),终点 (" << t / w << ", " << t % w << ")\n";
    cout << "图例: # 墙 . 平地(代价1) ~ 沼泽(代价5) S/E 起点/终点\n";

    vector<int> pathB;
    int steps = bfsShortest(maze, s, t, pathB);
    if (steps < 0) {
        cout << "BFS : 无解—起点到终点不可达(被墙封死)\n";
    } else {
        cout << "BFS : 最短步数 = " << steps << "\n 路径: ";
        printPathCoords(pathB, w);
        drawPath(maze, pathB, w);
    }

    vector<int> pathD;
    int cost = dijkstraCost(maze, s, t, pathD);
    if (cost < 0) {
        cout << "Dijkstra: 无解\n";
    } else {
        cout << "Dijkstra: 最低总代价 = " << cost
            << ",步数 = " << int(pathD.size()) - 1 << "\n 路径: ";
        printPathCoords(pathD, w);
        drawPath(maze, pathD, w);
    }

    if (steps >= 0 && cost >= 0) {
        int bfsCost = 0;
        for (int id : pathB) bfsCost += costOf(maze[id / w][id % w]);
        cout << "对比: BFS 最短步数 " << steps << "(路径代价 " << bfsCost
            << ") vs Dijkstra 最低代价路径步数 " << int(pathD.size()) - 1
            << "(代价 " << cost << ")\n";
    }
    return 0;
}

```

示例输入输出(真实运行结果):

```

$ ./lab9
迷宫 10 行 x 10 列,起点 (1,1),终点 (7,7)
BFS : 最短步数 = 12
    路径: (1,1) -> (1,2) -> (1,3) -> (1,4) -> (1,5) -> (1,6) -> (1,7) -> (2,7) -> (3,7) -> (4,7) -> (5,7) -> (6,7) -> (7,7)
Dijkstra: 最低总代价 = 18,步数 = 18
    路径: (1,1) -> (2,1) -> (3,1) -> (4,1) -> (5,1) -> (6,1) -> (7,1) -> (7,2) -> (7,3) -> (7,4) -> (6,4) -> (5,4) -> (4,4) -> (4,5) -> (4,6) -> (4,7) -> (5,7) -> (6,7) -> (7,7)
对比: BFS 最短步数 12(路径代价 21) vs Dijkstra 最低代价路径步数 18(代价 18)

$ ./lab9 sealed
BFS : 无解—起点到终点不可达(被墙封死)
Dijkstra: 无解

```

设计说明:① 顶点编号 $id = \text{行} \times \text{宽} + \text{列}$,一维数组存 dist/prev ——大图标准手法;② "四方向邻居"即边,不显式建邻接表,是"隐式图"代表;③ BFS 与 Dijkstra 共用前驱回溯,体现 §9.3/§9.4 的算法家族关系;④ INF 用 $1e9$ 而非 INT_MAX ,配合懒删除;⑤ 内置"无解"样例:两算法都检查终点距离,输出提示不崩溃。

下一章预告:第10章 算法设计技巧——贪心、分治、动态规划、回溯四大范式。Dijkstra 与 Prim 的贪心本质将在那里升华。

第10章 算法设计技巧

建议学时:5-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础(已读完第1-9章)

🎯 本章学习目标

- 掌握四大范式:贪心、分治、动态规划、回溯,说出适用场景
- 理解"局部最优 $\neq$ 全局最优",会用反例验证贪心猜想
- 能独立实现活动安排与哈夫曼编码(复用第6章二叉堆)
- 掌握 DP 两大要素(最优子结构、重叠子问题),独立写出 LCS 与 0-1 背包
- 掌握随机化:洗牌与 quickselect,完成 `rand()` $\rightarrow$ `<random>` 迁移
- 掌握回溯模板"选择-递归-撤销",独立实现八皇后

💡 从 C 到 C++:本章主题如何迁移

前九章学"给定结构,分析复杂度";本章反过来:给定问题,自己设计算法。四大范式是四套"解题姿势":见什么问题套什么姿势。

C 程序员的习惯是"两层循环 + 标志位"式穷举:n 小时没问题,一到 20, $O(2^n)$ 就彻底失败。四种范式改造穷举:贪心剪掉次优分支,分治切出独立小块,DP 缓存重复计算,回溯按需展开搜索。

C++ 给本章三样武器:① **lambda 与 `std::function`**——C 的回调要函数指针加 `void*` (第7章 `qsort`),C++ 的 `function<void(int)>` 捕获局部变量,把"搜索"与"结果处理"解耦;② `<random>` 库——`rand() % n` 的周期短、概率不均、全局状态三大问题全解决;③ `shuffle`、`nth_element`、`next_permutation`:手写版懂原理,标准库版做工程。

📦 前置知识自查

数学前置:递推式(第2章)、对数、均匀分布直觉。

C 语言前置:`malloc` 二维数组、函数指针(`qsort` 比较器)、递归函数。

依赖的前面章节:第2章(大 O 与分治复杂度)、第6章(二叉堆,哈夫曼用)、第7章(快排分割,quickselect 用)。

🚀 本章学习路线

1. **第一阶段(1.5 小时):**读 §10.1-§10.2,手写活动安排与哈夫曼建树
2. **第二阶段(1.5 小时):**读 §10.3,画 LCS 的 DP 表,默写背包
3. **第三阶段(1.5 小时):**读 §10.4-§10.5,验证洗牌均匀性,实现八皇后
4. **第四阶段(实验):**完成章末综合实验:背包双版本 + 八皇后 + 计时对比
5. **验收标准:**陌生问题 1 分钟内说出"范式、状态、复杂度"

本章知识导图

- §10.1 贪心:活动安排、哈夫曼、硬币反例
- §10.2 分治:最近点对与分治三要素
- §10.3 动态规划:重叠子问题与状态设计(记忆化、LCS、背包)
- §10.4 随机化:洗牌与第 k 小(Fisher-Yates、quickselect)
- §10.5 回溯:选择-递归-撤销(八皇后、子集和)
- §10.6 四大范式对比与选型
- §10.7 工程实践:DP 检查清单与随机化测试

§10.1 贪心:活动安排与哈夫曼编码

10.1.1 贪心思想:每步取局部最优

贪心(greedy) 一句话: 每一步选"当前看起来最好"的选择,希望最终全局最优。它不回头、不后悔,因此要求"无后效性"。经典案例:找零钱、活动安排、哈夫曼。注意: 贪心正确性必须逐题验证,没有通用模板——这是它最大的坑(误区1)。

10.1.2 活动安排问题

会议室只有一个,每个活动有开始与结束时间,同一时刻只能进行一个,问最多能安排几个?策略: 按结束时间最早排序,依次选不冲突的活动——**最早结束优先**。例:10 个活动排序后扫描,选中 (1,4)、(5,7)、(8,11),恰为最优。

示例 10-1 活动安排:最早结束优先

```
// 活动安排:按结束时间升序,依次选不冲突的活动
struct Act { int start, end; };           // 开始、结束时间
vector<Act> activitySelect(vector<Act> acts) {
    sort(acts.begin(), acts.end(),       // 贪心策略:最早结束优先
        [](const Act& a, const Act& b) { return a.end < b.end; });
    vector<Act> res;
    int lastEnd = -1;                     // 上一个选中活动的结束时间
    for (const Act& a : acts) {
        if (a.start >= lastEnd) {         // 不冲突:直接选
            res.push_back(a);
            lastEnd = a.end;
        }
    }
    return res;
}
```

10.1.3 为什么"最早结束"是对的

为什么不用"最早开始"或"持续时间最短"?直觉: 结束得越早,留给后面的空闲越多。严格证明叫**交换论证**(教材给出):把第一个活动换成"结束最早的活动"不会变差——它结束得不晚,后面所有活动仍放得下;逐步替换即可。注意直觉不是证明,后文有反例。

10.1.4 任务调度简介

同族另一个贪心:单机任务调度——已知各任务处理时间,使**平均完成时间最小**。策略:最短处理时间优先(SJF):短任务先完成,大家等待的总时间最小;交换论证同样成立。这是操作系统"短作业优先"的原型。复杂度 $O(n \log n)$ 。

10.1.5 哈夫曼编码:前缀码与二叉堆

变长编码:高频字符用短码、低频用长码。但变长有歧义: $a=0$, $b=01$ 时, 01 是 $a+b$ 还是 b ?靠**前缀码**解决:任何编码都不是另一个编码的前缀。哈夫曼算法构造**平均码长最短**的前缀码:每次合并频次最小的两棵(频次相加),直到剩一棵;频次越低,离根越远、码越长。取最小用第6章二叉堆:

示例 10-2 哈夫曼编码:最小堆建树与编码

```
// 哈夫曼结点:叶存字符,内部结点 ch 为 '\0'
struct HNode {
    char ch;
    int freq;
    HNode *left, *right;
    HNode(char c, int f)
        : ch(c), freq(f), left(nullptr), right(nullptr) {}
    bool isLeaf() const { return ch != '\0'; }
};

// 比较器:频次小者优先(默认大顶堆,取反)
struct Cmp {
    bool operator()(const HNode* a, const HNode* b) const {
        return a->freq > b->freq;
    }
};

// 建树:每次合并频次最小的两棵
HNode* buildHuffman(const map<char, int>& freq) {
    priority_queue<HNode*, vector<HNode*>, Cmp> pq;
    for (const auto& p : freq) pq.push(new HNode(p.first, p.second));
    while (pq.size() > 1) {
        HNode* a = pq.top(); pq.pop();    // 取两个最小频次
        HNode* b = pq.top(); pq.pop();
        HNode* p = new HNode('\0', a->freq + b->freq);    // 合并
        p->left = a; p->right = b;
        pq.push(p);
    }
    return pq.top();    // 只剩根
}

// DFS 生成编码:左 0 右 1,叶即编码
void genCodes(HNode* r, string path, map<char, string>& codes) {
    if (r->isLeaf()) { codes[r->ch] = path; return; }
    genCodes(r->left, path + "0", codes);
    genCodes(r->right, path + "1", codes);
}
```

C 的写法(手写最小堆)	C++ 的写法(priority_queue)
每次取最小线性扫描,建树 $O(n^2)$	堆取最小 $O(\log n)$,建树 $O(n \log n)$
数组 + 上滤下滤手写约 60 行	<code>priority_queue</code> + 自定义比较器几行
比较逻辑写死在代码里	比较器是模板参数,随意更换
结点手动 <code>free</code> ,易泄漏	可配合 <code>unique_ptr</code> 自动释放(第4章)

重点:为什么前缀码无歧义。哈夫曼编码是**树上的叶编码**:字符对应叶,编码是根到叶的路径。根到叶的路径不经过另一棵叶,任意两叶编码互不为前缀——解码器从左到右扫描,到叶就输出一个字符,绝不卡壳。字符必须全放叶上,放进内部结点就产生歧义。

⚠ 误区 1:贪心不是万能——硬币反例

硬币面值 {1, 3, 4} 凑 6 元。"尽量用大面值"给出 4+1+1 共 3 枚;最优是 3+3 共 2 枚。每步局部最优(先用 4 元)恰恰堵死了全局最优(两个 3 元)。贪心无通用证明:先想反例,再考虑交换论证;第9章 Dijkstra、Kruskal 敢用贪心,是因有严格证明。

§10.2 分治:最近点对与分治三要素

10.2.1 回顾:分治三要素与复杂度

分治在第2、7章反复出场(最大子序列和、归并、快排),归纳三要素:分(Divide)——切成更小的子问题;治(Conquer)——递归求解;合(Combine)——合并答案。归并与最大子序列和都是 $T(n) = 2T(n/2) + O(n)$:每层 $O(n)$ 、共 $\log_2 n$ 层,总计 $O(n \log n)$;另一形态 $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$ (二分)。遇到 $T(n) = aT(n/b) + f(n)$,画递归树看每层总代价。

🚀 记号说明(沿用第2章)

递推式用行内代码: $T(n) = 2T(n/2) + O(n)$ 表示"两个规模 $n/2$ 的子问题 + 线性合并"; n^2 、 2^n 、 $\log_2 n$ 同第2章;"平均复杂度"指输入等概率分布下的期望值。

10.2.2 最近点对:思想与结论

平面上 n 个点,求距离最近的一对。暴力两两枚举 $O(n^2)$ 。分治:① **分**:按 x 排序从中间切两半;② **治**:递归求两半各自最小距离得 d ;③ **合**(精髓):跨中线的点对 只需检查"距中线 d 以内"的条带——按 y 排序后每点至多与后面 7 个点比较($d \times 2d$ 矩形内至多塞下常数个点,证明见教材)。总复杂度 $O(n \log n)$ 。

示例 10-3 最近点对:分治骨架


```

struct Point { double x, y; }; // 平面点
double dist(const Point& a, const Point& b) {
    double dx = a.x - b.x, dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
}
// 最近点对:pts 已按 x 升序;返回最小距离
double closestPair(vector<Point> pts) {
    int n = int(pts.size());
    if (n <= 3) { // 基准情形:暴力
        double d = 1e18;
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                d = min(d, dist(pts[i], pts[j]));
        return d;
    }
    int mid = n / 2;
    double midX = pts[mid].x;
    double d = min(
        closestPair(vector<Point>(pts.begin(), pts.begin() + mid)),
        closestPair(vector<Point>(pts.begin() + mid, pts.end())));
    // 合:收集距中线 < d 的点,按 y 排序,每点至多查 7 个
    vector<Point> strip;
    for (const Point& p : pts)
        if (fabs(p.x - midX) < d) strip.push_back(p);
    sort(strip.begin(), strip.end(),
        [](const Point& a, const Point& b) { return a.y < b.y; });
    for (size_t i = 0; i < strip.size(); ++i)
        for (size_t j = i + 1; j < strip.size() && j <= i + 7; ++j)
            d = min(d, dist(strip[i], strip[j]));
    return d;
}

```

§10.3 动态规划:重叠子问题与状态设计

10.3.1 分治与 DP:重叠子问题

分治假设子问题**互不相交**。但递归分解常产生**大量相同子问题**:fib(5) 要算 fib(4) 与 fib(3),fib(4) 又要算 fib(3)——fib(3) 被重复算。**动态规划(DP)**:把子问题答案存进表格,用到查表,每个子问题只算一次。分治靠递归树,DP 靠表格,这是本质区别。

10.3.2 斐波那契:记忆化与自底向上

第1章的朴素递归是 $O(2^n)$,fib(50) 算到地老天荒。两种救法:**记忆化**(自顶向下:递归 + 查表)与**自底向上**(从 fib(0) 按序填表),都做到 $O(n)$;后者省掉递归开销,工程首选。

示例 10-4 斐波那契三版

```

// 版本一:朴素递归  $O(2^n)$ (第1章)
long long fibRec(int n) { return n <= 1 ? n : fibRec(n - 1) + fibRec(n - 2); }
// 版本二:记忆化(自顶向下):先查表再递归
long long fibMemo(int n, vector<long long>& memo) {
    if (memo[n] != -1) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}
// 版本三:自底向上:从小到大填表, $O(n)$ 
long long fibIter(int n) {
    if (n <= 1) return n;
    long long a = 0, b = 1;
    for (int i = 2; i <= n; ++i) { long long t = a + b; a = b; b = t; }
    return b;
}

```

10.3.3 最长公共子序列 LCS

子序列:按顺序挑出(可跳着)的字符序列。**LCS**:两字符串的最长公共子序列。状态: $dp[i][j]$ = s_1 前 i 个与 s_2 前 j 个的 LCS 长度。转移: ① 字符相等 $dp[i][j] = dp[i-1][j-1] + 1$; ② 否则取 $\max(dp[i-1][j], dp[i][j-1])$ 。填完表从右下角沿来路逆推还原出具体 LCS——**方案回溯**是 DP 常客。例:ABCBDAB 与 BDCABA 得长度 4,LCS 为 "BCBA"(不唯一)。

示例 10-5 LCS:DP 表构造与回溯

```

// 返回 LCS 长度并填好 dp 表
int buildLCSTable(const string& s1, const string& s2,
                 vector<vector<int>>& dp) {
    int m = int(s1.size()), n = int(s2.size());
    dp.assign(m + 1, vector<int>(n + 1, 0));
    for (int i = 1; i <= m; ++i)
        for (int j = 1; j <= n; ++j) {
            if (s1[i - 1] == s2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;           // 相等:左上+1
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]); // 不等:取较大
        }
    return dp[m][n];
}
// 回溯:从右下角沿来路逆推,收集 LCS 字符
string traceLCS(const string& s1, const string& s2,
               const vector<vector<int>>& dp) {
    string res;
    int i = int(s1.size()), j = int(s2.size());
    while (i > 0 && j > 0) {
        if (s1[i - 1] == s2[j - 1]) {           // 相等:这个字符属于 LCS
            res += s1[i - 1];
            --i; --j;
        } else if (dp[i - 1][j] >= dp[i][j - 1])
            --i;                                // 从上方来
        else
            --j;                                // 从左方来
    }
    reverse(res.begin(), res.end());
    return res;
}

```

10.3.4 0-1 背包:二维表与一维优化

0-1 背包: n 件物品,第 i 件重 $w[i]$ 、值 $v[i]$,容量 W ,每件要么整放要么不放,求最大总价值。状态: $dp[i][c]$ = 只用前 i 件、容量 c 的最大价值。转移:放第 i 件得 $dp[i-1][c-w[i-1]] + v[i-1]$,不放得 $dp[i-1][c]$,取最大。二维表 $O(nW)$ 时间空间;一维滚动数组把空间压到 $O(W)$,代价是容量必须逆序循环——正序时 $dp[c-w]$ 已被本轮更新,同一件被放多次(误区 2)。

示例 10-6 0-1 背包:二维与一维

```
// 二维版:dp[i][c] = 只用前 i 件、容量 c 的最大价值
int knap2D(const vector<int>& w, const vector<int>& v, int W) {
    int n = int(w.size());
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; ++i)
        for (int c = 1; c <= W; ++c)
            dp[i][c] = (c >= w[i - 1])
                ? max(dp[i - 1][c], dp[i - 1][c - w[i - 1]] + v[i - 1])
                : dp[i - 1][c];
    return dp[n][W];
}

// 一维版:滚动数组。容量必须逆序!正序变成完全背包
int knap1D(const vector<int>& w, const vector<int>& v, int W) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < int(w.size()); ++i)
        for (int c = W; c >= w[i]; --c) // 逆序是关键
            dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
```

C 的写法(手工二维数组)	C++ 的写法(vector)
malloc 每行再逐行 free,易错	<code>vector<vector<int>> dp(n+1, vector<int>(W+1, 0))</code> 自动清零释放
越界读写是未定义行为	<code>dp.at(i).at(c)</code> 越界抛异常,调试友好
维度要手把手传参	<code>dp.size()</code> 自带维度,范围 for 可遍历
换一维滚动数组要重写全部下标	两版结构一致,只差循环方向

⚠ 误区 2:一维背包的容量循环写成正序

一维版要求 $dp[c - w[i]]$ 还是“上一轮”的值。逆序保证更新 $dp[c]$ 时 $dp[c - w[i]]$ 尚未被本轮更新;正序则可能已被更新,第 i 件被用多次——正是完全背包的转移式。口诀:01 背包逆序,完全背包正序。验证:小样例手工跑,或与二维版对拍(实验任务 3)。

★ 重点:DP 解题五步(全章最重要的清单)

① **定义状态**:dp 下标代表什么?要完整描述局面且不冗余;② **写转移方程**:当前状态由哪些更小的状态转移来?③ **定初值与边界**:首行首列、空输入、容量 0 的答案?④ **定计算顺序**:依赖项先算好(通常从小到大);⑤ **方案回溯**:需要方案时记录选择或从终态逆推。

§10.4 随机化:洗牌与第 k 小

10.4.1 Fisher-Yates 洗牌

把数组随机打乱,朴素做法(任意位置交换)分布不均匀。**Fisher-Yates** 的关键:第 i 步只与 $[0, i]$ 中的随机位置交换(从后往前),交换后位置 i 被锁定。共 $n!$ 种等可能交换序列,与排列——对应。 `std::shuffle` 就是它。

示例 10-7 Fisher-Yates 与 `std::shuffle` 并排

```
// 手写 Fisher-Yates:从后往前,每个位置与 [0, i] 中随机位置交换
void fisherYates(vector<int>& a, mt19937& rng) {
    for (int i = int(a.size()) - 1; i >= 1; --i) {
        uniform_int_distribution<int> dist(0, i);    // 关键:上界是 i!
        swap(a[i], a[dist(rng)]);
    }
}
// 标准库写法:一行完成,内部就是 Fisher-Yates
void shuffleStd(vector<int>& a, mt19937& rng) {
    shuffle(a.begin(), a.end(), rng);
}
```

⚠ 误区 3:洗牌上界取 $[0, n-1]$

若每步从 n 个位置随机挑,共 n^n 种等可能序列,排列只有 $n!$ 种—— n^n 不被 $n!$ 整除,必有不均:如 $n=3$ 时 $(1,2,3)$ 概率 $2/9$ 。Fisher-Yates 每步可选数递减($1 \cdot 2 \cdot \dots \cdot n = n!$),——对应。实证:洗 10 万次统计排列分布。

10.4.2 quickselect:求第 k 小

求第 k 小(0 基)。排序后取第 k 位要 $O(n \log n)$ 。**quickselect** 复用第7章快排的**分割(partition)**:枢轴被放到最终位置 p ; $p == k$ 直接返回,否则只递归"包含第 k 小"的那一侧。期望代价 $n + n/2 + n/4 + \dots = 2n$,平均 $O(n)$;最坏(每次切最边上) $O(n^2)$ 。

示例 10-8 quickselect 与 `std::nth_element` 对照

```
// 快排的分割:把枢轴放到最终位置,返回其下标
int partition(vector<int>& a, int lo, int hi) {
    int pivot = a[hi];                // 简化:取末元素做枢轴
    int i = lo;
    for (int j = lo; j < hi; ++j)
        if (a[j] < pivot) swap(a[i++], a[j]);
    swap(a[i], a[hi]);
    return i;
}
// 第 k 小(0 基):分割后只递归"包含第 k 小"的那一侧
int quickSelect(vector<int>& a, int lo, int hi, int k) {
    int p = partition(a, lo, hi);
    if (p == k) return a[k];
    if (p > k) return quickSelect(a, lo, p - 1, k);
    return quickSelect(a, p + 1, hi, k);
}
// 标准库对照:nth_element 后第 k 位即答案(平均  $O(n)$ )
```

10.4.3 随机数:从 C 的 rand() 到现代随机库

C 的 `rand()/srand()` 三个硬伤:① 周期短(约 2^{31});② `rand() % n` 概率不均—— n 不整除 `RAND_MAX` 时余数偏斜,低位周期短;③ 全局状态,线程不安全、不可复制。C++ `<random>` 拆成两层:引擎(mt19937,周期 $2^{19937}-1$)产生原始随机位;分布(`uniform_int_distribution`)映射成均匀整数。种子固定则序列可复现,是复现性的基石;引擎与分布各建一次复用,勿在循环里反复构造。

C 的写法(rand 家族)	C++ 的写法(<random> 家族)
<code>srand(time(NULL)); rand();</code>	<code>mt19937 rng(seed);</code> 引擎与分布分离
<code>rand() % n</code> 概率不均	<code>uniform_int_distribution<int> d(0, n-1)</code> 均匀分布
全局状态,线程不安全,不可复制	引擎是普通对象,可复制、可传参、可并发
周期约 2^{31} ,质量差	mt19937 周期 $2^{19937}-1$,通过统计检验
洗牌/抽样要自己写	<code>std::shuffle</code> / <code>std::nth_element</code> 一行完成

10.4.4 随机算法的工程观

quickselect 平均 $O(n)$ 、最坏 $O(n^2)$ ——"数学概率上快"的含义:把最坏情形的概率压到工程上可忽略。随机选枢轴后,能触发最坏的输入几乎无法被构造;对单次最坏延迟有硬性要求(实时系统)则改用确定性算法。

§10.5 回溯:选择-递归-撤销

10.5.1 回溯模板:选择-递归-撤销

当搜索空间是"一步步做决定"的结构时,回溯(backtracking)是穷举的优雅写法:状态树上深度优先,核心三行——① 选择:决定写进状态;② 递归:进入下一层;③ 撤销:恢复状态,尝试下一个选择。撤销是灵魂,忘了它状态被污染。回溯的本质是带剪枝的深度优先穷举:排列空间 $n!$,子集空间 2^n ;八皇后不剪枝要试 64 选 8,剪枝后只访问约 1.5 万结点。剪枝两类:可行性剪枝(不可能通向合法解)与最优性剪枝(不可能优于已知最好解)——回溯"慢但保证对",贪心"快但可能错"。C++ 里递归用 `std::function` 包装的 lambda,见示例 10-9。

10.5.2 八皇后:完整实现

在 $n \times n$ 棋盘放 n 个皇后,任意两个不能同行、同列、同对角线。暴力枚举 $C(64,8)$ 是天文数字;回溯逐行放皇后, $O(1)$ 冲突检查:列 `col[c]`;主对角线 `d1[row+c]`(同一主对角线上 `row+c` 相同);副对角线 `d2[row-c+n-1]`(`row-c` 相同,加 $n-1$ 防负下标)。放不下就回退换列——"回溯"由此得名。

示例 10-9 八皇后:冲突检查与回调

```
// 通用回溯:每找到一个完整解,调用回调 visit(第7章函数指针的升级)
void nQueens(int row, vector<int>& pos, vector<bool>& col,
            vector<bool>& d1, vector<bool>& d2,
            const function<void(const vector<int>&)>& visit) {
    int n = int(pos.size());
    if (row == n) { visit(pos); return; } // 找到完整解
    for (int c = 0; c < n; ++c) {
        if (col[c] || d1[row + c] || d2[row - c + n - 1])
            continue; // 冲突剪枝
        pos[row] = c;
        col[c] = d1[row + c] = d2[row - c + n - 1] = true; // 选择
        nQueens(row + 1, pos, col, d1, d2, visit); // 递归
        col[c] = d1[row + c] = d2[row - c + n - 1] = false; // 撤销
    }
}
// 调用方:lambda 回调负责"计数与收集前 3 个解"
long long cnt = 0;
vector<vector<int>> first3;
vector<int> pos(8);
vector<bool> col(8), d1(15), d2(15);
nQueens(0, pos, col, d1, d2, [&](const vector<int>& sol) {
    ++cnt;
    if (first3.size() < 3) first3.push_back(sol);
});
// 结果:cnt == 92;前 3 解(1 基):(1,5,8,6,3,7,2,4)、(1,6,8,3,7,4,2,5)、(1,7,4,6,8,2,5,3)
```

10.5.3 子集和问题

给定正整数集合与 target,能否选出若干元素恰好和为 target?每件"选/不选"两个分支,走到叶判断命中。加**可行性剪枝**:已超 target 直接剪掉;总和都不够则不进搜索。子集和没有贪心可依赖(硬币反例同款),只能搜索——这就是范式选型的意义。

§10.6 四大范式对比与选型

10.6.1 四范式大表

范式	核心特征	适用场景	典型例题
贪心	每步局部最优;无后效性;正确性需证明	能构造策略且验证正确	活动安排、哈夫曼、Dijkstra
分治	子问题互不相交;分-治-合;递推式	能切成独立子问题	归并、快排、最近点对
动态规划	子问题重叠;最优子结构;状态与转移	有重叠子问题与最优子结构	斐波那契、LCS、0-1 背包
回溯	DFS 搜索;选择-递归-撤销;剪枝	无多项式思路,规模 ≤ 20	八皇后、子集和、数独

10.6.2 选型决策

- **先看规模**: $n \leq 20 \rightarrow$ 回溯; n 达 $10^6 \rightarrow$ 多项式算法
- **有贪心猜想?**策略可构造且反例想不出 $\rightarrow$ 试贪心并证明;有反例 $\rightarrow$ 放弃
- **能切成独立子问题?**左右互不影响 $\rightarrow$ 分治
- **有重复子问题?** $\rightarrow$ DP:定义状态、写转移、填表

- **都不行?**→ 回溯 + 剪枝,或近似/启发式

10.6.3 什么时候"想不出多项式算法":NP 问题的一句话

旅行商、哈密顿回路等问题至今**没人找到**多项式时间算法,普遍相信不存在——通常称 **NP 难**。遇到它们,心态不是"我再想想",而是:规模小用回溯 + 剪枝,规模大用近似或启发式,并明确告诉用户这是近似解。

§10.7 工程实践:状态检查、剪枝与随机化测试

10.7.1 DP 状态设计检查清单

- 状态参数能否完整描述"当前局面"?少一个参数转移就丢信息
- 转移方程引用的状态都"更小"吗?(防循环依赖)
- 初始值与边界是否齐全?(首行首列、空输入、容量 0)
- 计算顺序是否保证依赖项先算好?(背包容量、LCS 行列)
- 空间能否优化?二维表能否滚成一维?
- 需要输出方案吗?需要就保留完整表或记录选择

10.7.2 回溯剪枝的顺序

剪枝顺序影响常数:① 可行性剪枝放最前(冲突检查、超目标)——砍掉的子树最多;② 对称性剪枝(八皇后旋转/镜像可省约 7/8 搜索,选做);③ 最优性剪枝最后做(求最优解时用)。优先搜索分支更少的方向(如子集和先放大元素),也能显著加速。

10.7.3 随机化在测试中的应用

两个利器:① **随机数据 + 对拍**——用 `mt19937` 生成随机小样例,把"手写实现"与"暴力实现"逐一比对,十万个随机样例比十个手写边界更早暴露 bug(实验任务 3);② **固定种子**——回归可复现,换种子扩覆盖。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
活动安排(最早结束优先)	排序 + 线性扫描	$O(n \log n)$	$O(n \log n)$
任务调度(最短处理时间优先)	排序 + 线性扫描	$O(n \log n)$	$O(n \log n)$
哈夫曼建树	二叉堆(priority_queue)	$O(n \log n)$	$O(n \log n)$
最近点对	分治(条带合并)	$O(n \log n)$	$O(n \log n)$
最大子序列和(回顾)	分治(第2章算法三)	$O(n \log n)$	$O(n \log n)$
归并排序(回顾)	分治	$O(n \log n)$	$O(n \log n)$
快速排序(回顾)	分治 + 随机化枢轴	$O(n \log n)$	$O(n^2)$
斐波那契	朴素递归	$O(2^n)$	$O(2^n)$
斐波那契	记忆化 / 自底向上	$O(n)$	$O(n)$
LCS	动态规划	$O(mn)$	$O(mn)$
0-1 背包	动态规划(二维或一维)	$O(nW)$	$O(nW)$
洗牌	Fisher-Yates / std::shuffle	$O(n)$	$O(n)$
第 k 小	quickselect / std::nth_element	$O(n)$	$O(n^2)$
八皇后	回溯(冲突剪枝)	上界 $O(n!)$	上界 $O(n!)$
子集和	回溯(可行性剪枝)	$O(2^n)$	$O(2^n)$

最小必会清单

- 能说出贪心思想与"无后效性",举出活动安排、哈夫曼案例与硬币反例
- 能默写活动安排,说出复杂度 $O(n \log n)$
- 能默写哈夫曼建树(最小堆合并),解释前缀码解码无歧义
- 能说出分治三要素与最近点对合并思路,写出 $T(n) = 2T(n/2) + O(n)$
- 能说出分治与 DP 的本质区别(子问题独立 vs 重叠)与记忆化、自底向上
- 能独立写出 LCS 的 DP 表与回溯代码,解释 $dp[i][j]$ 的含义
- 能默写 0-1 背包两版,解释一维为何必须逆序
- 能默写 Fisher-Yates,说出"上界必须是 i ";会用 quickselect
- 能默写回溯模板"选择-递归-撤销",独立实现八皇后
- 会用 mt19937 + uniform_int_distribution 与 shuffle/nth_element

自测题

- Q 1. 活动安排为什么按"结束时间最早"贪心?按"开始时间最早"行吗?请举反例。**

✓ 参考答案

结束得越早,给后面留下的时间越多;交换论证保证任何最优解都能换成贪心解。按"最早开始"不行:活动 A(1,10) 霸占全天,(2,3)(4,5)(6,7) 全被挡住——贪心得 1 个,最优 3 个。

Q 2. 面值 {1, 3, 4} 凑 6 元,贪心给出什么?最优是什么?说明了什么?

✓ 参考答案

贪心 4+1+1 共 3 枚,最优 3+3 共 2 枚。局部最优 $\neq$ 全局最优:先用 4 元堵死了两个 3 元。贪心正确性必须逐题验证;人民币面值贪心成立,是"设计面值"的结果。

Q 3. 分治与动态规划的本质区别是什么?各举一个代表问题。

✓ 参考答案

分治的子问题互不相交(归并左半右半无重叠),复杂度来自递归树每层合并代价;DP 的子问题大量重叠(fib(3) 被重复计算),复杂度来自"每个状态只算一次"。代表:分治——归并;DP——LCS、0-1 背包。

Q 4. 0-1 背包的一维滚动数组为什么必须逆序?正序会怎样?

✓ 参考答案

逆序保证更新 $dp[c]$ 时 $dp[c-w[i]]$ 还是"没考虑第 i 件"的旧值;正序时它可能已被本轮更新,同一件物品被用多次,转移式变成完全背包。口诀:01 背包逆序,完全背包正序。验证:与二维版对拍(实验任务 3)。

Q 5. Fisher-Yates 每步的随机上界为什么必须是 i ?取 $[0, n-1]$ 会怎样?

✓ 参考答案

Fisher-Yates 每步可选数递减,共 $n!$ 种等可能交换序列,与排列一一对应,保证均匀。若每步从 n 个位置挑,共 n^n 种序列而排列只有 $n!$ 种, n^n 不被 $n!$ 整除,必然不均($n=3$ 时 (1,2,3) 概率 $2/9$)。实证:洗 10 万次统计分布。

章末实验题:0-1 背包与八皇后综合实验

实验 10:0-1 背包与八皇后综合实验

实验目标:综合运用动态规划与回溯两大范式:背包两版实现与验证、八皇后、计时对比。

实验要求:

- 任务 1:实现 knap2D 与 knap1D,均返回最大价值与方案下标(\$10.3.4 二维与一维、\$10.3.3 方案回溯)
- 任务 2:实现八皇后,输出解的个数与前 3 个解(\$10.5.2 选择-递归-撤销、冲突检查)
- 任务 3:固定种子随机数据($n=20$, $W=200$)验证两版价值与方案一致(\$10.7.3 随机化测试与对拍)
- 任务 4:计时对比一维与二维($n=1000$, $W=10000$),输出耗时与加速比(\$2.1 计时、\$10.3.4 空间优化)
- 任务 5:输入边界——容量 0、无物品时输出 0 而非崩溃(\$10.3.3 边界与初值)

实验环境:C++17。编译: `g++ -std=c++17 -O2 lab10.cpp -o lab10`;运行: `./lab10`。

第一步 写 knap2D(二维表 + 回溯选取),再写 knap1D(一维逆序 + pick 数组);两版判定相同:dp 值"严格变大"即选中

第二步 皇后复用示例 10-9 骨架:三个标记数组 + 回调;输出时列号加 1 转成 1 基

第三步 机数据用 mt19937 固定种子,权重 1-50、价值 1-100;同时比较价值与下标序列

第四步 小数据确认正确,再计时($n=1000$ 、 $W=10000$,结果累加防死代码消除);边界调用 knap1D(3,0,...) 与 knap1D(0,10,...) 确认返回 0

✅ 参考答案(完整可运行程序)

```
// lab10.cpp(第一段:背包两版)
#include <algorithm>
#include <chrono>
#include <functional>
#include <iostream>
#include <random>
#include <vector>
using namespace std;
using Clock = chrono::steady_clock;
// 任务 1a:二维 DP;返回 {价值, 方案下标}
pair<int, vector<int>> knap2D(int n, int W, const vector<int>& w,
                             const vector<int>& v) {
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; ++i)
        for (int c = 1; c <= W; ++c)
            dp[i][c] = (c >= w[i - 1])
                ? max(dp[i - 1][c], dp[i - 1][c - w[i - 1]] + v[i - 1])
                : dp[i - 1][c];
    vector<int> chosen; // 回溯选取方案
    for (int i = n, c = W; i >= 1; --i)
        if (dp[i][c] != dp[i - 1][c]) { chosen.push_back(i - 1); c -= w[i - 1]; }
    reverse(chosen.begin(), chosen.end());
    return {dp[n][W], chosen};
}
// 任务 1b:一维滚动数组(容量必须逆序!)
pair<int, vector<int>> knap1D(int n, int W, const vector<int>& w,
                             const vector<int>& v) {
    vector<int> dp(W + 1, 0);
    vector<vector<char>> pick(n, vector<char>(W + 1, 0));
    for (int i = 0; i < n; ++i)
        for (int c = W; c >= w[i]; --c) {
            int t = dp[c - w[i]] + v[i];
            if (t > dp[c]) { dp[c] = t; pick[i][c] = 1; }
        }
    vector<int> chosen;
    for (int i = n - 1, c = W; i >= 0; --i)
        if (pick[i][c]) { chosen.push_back(i); c -= w[i]; }
    reverse(chosen.begin(), chosen.end());
    return {dp[W], chosen};
}
```

lab10.cpp(第二段:八皇后与主程序)

```

// 任务 2:八皇后(std::function 回调)
long long queensCount = 0;
vector<vector<int>> firstSol; // 前 3 个解
void nQueens(int row, int n, vector<int>& pos, vector<bool>& col,
            vector<bool>& d1, vector<bool>& d2,
            const function<void(const vector<int>&)>& visit) {
    if (row == n) { visit(pos); return; }
    for (int c = 0; c < n; ++c) {
        if (col[c] || d1[row + c] || d2[row - c + n - 1]) continue;
        pos[row] = c;
        col[c] = d1[row + c] = d2[row - c + n - 1] = true; // 选择
        nQueens(row + 1, n, pos, col, d1, d2, visit); // 递归
        col[c] = d1[row + c] = d2[row - c + n - 1] = false; // 撤销
    }
}

int main() {
    mt19937 rng(2026); // 固定种子,可复现
    uniform_int_distribution<int> dw(1, 50), dv(1, 100);
    // 任务 3:随机数据验证两版
    int n = 20, W = 200;
    vector<int> w(n), v(n);
    for (int i = 0; i < n; ++i) { w[i] = dw(rng); v[i] = dv(rng); }
    auto r2 = knap2D(n, W, w, v), r1 = knap1D(n, W, w, v);
    cout << "随机数据 n=" << n << " W=" << W
        << " 价值: 二维=" << r2.first << " 一维=" << r1.first
        << " 方案一致: " << (r2 == r1 ? "是" : "否") << endl;
    // 任务 4:计时对比
    n = 1000; W = 10000;
    w.assign(n, 0); v.assign(n, 0);
    for (int i = 0; i < n; ++i) { w[i] = dw(rng); v[i] = dv(rng); }
    auto t0 = Clock::now();
    auto big2 = knap2D(n, W, w, v); // 累加防死代码消除
    double ms2 = chrono::duration<double, milli>(Clock::now() - t0).count();
    t0 = Clock::now();
    auto big1 = knap1D(n, W, w, v);
    double ms1 = chrono::duration<double, milli>(Clock::now() - t0).count();
    cout << "计时 n=" << n << " W=" << W << " 二维=" << ms2
        << " ms 一维=" << ms1 << " ms 加速比=" << ms2 / ms1 << endl;
    cout << "大样例价值一致: " << (big2.first == big1.first ? "是" : "否")
        << endl;
    // 八皇后(前 3 个解 + 总数)
    vector<int> pos(8);
    vector<bool> col(8), d1(15), d2(15);
    nQueens(0, 8, pos, col, d1, d2, [&](const vector<int>& sol) {
        ++queensCount;
        if (firstSol.size() < 3) firstSol.push_back(sol);
    });
    cout << "八皇后解的总数: " << queensCount << endl;
    for (size_t i = 0; i < firstSol.size(); ++i) {
        cout << "解 " << i + 1 << ": ";
        for (int x : firstSol[i]) cout << x + 1 << ' ';
        cout << endl;
    }
    // 任务 5:边界
    vector<int> wb = {2, 3, 4}, vb = {3, 4, 5};

```

```
cout << "容量 0: " << knap1D(3, 0, wb, vb).first  
    << " 无物品: " << knap1D(0, 10, wb, vb).first << endl;  
return 0;  
}
```

示例输出(本机真实运行;时间因机器而异):

```
随机数据 n=20 W=200 价值: 二维=807 一维=807 方案一致: 是  
计时 n=1000 W=10000 二维=28.0 ms 一维=11.4 ms 加速比=2.47  
大样例价值一致: 是  
八皇后解的总数: 92  
解 1: 1 5 8 6 3 7 2 4  
解 2: 1 6 8 3 7 4 2 5  
解 3: 1 7 4 6 8 2 5 3  
容量 0: 0 无物品: 0
```

设计说明:① 两版共用"dp 值严格变大即选中"的判定,同时比较价值与下标序列,比只比价值更严格;② 一维版用 char 型 pick 数组存选择历史,体积约为二维表的四分之一;计时中一维因缓存友好更快;③ 八皇后用 lambda 回调解耦"搜索"与"结果处理"——正是 §10.5 的 std::function 模式;④ 固定种子 2026 保证任意机器复现相同数据;⑤ 边界任务验证"初值与边界"检查项:容量 0 与无物品时转移自然失效,输出 0。

下一章预告:第11章 摊还分析——聚集法、会计法、势能法。第3章埋下的悬念"vector 倍增扩容为什么平均 $O(1)$ "将在那里得到严格证明;本章的哈夫曼建堆、quickselect 等算法的整体代价也会用摊还视角重新审视。

第11章 摊还分析

建议学时:4-5 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 分清三种"平均":最坏、平均、**摊还**——本章主题是第三种
- 掌握**聚集分析**(栈序列例证)与**会计方法**(动态数组"存 3 元"完整证明)
- 掌握**势能方法**:用 Φ 重推栈,理解通用框架与选 Φ 的直觉
- 会用摊还眼光看第 6 章的堆:buildHeap 为何 $O(n)$,斜堆 merge 为何摊还 $O(\log n)$
- 完成从 C 的 `realloc` 手动扩容到 C++ `vector` 自动倍增的迁移
- 能独立实测 `vector` 扩容拐点,验证"总拷贝 $\approx 2n$ "

💡 从 C 到 C++:本章主题如何迁移

本章是全书数学最重的一章,动机却非常"工程"。回忆 C 里的动态数组: `realloc` 扩容要么每次多开固定格数,要么自己"翻倍"。你凭直觉知道"翻倍更好"却说不出为什么,本章给出严格证明:倍增使 n 次插入总拷贝 $\approx 2n$,摊到每次平均 $O(1)$ 。

C++ 的 `vector` 把这套证明封装进语言: `push_back` 平均 $O(1)$ 、最坏 $O(n)$ 。C 里 `realloc` 后指针失效的坑,在 C++ 里是"迭代器失效"——同一机制的两副面孔。

数学工具不新: Σ 求和与等比级数(第 2 章)、堆与斜堆(第 6 章)。新的是**视角**:不看单个操作,看整个序列的"总账"——聚集、会计、势能三种方法,是同一本总账的三种记法。

📦 前置知识自查

数学前置:求和记号 Σ 、等差与等比级数: $\Sigma_{i=1}^n i = n(n+1)/2$, $\Sigma_{i=0}^n 2^i = 2^{n+1} - 1$;对数 $\log_2 n$ (见第 2 章)。

C 语言前置: `realloc` 手动扩容动态数组(第 3 章)、函数与循环。

依赖的前面章节:第 3 章 `vector` 倍增扩容与 `size/capacity`;第 6 章 buildHeap、斜堆 merge 与二项队列 (§11.5 的结论建立在它们结构上)。

🚀 本章学习路线

- 第一阶段(1.5 小时)**:读 §11.1–§11.2,建立"序列总账"直觉
- 第二阶段(1.5 小时)**:读 §11.3–§11.4,吃透"存 3 元"证明与势能框架
- 第三阶段(2 小时)**:读 §11.5–§11.6,把结论接到第 6 章的堆上;运行代码观察拐点,完成章末实验
- 验收标准**:能手写证明"倍增扩容总拷贝 $\approx 2n$ ",说清三种方法的区别与联系

本章知识导图

- §11.1 动机:摊还 vs 平均 vs 最坏
- §11.2 聚集分析:栈操作序列
- §11.3 会计方法:动态数组的"存3元"证明
- §11.4 势能方法:势能函数与通用框架
- §11.5 摊还在堆上的应用
- §11.6 工程印证:实测扩容拐点
- §11.7 三种方法的选用总结

§11.1 动机:摊还 vs 平均 vs 最坏

11.1.1 三种"平均"不是一回事

前几章我们反复说"vector 尾部插入平均 $O(1)$ 、最坏 $O(n)$ "。这个"平均"是什么意思?复杂度分析里有三种不同的说法,极易混为一谈:

说法	对什么取平均	需要什么假设	例子
最坏情形	不取平均,取最差输入	无假设,对所有输入成立	快排最坏 $O(n^2)$
平均情形	对输入的概率分布取平均	必须假设输入分布(如"等可能")	快排平均 $O(n \log n)$
摊还	对操作序列取平均	无概率假设,对任何序列成立	n 次 <code>push_back</code> 总代价 $O(n)$

摊还分析(amortized analysis):数据结构上任意一个由 n 个操作组成的序列,若总代价为 $O(n \cdot f(n))$,就说每个操作摊还代价为 $O(f(n))$ 。它不假设任何概率分布,对最恶劣的序列也成立。

11.1.2 为什么需要摊还分析

回头看前几章,至少有三个结论是"背下来"而没证明的:

- **第3章**:vector 倍增扩容使 `push_back` "平均 $O(1)$ "——没证明,也没说清是摊还;
- **第6章**:`buildHeap` 是 $O(n)$ ——逐个 `insert` 明明 $O(n \log n)$,整体下滤为何线性?
- **第6章**:斜堆 `merge` 摊还 $O(\log n)$ ——单个 `merge` 最坏 $O(n)$,教材直接给结论;

三个结论分别对应本章的三种方法:聚集证 `buildHeap`,会计证动态数组,势能证斜堆——学完本章,"背诵"变"推导"。摊还分析是算法设计的"定价"工具:一次操作贵没关系,整个序列便宜就行。

⚠ 误区 1:把摊还 $O(1)$ 当单次 $O(1)$

摊还 $O(1)$ 的意思是" n 次操作总代价 $O(n)$ ",**不保证单次操作是 $O(1)$** 。vector 那次触发扩容的 `push_back`,单次代价仍是 $O(n)$ ——只是被前面的便宜 `push` 平摊了。同理,"斜堆 `merge` 摊还 $O(\log n)$ "不代表某一次 `merge` 不会花 $O(n)$ 。实时系统里单次延迟上限有时更关键,这时需要第12章平衡树那种"每次最坏 $O(\log n)$ "的结构。

11.1.3 摊还代价的定义

设操作序列共 n 个操作,第 i 个操作的实际代价记为 c_i ,总代价 $T(n) = \sum_{i=1}^n c_i$ 。若对任意长度为 n 的合法序列都有 $T(n) = O(n \cdot f(n))$,则称每个操作的**摊还代价**为 $O(f(n))$ 。三种方法回答同一问题——"怎么证 $T(n)$ 的上界"——只是记账

方式不同:

方法	一句话	数学工具
聚集分析	先算总账,再平均	Σ 求和
会计方法	每个操作预存,贵的取款	账户余额非负
势能方法	把"存款"记在数据结构状态上	势能函数 Φ 的差

 记号说明

本章公式密集,记号统一: c_i 第 i 个操作的**实际代价**, a_i 其**摊还代价**; $\Phi(D_i)$ 第 i 个操作结束后结构的**势能**(读"Phi"), $\Delta\Phi = \Phi(D_i) - \Phi(D_{i-1})$ 。

§11.2 聚集分析:栈操作序列

11.2.1 方法:先求和,再平均

聚集分析最直接:把 n 个操作的实际代价**全部加起来**得总代价 $T(n)$,除以 n 即摊还代价。难点只在"证总代价上界"——通常靠巧妙的计数观察,而非逐个求和。

11.2.2 经典例子:栈的 push/pop/multiPop

考虑一个从空栈开始的栈,三个操作:push(代价 1)、pop(代价 1)、multiPop(k)(连续弹出 k 个元素,不够时弹到空,代价 = 实际弹出个数,最坏 $O(n)$)。问:任意 n 个操作,总代价是多少?

直接求和会卡住:某次 multiPop 单次就花 $O(n)$,但序列里不可能全是 multiPop——弹出的元素**必须先被 push 进去**,而 n 个操作最多 push n 次。 **关键观察:每个元素至多入栈一次、至多出栈一次**,于是:

出栈总次数 $\leq$ 入栈总次数 $\leq n$,总代价 $\leq 2n$,摊还代价 $O(1)$ 。证明没做任何输入假设: **任何** 长度为 n 的序列总代价都 $\leq 2n$ 。若问"全是 multiPop 怎么办?"——这种序列根本不存在:栈空了就没得弹。这个"不可能"正是证明核心。

11.2.3 代码验证:总代价 $\leq 2n$

示例 11-1 统计栈操作序列的实际总代价


```

#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> st;
    long long total = 0;
    int n = 10000;
    for (int i = 0; i < n; ++i) {
        st.push_back(i);           // push:代价 1
        total += 1;
        if (i % 1000 == 999) {    // 每 1000 次 push 来一个大 multiPop
            while (!st.empty()) { // 弹空:每个元素只出栈一次
                st.pop_back();
                total += 1;
            }
        }
    }
    cout << "操作数 n = " << n
         << ", 总代价 = " << total
         << " ( $\leq 2n =$ " << 2 * n << ") "
         << ", 摊还 = " << (double)total / n << endl;
    return 0;
}

```

§11.3 会计方法:动态数组的"存 3 元"证明

11.3.1 方法:预存存款

会计方法把摊还代价看作"收费":每个操作收摊还代价 a_i 元,其中 c_i 元支付该操作本身, $a_i - c_i$ 元存入**账户**,留给未来昂贵操作取用。唯一要求是:

任何时刻账户余额非负:对每个前缀 $\sum_{i=1}^k (a_i - c_i) \geq 0$ 。

余额非负 $\Rightarrow$ 累加得 $\sum_{i=1}^n a_i \geq \sum_{i=1}^n c_i$,总摊还代价是总实际代价的**上界**。故每个操作收费 $O(1) \Rightarrow$ 总实际代价 $O(n)$ 。

11.3.2 动态数组:插入收 3 元

回到第 3 章的动态数组:push_back 平时只写一个元素(实际代价 1);容量满时倍增扩容要拷贝全部旧元素(实际代价 $O(n)$)。定价 **3 元**:

- 1 元:支付本次写入;
- 2 元:存入账户,留给"未来某次扩容时本元素的拷贝";

扩容时每拷贝一个旧元素从账户支付 1 元。直觉:设扩容前容量为 m (已满),拷贝费 m 元;这 m 个元素中后 $m/2$ 个是**上次**扩容后才插入的,每个带 2 元存款,合计恰好 m 元;前 $m/2$ 个"老元素"的存款原封留存。归纳下去,账户永不_为负。

11.3.3 存款非负的 Σ 推导

严格版本用总账。设元素个数 s 、容量 $c = 2^k$,累计扩容拷贝费是等比级数: $\sum_{j=0}^{k-1} 2^j = c - 1$ 。总收费 $3s$,总花费 s (写入) + $(c - 1)$ (拷贝),余额 = $3s - s - (c - 1) = 2s - c + 1$ 。数组满才扩容,刚扩完 $s = c/2$,故 $s \geq c/2$,余额 $\geq 1 > 0$ 。

余额恒正 $\Rightarrow$ 总摊还 $\geq$ 总实际 $\Rightarrow$ 总代价 $\leq 3n$, 摊还 $O(1)$ 。证明只需"容量按常数因子增长", 1.5 倍、1.125 倍都行; 唯独"每次 +1"不行(摊还 $O(n)$)。

C 的 realloc 手动扩容	C++ 的 vector 自动倍增
realloc 扩容, size 变化自己记账	push_back 自动扩容, size() / capacity() 自带
扩容策略自己写, 每次 +1 就退化到 $O(n^2)$	倍增策略内建, 摊还 $O(1)$ 由标准库保证
失败返回 NULL, 旧指针要自己处理	分配失败抛 bad_alloc, 旧内存由 RAII 释放
扩容后旧指针全部失效	扩容后迭代器失效, 下标访问安全(见 §3.3.3)

11.3.4 代码验证: 账户余额永不为负

示例 11-2 会计方法视角的"存 3 元"动态数组

```
// 每次 push 收 3 元(1 元自用 + 2 元存款), 扩容拷贝从账户支付。
class BankArray {
    int* data_ = nullptr;
    size_t size_ = 0, cap_ = 0;
public:
    long long balance = 0;           // 账户余额
    long long minBalance = 0;       // 全程最低余额
    void push_back(int x) {
        balance += 3;               // 收 3 元
        balance -= 1;               // 支付本次写入
        if (size_ == cap_) {        // 容量满: 倍增
            size_t newCap = cap_ == 0 ? 1 : cap_ * 2;
            int* p = new int[newCap];
            for (size_t i = 0; i < size_; ++i) { // 每拷 1 个付 1 元
                p[i] = data_[i];
                balance -= 1;
            }
            delete[] data_;
            data_ = p;
            cap_ = newCap;
        }
        data_[size_++] = x;
        if (balance < minBalance) minBalance = balance;
    }
    ~BankArray() { delete[] data_; }
};
```

§11.4 势能方法: 势能函数与通用框架

11.4.1 方法: 势能函数 Φ

势能方法(potential method)把会计方法的"账户"从具体元素上抽象出来, 记在**数据结构的状态**上。定义势能函数 $\Phi(D)$ 为状态 D 的一个数值属性(如"栈中元素个数"); 设 D_i 为第 i 个操作结束后的状态, D_0 为初始状态, 则第 i 个操作的摊还代价定义为:

$a_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$, 即 **摊还代价 = 实际代价 + 势能变化 $\Delta\Phi$** 。

两边求和,中间项错位相消(望远镜求和): $\sum_{i=1}^n a_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$ 。

只要 $\Phi(D_0) = 0$ 且 $\Phi(D_i) \geq 0$ 对所有 i 成立,就有 $\sum a_i \geq \sum c_i$,总摊还是总实际代价的上界。与会计方法唯一区别:会计把存款绑在具体元素上,势能把它绑在整个结构的状态上——会计方法是势能方法的特例。

11.4.2 栈的例子重推

取 $\Phi(D)$ = 栈中元素个数, $\Phi(D_0) = 0$,且元素个数永不为负,条件满足。逐个操作算摊还代价:

- **push**:实际 1, $\Delta\Phi = +1$,摊还 = 2;
- **pop**:实际 1, $\Delta\Phi = -1$,摊还 = 0;
- **multiPop(k)**:弹出 k 个,实际 k , $\Delta\Phi = -k$,摊还 = 0;

总摊还 = $2 \times \text{push 次数} \leq 2n$,与聚集分析一致。"贵的"multiPop 摊还代价是 0——它消耗的正是 push 积累的势能。

⚠ 误区 2:把平均情形分析当摊还分析

"快排平均 $O(n \log n)$ "与"vector 插入摊还 $O(1)$ "是两回事:前者对**输入排列**取平均,前提是"输入等可能",数据分布一变结论就失效;后者对**操作序列**取平均,对任何序列都成立——攻击者精心构造"push 到满再 push"的序列,总拷贝也 $\leq 2n$ 。口诀:"随机""等可能""期望"是平均分析;"任意序列""总代价上界"是摊还分析。

11.4.3 自调整表(move-to-front):结论与直觉

自调整表:元素排成一列,每次访问后移到表头(move-to-front),频繁访问者越用越靠前。访问位置 k 的元素实际代价 $O(k)$ (要挪动 $k-1$ 个元素),但**摊还代价 $O(\log n)$** : n 次任意访问的总代价 $O(n \log n)$ 。

证明思路(从简,见教材):取势能 Φ = 表中逆序对个数。把被访问元素从位置 k 提到表头,破坏约 k 个逆序对(势能下降),新产生的逆序对有界;势能变化与移动代价大体抵消,余项用对数界住。结论:摊还 $O(\log n)$ 。自调整思想在工程中遍地开花:LRU 缓存、"最近使用的放前面"的接口设计都是亲戚。

11.4.4 势能方法的通用框架

★ 重点:势能方法四步框架

1. **选 Φ** :只依赖数据结构的状态(元素个数、逆序对数.....);
2. **验证**: $\Phi(D_0)$ 有界且 $\Phi(D_i) \geq 0$;
3. **逐个算**: $a_i = c_i + \Delta\Phi$,昂贵操作 $\Delta\Phi$ 为负、便宜操作为正;
4. **累加得界**: $\sum a_i = O(nf(n))$ 则摊还 $O(f(n))$ 。

选 Φ 的直觉: Φ 应度量"结构中蕴藏的、未来必然要做的活儿"——待弹出的元素、待拷贝的元素、待"纠正"的逆序对。消耗这些"存量"时,实际代价由势能下降支付。

⚠ 误区 3:势能函数选错,存款为负

Φ 必须非负,这是势能方法的前提。反例:栈取 $\Phi(D) = -(\text{元素个数})$, $\Phi(D_0) = 0$ 但 $\Phi(D_i) < 0$,此时 $\sum a_i < \sum c_i$,总摊还代价**低于**总实际代价,不再是上界,"总代价 $O(n)$ "直接失效。更隐蔽的错误:选 Φ 依赖操作历史(如" Φ 取 i 的奇偶"),望远镜求和不再成立。选好 Φ 后务必检查"非负"与"只依赖状态"两条。

§11.5 摊还在堆上的应用

11.5.1 斜堆:merge 摊还 $O(\log n)$

第6章讲过斜堆:没有平衡信息,merge时无条件交换左右子堆,单次merge最坏 $O(n)$,但merge摊还 $O(\log n)$ 。证明用势能法(从简):称"重"结点为右子堆大小 $\geq$ 左子堆一半者,势能 Φ = 重结点总数。merge的实际代价 = 右路径步数 = 轻边数 + 重边数;轻边数 $\leq \log_2 n$ (每经一条轻边子树至少减半),每条重边由势能下降1支付,交换后至多产生 $\log n$ 个新重结点,两抵得 $O(\log n)$ 。斜堆是"摊还换来简单"的典型:无平衡信息、代码最短。

11.5.2 二项队列:insert 摊还 $O(1)$

二项队列的insert可看作"二进制加法":二项树大小是2的幂,合并两个同大小树就像进位 $1+1=10$,于是借用二进制计数器的经典结论:从0加1共 n 次,总翻转位数 $\leq 2n$,摊还 $O(1)$ 。故insert摊还 $O(1)$ 、最坏 $O(\log n)$;deleteMin与merge摊还 $O(\log n)$ 。对比二叉堆:insert最坏就是 $O(\log n)$,不必摊还——摊还分析只在结构把贵操作推迟分摊时,才比最坏更紧。

11.5.3 二叉堆 buildHeap:聚集分析回顾

第6章说buildHeap是 $O(n)$,现在证明。buildHeap从第 $\lfloor n/2 \rfloor$ 个结点起逐个下滤,关键是把结点按高度而非按深度计数:完全二叉树中高度为 h 的结点至多有 $n/2^{h+1}$ 个,每个下滤 $\leq h$ 层,总代价:

$$T(n) \leq \sum_{h=0}^{\log n} (n/2^{h+1}) \cdot h = (n/2) \cdot \sum_{h=1}^{\infty} h/2^h = (n/2) \cdot 2 = O(n).$$

级数 $\sum_{h=1}^{\infty} h/2^h = 2$ 。这就是"逐个insert是 $O(n \log n)$ 、整体下滤只要 $O(n)$ "的原因:多数结点在底层、下滤路径短;高层结点上滤深,但数量指数级地少。下面代码统计真实步数验证。

示例 11-3 统计 buildHeap 的总下滤步数(验证 $\approx n$)

```

// 从第 n/2 个结点向前逐个下滤,统计总步数
#include <iostream>
#include <vector>
using namespace std;

long long heapify(vector<int>& a) {
    int n = int(a.size());
    long long steps = 0;
    auto down = [&](int i) {          // 下滤
        while (true) {
            int l = 2 * i + 1, r = 2 * i + 2, m = i;
            if (l < n && a[l] > a[m]) m = l;
            if (r < n && a[r] > a[m]) m = r;
            if (m == i) break;
            swap(a[i], a[m]);
            ++steps;                    // 交换一层记 1 步
            i = m;
        }
    };
    for (int i = n / 2 - 1; i >= 0; --i) down(i);
    return steps;
}

int main() {
    int n = 65535;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) a[i] = i + 1;    // 升序:对最大堆最不利
    long long s = heapify(a);
    cout << "n = " << n << ", 总下滤步数 = " << s
        << " (理论 O(n),实测 ≈ " << s * 100 / n << "% n)" << endl;
    return 0;
}
// 输出:n = 65535, 总下滤步数 = 65519 (理论 O(n),实测 ≈ 99% n)

```

§11.6 工程印证:实测扩容拐点

11.6.1 手写倍增 vs 标准库:观察扩容拐点

理论说"容量翻倍", `vector` 真的翻倍吗?打印每次扩容的 (size, capacity) 拐点即可见分晓。下面把 手写实现与标准库写法并排对照。

示例 11-4 手写倍增与 `std::vector` 并排对照:观察 capacity 拐点

```

// 手写倍增与 std::vector 并排:打印扩容拐点(size → capacity)
#include <iostream>
#include <vector>
using namespace std;

class MyVector {                                // 手写倍增动态数组
    int* data_ = nullptr; size_t size_ = 0, cap_ = 0;
public:
    void push_back(int x) {
        if (size_ == cap_) {
            size_t newCap = cap_ == 0 ? 1 : cap_ * 2;    // 倍增
            int* p = new int[newCap];
            for (size_t i = 0; i < size_; ++i) p[i] = data_[i];
            delete[] data_;
            data_ = p;
            cap_ = newCap;
            cout << size_ << "→" << cap_ << " ";    // 打印拐点
        }
        data_[size_++] = x;
    }
    ~MyVector() { delete[] data_; }
};

int main() {
    cout << "手写倍增: ";
    MyVector v;
    for (int i = 0; i < 1024; ++i) v.push_back(i);
    cout << "\nstd::vector: ";
    vector<int> sv;                                // 标准库:同样的拐点观察
    size_t prev = 0;
    for (int i = 0; i < 1024; ++i) {
        sv.push_back(i);
        if (sv.capacity() != prev) {    // 容量变化 = 扩容拐点
            cout << sv.size() << "→" << sv.capacity() << " ";
            prev = sv.capacity();
        }
    }
    cout << endl;
    return 0;
}

```

C 的 malloc 数组	C++ 的手写倍增类 / vector
没有 size/capacity 概念,长度、容量全靠变量自管	size() 与 capacity() 是内建接口,拐点一目了然
扩容写错(如每次 +1)代价是 $O(n^2)$,且只有运行慢时才发现	倍增由库保证,摊还 $O(1)$;手写类也要照抄倍增骨架
free 忘记调用即泄漏	析构函数自动释放(RAII)

11.6.2 实测数据解读

本机实测(g++ 13,libstdc++):vector 连续 push_back 1024 次,拐点序列 1→1, 2→2, 3→4, 5→8, 9→16, ..., 513→1024,从 2 起每次翻倍。核账:总拷贝 = 1+2+4+...+512 = 1023 ≈ n,共 10 次扩容。各实现初始容量常数不同,但"几何增长"的摊还结论完全相同。

🚀 工程建议:管理扩容的四个招式

- **预判容量:**知道要装多少时先 `v.reserve(n)`,把扩容次数降为 0——摊还分析告诉你"扩容不贵",但一次都别扩更便宜;
- **只读不扩:**循环里先 `reserve` 再填,迭代器失效问题就此消失;
- **缩容要谨慎:** `shrink_to_fit` 释放多余容量,但下次插入又要重新扩容——"缩了又扩"是双倍浪费,只适合"装完就长期只读"的场景;
- **增长因子不是越大越好:**2 倍省拷贝但峰值内存达 2 倍;1.5 倍更省内存、对缓存更友好。只要因子 > 1,摊还就仍是 $O(1)$ 。

C 的手写扩容代码	C++ 的标准库接口
<code>realloc</code> 每次扩容都是裸内存搬运,策略(倍增还是 +1)写死在调用点	<code>push_back</code> 自动倍增;策略由实现决定, <code>capacity()</code> 把拐点暴露成可观测接口
想预分配要自己写"capacity 变量 + 判断"样板	<code>reserve(n)</code> 一行预分配,扩容次数归零
<code>realloc</code> 返回 NULL 要检查,失败后旧指针仍有效	分配失败抛 <code>bad_alloc</code> ,异常安全由标准库保证

⚠ 误区 4:扩容增量太小,摊还证明失效

"每次 +1"的扩容不是常数因子增长:第 i 次扩容拷贝 $i-1$ 个元素,总拷贝 $\sum_{i=1}^n i \approx n^2/2$,摊还 $O(n)$ ——插入 10 万元素就要拷约 50 亿次。vector 扩容系数是 2,Python 的 list 约 1.125,都是几何增长;"每次多开固定格数"的扩容是平方级灾难。写 C 的 `realloc` 时这是最常见的性能坑:功能正确、规模一大就慢十倍百倍,只靠计时才能发现。

§11.7 三种方法的选用总结

11.7.1 一张表看清三种方法

方法	核心思路	优点	局限	经典用例
聚集分析	总代价求和再平均	最直接	整体求和要可做	栈序列、buildHeap、二进制计数器
会计方法	每操作预存、贵的取款	最直观	要设计收费方案	动态数组"存 3 元"、二项队列 insert
势能方法	势能函数挂在状态上	最强大	选 Φ 靠经验	斜堆 merge、自调整表、splay 树

11.7.2 选型建议

三个方法算的是同一笔账,结论互相印证。选型:结构简单、总量可数,用聚集分析(栈、buildHeap);要给每个操作"讲明白为什么便宜",用会计方法(动态数组);结构复杂、代价藏在状态里,用势能方法(斜堆、自调整表)。摊还分析不是课堂玩具:数据库批量提交、C++ 的 vector、Python 的 list,全是"贵操作由便宜操作平摊"的生意经。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
尾部插入 push_back	动态数组(倍增扩容)	摊还 $O(1)$	$O(n)$ (扩容那次)
尾部插入	动态数组(每次 +1 扩容)	$O(n)$	$O(n)$
push	栈(数组或链表)	$O(1)$	$O(1)$
pop	栈	$O(1)$	$O(1)$
multiPop(k)	栈	摊还 $O(1)$	$O(n)$ (单次)
n 次 push/pop/multiPop 序列	栈(聚集分析)	总代价 $O(n)$, 每个元素至多入出栈一次	
插入(存 3 元)	动态数组(会计方法)	总摊还 $\leq 3n$, 账户余额恒 ≥ 1	
加 1 / insert	二进制计数器 / 二项队列	摊还 $O(1)$	$O(\log n)$ (单次)
访问(移到表头)	自调整表(move-to-front)	摊还 $O(\log n)$	$O(n)$
insert	二叉堆	$O(\log n)$	$O(\log n)$
buildHeap	二叉堆(聚集分析)	$O(n)$	$O(n)$
merge	斜堆(势能方法)	摊还 $O(\log n)$	$O(n)$
merge / deleteMin	二项队列	摊还 $O(\log n)$	$O(\log n)$
会计方法	通用框架	余额非负 $\Rightarrow$ 总摊还 $\geq$ 总实际	
势能方法	通用框架	$a_i = c_i + \Delta\Phi$, Φ 非负 $\Rightarrow$ 摊还为上界	

最小必会清单

- 能独立说出摊还、平均、最坏三种分析的区别,并各举一例
- 能独立证明:任意 n 个 push/pop/multiPop 操作总代价 $\leq 2n$
- 能独立默写"插入收 3 元"的会计证明,并推导余额 $= 2s - c + 1 \geq 1$
- 能独立解释"每次 +1"扩容为何 $O(n^2)$,几何扩容为何摊还 $O(1)$
- 能独立写出势能定义式 $a_i = c_i + \Delta\Phi$,并解释 Φ 为何必须非负
- 能用势能方法(Φ = 元素个数)重推栈:push 2、pop 0、multiPop 0
- 能独立用聚集分析证明 buildHeap 是 $O(n)$ (按高度计数)
- 能独立说出斜堆 merge 摊还 $O(\log n)$ 、二项队列 insert 摊还 $O(1)$

自测题

Q 1. 摊还分析与平均情形分析的根本区别是什么?

✓ 参考答案

平均情形分析对**输入**的分布取平均(如"快排平均 $O(n \log n)$ "假设排列等可能),分布一变结论即失效;摊还分析对**操作序列**取平均,不假设任何概率——" n 次 push_back 总代价 $O(n)$ "对任何序列都成立。

Q 2. 为什么说"全 multiPop 的序列"不存在?这对证明意味着什么?

✓ 参考答案

multiPop 要弹出元素,而元素必须先 push 进去;从空栈开始的序列里,弹出总次数 $\leq$ 入栈总次数 $\leq n$,multiPop 的总代价被 push 次数封顶——昂贵操作在结构上不可能频繁出现,总代价 $\leq 2n$ 对任何序列成立。

Q 3. 动态数组扩容"存 1 元"为什么不够?给出会破产的序列。

✓ 参考答案

存 1 元 = 每次插入收费 2(1 元自用 + 1 元存款)。设 size = s 、容量 $c = 2^k$,总收费 $2s$,总花费 $s + (c - 1)$,余额 = $s - c + 1$ 。数组一满($s = c$)时余额只剩 1;此时扩容:收 2 元、付 1 元自用、再付 c 元拷贝,余额 = $1 + 2 - 1 - c = 2 - c < 0$ —— $c \geq 4$ 即破产。而"存 2 元"(收 3 元)时余额 = $2s - c + 1$,利用 $s \geq c/2$ 得余额 ≥ 1 恒正。扩容是一次性大额开销,存款必须按"每个元素都积累足够配额"定价,2 元是最低安全价。

Q 4. buildHeap 为什么是 $O(n)$ 而不是 $O(n \log n)$?差在哪一步?

✓ 参考答案

逐个 insert 到空堆是 $O(n \log n)$,因为每个元素都要从底部上滤 $\log n$ 层;buildHeap 整体下滤时,多数结点在底层、下滤路径短:高度 h 的结点只有 $n/2^{h+1}$ 个,每个下滤 $\leq h$ 层,总代价 $(n/2) \cdot \sum h/2^h = O(n)$ 。差在"按高度计数"而非"按深度计数",同一棵树的两种视角。

Q 5. 为什么 vector 扩容后迭代器全部失效,而 std::list 不会?这与摊还分析有何联系?

✓ 参考答案

vector 扩容要整体搬家到新内存,旧地址上的一切指针/迭代器随之作废——这是"倍增拷贝"的直接后果,也是摊还 $O(1)$ 付出的代价(总拷贝 $\approx 2n$ 由摊还证明兜底);list 的每个结点独立分配,插入/删除只改指针不搬数据,迭代器天然稳定,代价是按索引访问 $O(n)$ 。选型时,"迭代器稳定"是链表唯一的决胜点。

章末实验题:动态数组扩容的摊还代价实测

实验 11:动态数组扩容的摊还代价实测

实验目标:手写带拷贝计数器的动态数组,用真实数据验证"倍增扩容总拷贝 $\approx 2n$ 、每次 +1 扩容总拷贝 $\approx n^2/2$ ",并用会计方法解释"存 3 元"为什么够。

实验要求:

- 任务 1:手写倍增动态数组 DoublingArray,在扩容处累计真实拷贝次数,提供 totalCopies/lastCost/maxCost 三个统计量(知识点:\$11.3 会计方法、第 3 章倍增扩容)
- 任务 2:连续插入 $n = 10000$ 个元素,统计总拷贝,验证倍增 $\approx 2n$ vs 每次 +1 的笨扩容 $\approx n^2/2$ (知识点:\$11.2)
- 任务 3:对比两种策略的"单次最大代价"与"平均代价",解释摊还 $O(1)$ 与"单次最坏 $O(n)$ "并存(知识点:\$11.1)
- 任务 4:用会计方法注释 BankArray 收费规则,验证全程最低余额 ≥ 0 ,写出余额公式 $2s - c + 1$ (知识点:\$11.3)
- 任务 5:观察标准库 vector 的 capacity 拐点序列,与手写倍增对照(知识点:\$11.6)

实验环境:C++17,需 g++ 或 MSVC。编译: `g++ -std=c++17 -O2 -o lab11 lab11.cpp`;运行 lab11 (Windows 为 lab11.exe)。

第一步 个类共用同一骨架:扩容时先分配新数组,循环拷贝旧元素并更新计数——拷贝计数必须放在真实 for 循环里,不能用 `memcpy` 或 `std::copy`,否则统计失真

第二步 扩容 $n = 10000$ 时总拷贝约 5000 万次,int 会溢出,计数器一律用 long long

第三步 任务 4 的余额更新放在 push_back 末尾统一做,并记录全程最低余额;理论下界 1,实测应 ≥ 1

第四步 任务 5:记录上一次 capacity(),每次 push 后比较,变化即打印 size→capacity

✅ 参考答案(完整可运行程序,分三段)

```
// lab11.cpp — 实验 11:动态数组扩容的摊还代价实测(第一段:两种策略)
#include <iostream>
#include <vector>
using namespace std;

// 任务 1:带真实拷贝计数器的倍增动态数组
class DoublingArray {
    int* data_ = nullptr;
    size_t size_ = 0, cap_ = 0;
public:
    long long totalCopies = 0, lastCost = 0, maxCost = 0;
    void push_back(int x) {
        lastCost = 0;
        if (size_ == cap_) { // 满:倍增扩容
            size_t newCap = cap_ == 0 ? 1 : cap_ * 2;
            int* p = new int[newCap];
            for (size_t i = 0; i < size_; ++i) p[i] = data_[i];
            lastCost = (long long)size_; // 本次 push 的拷贝代价
            totalCopies += size_;
            if (lastCost > maxCost) maxCost = lastCost;
            delete[] data_;
            data_ = p;
            cap_ = newCap;
        }
        data_[size_++] = x;
    }
    ~DoublingArray() { delete[] data_; }
};

// 任务 1:每次只多 1 格的"笨扩容"(对照策略)
class LinearArray {
    int* data_ = nullptr;
    size_t size_ = 0, cap_ = 0;
public:
    long long totalCopies = 0, lastCost = 0, maxCost = 0;
    void push_back(int x) {
        lastCost = 0;
        if (size_ == cap_) { // 满:只多开 1 格
            int* p = new int[cap_ + 1];
            for (size_t i = 0; i < size_; ++i) p[i] = data_[i];
            lastCost = (long long)size_;
            totalCopies += size_;
            if (lastCost > maxCost) maxCost = lastCost;
            delete[] data_;
            data_ = p;
            ++cap_;
        }
        data_[size_++] = x;
    }
    ~LinearArray() { delete[] data_; }
};
```

lab11.cpp(第二段:会计方法"存 3 元"动态数组)

```
// 任务 4:每次 push 收 3 元(1 元自用 + 2 元存款),扩容每拷 1 个付 1 元。
// 余额 = 总收费 3s - 总花费 (s + c - 1) = 2s - c + 1 ≥ 1,恒正 → 摊还 O(1)。
class BankArray {
    int* data_ = nullptr;
    size_t size_ = 0, cap_ = 0;
public:
    long long balance = 0;           // 账户余额
    long long minBalance = 1LL << 60; // 全程最低余额(应恒 ≥ 0)
    void push_back(int x) {
        balance += 3;                // 收 3 元
        balance -= 1;                // 支付本次写入
        if (size_ == cap_) {         // 扩容:每拷 1 个元素付 1 元
            size_t newCap = cap_ == 0 ? 1 : cap_ * 2;
            int* p = new int[newCap];
            for (size_t i = 0; i < size_; ++i) {
                p[i] = data_[i];
                balance -= 1;
            }
            delete[] data_;
            data_ = p;
            cap_ = newCap;
        }
        data_[size_++] = x;
        if (balance < minBalance) minBalance = balance;
    }
    ~BankArray() { delete[] data_; }
};
```

lab11.cpp(第三段:主程序)

```

int main() {
    const int N = 10000;    // 连续插入的元素个数

    // ---- 任务 2、3:拷贝统计与代价对比 ----
    DoublingArray d;
    for (int i = 0; i < N; ++i) d.push_back(i);

    cout << "[任务2/3] 倍增策略:连续插入 " << N << " 个元素\n"
        << " 总拷贝次数 = " << d.totalCopies
        << " (理论  $\approx 2n =$  " << 2 * N << ") \n"
        << " 单次最大代价 = " << d.maxCost
        << " 次拷贝(仅发生在扩容的那一次 push) \n"
        << " 平均代价 = " << (double)d.totalCopies / N
        << " 次拷贝/push \n\n";

    LinearArray l;
    for (int i = 0; i < N; ++i) l.push_back(i);
    cout << "[任务2/3] 笨扩容(每次 +1):连续插入 " << N << " 个元素\n"
        << " 总拷贝次数 = " << l.totalCopies
        << " (理论  $\approx n^2/2 =$  " << (long long)N * N / 2 << ") \n"
        << " 单次最大代价 = " << l.maxCost
        << " 次拷贝(最后一次扩容) \n"
        << " 平均代价 = " << (double)l.totalCopies / N
        << " 次拷贝/push \n\n";

    // ---- 任务 4:验证"存 3 元"够不够 ----
    BankArray b;
    for (int i = 0; i < N; ++i) b.push_back(i);
    cout << "[任务4] 会计方法:存 3 元策略运行 " << N << " 次 push 后\n"
        << " 账户余额 = " << b.balance
        << ", 全程最低余额 = " << b.minBalance
        << " (非负即证明有效) \n\n";

    // ---- 任务 5:vector 的 capacity 拐点 ----
    vector<int> v;
    size_t prev = 0;
    cout << "[任务5] std::vector 扩容拐点(size  $\rightarrow$  capacity): \n ";
    for (int i = 1; i <= 2048; ++i) {
        v.push_back(i);
        if (v.capacity() != prev) {        // 容量变化 = 一次扩容拐点
            cout << v.size() << "  $\rightarrow$  " << v.capacity() << " ";
            prev = v.capacity();
        }
    }
    cout << "\n\n结论: vector 采用倍增策略, 拐点序列 1,2,4,8,..., "
        << "总拷贝  $\approx 2n$ , 与本章理论完全一致。" << endl;
    return 0;
}

```

编译与运行,以及本机实测输出:

```
g++ -std=c++17 -O2 -o lab11 lab11.cpp
./lab11
```

[任务2/3] 倍增策略:连续插入 10000 个元素

总拷贝次数 = 16383 (理论 $\approx 2n = 20000$)

单次最大代价 = 8192 次拷贝(仅发生在扩容的那一次 push)

平均代价 = 1.6383 次拷贝/push

[任务2/3] 笨扩容(每次 +1):连续插入 10000 个元素

总拷贝次数 = 49995000 (理论 $\approx n^2/2 = 50000000$)

单次最大代价 = 9999 次拷贝(最后一次扩容)

平均代价 = 4999.5 次拷贝/push

[任务4] 会计方法:存 3 元策略运行 10000 次 push 后

账户余额 = 3617, 全程最低余额 = 2 (非负即证明有效)

[任务5] `std::vector` 扩容拐点(size $\rightarrow$ capacity):

1 $\rightarrow$ 1 2 $\rightarrow$ 2 3 $\rightarrow$ 4 5 $\rightarrow$ 8 9 $\rightarrow$ 16 17 $\rightarrow$ 32 33 $\rightarrow$ 64 65 $\rightarrow$ 128 129 $\rightarrow$ 256 257 $\rightarrow$ 512 513 $\rightarrow$ 1024 1025 $\rightarrow$ 2048

结论: `vector` 采用倍增策略,拐点序列 1,2,4,8,...,总拷贝 $\approx 2n$,与本章理论完全一致。

设计说明:① 三个类共用"满则扩容、循环拷贝"的骨架,差异只在扩容因子($\times 2$ 与 $+1$),暴露摊还分析的唯一变量;② 拷贝计数放在真实 for 循环而非 `memcpy`,统计语义代价而非内存搬运字节;③ 任务 4 用可运行的"余额"把抽象证明变成可观察的量,实测最低余额 2 与理论下界 1 吻合;④ 任务 5 对照标准库 `vector`,把"平均 $O(1)$ "落到实地:拐点正是 §11.3 证明的倍增策略。

下一章预告:第12章 高级数据结构——红黑树、Treap、跳表、k-d 树;以及标准库选择指南。本章的势能方法是第12章伸展树(splay tree)摊还分析的直接武器;红黑树的摊还/最坏界限对比、Treap 的随机化分析,都会用到本章的三种方法。

第12章 高级数据结构

建议学时:4-5 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解红黑树五条性质与插入修正,知道 `std::map / set` 底层就是红黑树
- 掌握 Treap 与跳表的随机化平衡思想,能完整实现插入/删除/查找
- 掌握 k-d 树最近邻剪枝查询,理解配对堆摊还 $O(1)$ 合并
- 学会 map vs unordered_map 选型,并实现 10^4 级跳表字典与 `std::map` 实测对比

💡 从 C 到 C++:本章主题如何迁移

前 11 章最大的缺口是"平衡":BST 在有序输入下退化成链表,AVL 实现繁琐。本章五个结构给出三种哲学:红黑树用确定不变式强制平衡;Treap 与跳表把平衡交给随机性(期望 $O(\log n)$);配对堆靠摊还分析兜底。

在 C 里做"有序字典"很痛苦:手写 AVL/红黑树要几百行,一个修正分支写错就悄悄失衡;C++ 的答案是一行 `std::map` ——内部就是红黑树。

跳表与 Treap 的价值在"实现简单":代码量接近链表,却拿到期望 $O(\log n)$ ——用"概率上几乎不可能坏"换"写起来几乎不可能错"。

📦 前置知识自查

数学前置:概率期望直觉(几何分布)、对数运算、摊还分析概念(第11章)。

C 语言前置:`struct` 与指针、链表结点增删(第3章)、函数指针、`malloc / free` 对照 `new / delete`。

依赖的前面章节:第4章树(BST、旋转、AVL)、第5章散列、第6章堆、第11章摊还分析。

🚀 本章学习路线

1. **第一阶段(2 小时):**读 §12.1–§12.4,背下修正口诀,手写 Treap、跳表与 k-d 树
2. **第二阶段(1 小时):**读 §12.5–§12.6,理解配对堆与选型决策表
3. **第三阶段(实验,1 小时):**完成章末实验:跳表字典的实现与对比
4. **验收标准:**能随口说出"红黑树高度 $O(\log n)$ " "跳表期望 $O(\log n)$ "

本章知识导图

- §12.1 红黑树:五条性质与插入修正
- §12.2 Treap:随机优先级的树堆
- §12.3 跳表:多层链表的概率平衡
- §12.4 k-d 树:多维空间切分与最近邻
- §12.5 配对堆:摊还 $O(1)$ 合并的实用堆
- §12.6 标准库选型指南
- §12.7 全书总结:十二章知识地图

§12.1 红黑树:五条性质与插入修正

12.1.1 五条性质与"黑高"

红黑树是一棵二叉查找树,每个结点带红/黑一色,满足五条性质:① 结点非红即黑;② 根是黑色;③ 叶结点(NIL)是黑色;④ 红结点的子结点都是黑色——"红结点不能有红孩子";⑤ 从任一结点到其所有后代叶的路径上,黑色结点数相同,这个数叫**黑高**。

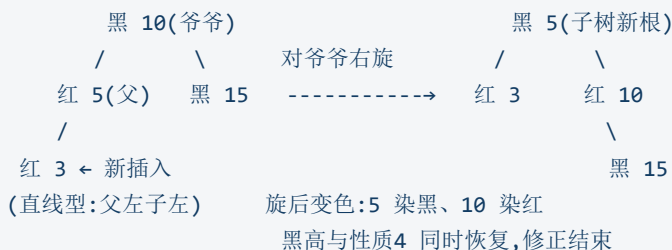
关键保证:设黑高 h_b ,最短路径(全黑)长 h_b ,性质4 又禁红红相邻,最长路径至多 $2h_b$ ——最长 $\leq$ 最短的 2 倍,树高 $O(\log n)$,三操作最坏 $O(\log n)$ 。这就是"弱平衡":不追求 AVL 的高度差 ≤ 1 。

12.1.2 插入修正:变色与旋转

插入先染**红**——红结点不影响性质5(黑高不变),只可能破坏性质4(红父红子)。修正只看叔叔颜色:叔叔红 $\rightarrow$ 变色;叔叔黑 $\rightarrow$ 旋转。

示例 12-1 插入修正:叔叔为黑 $\rightarrow$ 旋转 + 变色

红黑树插入修正之二:叔叔为黑(或空),旋转 + 变色(LL 型为例)



叔叔为红时,父、叔变黑,爷爷变红,冲突上移一层继续检查;爷爷若是根最后染黑。叔叔为黑时按 AVL 的四种形态旋转:LL/RR 直线一次旋转,LR/RL 之字形两次旋转(先转父再转爷爷);旋转后被提上来的结点染黑、爷爷染红。

★ 重点:插入易、删除难

插入只处理"红父红子"一处冲突,沿路径上移 $O(\log n)$ 层;删除要把黑高"挖掉"一层,处理"双黑"问题——红黑树"实现极复杂、接口极简单",删除从不手写,因此适合内建在标准库。

12.1.3 与 2-3-4 树的关系

一句话:**红黑树是 2-3-4 树的二叉表示**——把 2-3-4 树一个结点内的 2~3 个键"压扁"成红黑交替的二叉链。

12.1.4 C++ 视角:STL 的 map/set 就是红黑树

`std::map` / `std::set` 底层就是红黑树,于是 map 有三个赠品:① 中序遍历即按键升序;② 插入/删除其他元素不使既有迭代器失效;③ `lower_bound` / `upper_bound` $O(\log n)$ 定位区间。

C 的手写有序查找表	C++ 的标准库
手写 AVL/红黑树:数百行,修正分支众多	<code>std::map</code> / <code>std::set</code> :一行声明即用
有序遍历要自己写中序递归	范围 <code>for</code> 直接按键序输出
范围查询手写"第一个 $\geq x$ 的结点"	<code>lower_bound</code> / <code>upper_bound</code> 现成
忘记 <code>free</code> 整棵树即泄漏	析构自动释放全部结点

⚠ 误区 1:性质4 的方向与口诀记反

性质4 是"红结点的孩子必须黑",反过来"黑结点的孩子必须红"是错的——黑结点孩子颜色任意。高频错误还有口诀记反:叔叔红时去旋转,树永远修不好。

§12.2 Treap:随机优先级的树堆

12.2.1 思想:BST + 堆序

BST 退化的根源是"输入顺序决定树形"。Treap 换一条路:**Treap**(tree+heap)让每个结点携带一个**随机优先级**,并额外要求优先级满足**堆序**(小顶堆:父 $\leq$ 子)。键决定"在哪里",优先级决定"谁当根"——优先级随机,树形就随机,期望高度 $O(\log n)$ 。把平衡从"旋转纠错"换成"随机决定",实现从几百行降到几十行。

12.2.2 插入与旋转

插入:先按 BST 规则插到叶位置,再沿路径向上旋转,直到父的优先级 $\leq$ 子——与二叉堆"上滤"异曲同工。删除同理:把目标旋转着推下去,变叶子再摘除。

示例 12-5 Treap 完整实现(第一段:结构、旋转、插入)

```

// Treap:键满足 BST 序,随机优先级满足小顶堆序
#include <iostream>
#include <random>
using namespace std;

struct Node {
    int key, pri;
    Node* left = nullptr;
    Node* right = nullptr;
    Node(int k, int p) : key(k), pri(p) {}
};

Node* rotateRight(Node* t) {    // 右旋:提升左孩子
    Node* x = t->left;
    t->left = x->right;
    x->right = t;
    return x;
}

Node* rotateLeft(Node* t) {    // 左旋:提升右孩子
    Node* x = t->right;
    t->right = x->left;
    x->left = t;
    return x;
}

Node* insert(Node* t, int key, int pri) {
    if (!t) return new Node(key, pri);
    if (key < t->key) {
        t->left = insert(t->left, key, pri);
        if (t->left->pri < t->pri) t = rotateRight(t); // 堆序上滤
    } else if (key > t->key) {
        t->right = insert(t->right, key, pri);
        if (t->right->pri < t->pri) t = rotateLeft(t);
    }
    return t;
}

```

示例 12-6 Treap 完整实现(第二段:删除与查找)

```

Node* erase(Node* t, int key) {
    if (!t) return nullptr;
    if (key < t->key) t->left = erase(t->left, key);
    else if (key > t->key) t->right = erase(t->right, key);
    else if (!t->left) { Node* r = t->right; delete t; return r; }
    else if (!t->right) { Node* l = t->left; delete t; return l; }
    else if (t->left->pri < t->right->pri) {
        Node* x = rotateRight(t);
        x->right = erase(t, key);
        return x;
    } else {
        Node* x = rotateLeft(t);
        x->left = erase(t, key);
        return x;
    }
    return t;
}

bool find(Node* t, int key) {
    while (t && t->key != key)
        t = key < t->key ? t->left : t->right;
    return t != nullptr;
}

```

12.2.3 期望 $O(\log n)$ 的直觉与缺陷

直觉:根是整棵树优先级最小的结点,优先级随机,所以"根是第 k 小的键"的概率均匀为 $1/n$,左右子树递归同构——与快排"随机枢轴"的分析同构:Treap 就是快排的树形版本。最坏情形(优先级恰好按键单调)概率 $1/n!$,现实不会出现。优先级相等时须约定比较方向,否则可能破坏 BST 序。

⚠ 误区 2:堆序方向与旋转方向配错

本例用小顶堆(父 $\leq$ 子),口诀"左小右旋、右小左旋";若定义成大顶堆,所有比较都要取反。实现后用有序序列插入再中序遍历:输出必须升序,且每个结点优先级 $\leq$ 子结点。

§12.3 跳表:多层链表的概率平衡

12.3.1 多层链表与几何分布

有序链表查找 $O(n)$,因为每步只能走一个结点。跳表(skip list)的思路:多建几层链表,上层是下层的"快速通道"。每个新结点先进入最底层,再以概率 $1/2$ 晋升,再以概率 $1/2$ 继续.....层数 k 满足几何分布: $P(\text{层} \geq k) = 2^{-(k-1)}$,每层结点数期望 $n/2^{k-1}$ 。查找从最高层出发,每层右移到"下一个更大"结点前再下降,共 $O(\log n)$ 层。

12.3.2 三操作:查找、插入、删除

三操作共享同一"下探"骨架:从最高层向右试探,把每层"最后一个 $< \text{key}$ 的结点"记进 `update` 数组。**查找**:下探后看 `update[0]->next[0]` 是否为 `key`。**插入**:随机层数 `lvl`,若高于当前最高层先补 `update` 头指针,再逐层链入。**删除**:下探后若第 0 层后继即目标,逐层跳过并 `delete`,最后回收空高层。

示例 12-7 跳表完整实现(第一段:结构、查找、插入)

```

#include <iostream>
#include <random>
#include <vector>
using namespace std;

struct Node {
    int key;
    vector<Node*> next;          // next[i]:第 i 层的后继
    Node(int k, int lvl) : key(k), next(lvl + 1, nullptr) {}
};

class SkipList {
    Node* head_;                // 头结点:全部层的起点
    int level_ = 0;
    mt19937 rng_{12345};        // 固定种子
public:
    SkipList(int maxLvl = 16) : head_(new Node(0, maxLvl)) {}
    ~SkipList() {
        Node* p = head_>next[0];
        while (p) { Node* t = p; p = p->next[0]; delete t; }
        delete head_;
    }
    int randomLevel() {          // 几何分布:每层以 1/2 概率继续
        int lvl = 1;
        while ((rng_() & 1) && lvl < int(head_>next.size()) - 1) ++lvl;
        return lvl;
    }
    bool find(int key) const {
        Node* cur = head_;
        for (int i = level_; i >= 0; --i)
            while (cur->next[i] && cur->next[i]->key < key)
                cur = cur->next[i];
        Node* x = cur->next[0];
        return x != nullptr && x->key == key;
    }
    void insert(int key) {
        if (find(key)) return;
        vector<Node*> update(head_>next.size());
        Node* cur = head_;
        for (int i = level_; i >= 0; --i) {
            while (cur->next[i] && cur->next[i]->key < key)
                cur = cur->next[i];
            update[i] = cur;
        }
        int lvl = randomLevel();
        if (lvl > level_) {
            for (int i = level_ + 1; i <= lvl; ++i) update[i] = head_;
            level_ = lvl;
        }
        Node* x = new Node(key, lvl);
        for (int i = 0; i <= lvl; ++i) {
            x->next[i] = update[i]->next[i];
            update[i]->next[i] = x;
        }
    }
};

```

```

    }
}

```

示例 12-8 跳表完整实现(第二段:删除与按序遍历)

```

bool erase(int key) {
    vector<Node*> update(head_>next.size());
    Node* cur = head_;
    for (int i = level_; i >= 0; --i) {
        while (cur->next[i] && cur->next[i]->key < key)
            cur = cur->next[i];
        update[i] = cur;
    }
    Node* x = cur->next[0];
    if (!x || x->key != key) return false;
    for (int i = 0; i <= level_; ++i) {
        if (update[i]->next[i] != x) break;
        update[i]->next[i] = x->next[i];
    }
    delete x;
    while (level_ > 0 && head_>next[level_] == nullptr)
        --level_;
    return true;
}

vector<int> keys() const {
    vector<int> v;
    for (Node* p = head_>next[0]; p; p = p->next[0]) v.push_back(p->key);
    return v;
}

size_t size() const { return keys().size(); }
};

```

12.3.3 与平衡树对比:期望 vs 确定

对比项	跳表	红黑树 / AVL
平衡机制 / 实现难度	随机层数, ~ 链表级, 几十行	旋转 + 不变式, 高, 数百行
高度	期望 $O(\log n)$, 最坏 $O(n)$	最坏保证 $O(\log n)$
每结点内存	平均约 2 个指针	2 个指针 + 颜色位
工程使用	Redis 有序集合 zset、LevelDB	std::map/set、Java TreeMap

C 的手写多级链表	C++ 的跳表类
每层一个 Node* next[k] 数组要手写分配	vector<Node*> next 自动管理层指针
释放多层结点容易重复 free 或泄漏	析构沿第 0 层逐个 delete, RAIL 兜底
随机层数用 rand(), 可移植性差	mt19937 质量与可复现性都有保证

⚠ 误区 3:随机层数用错了分布

层数必须**几何分布**(每层 1/2 概率继续)。三个常见错误:① 固定层数让高层退化成线性扫描 $O(n)$;② 删除后不回收空高层;③ 用 `time()` 做种子,同一秒插入的结点层数高度相关。调试:逐层打印链表,验证"第 0 层完整、高层稀疏"。

§12.4 k-d 树:多维空间切分与最近邻

12.4.1 交替维度切分建树

k-d 树递归地选一个维度取**中位数**作切分点,左子树为小于切分点的点集,右子树反之;下一层换另一个维度。对二维点,第 0 层按 x 切、第 1 层按 y 切、第 2 层再按 x 维度交替,保证每个区域在两个方向都被"削"过,树高 $O(\log n)$ 。

示例 12-9 k-d 树:交替维度建树(二维)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Point { double x, y; };

struct KdNode {
    Point p;
    int dim;           // 切分维度:0 按 x,1 按 y
    KdNode* left = nullptr;
    KdNode* right = nullptr;
    KdNode(const Point& pt, int d) : p(pt), dim(d) {}
};

// nth_element:O(n) 选出中位数并原地分区
KdNode* build(vector<Point>& pts, int lo, int hi, int depth) {
    if (lo > hi) return nullptr;
    int dim = depth % 2;
    int mid = (lo + hi) / 2;
    nth_element(pts.begin() + lo, pts.begin() + mid, pts.begin() + hi + 1,
        [dim](const Point& a, const Point& b) {
            return dim == 0 ? a.x < b.x : a.y < b.y;
        });
    KdNode* node = new KdNode(pts[mid], dim);
    node->left = build(pts, lo, mid - 1, depth + 1);
    node->right = build(pts, mid + 1, hi, depth + 1);
    return node;
}
```

12.4.2 最近邻查询与剪枝

给定查询点 q ,找点集中距离最近者。先沿树下探到 q 所在区域,得到"当前最佳距离" d_{best} ;回溯时——若 q 到分割线的距离 $\geq d_{\text{best}}$,另一侧任何点都不可能更近,整棵子树剪掉。

示例 12-10 k-d 树:最近邻查询(带剪枝)

```
double dist2(const Point& a, const Point& b) {
    double dx = a.x - b.x, dy = a.y - b.y;
    return dx * dx + dy * dy;
}

void nearest(KdNode* node, const Point& q, Point& best, double& bestD) {
    if (!node) return;
    double d = dist2(node->p, q);
    if (d < bestD) { bestD = d; best = node->p; }

    int dim = node->dim;
    double cut = dim == 0 ? node->p.x : node->p.y;
    double qc = dim == 0 ? q.x : q.y;
    KdNode* near = (qc < cut) ? node->left : node->right;
    KdNode* far = (qc < cut) ? node->right : node->left;
    nearest(near, q, best, bestD);

    double dCut = (qc - cut) * (qc - cut);
    if (dCut < bestD)
        nearest(far, q, best, bestD);
}
```

记号说明:距离平方省去开根号

最近邻只需比较大小, `dist2` 返回距离平方即可,省掉每步一次 `sqrt`;剪枝条件用平方比较数学等价。

12.4.3 应用与退化警告

k-d 树是"多维点集查询"的默认答案:游戏引擎的"最近敌人"查询替代逐对检测的 $O(n^2)$,光线追踪与 k-NN 分类也建立在"空间切分 + 剪枝"上。数据挤在一条直线上时剪枝失效,查询最坏 $O(n)$;生产级替代是 R 树或网格索引。

误区 4:剪枝条件漏写或写反

最近邻两种经典错误:① 漏掉 `if (dCut < bestD)` 直接递归两侧——正确但退化成全树搜索 $O(n)$;② 比较方向反了——可能跳过真正更近的点。调试:拿 5 点小数据集逐结点打印"近侧/远侧/是否剪枝"核对。

§12.5 配对堆:摊还 $O(1)$ 合并的实用堆

12.5.1 配对堆是什么

第6章的二叉堆支持 `insert` 与 `deleteMin`,但**合并两个堆**要 $O(n)$ 。左式堆 `merge` 是 $O(\log n)$ 但实现复杂;**配对堆** (pairing heap)走实用路线: `merge` 只比两个根的键,把小根挂到大根的孩子链表里,摊还 $O(1)$; `deleteMin` 把根的孩子两两配对合并,摊还 $O(\log n)$ 。

示例 12-11 配对堆:`merge`、`insert`、`deleteMin`

```

// 配对堆:孩子用"左孩子 + 右兄弟"链表表示,根最小
struct PHNode {
    int val;
    PHNode* child = nullptr;    // 第一个孩子,next 为兄弟链
    PHNode* next = nullptr;
    PHNode(int v) : val(v) {}
};

PHNode* merge(PHNode* a, PHNode* b) {    // 摊还  $O(1)$ 
    if (!a) return b;
    if (!b) return a;
    if (a->val > b->val) swap(a, b);    // 键小的做根
    b->next = a->child;
    a->child = b;
    return a;
}

PHNode* mergePairs(PHNode* first) {    // 两趟合并
    if (!first || !first->next) return first;
    PHNode* a = first;
    PHNode* b = first->next;
    PHNode* rest = b->next;
    return merge(merge(a, b), mergePairs(rest));
}

PHNode* deleteMin(PHNode* root) {
    PHNode* children = root->child;
    delete root;
    return mergePairs(children);
}

// insert = merge(现堆, 单结点堆),摊还  $O(1)$ 

```

12.5.2 三堆选择的一句话

只做优先队列(不合并):二叉堆;需要合并且代码量敏感:配对堆;需要最坏保证的合并:左式堆。代价是摊还而非最坏:单次 `deleteMin` 可能偶发 $O(n)$,长时间平均 $O(\log n)$ 。

§12.6 标准库选型指南

12.6.1 map/set vs unordered_map/set:决策表

`map / set` 底层红黑树, `unordered_map / unordered_set` 底层散列表(第5章)。选哪个:

需求	map / set(红黑树)	unordered_map / set(散列)
按键升序遍历	支持(中序遍历)	不支持(顺序随机)
范围查询 [lo, hi]	支持(lower_bound/upper_bound)	不支持
平均查找速度	$O(\log n)$	$O(1)$,常数更小
最坏查找	$O(\log n)$ 有保证	$O(n)$ (散列冲突)
迭代器失效	插入不失效,删除只失效被删者	再散列(rehash)时全部失效

12.6.2 自定义比较器与自定义哈希

键是自定义结构时,map 需要"严格弱序"(等价的键互比为假),unordered_map 需要哈希与判等:

示例 12-12 自定义比较器与自定义哈希

```
#include <map>
#include <unordered_map>
#include <string>
using namespace std;

struct Person { string name; int age; };

struct ByAge {                                // 自定义比较器:按年龄排序
    bool operator()(const Person& a, const Person& b) const {
        return a.age < b.age;                // 严格弱序:age 相同视为等价
    }
};
map<Person, int, ByAge> ageRank;

struct PersonHash {                            // 自定义哈希:让 Person 可散列
    size_t operator()(const Person& p) const {
        return hash<string>()(p.name) ^ (hash<int>()(p.age) << 1);
    }
};
unordered_map<Person, int, PersonHash> phone;
```

C 的做法	C++ 的做法
qsort(a, n, sz, cmp) 传函数指针,比较逻辑与容器分离	map<K, V, ByAge> 传类型(函数对象/lambda),随容器封装
手写散列表,冲突处理自己维护	unordered_map<K, V, Hash> 只需提供哈希类型
比较器写错静默产生错误排序	严格弱序违规是未定义行为,用断言检查等价性

12.6.3 手写 vs 标准库:有序字典并排

把 §12.3 的跳表包一层,接口与 std::map 对齐——第3章 ADT 的威力:

示例 12-13 手写跳表字典 vs std::map,接口并排

```
// ---- 手写版:Skiplist 包成有序字典 ----
class SkipDict {
    SkipList sl_;
public:
    void insert(int k) { sl_.insert(k); }
    bool find(int k)   { return sl_.find(k); }
    bool erase(int k)  { return sl_.erase(k); }
    vector<int> keys() { return sl_.keys(); }
};

SkipDict dict;
dict.insert(5); dict.insert(3);
vector<int> ord = dict.keys();           // [3, 5]

// ---- 标准库版:同一套接口,红黑树 ----
map<int, string> m;
m[5] = "five"; m[3] = "three"; m[8] = "eight";
for (auto& [k, v] : m) { /* 按序遍历 */ }
auto it = m.lower_bound(4);
```

12.6.4 工程选型建议

- 一次性有序输出:vector + `sort`
- 持续插入 + 需要有序:map(手写红黑树永远不是选项)
- 只查不遍历:unordered_map;冲突多时换更均匀的哈希
- Python 同学没有 map:list+bisect、dict+sorted、sortedcontainers(见 Python 版 §12.6)

§12.7 全书总结:十二章知识地图

12.7.1 十二章知识地图

- 第1章 绪论:递归与数学 → 第2章 算法分析: $O/\Omega/\Theta$
- 第3章 表栈队列:数组与链表 → 第4章 树:BST、AVL、遍历
- 第5章 散列:冲突处理 → 第6章 堆:二叉堆、上滤下滤
- 第7章 排序:八大排序 → 第8章 不相交集:union/find
- 第9章 图:最短路与 MST → 第10章 算法设计:贪心、分治、DP
- 第11章 摊还分析:三方法 → 第12章 高级结构:平衡与期望

12.7.2 "遇到问题选什么结构"终极决策表

需求场景	推荐结构	理由
只做等值查找/去重	散列(unordered_map / dict)	平均 $O(1)$, 常数最小
需要按键有序遍历	红黑树(map)或跳表	中序/底层链即有序序列
范围查询 [lo, hi]	map 的 lower_bound / 有序数组二分	$O(\log n)$ 定位 + $O(k)$ 输出
优先队列(含合并)	二叉堆 / 配对堆	$O(\log n)$ / 合并摊还 $O(1)$
多维点集最近邻	k-d 树	平均 $O(\log n)$ 剪枝查询

12.7.3 下一步学习方向

学完十二章,你已具备"手写任何基础结构 + 正确选型"的能力。三个方向任选:① **算法竞赛**——跳表与 Treap 常作为平衡树的快速替代;② **系统设计**——缓存用散列、排行榜用跳表、范围索引用 B+ 树;③ **数据库索引**——B+ 树是红黑树的"磁盘亲戚"。最后记住:数据结构没有银弹,选型的本质是拿时间复杂度、实现复杂度与最坏保证做三角权衡。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
查找 / 插入 / 删除	红黑树(map/set 底层)	$O(\log n)$	$O(\log n)$
有序遍历全部元素	红黑树中序遍历	$O(n)$	$O(n)$
前驱 / 后继	红黑树迭代器 ++/--	$O(\log n)$	$O(\log n)$
查找 / 插入 / 删除	Treap	期望 $O(\log n)$	$O(n)$ (优先级恰好有序)
查找 / 插入 / 删除	跳表	期望 $O(\log n)$	$O(n)$ (全部同层)
建树	k-d 树(nth_element 切分)	$O(n \log n)$	$O(n \log n)$
最近邻查询	k-d 树	平均 $O(\log n)$	$O(n)$ (数据退化)
合并两个堆	配对堆	摊还 $O(1)$	摊还 $O(1)$
插入(单结点)	配对堆	摊还 $O(1)$	摊还 $O(1)$
deleteMin	配对堆	摊还 $O(\log n)$	摊还 $O(\log n)$
查找	map / set	$O(\log n)$	$O(\log n)$
查找	unordered_map / unordered_set	平均 $O(1)$	$O(n)$ (散列冲突)
范围查询 [lo, hi]	map 的 lower_bound/upper_bound	$O(\log n) + O(k)$	同左(k 为输出个数)
二分查找	有序数组 + std::binary_search	$O(\log n)$	$O(\log n)$
插入到有序数组	vector + insert	$O(n)$	$O(n)$

最小必会清单

- 能默写红黑树五条性质,并解释"性质4+性质5 $\rightarrow$ 高度 $O(\log n)$ "
- 能画出插入修正两个分支(叔叔红变色、叔叔黑旋转)与四种旋转形态
- 能说出"红黑树是 2-3-4 树的二叉表示"
- 能独立实现 Treap 与跳表的插入/删除/查找,说出期望为何 $O(\log n)$
- 能说出跳表与红黑树的取舍(期望 vs 确定、实现难度、内存、并发)
- 能独立实现 k-d 树交替维度建树与最近邻剪枝查询
- 能说出配对堆合并摊还 $O(1)$ 、deleteMin 摊还 $O(\log n)$,一句话对比三堆
- 能背出 map vs unordered_map 决策表,写出自定义比较器(严格弱序)与自定义哈希

自测题

Q 1. 为什么性质4 与性质5 一起能保证红黑树高度 $O(\log n)$?

✓ 参考答案

性质5 说所有路径黑高相同,设黑高 h_b ;性质4 禁红红相邻,最长路径至多 $2h_b$,最短路径长 h_b ——最长 $\leq 2 \times$ 最短。而黑高 h_b 的树至少 $2^{h_b}-1$ 个内部结点,故树高 $\leq 2 \log_2(n+1) = O(\log n)$ 。

Q 2. Treap 的期望高度为什么是 $O(\log n)$?最坏情形是什么?

✓ 参考答案

优先级随机,根是"优先级最小结点"的概率均匀为 $1/n$,左右子树递归同构——与快排随机枢轴同构。最坏是优先级恰好按键单调,高度 $O(n)$,概率 $1/n!$ 。

Q 3. 跳表查找从第几层开始?层数为什么用几何分布而不是固定层数?

✓ 参考答案

从当前最高层开始逐层下降。几何分布(每层 $1/2$ 概率晋升)使每层期望结点数 $n/2^k$,查找每层只走常数步,期望 $O(\log n)$ 。固定层数则退化成线性扫描。

Q 4. k-d 树最近邻查询的剪枝条件是什么?漏写会怎样?

✓ 参考答案

回溯时,若查询点到分割线的距离平方 $\geq$ 当前最佳距离平方,另一侧不可能更近,整棵子树剪掉;只有严格小于才进入。漏写则退化成全树搜索 $O(n)$;方向写反则可能漏掉更近的点。

Q 5. 向 map 与 unordered_map 各插入一个元素,既有迭代器分别会怎样?工程含义?

✓ 参考答案

map(红黑树)插入只改指针不搬内存,除被删除者外所有迭代器保持有效;unordered_map 插入可能触发再散列,全部迭代器失效。"遍历中插入"或"缓存迭代器"必须优先 map——§3.3 迭代器失效思想再现。

章末实验题:跳表字典的实现与对比

实验 12:跳表字典的实现与对比

实验目标:实现完整跳表,封装成有序字典,与 `std::map` 对照验证正确性并实测 10^4 级随机键耗时。

实验要求:

- 任务 1:实现跳表的查找/插入/删除与随机层数(知识点:§12.3 三操作与几何分布)
- 任务 2:封装成有序字典接口 `insert/find/erase/keys`(知识点:§12.3 底层链有序 + 第3章 ADT)
- 任务 3:用 `std::map` 做同一组随机操作,验证输出一致(知识点:§12.6 标准库对照)
- 任务 4:对 10^4 级随机键分别计时插入与查找(知识点:§12.3 期望复杂度 + 第2章计时)
- 任务 5:删除全部元素,验证跳表为空(知识点:§12.3 删除与空层回收)

实验环境:C++17。编译: `g++ -std=c++17 -O2 lab12.cpp -o lab12`;运行 `lab12`。

第一步 示例 12-7/12-8 实现 SkipList,升/降/乱序三组输入冒烟测试,遍历必须升序

第二步 任务 3 用同一种子生成随机操作序列,插入前先 find 去重;任务 4 洗牌生成 10^4 个不重复键,结果累加输出防死代码消除

✔ 参考答案(完整可运行程序,约 140 行,分四段)

```

// lab12.cpp — 跳表字典的实现与对比(第一段:任务1 跳表核心)
#include <iostream>
#include <vector>
#include <random>
#include <map>
#include <algorithm>
#include <chrono>
using namespace std;
using Clock = chrono::steady_clock;

struct Node {
    int key;
    vector<Node*> next;
    Node(int k, int lvl) : key(k), next(lvl + 1, nullptr) {}
};

class SkipList {
    // 任务 1:三操作 + 随机层数
    Node* head_;
    int level_ = 0;
    mt19937 rng_{12345}; // 固定种子:可复现
public:
    SkipList(int maxLvl = 16) : head_(new Node(0, maxLvl)) {}
    ~SkipList() {
        Node* p = head_>next[0];
        while (p) { Node* t = p; p = p->next[0]; delete t; }
        delete head_;
    }
    int randomLevel() { // 几何分布:每层 1/2 概率晋升
        int lvl = 1;
        while ((rng_() & 1) && lvl < int(head_>next.size()) - 1) ++lvl;
        return lvl;
    }
    bool find(int key) const {
        Node* cur = head_;
        for (int i = level_; i >= 0; --i)
            while (cur->next[i] && cur->next[i]->key < key)
                cur = cur->next[i];
        Node* x = cur->next[0];
        return x != nullptr && x->key == key;
    }
    void insert(int key) {
        if (find(key)) return; // 字典语义:键唯一
        vector<Node*> update(head_>next.size());
        Node* cur = head_;
        for (int i = level_; i >= 0; --i) {
            while (cur->next[i] && cur->next[i]->key < key)
                cur = cur->next[i];
            update[i] = cur;
        }
        int lvl = randomLevel();
        if (lvl > level_) {
            for (int i = level_ + 1; i <= lvl; ++i) update[i] = head_;
            level_ = lvl;
        }
        Node* x = new Node(key, lvl);
    }
};

```

```

        for (int i = 0; i <= lvl; ++i) {
            x->next[i] = update[i]->next[i];
            update[i]->next[i] = x;
        }
    }
}

```

lab12.cpp(第二段:删除、遍历 + 任务2 字典封装)

```

bool erase(int key) {
    vector<Node*> update(head_->next.size());
    Node* cur = head_;
    for (int i = level_; i >= 0; --i) {
        while (cur->next[i] && cur->next[i]->key < key)
            cur = cur->next[i];
        update[i] = cur;
    }
    Node* x = cur->next[0];
    if (!x || x->key != key) return false;
    for (int i = 0; i <= level_; ++i) {
        if (update[i]->next[i] != x) break;
        update[i]->next[i] = x->next[i];
    }
    delete x;
    while (level_ > 0 && head_->next[level_] == nullptr) --level_;
    return true;
}

vector<int> keys() const {
    vector<int> v;
    for (Node* p = head_->next[0]; p; p = p->next[0]) v.push_back(p->key);
    return v;
}

size_t size() const { return keys().size(); }
};

class SkipDict { // 任务 2:有序字典接口
    SkipList sl_;
public:
    void insert(int k) { sl_.insert(k); }
    bool find(int k) { return sl_.find(k); }
    bool erase(int k) { return sl_.erase(k); }
    vector<int> keys() { return sl_.keys(); }
    size_t size() const { return sl_.size(); }
};

```

lab12.cpp(第三段:任务3 一致性对照)


```

int main() {
    // 任务 3:与 std::map 对照,输出必须一致
    cout << "== 任务3:跳表字典 vs std::map 一致性 ==" << endl;
    SkipDict dict;
    map<int, int> m;
    mt19937 rng(12345);
    uniform_int_distribution<int> dist(0, 99999);
    for (int i = 0; i < 20000; ++i) {
        int k = dist(rng);
        int op = i % 4;           // 0/1 插,2 删,3 查
        if (op <= 1) {
            if (!m.count(k)) dict.insert(k);
            m[k] = k;
        } else if (op == 2) {
            dict.erase(k);
            m.erase(k);
        } else {
            bool a = dict.find(k), b = m.count(k) > 0;
            if (a != b) { cout << "不一致! key=" << k << endl; return 1; }
        }
    }
    vector<int> dk = dict.keys(), mk;
    for (auto& p : m) mk.push_back(p.first);
    cout << "20000 次随机操作后:跳表 size=" << dict.size()
         << ",map size=" << m.size() << endl;
    cout << "跳表前 10 键: ";
    for (int i = 0; i < 10; ++i) cout << dk[i] << ' ';
    cout << "\n按序遍历顺序一致: " << (dk == mk ? "是" : "否") << endl;
}

```

lab12.cpp(第四段:任务4 计时 + 任务5 删除全部)

```

// 任务 4:10^4 随机键计时对比
const int N = 10000;
vector<int> ks(N);
for (int i = 0; i < N; ++i) ks[i] = i;
shuffle(ks.begin(), ks.end(), rng);
SkipDict s2;
auto t0 = Clock::now();
for (int k : ks) s2.insert(k);
double s1Ins = chrono::duration<double, milli>(Clock::now() - t0).count();
map<int, int> m2;
t0 = Clock::now();
for (int k : ks) m2[k] = k;
double m1Ins = chrono::duration<double, milli>(Clock::now() - t0).count();
long long hits = 0;
t0 = Clock::now();
for (int k : ks) hits += s2.find(k);
double s1Find = chrono::duration<double, milli>(Clock::now() - t0).count();
t0 = Clock::now();
for (int k : ks) hits += m2.count(k);
double m1Find = chrono::duration<double, milli>(Clock::now() - t0).count();
cout << "\n== 任务4:10^4 随机键计时 ==" << endl;
cout << "插入 " << N << " 键:跳表 " << s1Ins << " ms,map " << m1Ins << " ms" << endl;
cout << "查找 " << N << " 键:跳表 " << s1Find << " ms,map " << m1Find << " ms" << endl;
cout << "(命中数 " << hits << ",防止死代码消除)" << endl;

// 任务 5:删除全部元素
for (int k : ks) s2.erase(k);
cout << "\n== 任务5:删除全部后 ==" << endl;
cout << "跳表 size=" << s2.size() << ",为空=" << (s2.size() == 0 ? "是" : "否")
    << ",find(0)=" << s2.find(0) << endl;
return 0;
}

```

示例输出(真实运行,机器不同数值不同,量级关系一致):

```

== 任务3:跳表字典 vs std::map 一致性 ==
20000 次随机操作后:跳表 size=9293,map size=9293
跳表前 10 键: 5 22 38 58 68 78 95 96 101 127
按序遍历顺序一致: 是

== 任务4:10^4 随机键计时 ==
插入 10000 键:跳表 3.1292 ms,map 1.2555 ms
查找 10000 键:跳表 1.188 ms,map 0.6599 ms
(命中数 20000,防止死代码消除)

== 任务5:删除全部后 ==
跳表 size=0,为空=是,find(0)=0

```

设计说明:① update 数组统一"下探路径",插入/删除共享骨架;② 插入前先 find 去重,与 map 键唯一语义对齐;③ 任务3 用同一随机操作序列同时驱动两者,任何一次查找不一致立即退出;④ 同一份键序列计时,两者同为 $O(\log n)$,map 常数更小;删除后回收空高层,size=0 验证删除路径正确。

全书十二章到此结束:第1-2章打下递归与复杂度地基,第3-9章建起线性结构、树、散列、堆、排序、图的工具箱,第10-11章学会设计范式与摊还分析,本章补上最后一块拼图——平衡与概率。数据结构是算法工程师的母语,愿你在竞赛、系统设计或工程实践中带着这十二章的判断力走得更远。